
Bases de Datos

Módulo: 2º DAM Desarrollo de Interfaces





Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Índice

**THINK TWICE
CODE ONCE!**

1. Acceso a una BBDD desde Java
 1. El conector JDBC
 1. Carga del Driver
 2. Establecer y manipular la conexión
 3. Ejecución de instrucciones SQL
 1. Statement
 2. PreparedStatement
 3. CallableStatement
 4. Juegos de registros con ResultSet
 1. Posicionamiento
 2. Lectura de datos
 3. Modificación de los datos
 4. Supresión de los datos
 5. Inserción de los datos
 5. Gestión de las transacciones
 1. Puesta en marcha
 2. SavePoints o puntos de salvaguarda
 3. Niveles de aislamiento

Acceso a una BBDD desde Java

- Cuando se desea manipular una base de datos a partir de un lenguaje de programación, se dispone de **dos soluciones**:
 - Comunicarse directamente con la base de datos.
 - Utilizar una capa de software que asegure el diálogo con la base de datos.
- La primera solución comporta varios requisitos.
 - Debe dominar perfectamente la programación de red.
 - También debe conocer en detalle el protocolo utilizado por la base de datos.
 - Este tipo de desarrollo suele ser muy largo y lleno de trabas. Por ejemplo, ¿puede acceder a las especificaciones del protocolo?
 - Deberá empezar de nuevo todo su trabajo si cambia de tipo de base de datos ya que por supuesto los protocolos no son compatibles de una base de datos a otra. O incluso peor, de una versión a otra de una misma base de datos.

Acceso a una BBDD desde Java

- Por supuesto, la **segunda solución** es preferible y es la que los diseñadores de Java han elegido. Para ello, desarrollaron la **API JDBC** (Java DataBase Connectivity) para el acceso a una base de datos.
- Precisamente, la API JDBC se compone de dos partes. La primera parte, contenida en el paquete **java.sql**, se compone esencialmente de interfaces. Estas interfaces están implementadas por los **drivers JDBC**.
- Estos drivers (o puentes) no están desarrollados por Oracle sino, en general, por la empresa diseñadora de la base de datos. Efectivamente, es el propio fabricante quien domina mejor la técnica para comunicarse con su base de datos. Existen **cuatro tipos** de drivers JDBC con características y rendimiento diferentes.

Acceso a una BBDD desde Java

Tipos de Drivers:

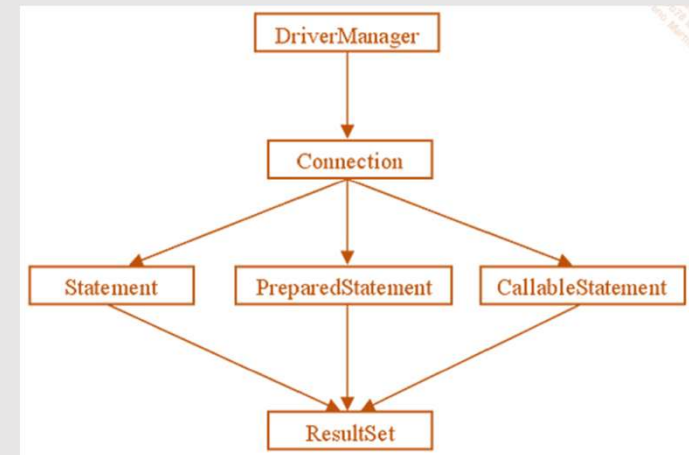
- **Tipo 1: Driver JDBC-ODBC.** Este tipo de puente no es específico de una base de datos concreta, sino que traduce simplemente las instrucciones JDBC en instrucciones ODBC. A continuación, es el driver ODBC quien asegura la comunicación con la base de datos. Esta solución solo ofrece resultados mediocres porque dependen del número de capas de software utilizadas. Las funcionalidades JDBC también están limitadas por las de la capa ODBC. Este tipo de controlador no está disponible a partir de la versión 8 de Java. Oracle recomienda utilizar el controlador específico de la base de datos con la que desee trabajar. Esta, generalmente, disponible en el sitio del fabricante de la base de datos.
- **Tipo 2: Driver nativo:** Este tipo de puente no está escrito completamente en Java. La parte de este driver escrita en Java efectúa sencillamente llamadas hacia funciones del driver nativo. Se efectúan estas llamadas mediante la API JNI (Java Native Interface). De la misma forma que para los puentes de tipo 1, hace falta una traducción entre el código Java y la base de datos. Sin embargo, este tipo sigue siendo más eficaz que los drivers JDBC-ODBC.

Acceso a una BBDD desde Java

- **Tipo 3: Driver que utiliza un servidor intermediario:** Este puente de protocolo de red, desarrollado completamente en Java, no se comunica directamente con la base de datos, sino que se dirige a un servidor intermediario. El diálogo entre el driver y el servidor intermediario es estándar sea cual sea el tipo de base de datos. A continuación, es este servidor intermediario quien transmite las peticiones utilizando un protocolo específico para la base de datos.
- **Tipo 4: Driver completamente escrito en Java:** Este tipo de driver representa la solución ideal, ya que no hay intermediario alguno. El driver transmite directamente las peticiones a la base de datos utilizando el protocolo propio de la base de datos. La mayoría de los puentes actuales son de este tipo.

Presentación de JDBC

- La API JDBC proporciona un conjunto de clases y sobre todo de interfaces que permiten la manipulación de una base de datos. Estos elementos representan todo lo necesario para acceder a los datos desde una aplicación de Java.
- El esquema siguiente retoma el camino lógico desde el driver hasta los datos.
- La clase **DriverManager** es nuestro punto de partida. Es ella quien asegura el vínculo con el driver. A través de ella, podemos obtener una conexión hacia la base de datos. Está representada por una instancia de clase que implementa la interfaz **Connection**. A continuación, se utiliza esta conexión para transmitir instrucciones a la base de datos. Las peticiones simples se ejecutan mediante la interfaz **Statement**; las peticiones con parámetros, con la interfaz **PreparedStatement**, y los procedimientos almacenados, con la interfaz **CallableStatement**.
- Los eventuales registros seleccionados por la instrucción SQL se recogen y devuelven en forma de un elemento **ResultSet**.

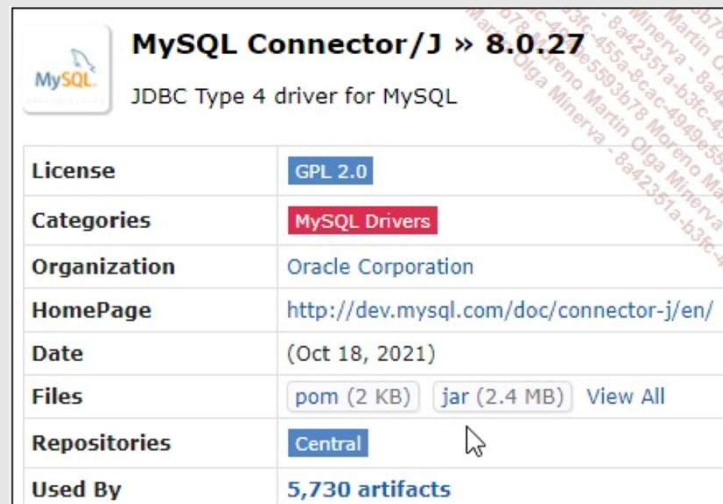


Carga del driver

- La primera etapa indispensable es obtener general, este controlador está disponible para su descarga en el sitio del diseñador de la base de datos. Para nuestros ejemplos, utilizaremos el controlador facilitado por Oracle para acceder a un servidor de base de datos MySQL. La versión 8.0 del driver (llamado **Connector/J**) es necesario para utilizar MySQL 8.0.
- Es posible descargarlo en la dirección URL siguiente: <https://dev.mysql.com/downloads/connector/j/>
- Es suficiente con realizar la descarga adaptada en función de nuestro sistema operativo.
- También es posible descargar de manera más sencilla este driver en el store Maven, en la siguiente dirección: <https://mvnrepository.com/artifact/mysql/mysql-connector-java/8.0.27>
- Se muestra la siguiente diapositivas. Permite la descarga directa del driver, pulsando en **jar (2.4MB)** o usando un administrador de dependencias, como Maven o Gradle. el driver JDBC adaptado a su base de datos.

Carga del driver

- El archivo jar obtenido ([mysql-connector-java-8.0.27.jar](#)) contiene las clases desarrolladas por Oracle que implementan las diferentes interfaces JDBC. Por supuesto, este archivo deberá estar accesible en el momento de la compilación y de la ejecución de la aplicación.

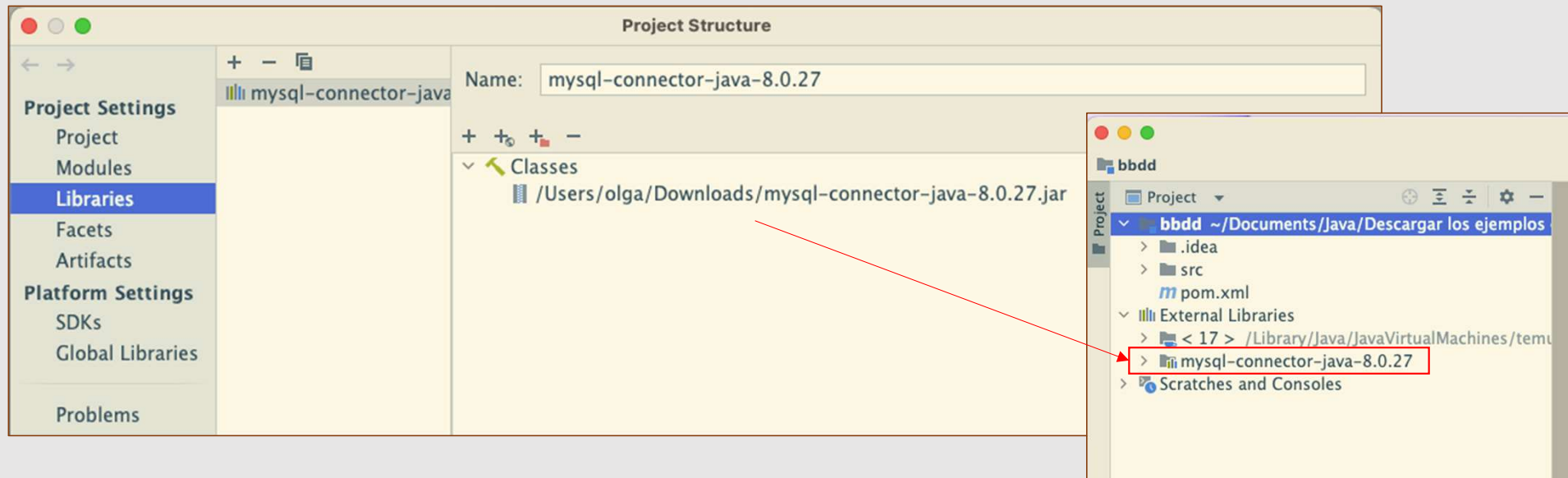


The screenshot shows the Maven Central artifact page for MySQL Connector/J version 8.0.27. The page includes a table with metadata such as License (GPL 2.0), Categories (MySQL Drivers), Organization (Oracle Corporation), and Date (Oct 18, 2021). It also lists available files (pom and jar) and indicates that 5,730 artifacts use this dependency.

 MySQL Connector/J » 8.0.27 JDBC Type 4 driver for MySQL	
License	GPL 2.0
Categories	MySQL Drivers
Organization	Oracle Corporation
HomePage	http://dev.mysql.com/doc/connector-j/en/
Date	(Oct 18, 2021)
Files	pom (2 KB) jar (2.4 MB) View All
Repositories	Central
Used By	5,730 artifacts

Carga del driver

- Para ello, hay que añadir el jar a nuestro proyecto:
- Abrir → File → Project Structure y vamos a la pestaña Libraries para ver la siguiente pantalla, donde pulsando + y buscando la ubicación del archivo en el ordenador lo añadiremos:



Establecer y manipular la conexión

- Una vez descargado e instanciado, el driver es capaz de facilitar una conexión hacia el servidor de base de datos.
- **Establecer la conexión:**
- El método `getConnection` de la clase `DriverManager` se encarga de esta tarea y propone tres soluciones para establecer la conexión. Cada una de las versiones de este método recibe como parámetro una cadena de caracteres que representa una URL que contiene los datos necesarios para establecer la conexión. El contenido de esta cadena de caracteres es específico para cada driver y hay que consultar de nuevo la documentación para obtener la sintaxis adecuada. Sin embargo, el principio de esta cadena de caracteres es estándar y presenta la forma siguiente `jdbc:nombreDelProtocolo`. El nombre del protocolo es propio para cada driver y gracias a este nombre el método `getConnection` es capaz de identificar el driver correcto. El resto de la cadena es específica de cada driver. En general, contiene los datos que permiten identificar el servidor y la base de datos en este servidor, hacia la cual debe establecerse la conexión. Para el driver de MySQL, la sintaxis básica es la siguiente:
`jdbc:mysql://direccionIP/nombreDeLaBaseDeDatos?user=nombreUsuario&password=contraseñaUsuario;`
- En caso de éxito, el método `getConnection` devuelve una instancia de una clase que implementa la interfaz `Connection`. En algunos casos, es conveniente emplear la segunda versión del método `getConnection`, ya que permite indicar el nombre y la contraseña utilizados para establecer la conexión como parámetros separados de la URL de conexión.

Establecer y manipular la conexión - Ejemplo

- Este ejemplo supone que tiene un servidor MySQL instalado en la máquina de desarrollo con una base de datos llamada `nombreDeLaBaseDeDatos` y una cuenta de `usuario` root identificada por la `contraseña` -----.

```
public class TestConnection {
    public static void main(String[] args) {
        String url="jdbc:mysql://localhost/nombreDeLaBaseDeDatos";
        String user="root";
        String password="-----";
        try {
            Connection cnx = DriverManager.getConnection(url,
                user, password);
            System.out.println("Estado de la conexión:");
            System.out.println(cnx.isClosed()? "cerrada": "abierta");
        } catch (SQLException e) {
            System.out.println("Un error se ha producido durante la conexión a la base de datos");
            e.printStackTrace();
        }
    }
}
```

Ejecución de instrucciones SQL : Statement

- Antes de la ejecución de una instrucción SQL, debemos elegir el tipo de objeto JDBC más apropiado. Las diapositivas siguientes describen los tres tipos de objetos disponibles y su utilización.

a. Ejecución de instrucciones básicas con el objeto de tipo Statement

- Un objeto de este tipo se obtiene mediante el método `createStatement` de la conexión. JDBC cuenta con dos versiones de este método; la primera no recibe ningún parámetro. En este caso, si se utiliza el objeto `Statement` para ejecutar una instrucción SQL que genera un juego de registros (`select`), este estará en solo lectura y con un desplazamiento de los datos hacia delante únicamente.
- Los datos presentes en este juego de registros no podrán modificarse y solo se podrá hacer el recorrido del juego de registros desde el primero hacia el último. La segunda versión permite elegir las características del juego de registros generado. Recibe dos parámetros. El primero determina el tipo del juego de registros.

Ejecución de instrucciones SQL : Statement

- Se definen las constantes siguientes de la clase `ResultSet`:
 - `ResultSet.TYPE_FORWARD_ONLY` : el juego de registros será de desplazamiento hacia delante únicamente.
 - `ResultSet.TYPE_SCROLL_INSENSITIVE`: el juego de registros podrá recorrerse en ambos sentidos, pero será insensible a los cambios realizados en la base de datos por otros usuarios.
 - `ResultSet.TYPE_SCROLL_SENSITIVE`: el juego de registros podrá recorrerse en ambos sentidos y será sensible a los cambios realizados en la base de datos por otros usuarios.
- El segundo argumento determina las posibilidades de modificación de los datos contenidos en el juego de los registros. Las dos constantes siguientes están definidas:
 - `ResultSet.CONCUR_READ_ONLY` : los registros son de solo lectura.
 - `ResultSet.CONCUR_UPDATABLE`: los registros pueden modificarse dentro del juego de registros.

Ejecución de instrucciones SQL : Statement

- Este objeto es el más elemental que permite la ejecución de instrucciones SQL. Puede encargarse de la ejecución de cualquier instrucción SQL. Por lo tanto, puede ejecutar tanto instrucciones **DDL** (Data Definition Language) como instrucciones **DML** (Data Manipulation Language). Solo hace falta elegir en este objeto el método mejor adaptado para la ejecución del código SQL. Debería seleccionar un método u otro en función del tipo de resultado que devuelve la instrucción SQL. Hay cuatro métodos disponibles:
- **public boolean execute(String sql)**: este método permite ejecutar cualquier instrucción SQL. El boolean devuelto por este método indica si se ha generado algún juego de registros (true) o si simplemente la instrucción modificó registros en la base de datos (false). Si se genera un juego de registros, puede obtenerse mediante el método **getResultSet**. Si ha habido registros modificados en la base de datos, el método **getUpdateCount** devuelve el número de registros modificados. La función siguiente ilustra el uso de estos métodos.

```

public class TestStatement {
    public static void main(String[] args) {
        try (Connection cnx = getConnection();
            Statement stm = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                ResultSet.CONCUR_READ_ONLY)) {
            BufferedReader br =
                new BufferedReader(new InputStreamReader(System.in));
            System.out.println("introducir su instrucción SQL:");
            String sentencia = br.readLine();
            boolean resultado = stm.execute(sentencia);
            //Si es verdadero, ha habido un resultado
            if (resultado) {
                System.out.println("Su sentencia ha generado un juego de registros");
                //Recuperación del juego de resultados
                ResultSet rs = stm.getResultSet();
                //Posicionar en el último registro
                rs.last();
                //Lectura del número de línea
                System.out.println("contiene " + rs.getRow() + " registros");
                //si es falso, puede que haya una actualización
            } else {
                System.out.println("Su sentencia ha modificado registros en la base");
                System.out.println("Registros modificados:" + stm.getUpdateCount());
            }
        } catch (SQLException e) {
            System.out.println("Su sentencia no ha funcionado correctamente");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static Connection getConnection() throws SQLException {
        String url = "jdbc:mysql://localhost/bbdd?serverTimezone=UTC";
        String user = "root";
        String password = "-----";
        return DriverManager.getConnection(url, user, password);
    }
}

```

Ejecución de instrucciones SQL: : Statement

- Para el ejemplo, se ha creado una tabla PERSONAS con el siguiente script:

```
create database demo_java;  
use demo_java;  
create table PERSONAS(  
id int auto_increment  
primary key,  
nombre varchar(30) not null,  
apellido varchar(30) not null,  
fechaDenacimiento date  
);
```

- Se ha añadido un registro:

```
insert into  
PERSONAS(nombre,apellido,fechaDenacimiento)  
values "McQueen" "Steeve" "1930-03-24");
```

Ejecución de instrucciones SQL : Statement

- `public Resultset executeQuery(String sentencia)`: este método fue diseñado especialmente para la ejecución de instrucciones `select`. El juego de los registros está disponible directamente mediante el valor devuelto por la función.
- `public int executeUpdate(String sentencia)`: este método se adapta perfectamente a la ejecución de instrucciones que modifican el contenido de la base de datos, como son las instrucciones `insert`, `update`, `delete`. El valor entero devuelto por esta función indica el número de registros afectados por la modificación.
- `public int[] executeBatch()` : este método permite ejecutar un conjunto de instrucciones SQL por lote. El lote de instrucciones que se va a ejecutar debe prepararse previamente mediante los métodos `addBatch` y `clearBatch` . El primero recibe como parámetro una cadena de caracteres que representa una instrucción SQL que hay que añadir al lote. El segundo permite reinicializar el lote de instrucciones. No es posible suprimir una instrucción particular del lote. Conviene no añadir al lote ninguna instrucción SQL que genere un juego de resultados, ya que en este caso el método `updateBatch` produce una excepción de tipo `BatchUpdateException` . Esta función devuelve una matriz de enteros que permite obtener información sobre la ejecución de cada una de las peticiones del lote. Cada celda de la matriz contiene un valor entero que representa el resultado de la ejecución de la petición correspondiente en el lote. Un valor superior o igual a 0 indica un funcionamiento correcto de la instrucción y representa el número de registros modificados. Un valor igual a la constante `Statement.EXECUTE_FAILED` indica que la ejecución de la instrucción falló. En este caso, algunos drivers detienen la ejecución del lote, mientras que otros siguen con la instrucción siguiente del lote. Un valor igual a la constante `Statement.SUCCESS_NO_INFO` indica que la instrucción ha sido ejecutada correctamente, pero que no se puede determinar el número de registros modificados. Este método es muy práctico para ejecutar modificaciones en varias tablas relacionadas. Este podría ser, por ejemplo, el caso en una aplicación de gestión comercial con una tabla para las solicitudes y otra para las líneas de detalle de cada solicitud. La supresión de un comando debe causar la supresión de todas las líneas correspondientes. La función siguiente le permite introducir varias instrucciones SQL y ejecutarlas en lote.

```

public class TestStatementBatch {

    public static void main(String[] args) {
        try (Connection cnx = getConnection();
            Statement stm = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_READ_ONLY)) {

            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            System.out.println("rellenar sus instrucciones SQL y ejecutarlas en lote :");
            String consulta = br.readLine();
            while (!consulta.equalsIgnoreCase("run")) {
                stm.addBatch(consulta);
                consulta = br.readLine();
            }
            System.out.println("ejecución del lote de instrucciones");
            int[] resultados = stm.executeBatch();
            for (int i = 0; i < resultados.length; i++) {
                switch (resultados[i]) {
                    case Statement.EXECUTE_FAILED:
                        System.out.println("la ejecución de la instrucción " + i + " ha fallado");
                        break;
                    case Statement.SUCCESS_NO_INFO:
                        System.out.println("la ejecución de la instrucción " + i + " ha tenido éxito");
                        System.out.println("el número de registros modificados es desconocido");
                        break;
                    default:
                        System.out.println("la ejecución de la instrucción " + i + " ha tenido éxito");
                        System.out.println("ha modificado " + resultados[i] + " registros");
                        break;
                }
            }

        } catch (SQLException e) {
            System.out.println("su batch no ha funcionado correctamente");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```


Ejecución de instrucciones SQL : Statement

Otros métodos van a intervenir en el comportamiento del objeto Statement :

- La ejecución de código con instrucciones SQL válidas da el siguiente resultado:
- `public void setQueryTimeOut(int duracion)`: este método indica la duración máxima asignada para la ejecución de una instrucción SQL antes de producir una excepción.
- `public void close()`: cuando un objeto `Statement` ya no es útil en una aplicación, es preferible cerrarlo explícitamente invocando este método. Esto provoca la liberación de todos los recursos usados por este objeto. El cierre de un objeto `Statement` provoca también el cierre del juego de registros asociado (`ResultSet`). Al igual que la interfaz `Connection`, la interfaz `Statement` hereda de la interfaz `AutoCloseable`. Por lo tanto, es posible (como muestran los ejemplos) inicializar este tipo de objeto en una instrucción *try with resources*, para provocar su cierre automático al final del bloque *try*.
- `public void setMaxRows(int número)`: este método limita el número de líneas de los juegos de registros generados por este objeto `Statement` . Si una instrucción SQL genera un juego de registros que contiene más líneas, las líneas de más simplemente se ignoran (sin más información).
- `public void setMaxFieldSize(int tamaño)`: este método limita el tamaño de algunos tipos de campos en el juego de registros. Los tipos de campos afectados son los campos de una base de datos que pueden tener un tamaño variable, como por ejemplo los campos compuestos por cadenas de caracteres o datos binarios. Los datos de más simplemente se ignoran. Los campos afectados pueden ser de hecho inutilizables en la aplicación.
- `public void setFetchSize(int numLineas)`: cuando una instrucción SQL genera un juego de registros, los datos correspondientes se transfieren desde el servidor de base de datos hacia la memoria de la aplicación de Java. Este traslado se efectúa por bloques según el uso de los datos. Este método indica al driver el número de líneas de cada bloque transferido desde la base de datos hacia la aplicación.
- `public boolean getMoreResults()`: si se utiliza el método `execute` para ejecutar varias instrucciones SQL, por ejemplo dos instrucciones `select`, en este caso, se generan dos juegos de registros. El primero se obtiene con el método `getResultSet` . El segundo solo será accesible después de haber llamado el método `getMoreResults` . Esta función permite desplazar el puntero sobre el resultado siguiente y devuelve un booleano igual a `true` si el resultado siguiente es un juego de registros. En tal caso, dicho juego de registros también es accesible mediante el método `getResultSet` . Si la función `getMoreResults` devuelve un booleano `false`, es que el resultado siguiente no es un juego de registros o que sencillamente no hay ningún resultado siguiente. Para resolver la duda, hay que invocar la función `getUpdateCount` y comprobar el valor devuelto por esta función. Si este valor es igual a `-1`, es que no hay ningún resultado siguiente; en caso contrario, el valor devuelto representa el número de registros modificados en la base de datos.

```

public class TestStatementGetMoreResults {
    public static void main(String[] args) {
        boolean resultado;
        ResultSet rs;
        try (Connection cnx = getConnexion();
            Statement stm = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_READ_ONLY)) {

            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            System.out.println("rellenar sus instrucciones SQL separadas por ; :");
            String consulta = br.readLine();
            resultado = stm.execute(consulta);
            int i = 1;
            // tratamiento del resultado generado por la primera instrucción
            if (resultado) {
                System.out.println("su instrucción N° " + i + " ha generado un juego de registros");
                rs = stm.getResultSet();
                rs.last();
                System.out.println("contiene " + rs.getRow() + " registros");
            } else {
                System.out.println("su instrucción N° " + i + " ha modificado los registros en la base");
                System.out.println("número de registros modificados : " + stm.getUpdateCount());
            }
            i++;
            // movimiento del puntero sobre el eventual resultado siguiente
            resultado = stm.getMoreResults();
            // bucle muestra que haya un resultado de tipo de juego de registro -> resultado==true
            // o que haya un resultado de tipo número de registros modificados -> getUpdateCount != -1
            while (resultado || stm.getUpdateCount() != -1) {
                if (resultado) {
                    System.out.println("su instrucción N° " + i + " ha generado un juego de registros");
                    rs = stm.getResultSet();
                    rs.last();
                    System.out.println("contiene " + rs.getRow() + " registros");
                } else {
                    System.out.println("su instrucción N° " + i + " ha modificado los registros en la base");
                    System.out.println("número de registros modificados : " + stm.getUpdateCount());
                }
                i++;
                // movimiento del puntero sobre el eventual resultado siguiente
                resultado = stm.getMoreResults();
            }
        } catch (SQLException e) {
            System.out.println("su instruccion no ha funcionado correctamente");
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static Connection getConnexion() throws SQLException {
        String url = "jdbc:mysql://localhost/demo_java?serverTimezone=UTC&allowMultiQueries=true";
        //Continua...
    }
}

```

Ejecución de instrucciones SQL : PreparedStatement

b. Ejecución de instrucciones configuradas con el objeto PreparedStatement

- Ocurre a menudo que hay que ejecutar varias veces una petición SQL con solo una pequeña modificación entre dos ejecuciones. El ejemplo clásico corresponde a una petición de selección con una restricción.

```
String petition = "select * from PERSONAS where nombre = 'McQueen'";
```

- El valor sobre el cual recae la restricción suele introducirlo el usuario de la aplicación y en este caso está disponible en una variable. La primera solución que viene a la mente consiste en construir la petición SQL por concatenación de varias cadenas de caracteres.

```
String petition = "select * from PERSONAS where nombre = " + nombreIntroducido + "';
```

- Esta solución presenta varios inconvenientes: La concatenación de cadenas de caracteres consume muchos recursos; el servidor debe analizar cada vez una nueva petición y la sintaxis se va a volver compleja rápidamente si es necesario concatenar varias cadenas. Siempre hay que recordar que al final debemos obtener una instrucción SQL correcta.
- Finalmente, la concatenación puede conducir a riesgos de inyecciones SQL que desvirtuarían la función inicial de la consulta. Imagine la búsqueda para identificar a una persona por su nombre de usuario y contraseña, introducidos en un formulario. Podría ser de la siguiente forma:

```
String petition = "select * from PERSONAS where login = " + loginIntroducido + " and contrasenia = " + contraseniaIntroducida + "';
```

Ejecución de instrucciones SQL : PreparedStatement

- Si el usuario rellena los valores «admin» y «mdp1234», entonces la consulta generada es la siguiente:

```
Select * from PERSONAS where login = 'admin' and contrasenia = 'mdp1234'
```

- En consecuencia, si el usuario introduce los valores «admin» y «' or 'a'='a' », entonces la consulta generada es la siguiente:

```
Select * from PERSONAS where login = 'admin' and contrasenia = " or 'a'='a' ;
```

- La consulta adquiere otro aspecto y el comportamiento cambia. Un atacante puede iniciar sesión sin conocer la contraseña del usuario. **Esto se llama inyección SQL**. El error de seguridad en este ejemplo es interpretar las comillas introducidas por el usuario como un elemento de sintaxis SQL. Debe escapar este carácter para indicar que se tiene que interpretar solo como un carácter de texto.

Los diseñadores de JDBC tienen prevista una solución eficaz para resolver estos inconvenientes. El objeto **PreparedStatement** aporta una solución eficaz a este problema al permitirnos crear peticiones con parámetros. En este tipo de petición, se sustituyen los parámetros por signos de interrogación. Antes de la ejecución de la petición, hay que facilitar al objeto **PreparedStatement** los valores que debe utilizar para remplazar los diferentes signos de interrogación. Se puede crear un objeto **PreparedStatement** utilizando el mismo principio que para la creación de un objeto **Statement**.

Ejecución de instrucciones SQL : PreparedStatement

- El método `prepareStatement` accesible a partir de una conexión devuelve un objeto `PreparedStatement`. Este método está disponible bajo dos formas. La primera recibe como argumento una cadena de caracteres que representa la petición SQL. En esta petición, la ubicación de los parámetros debe estar reservada por signos de interrogación. Si se crea un juego de registros con la ejecución de esta petición, será de solo lectura y de desplazamiento únicamente hacia delante. La segunda forma del método `prepareStatement` recibe, además de la cadena de caracteres, un argumento que indica el tipo de juego de registros y otro que determina las posibilidades de modificación de los datos contenidos en el juego de registros.
- Se pueden utilizar las mismas constantes que para el método `createStatement`.

```
PreparedStatement pstmt=cnx.prepareStatement("select * from PERSONAS where nombre=?",  
                                             ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_READ_ONLY);
```

Antes de la ejecución de la instrucción SQL, debemos facilitar un valor para cada uno de los signos de interrogación que representan un parámetro. Para ello, el objeto `PreparedStatement` dispone de numerosos métodos que permiten asignar un valor a un parámetro. Cada uno de estos métodos corresponde al tipo de datos SQL que se ha de insertar en lugar de cada signo de interrogación. Estos métodos siguen la siguiente nomenclatura: `setXxx`, donde `Xxx` representa un tipo de datos SQL. Cada uno de estos métodos recibe como primer argumento un entero que corresponde al rango del parámetro en la instrucción SQL. El primer parámetro se sitúa en el rango 1. El segundo argumento corresponde al valor que se ha de transferir en el parámetro. El tipo de este argumento corresponde por supuesto al tipo de datos SQL que hay que trasladar hacia el parámetro. El driver JDBC convierte a continuación el tipo Java en tipo SQL.

Ejecución de instrucciones SQL : PreparedStatement

- Por ejemplo, el método `setInt (int indiceParam, int value)` efectúa una conversión del tipo `int` java en tipo `INTEGER` SQL. Cada proveedor de un controlador JDBC proporciona documentación que enumera las relaciones entre los tipos Java y los tipos SQL. Esta documentación para el controlador JDBC está disponible en la siguiente dirección:
<https://dev.mysql.com/doc/connector-j/8.0/en/connector-j-reference-type-conversions.html>
- Se conservan los valores almacenados en los parámetros de una ejecución de la instrucción SQL a la siguiente. El método `clearParameters` permite reinicializar el conjunto de los parámetros.
- Se pueden utilizar los parámetros en una instrucción SQL en sustitución de valores, pero nunca para sustituir el nombre de un campo, y aun menos de un operador. Por supuesto, la sintaxis siguiente está prohibida:

```
PreparedStatement pstmt = cnx.prepareStatement("select * from PERSONAS where ? = ?", ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_READ_ONLY);

pstmt.setString(1,"nombre");
pstmt.setString(2,"McQueen");
rs=pstmt.executeQuery();
```

Los demás métodos disponibles con un objeto PreparedStatement son idénticos a los definidos para un objeto Statement , ya que la interfaz PreparedStatement hereda directamente de la interfaz Statement . El ejemplo de la siguiente diapositiva muestra la lista de personas que responden al nombre proporcionado por el usuario de la aplicación.

Ejecución de instrucciones SQL : PreparedStatement - Ejemplo

```
public class TestPreparedStatement {

    public static void main(String[] args) {
        try (Connection cnx = getConnexion());
            PreparedStatement pstmt = cnx.prepareStatement("select * from PERSONAS where nombre = ?",
                ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_READ_ONLY)) {

            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            System.out.println("rellenar el nombre de la persona buscada :");
            String nombre = br.readLine();
            //Aplicar los parámetros
            pstmt.setString(1, nombre);
            //Ejecutar la consulta y obtener el resultado
            ResultSet rs = pstmt.executeQuery();
            //Mientras hay registros
            while (rs.next()) {

                for (int i = 1; i <= rs.getMetaData().getColumnCount(); i++) {
                    System.out.print(rs.getString(i) + "\t");
                }
                System.out.println();
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Ejecución de instrucciones SQL : PreparedStatement

- El tipo `PreparedStatement` también se usa mucho para las operaciones de escritura (insertar, actualizar, eliminar). Dentro del marco de la inserción, una funcionalidad muy apreciable hace posible repetir la posible clave autogenerada por la base de datos (este suele ser el caso de una clave primaria del tipo de incremento automático).
- Para recuperar la clave generada automáticamente, debe usar la segunda firma del método `prepareStatement` del objeto de tipo `Connection`. Este método espera como parámetro una cadena de caracteres que corresponde a la consulta parametrizada, seguida de un número entero que indica si el desarrollador desea recuperar la clave autogenerada por la base de datos. La constante `Statement.RETURN_GENERATED_KEYS` se usa para especificar esto.
- La ejecución de la consulta parametrizada con el método `executeUpdate` de la interfaz `PreparedStatement` permite evaluar un `ResultSet` que contiene un único registro con la clave autogenerada. El acceso a este `ResultSet` se hace a través del método `getGeneratedKeys` de la interfaz `Statement`, de la que deriva la interfaz `PreparedStatement`. La clave está disponible en la primera columna del único registro de `ResultSet`.

Ejecución de instrucciones SQL : PreparedStatement - Ejemplo

```
public class TestInsertionViaPreparedStatement {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion());
            //Consulta de insercion con los campos obligatorios
            PreparedStatement pstmt = cnx.prepareStatement("insert into PERSONAS(nombre, apellido) values(?,?)",
                Statement.RETURN_GENERATED_KEYS)) {
                BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
                System.out.println("introducir el nombre de la persona a agregar :");
                String nombre = br.readLine();
                System.out.println("introducir el apellido de la persona a agregar :");
                String apellido = br.readLine();
                //Aplicar los parámetros
                pstmt.setString(1, nombre);
                pstmt.setString(2, apellido);
                //Ejecutar la consulta y obtener el resultado
                pstmt.executeUpdate();
                //Recuperar el ResultSet que eventualmente contiene una clave
                ResultSet rsClaveGenerada = pstmt.getGeneratedKeys();
                //Si hay un registro, es el que tiene una clave
                if(rsClaveGenerada.next()){
                    System.out.println("La clave de la registro generado es " + rsClaveGenerada.getInt(1));
                }
            } catch (SQLException e) {
                e.printStackTrace();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Ejecución de instrucciones SQL : CallableStatement

c. Ejecución de procedimientos almacenados con el objeto CallableStatement

- Un procedimiento almacenado representa código SQL almacenado en el servidor. Este planteamiento ofrece varias ventajas: los tratamientos y los tiempos de respuesta se han mejorado; y el código SQL se puede compartir entre varias aplicaciones.
- A cambio, las aplicaciones se vuelven mucho más dependientes del servidor de base de datos. El cambio de servidor le obligará seguramente a escribir de nuevo los procedimientos almacenados, ya que la sintaxis de un procedimiento almacenado es propia de cada servidor. Los procedimientos almacenados son accesibles desde Java mediante el objeto `CallableStatement`.
- Como para los objetos `Statement` y `PreparedStatement`, sigue siendo la conexión la que facilitará una instancia de esta clase. En este caso, se debe utilizar el método `prepareCall`. Este método recibe como argumento una cadena de caracteres que identifica el procedimiento almacenado que hay que invocar. Por el contrario, la sintaxis de esta cadena de caracteres es un poco especial, ya que no es suficiente indicar el nombre del procedimiento almacenado.

{call nombreProcedimiento(?, ?, ...)}

- En esta sintaxis, los signos de interrogación representan los parámetros que recibe el procedimiento almacenado. Como para el objeto `PreparedStatement`, los valores de estos parámetros deben facilitarse mediante los métodos `setXxx` correspondientes al tipo del parámetro.

Ejecución de instrucciones SQL : CallableStatement

- Para los argumentos utilizados como salida del procedimiento almacenado, (argumentos etiquetados con la palabra clave out), su ejecución almacenará un valor en él, y debemos informar el objeto invocando el método `registerOutParameter`. Este método recibe como `CallableStatement` primer argumento el índice del parámetro 1 para el valor de retorno del procedimiento almacenado, y luego como segundo argumento el tipo SQL del parámetro. Se puede indicar este tipo con una de las constantes definidas en la interfaz `java.sql.Types`. Tras la ejecución del procedimiento almacenado, el valor de los parámetros utilizados como salida está accesible mediante los métodos `getXxx`, donde Xxx representa el tipo SQL del parámetro. Estos métodos reciben como argumento el índice del parámetro en la llamada al procedimiento almacenado.
- Como para los métodos `createStatement` y `prepareStatement`, el método `prepareCall` propone una segunda sintaxis que permite indicar las características de un eventual juego de registros generado por la ejecución del procedimiento almacenado.
- Para ilustrar el uso del objeto `CallableStatement`, vamos a emplear el procedimiento almacenado siguiente:

```
CREATE DEFINER=`mysqluser`@`localhost` PROCEDURE `numeroPersonasPorNombre`(in ordenar varchar(4),out
numeroTotal integer)
BEGIN
    select count(*) into numeroTotal from PERSONAS;
    if(ordenar='ASC')then
        select count(*) as numeroPersonas, nombre from PERSONAS group by nombre asc;
    else
        select count(*) as numeroPersonas, nombre from PERSONAS group by nombre desc;
    end if,
END
```

- El código de la siguiente diapositiva se utiliza para ejecutar este procedimiento almacenado y explotar los resultados:

Ejecución de instrucciones SQL : CallableStatement

```
public class TestCallableStatement {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion();
            CallableStatement cstm = cnx.prepareCall("{call numeroPersonasPorNombre ( ?, ? )}",
                ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_READ_ONLY)) {
            System.out.println("rellenar la ordenación del resultado (ASC o DESC :)");
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            String ordenar= br.readLine();
            // Aplicación de las parámetros
            cstm.setString(1, ordenar);
            cstm.registerOutParameter(2, java.sql.Types.INTEGER);
            // Ejecución de la consulta
            boolean existeResultado = cstm.execute();
            // Recuperación del parámetro del tipo out
            int numeroPersonas = cstm.getInt(2);
            System.out.println("número de personas: " + numeroPersonas);

            if (existeResultado) {
                ResultSet rs = cstm.getResultSet();
                while (rs.next()) {
                    System.out.println(rs.getString("nombre") + ": " + rs.getInt("numeroPersonas") + " personas");
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Juegos de registros con la interfaz ResultSet

- Cuando se ejecuta la instrucción SQL `select` con el método `executeQuery` de un objeto `Statement`, `PreparedStatement` o `CallableStatement`, este devuelve un objeto `ResultSet`. A través de este objeto `ResultSet` vamos a poder intervenir sobre el juego de registros. Nuestras posibilidades de acción sobre este juego de registros están determinadas por las características del objeto `ResultSet`. Se fijan estas características en el momento de la creación de los objetos `Statement`, `PreparedStatement` o `CallableStatement` en función de los argumentos pasados durante la llamada a los métodos `createStatement`, `prepareStatement` o `prepareCall`.
- El primer argumento determina el tipo del juego de registros. Se definen las constantes siguientes:
 - `ResultSet.TYPE_FORWARD_ONLY`: el juego de registros se desplazará solo hacia delante.
 - `ResultSet.TYPE_SCROLL_INSENSITIVE`: el juego de registros podrá recorrerse en ambos sentidos, pero será insensible a los cambios realizados en la base de datos por otros usuarios.
 - `ResultSet.TYPE_SCROLL_SENSITIVE`: el juego de registros podrá recorrerse en ambos sentidos y será sensible a los cambios realizados en la base de datos por otros usuarios.
- El segundo argumento determina las posibilidades de modificación de los datos contenidos en el juego de registros. Se definen las dos constantes siguientes:
 - `ResultSet.CONCUR_READ_ONLY`: los registros son de solo lectura.
 - `ResultSet.CONCUR_UPDATABLE`: los registros pueden modificarse en el juego de registros.

Juegos de registros con la interfaz ResultSet

```
private static void mostrarTypeResultSet(ResultSet rs) {
    try {
        switch (rs.getType()) {
            case ResultSet.TYPE_FORWARD_ONLY:
                System.out.println("el ResultSet solo se desplaza hacia adelante");
                break;
            case ResultSet.TYPE_SCROLL_INSENSITIVE:
                System.out.println("el ResultSet se puede recorrer en ambos sentidos");
                System.out.println("no es sensible a las modificaciones hechas por los otros usuarios");
                break;
            case ResultSet.TYPE_SCROLL_SENSITIVE:
                System.out.println("el ResultSet se puede recorrer en ambos sentidos");
                System.out.println("es sensible a las modificaciones hechas por los otros usuarios");
                break;
        }
        switch (rs.getConcurrency()) {
            case ResultSet.CONCUR_READ_ONLY:
                System.out.println("los datos contenidos en el ResultSet son de solo lectura ");
                break;
            case ResultSet.CONCUR_UPDATABLE:
                System.out.println("los datos contenidos en el ResultSet son modificables");
                break;
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

- Por supuesto, hace falta que las acciones ejecutadas sobre el objeto ResultSet sean compatibles con estas características; en caso contrario, se producirá una excepción. Es posible comprobar las características de un objeto ResultSet utilizando los métodos `getType` y `getConcurrency` . El siguiente código muestra las características del ResultSet. Está disponible en la clase `TestTypeResultSet` del paquete `package org.example.consulta`.

Juegos de registros con la interfaz ResultSet

a. Posicionamiento en un ResultSet

- El objeto ResultSet gestiona un puntero de registro que determina sobre qué registro van a intervenir los métodos ejecutados sobre el propio ResultSet. Este registro se llama a veces registro activo o registro en curso. El objeto ResultSet contiene siempre dos registros ficticios que sirven de referencia para el principio del ResultSet (BOF – Beginning Of File) y para el final del ResultSet (EOF – End Of File). El puntero de registro se puede colocar sobre uno de estos dos registros, pero nunca antes del registro BOF ni después del registro EOF. Estos registros no contienen datos y una operación de lectura o escritura sobre ellos genera una excepción. En el momento de la creación del ResultSet, el puntero se coloca antes del primer registro (BOF). Cuando el conjunto de registros no contiene resultados, necesariamente hay estos dos registros ficticios.
- Existen muchos métodos disponibles para gestionar el puntero de registros:
 - **boolean absolute(int position)**: desplaza el puntero de registro sobre el registro especificado. La numeración de los registros empieza en 1. Si el valor del argumento posición es negativo, se efectúa el desplazamiento partiendo del final del ResultSet. Si el número de registro no existe, se coloca el puntero sobre el registro BOF si el valor es negativo e inferior al número de registros, o sobre el registro EOF si el valor es positivo y superior al número de registros. Este método devuelve true si el puntero está colocado en un registro válido y false en el caso contrario (BOF o EOF).
 - **boolean relative(int displacement)**: desplaza el puntero del número de registros especificados por el argumento displacement. Si el valor de este argumento es positivo, el cursor baja en el juego de registros, y si el valor es negativo, el cursor sube en el juego de registros. Este método devuelve true si el puntero está colocado sobre un registro válido y false en el caso contrario (BOF o EOF).

Juegos de registros con la interfaz ResultSet

- `void beforeFirst()`: desplaza el puntero de registros antes del primer registro (BOF).
 - `void afterLast()`: desplaza el puntero de registros después del último registro (EOF).
 - `boolean first()`: desplaza el puntero de registros sobre el primer registro. Este método devuelve true si hay un registro en el ResultSet y false en caso contrario.
 - `boolean last()`: desplaza el puntero de registros sobre el último registro. Este método devuelve true si hay un registro en el ResultSet y false en caso contrario.
 - `boolean next()`: desplaza el puntero de registros sobre el registro que sigue al registro actual. Este método devuelve true si el puntero está en un registro válido y false en caso contrario (EOF).
 - `boolean previous()`: desplaza el puntero de registros sobre el registro anterior al registro actual. Este método devuelve true si el puntero está sobre un registro válido y false en caso contrario (BOF).
- Para todos estos métodos, excepto para el método `next`, es absolutamente necesario que el `ResultSet` sea de tipo `SCROLL_SENSITIVE` o `SCROLL_INSENSITIVE`. Si el `ResultSet` es de tipo `FORWARD_ONLY`, solo el método `next` funciona y en este caso los demás métodos producen una excepción.

Juegos de registros con la interfaz ResultSet

- Los métodos siguientes permiten comprobar la posición del puntero de registros:
 - `boolean isBeforeFirst()`: devuelve true si el puntero está situado antes del primer registro (BOF).
 - `boolean isAfterLast()`: devuelve true si el puntero está situado después del último registro (EOF).
 - `boolean isFirst()`: devuelve true si el puntero está situado en el primer registro.
 - `boolean isLast()`: devuelve true si el puntero está situado en el último registro.
 - `int getRow()` : devuelve el número del registro sobre el cual se encuentra el puntero de registros. Si no existe un registro en curso (BOF o EOF), devuelve el valor 0.
- El programa de la siguiente diapositiva permite mostrar la posición del puntero en el `ResultSet`.

Juegos de registros con la interfaz ResultSet

```
public class TestPositionPunteroResultSet {
    public static void main(String[] args) {
        try (Connection cnx = getConnexion();
            Statement stm =
cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_UPDATABLE)) {
            // Ejecutar la consulta y obtener el resultado
            ResultSet rs = stm.executeQuery(
                "select * from PERSONAS");
            mostrarPosicionPuntero(rs);
            // Mientras hay registros
            while (rs.next()) {
                mostrarPosicionPuntero(rs);
            }
            mostrarPosicionPuntero(rs);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```
private static void mostrarPosicionPuntero(ResultSet rs) {
    try {
        if (rs.isBeforeFirst()) {
            System.out.println("el puntero está antes del primer registro");
        }
        if (rs.isAfterLast()) {
            System.out.println("el puntero está después del último registro");
        }
        if (rs.isFirst()) {
            System.out.println("el puntero está en el primer registro");
        }
        if (rs.isLast()) {
            System.out.println("el puntero está en el último registro");
        }
        int position;
        position = rs.getRow();
        if (position != 0) {
            System.out.println("es el registro número " + position);
        }
    } catch (SQLException e) {
        // TODO Bloque catch autogenerado
        e.printStackTrace();
    }
}
```

Juegos de registros con la interfaz ResultSet

b. Lectura de los datos en un ResultSet

- El objeto **ResultSet** facilita numerosos métodos que permiten la lectura de los campos de un registro.
- Cada uno de estos métodos es específico de un tipo de datos SQL. Por supuesto, hay que elegir adecuadamente el método de lectura según el tipo de datos del campo cuyo valor se quiere obtener. Sin embargo, algunos de estos métodos son relativamente flexibles y permiten la lectura de varios tipos de datos. El cuadro siguiente presenta los principales tipos de datos SQL y los métodos que permiten leer el contenido de un ResultSet .

Juegos de registros con la interfaz ResultSet

- Los métodos marcados con el símbolo 😊 son los métodos más aconsejados. Los métodos marcados con el símbolo 😬 podrían utilizarse, aunque conllevan riesgos de pérdida de información.

	TINYINT	SMALLINT	INTEGER	BIGINT	REAL	FLOAT	DOUBLE	DECIMAL	NUMERIC	BIT	CHAR	VARCHAR	LONGVARCHAR	BINARY	VARBINARY	LONGVARBINARY	DATE	TIME	TIMESTAMP	CLOB	BLOB	ARRAY	REF	STRUCT	JAVA OBJECT
getBytes	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊												
getShort	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊												
getInt	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊												
getLong	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊												
getFloat	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊												
getDouble	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊												
getBigDecimal	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊												
getBoolean	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊												
getString	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊						
getBytes														😊	😊	😊	😊	😊							
getDate											😊	😊	😊				😊		😊						
getTime											😊	😊	😊					😊	😊						
getTimeStamp											😊	😊	😊				😊	😊	😊						
getAsciiStream											😊	😊	😊	😊	😊	😊									
getUnicodeStream											😊	😊	😊	😊	😊	😊									
getBinaryStream														😊	😊	😊									
getClob																			😊						
getBlob																				😊					
getArray																					😊				
getRef																						😊			
getCharacterStream											😊	😊	😊	😊	😊	😊									
getObject	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊	😊

Juegos de registros con la interfaz ResultSet

- Cada uno de estos métodos se presenta bajo dos formas. La primera recibe como argumento el número de la columna de la que se quiere obtener el valor. La numeración empieza con el 1. La segunda versión recibe una cadena de caracteres que representa el nombre de la columna en la base de datos. Si la petición utilizada para crear el `ResultSet` contiene alias, entonces las columnas llevan el nombre del alias y no el nombre del campo en la base de datos. Para una mejor legibilidad del código, es preferible usar los nombres de las columnas en vez de sus índices. Cuando en la base de datos un campo no contiene valores (`NULL`), los métodos devuelven un valor igual a 0 para los campos numéricos, un valor `false` para los campos booleanos y un valor `null` para los demás tipos. En algunos casos, podrían existir ciertas dudas relativas al valor realmente almacenado en la base de datos.
- Por ejemplo, el método `getInt` puede devolver un valor igual a cero porque está realmente almacenado este valor en la base de datos o porque este campo admite valores nulos en la base de datos. Para resolver la duda, el método `wasNull` devuelve un booleano igual a `true` si el campo donde se hizo la última operación de lectura sobre el `ResultSet` contenía efectivamente un valor `null`.

Juegos de registros con la interfaz ResultSet

- El código de la siguiente diapositiva resalta la manipulación de los tipos Java y SQL al escribir y leer información. Se enfatiza la manipulación de fechas.
- Para escribir una fecha en una consulta parametrizada, es posible usar el método `setDate`, que espera en el segundo parámetro un objeto de tipo `java.sql.Date`. Un objeto de este tipo no debe manipularse en el resto de la aplicación. Es preferible utilizar un objeto de tipo `LocalDate` (si la parte horaria no es útil) o un objeto de tipo `LocalTime`. El método `java.sql.Date.valueOf` espera como parámetro un objeto de tipo `LocalDate` para obtener un objeto de tipo `java.sql.Date`.
- Para escribir una fecha en una consulta parametrizada, también es posible usar el método `setObject`, que acepta un objeto como segundo parámetro. Este objeto puede ser de tipo `LocalDate`. El controlador JDBC podrá usar este objeto para guardarlo en una columna de tipo Date.
- Para leer los datos, se adaptan los métodos `getXxx` de la interfaz `ResultSet`. La lectura de una fecha se realiza con el método `getDate`. Devuelve un objeto de tipo `java.sql.Date`.
- Es posible obtener un objeto de tipo `LocalDate` a partir de un objeto de tipo `java.sql.Date` utilizando el método `toLocalDate`.

Juegos de registros con la interfaz ResultSet

c. Modificación de los datos en un ResultSet

- La modificación de los datos se efectúa simplemente utilizando los métodos `updateXxx`, donde Xxx corresponde al tipo de datos de la columna que se ha de actualizar. Como para el caso de los métodos `getXxx`, estos disponen de dos versiones: una recibe como argumento el índice de la columna que se ha de actualizar, la segunda recibe como argumento una cadena de caracteres que representa el nombre de la columna en la base de datos. Si la petición utilizada para crear el ResultSet contiene alias, entonces las columnas llevan el nombre del alias y no el nombre del campo en la base de datos. Para una mejor legibilidad del código, es preferible por supuesto usar los nombres de las columnas en vez de sus índices.
- El tipo del segundo argumento esperado por estos métodos corresponde, lógicamente, al tipo de datos que hay que actualizar en el ResultSet. A continuación, se deben validar las modificaciones con el método `updateRow` o cancelar con el método `cancelRowUpdates`. El ResultSet debe ser obligatoriamente de tipo `CONCUR_UPDATABLE` para poder ser modificado. En caso contrario, la ejecución de estos métodos genera una excepción.
- El siguiente ejemplo actualiza los nombres de las personas que usan ResultSet. Por supuesto, en un contexto profesional, sería mejor usar una instrucción SQL para realizar esta tarea.

Juegos de registros con la interfaz ResultSet

```
public class TestModificationViaResultSet {

    public static void main(String[] args) {
        try (Connection cnx = getConnexion());
            Statement stmModif = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
            ResultSet.CONCUR_UPDATABLE);
            Statement stmReading = cnx.createStatement() {
                leerYModificarPersonas(stmModif);
                leerPersonas(stmReading);
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }

    private static void leerYModificarPersonas(Statement stmModif) throws SQLException {
        ResultSet rs = stmModif.executeQuery("select id, nombre, apellido, fechaNacimiento from
PERSONAS");
        while(rs.next()){
            //Modificación del nombre poniéndolo en mayúsculas
            rs.updateString("nombre", rs.getString("nombre").toUpperCase());
            rs.updateRow();
        }
    }
}
```

```
private static void leerPersonas(Statement stm) throws SQLException {  
    ResultSet rs = stm.executeQuery("select id, nombre, apellido, fechaNacimiento from  
PERSONAS");  
    System.out.println("id\t\tnombre\t\tapellido\t\tfecha de nacimiento");  
    while(rs.next())  
    {  
        int id = rs.getInt("id");  
        System.out.print(rs.isNull()? "desconocido":id);  
        System.out.print("\t\t");  
  
        String nombre = rs.getString("nombre");  
        System.out.print(rs.isNull()? "desconocido":nombre);  
        System.out.print("\t\t");  
  
        String apellido = rs.getString("apellido");  
        System.out.print(rs.isNull()? "desconocido":apellido);  
        System.out.print("\t\t");  
  
        Date sqlDate = rs.getDate("fechaNacimiento");  
  
        System.out.println(rs.isNull()? "desconocida":DateTimeFormatter.ofPattern("dd/MM/yyyy").format(sqlDate.toLocalDate()));  
    }  
}
```

Juegos de registros con la interfaz ResultSet

d. Supresión de datos en un ResultSet

- La supresión de una línea se realiza de manera sencilla al situar el puntero sobre la línea que se ha de suprimir y llamando, a continuación, al método `deleteRow`. La línea se elimina inmediatamente del ResultSet y de la base de datos. La posición del puntero de registros después del borrado depende del driver usado. Algunos drivers desplazan el puntero al registro siguiente, otros lo desplazan al anterior, e incluso algunos no modifican la posición del puntero. En este caso, hay que utilizar alguno de los métodos de desplazamiento para situar el puntero sobre un registro utilizable. El ResultSet debe ser obligatoriamente de tipo `CONCUR_UPDATABLE` para poder suprimir datos en él. En caso contrario, la ejecución de un método `deleteRow` produce una excepción.

Juegos de registros con la interfaz ResultSet

```
public class TestSuppressionViaResultSet {

    public static void main(String[] args) {
        try (Connection cnx = getConnexion());
            Statement stmDelete = cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_UPDATABLE);
            Statement stmReading = cnx.createStatement()) {
                leerYEliminarPersonas(stmDelete);
                leerPersonas(stmReading);
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }

    private static void leerYEliminarPersonas(Statement stmDelete) throws SQLException {
        ResultSet rs = stmDelete.executeQuery("select id, nombre, apellido, fechaNacimiento from
PERSONAS");
        while(rs.next()){
            //Delete el registro si la personas se llama Eastwood
            System.out.println("Persona actual : " + rs.getString("nombre"));
            if("Eastwood".equalsIgnoreCase(rs.getString("nombre")))
            {
                System.out.println("Elimino la persona");
                rs.deleteRow();
            }
        }
    }
}
```

[illegible]

Juegos de registros con la interfaz ResultSet

e. Inserción de datos en un ResultSet

- Cada objeto `ResultSet` contiene una línea especial destinada a la inserción de datos. El puntero de registro se debe situar, previamente, en esta línea especial con la instrucción `moveToInsertRow`. Una vez situado el puntero sobre esta línea, es posible actualizarla mediante los métodos `updateXXX`. A continuación, se debe validar la inserción de la línea mediante el método `insertRow`. Este método provoca la actualización de la base de datos. La línea de inserción se convierte en este momento en una línea normal del `ResultSet` y el puntero de registro se posiciona en esta línea. Puede regresar a la línea en la que estaba antes de la inserción gracias al método `moveToCurrentRow`. Si no proporcionamos valores para todas las columnas, se insertarán valores `null` en la base de datos para las columnas que acepten valores `null`. La base de datos debe aceptar los valores nulos para estos campos porque, en caso contrario, se produce una excepción. El `ResultSet` debe ser obligatoriamente de tipo `Statement` para poder insertar datos en él. En caso contrario, la ejecución del método genera una excepción.

Juegos de registros con la interfaz ResultSet

```
public class TestInsertionViaResultSet {

    public static void main(String[] args) {
        try (Connection cnx = getConnexion());
            Statement stmInsertion =
cnx.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_UPDATABLE);
            Statement stmReading = cnx.createStatement() {
                agregarPersona(stmInsertion);
                leerPersonas(stmReading);
            } catch (SQLException e) {
                e.printStackTrace();
            }
    }

    private static void agregarPersona(Statement stmInsertion) throws SQLException {
        ResultSet rs = stmInsertion.executeQuery("select id, nombre, apellido,
fechaNacimiento from PERSONAS");
        rs.moveToInsertRow();
        rs.updateString("nombre", "BESSON");
        rs.updateString("apellido", "Luc");
        rs.insertRow();
    }
}
```

```
private static void leerPersonas(Statement stm) throws SQLException {  
    ResultSet rs = stm.executeQuery("select id, nombre, apellido, fechaNacimiento from PERSONAS");  
    System.out.println("id\tnombre\tapellido\tfecha de nacimiento");  
    while(rs.next())  
    {  
        int id = rs.getInt("id");  
        System.out.print(rs.isNull()? "desconocido":id);  
        System.out.print("\t\t");  
  
        String nombre = rs.getString("nombre");  
        System.out.print(rs.isNull()? "desconocido":nombre);  
        System.out.print("\t\t");  
  
        String apellido = rs.getString("apellido");  
        System.out.print(rs.isNull()? "desconocido":apellido);  
        System.out.print("\t\t");  
  
        Date sqlDate = rs.getDate("fechaNacimiento");  
  
        System.out.println(rs.isNull()? "desconocida":DateTimeFormatter.ofPattern("dd/MM/yyyy").format(sqlDate.  
            toLocalDate()));  
    }  
}
```

Gestión de las transacciones

- Las transacciones van a permitir asegurar que un conjunto de instrucciones SQL se ejecuten todas con éxito. Si una de ellas falla, las instrucciones que se completaron con éxito de previamente simplemente se cancelan. Esto hace posible mantener consistente la información en la base de datos. La transferencia de un importe de dinero entre dos cuentas bancarias representa el ejemplo clásico en el cual se necesita una transacción. Imagínese la situación siguiente: nuestro banco debe realizar el cobro de un cheque de 1000 € a adeudar en la cuenta 12345 y a abonar en la cuenta 67890. Por razones de seguridad, después de cada operación efectuada en una cuenta (débito o crédito), se edita un informe.

Gestión de las transacciones

```
public static void movimiento(String cuentaDebito,String cuentaCredito,double importe){
    try{
        Connection cnx=...;
        PreparedStatement stm;
        stm=cnx.prepareStatement("update cuentas set saldo=saldo + ? where numero=?");
        stm.setDouble(1,importe * -1);
        stm.setString(2,cuentaDebito);
        stm.executeUpdate();
        impresionInforme(cuentaDebito, importe);
        stm.setDouble(1,importe);
        stm.setString(2,cuentaCredito);
        stm.executeUpdate();
        impresionInforme(cuentaCredito, importe);
    }catch (Exception e){
        e.printStackTrace();
    }
}
```

Para el 99,9999 % de los movimientos, no hay ningún problema; pero un buen día la impresora que se encarga de las ediciones se bloquea y este bloqueo genera una excepción en el método `impresionInforme`. A priori, esta excepción no es un problema, ya que la llamada a este método está ubicada dentro de un bloque `try` y existe un bloque `catch` encargado de gestionar la excepción. Sin embargo, no debemos olvidar que, si se produce una excepción y se ejecuta un bloque `catch`, la ejecución continua por la instrucción que sigue al bloque `catch`. En este supuesto, las instrucciones ubicadas entre la que produjo la excepción y el final de bloque `try` simplemente no se ejecutan. En nuestro caso, eso puede resultar problemático si esta excepción se produce en el método `impresionInforme` ejecutado justo después de la operación de débito. La operación de crédito, simplemente, no se lleva a cabo. Por lo tanto, se pierde el importe. Una solución consiste en cancelar las

Gestión de las transacciones

a. Puesta en marcha de las transacciones

- Hasta ahora, no nos hemos preocupado por las transacciones y, sin embargo, las estamos realizando desde nuestra primera instrucción SQL ejecutada mediante JDBC. El funcionamiento por defecto de la API JDBC consiste efectivamente en incluir cada instrucción ejecutada en una transacción y, a continuación, validar la transacción si la instrucción se ha ejecutado correctamente (commit) o cancelarla en el caso contrario (rollback). Este modo de funcionamiento se llama modo autoCommit. Si desea gestionar usted mismo el fin de una transacción validando o anulando todas las instrucciones que contiene, debe desactivar el modo autoCommit invocando el método setAutoCommit(false) sobre el objeto Connection. En lo sucesivo, será responsable de poner fin a las transacciones. Los métodos commit y rollback del objeto Connection permiten validar o cancelar las instrucciones ejecutadas desde el principio de la transacción. Una nueva transacción empieza automáticamente tras el fin de la anterior o desde la apertura de la conexión. El código que permite transferir un importe entre dos cuentas debería tener la forma siguiente.
- Para que el siguiente programa funcione, es necesario crear una tabla con un conjunto de prueba. Aquí está el script para crear la tabla CUENTAS y agregar dos cuentas:

```
create tabla CUENTAS(  
numero char(9) primary key,  
saldo integer not null  
);  
insert into CUENTAS(numero, saldo) values ("123456789", 2500);  
insert into CUENTAS(numero, saldo) values ("987654321", 10000);
```

Ahora que la base de datos está configurada, el código de la siguiente diapositiva permite destacar el funcionamiento de las transacciones:

Gestión de las transacciones

```
public class TestTransaction {
    public static void main(String[] args) {
        movimiento("123456789", "987654321", 1000);
    }
    public static void movimiento(String cuentaDebito, String cuentaCredito, double suma) {
        try (Connection cnx = getConnexion()) {
            try (PreparedStatement pstm = cnx.prepareStatement("update CUENTAS set saldo=saldo + ? where numero=?");) {
                cnx.setAutoCommit(false);
                pstm.setDouble(1, suma * -1);
                pstm.setString(2, cuentaDebito);
                pstm.executeUpdate();
                impresionInforme(cuentaDebito, suma * -1);
                pstm.setDouble(1, suma);
                pstm.setString(2, cuentaCredito);
                pstm.executeUpdate();
                impresionInforme(cuentaCredito, suma);
                System.out.println("Validación de la transacción"); //Validación de las operaciones
                cnx.commit();
            } catch (Exception e) {
                try {
                    System.out.println("Anulación de la transacción"); //Anulación de las operaciones
                    cnx.rollback();
                } catch (SQLException e1) {
                    e1.printStackTrace();
                }
                e.printStackTrace();
            }
        } catch (SQLException e2) {
            // TODO Auto-generated catch block
            e2.printStackTrace();
        }
    }

    private static void impresionInforme(String cuenta, double suma) throws Exception {
        System.out.println(cuenta + " : " + suma);
        //Descomente la siguiente línea para provocar la anulación de las operaciones
        //throw new Exception("Error en la edición del informe");
    }
}
```

Gestión de las transacciones

b. Puntos de salvaguarda

- En el momento de la llamada al método `rollback`, se cancela el conjunto de instrucciones SQL ejecutadas desde el principio de la transacción. Este método propone una segunda versión que recibe como parámetro un objeto `SavePoint`. Este objeto representa una referencia en la ejecución de las instrucciones SQL. Lo crea el método `setSavePoint`. La llamada al método `rollback` pasando como argumento un objeto `SavePoint` provoca la cancelación de todas las instrucciones ejecutadas hasta este punto de salvaguarda.

Gestión de las transacciones

c. Niveles de aislamiento

- Mientras una transacción está activa, los datos modificados por las instrucciones ejecutadas dentro de la transacción pueden bloquearse en la base de datos para evitar conflictos e incoherencias. Las bases de datos cuentan con diferentes tipos de bloqueos para una transacción. Para entender sus efectos de forma correcta, hay que identificar previamente los tipos de problemas que podemos encontrar cuando una transacción está activa.
- **Lectura errónea:** se produce esta anomalía cuando una aplicación accede a datos que están siendo modificados por una transacción que aún no ha sido validada.
- **Lectura no reproducible:** ocurre esta anomalía cuando las ejecuciones sucesivas de una misma instrucción select no producen el mismo resultado. Este caso se da cuando los datos que está leyendo están siendo modificados por otra transacción.
- **Lectura fantasma:** ocurre esta anomalía si ejecuciones sucesivas de una misma petición devuelven datos de más o de menos. Puede darse el caso si otra transacción está suprimiendo o añadiendo datos a la tabla.
- La API JDBC prevé varios niveles de aislamiento, que determinan la manera en que se bloquean los datos durante la transacción. Este bloqueo puede ser de lectura, de escritura, o bien de lectura y escritura sobre los datos a los que se accede mediante las instrucciones de la transacción.

Gestión de las transacciones

- El método `setTransactionIsolationLevel` permite definir los tipos de problemas que se pueden evitar. Este método recibe como parámetro alguna de las constantes presentadas en el cuadro siguiente. Este cuadro presenta también el efecto de cada nivel de aislamiento en los datos.

constante	Lectura errónea	Lectura no reproducible	Lectura fantasma
TRANSACTION_READ_UNCOMMITTED	posible	posible	posible
TRANSACTION_READ_COMMITTED	imposible	posible	posible
TRANSACTION_REPEATABLE_READ	imposible	imposible	posible
TRANSACTION_SERIALIZABLE	imposible	imposible	imposible

- El hecho de especificar un nivel de aislamiento para una transacción puede tener efectos sobre otras aplicaciones que accedan a los datos manipulados durante la transacción, ya que estos pueden estar bloqueados y, por lo tanto, ser inaccesibles para las demás aplicaciones.



¿Preguntas?

Aplicaciones gráficas Java Swing

Módulo: 2º DAM Desarrollo de Interfaces





Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Índice

**THINK TWICE
CODE ONCE!**

1. Introducción
2. Bibliotecas gráficas
 1. AWT
 2. Java Swing
3. Java Swing
 1. Diseño de una interfaz gráfica
 2. Las ventanas
 3. El thread EDT
 4. La gestión de los eventos
 5. Aspecto de los componentes
 6. El posicionamiento de los componentes
 1. FlowLayout
 2. BorderLayout
 3. GridLayout
 4. BoxLayout
 5. GridBagLayout
 6. Sin renderizador
7. Los componentes gráficos
 1. JComponent
 2. JLabel
 3. JProgressBar
 4. JTextField
 5. JPasswordField
 6. JTextArea
 7. JButton
 8. JMenu, JMenuBar, JMenuItem, JPopupMenu, Jseparator
 9. JToolBar
 10. JCheckBox
 11. JRadioButton
 12. JList
 13. JComboBox
8. Los cuadros de diálogo

Introducción

Hasta ahora, todos los ejemplos de código que hemos realizado funcionan exclusivamente en modo texto. La información se visualiza en una consola y también se informa desde dicha consola. La sencillez de este modo de funcionamiento supone una ventaja innegable para el aprendizaje de un lenguaje. Sin embargo, la mayoría de los usuarios de las futuras aplicaciones seguramente esperan disponer de una interfaz un poco menos “deprimente” que una pantalla en modo texto.

En esta UD vamos a estudiar cómo funcionan las interfaces gráficas con Java. El diseño de interfaces gráficas en Java no es tan sencillo y requiere escribir muchas líneas de código. En la práctica, contará con varias herramientas de desarrollo, capaces de encargarse de la generación de una gran parte de este código según el diseño gráfico de la aplicación que esté dibujando. Sin embargo, es importante entender correctamente los principios de funcionamiento de este código para intervenir en él y eventualmente optimizarlo. En este capítulo, no emplearemos ninguna herramienta específica.

Las bibliotecas gráficas

El lenguaje Java propone dos bibliotecas dedicadas al diseño de interfaces gráficas: la biblioteca AWT y la biblioteca SWING. Los fundamentos de uso son casi idénticos en ambas bibliotecas. El uso simultáneo de las dos bibliotecas en una misma aplicación puede provocar problemas de funcionamiento y por ello debería evitarse. Hay una tercera biblioteca más reciente llamada Java FX.

a. La biblioteca AWT

Esta biblioteca es la primera disponible para el desarrollo de interfaces gráficas. Contiene una multitud de clases e interfaces que permiten la definición y la gestión de interfaces gráficas. En realidad, esta biblioteca utiliza las funcionalidades gráficas del sistema operativo. Por lo tanto, no es el código presente en esta biblioteca el que asegura el resultado gráfico de los diferentes componentes. Este código hace de intermediario con el sistema operativo. Su uso ahorra bastante recursos, pero presenta varios inconvenientes:

Al estar relacionado el aspecto visual de cada componente con la representación que el sistema operativo hace de él, puede resultar delicado desarrollar una aplicación que tenga una apariencia coherente en todos los sistemas. El tamaño y la posición de los diferentes componentes son los dos elementos que se ven principalmente afectados por este problema.

Para que esta biblioteca sea compatible con todos los sistemas operativos, los componentes que contiene están limitados a los más corrientes (botones, zonas de texto, listas...).

Las bibliotecas gráficas

b. La biblioteca Swing

- Esta biblioteca se diseñó para resolver las principales carencias de la biblioteca AWT. Se obtuvo esta mejora al escribir completamente la biblioteca en Java sin apenas recurrir a los servicios del sistema operativo. Únicamente algunos elementos gráficos (ventanas y cuadros de diálogo) siguen relacionados con el sistema operativo. Para los demás componentes, es el código de la biblioteca Swing el encargado de determinar completamente su aspecto y su comportamiento.
- La biblioteca Swing contiene por lo tanto una cantidad impresionante de clases que sirven para redefinir los componentes gráficos. Sin embargo, no debemos pensar que la biblioteca Swing convierte la biblioteca AWT en algo completamente obsoleto. De hecho, Swing recupera muchos de los elementos de la biblioteca AWT. En el resto de la unidad emplearemos esencialmente esta biblioteca.



Diseño de una interfaz gráfica

Cualquier aplicación gráfica se compone de, al menos, un **contenedor de primer nivel**. La biblioteca Swing dispone de tres clases que permiten llevar a cabo este papel:

- **JApplet** : representa una ventana gráfica incluida en una página HTML para que un navegador se haga cargo de ella. Esta clase ya no es útil desde la plataforma Java 11 porque la tecnología de applets ha sido abandonada.
- **JWindow** : representa la ventana gráfica más rudimentaria que pueda existir. No dispone de ninguna barra de título, ningún menú de sistema, ningún borde: en realidad es un mero rectángulo. Esta clase se utiliza rara vez, excepto para la visualización de una pantalla de inicio en el momento del arranque de una aplicación (splash screen).
- **JFrame**: representa una ventana gráfica completa y plenamente funcional. Dispone de una barra de título, de un menú de sistema y de un borde. Puede fácilmente contener un menú y, por supuesto, es el elemento que vamos a emplear en la mayoría de los casos.

Las ventanas

- La clase `JFrame` es un elemento indispensable en cualquier aplicación gráfica. Como en el caso de una clase normal, debemos crear una instancia, modificar eventualmente las propiedades y utilizar los métodos. A continuación se muestra el código de la primera aplicación gráfica.
- Volviendo al programa, cabe admitir que es fácil de usar y muy eficaz. De hecho, es tan eficaz que no se puede parar la aplicación. En efecto, incluso si el usuario cierra la ventana, este cierre no provoca la supresión de la instancia de `JFrame` de la memoria. La única solución para detener la aplicación es apagar la máquina virtual Java con la combinación de teclas `[Ctrl] C`. Ante esto, se recomienda proporcionar otra solución para detener más fácilmente la ejecución de la aplicación, es decir, junto con el cierre de la ventana

```
public class Principal{
    public static void main(String[] args) {
        JFrame ventana;
        // creación de la instancia de la clase JFrame
        ventana=new JFrame();
        // modificación de la posición y de la tamaño de la ventana
        ventana.setBounds(0,0,300,400);
        // modificación del título de la ventana
        ventana.setTitle("primera ventana en JAVA");
        // visualizacion de la ventana
        ventana.setVisible(true);
    }
}
```

Las ventanas

- Una primera solución consiste en gestionar los eventos que se producen en el momento del cierre de la ventana y, en uno de ellos, provocar la detención de la aplicación. Se estudiará esta solución en la sección dedicada a la gestión de los eventos.
- La segunda solución utiliza comportamientos predefinidos para el cierre de la ventana. Estos comportamientos están determinados por el método `setDefaultCloseOperation`. Se definen varias constantes para determinar la acción emprendida al cierre de la ventana.
- `DISPOSE_ON_CLOSE`: esta opción provoca la detención de la aplicación en el momento del cierre de la última ventana controlada por la máquina virtual.
- `DO_NOTHING_ON_CLOSE`: con esta opción, no ocurre nada cuando el usuario pide el cierre de la ventana. En este caso, es obligatorio gestionar los eventos para que la acción del usuario tenga algún efecto sobre la ventana o la aplicación.
- `EXIT_ON_CLOSE`: esta opción provoca la detención de la aplicación incluso si otras ventanas siguen visibles.
- `HIDE_ON_CLOSE`: con esta opción la ventana simplemente queda oculta como consecuencia de una llamada a su método `setVisible(false)`.

Las ventanas

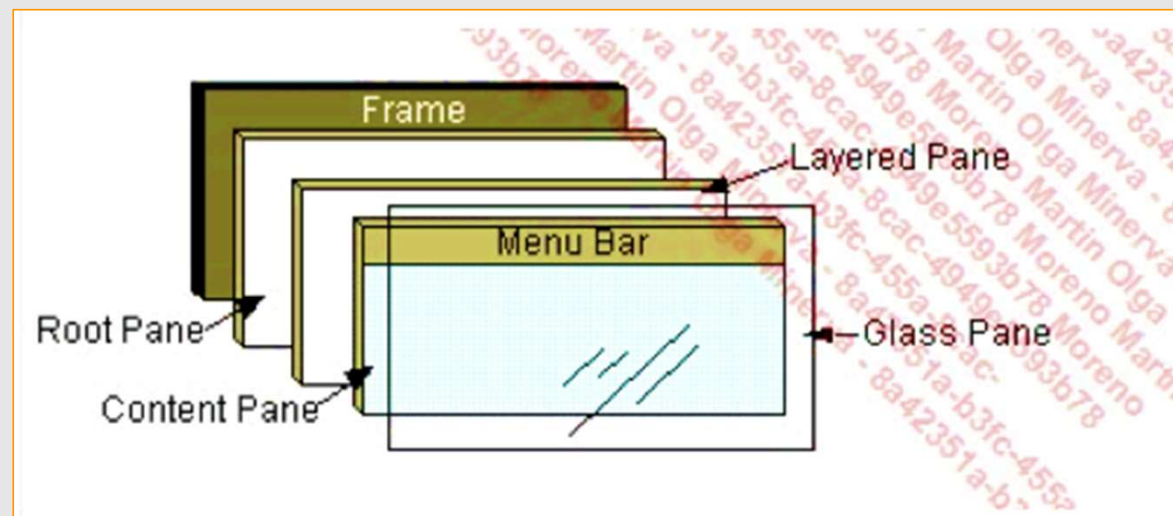
- La clase **JFrame** se encuentra al final de una jerarquía de clases bastante importante e implementa numerosas interfaces. Por este motivo, dispone de varios métodos y atributos.



- La meta de estas diapositivas no es retomar toda la documentación del JDK, de modo que no recorreremos todos los métodos disponibles, sino sencillamente los más utilizados según las necesidades. Sin embargo, puede resultar interesante revisar la documentación antes de realizar el diseño de un método para determinar si lo que queremos diseñar no ha sido ya previsto por los diseñadores de Java. La javadoc del paquete **java.swing** está disponible en la dirección siguiente:
- <https://docs.oracle.com/en/java/javase/18/docs/api/java.desktop/javax/swing/package-summary.html>

Las ventanas

- Ahora que somos capaces de visualizar una ventana, el grueso del trabajo va a consistir en añadirle un contenido. Antes de poder añadir algo a una ventana, conviene entender bien su estructura, que resulta relativamente compleja. Un objeto **JFrame** se compone de varios elementos superpuestos, cada uno con un papel muy específico en la gestión de la ventana.



Las ventanas

- El elemento **RootPane** corresponde al contenedor de los otros tres elementos. El elemento es el responsable de la gestión de la posición de los elementos tanto en los ejes X e Y como en el eje Z, lo que permite la superposición de diferentes elementos. El elemento **ContentPane** es el contenedor básico de todos los elementos añadidos en la ventana. A él vamos a confiarle, por esta razón, los diferentes componentes de la interfaz de la aplicación. Por encima del **ContentPane** se superpone el **GlassPane**, como es posible hacer con un cristal sobre una foto. De hecho, presenta muchas similitudes con el cristal.
 - Es transparente por defecto.
 - Lo dibujado en el GlassPane esconde los demás elementos.
 - Es capaz de interceptar los eventos relacionados con el ratón antes de que estos hayan alcanzado los demás componentes.

Las ventanas

- De todos estos elementos, es sin duda el `ContentPane` el que vamos a utilizar con mayor frecuencia.
- Podemos acceder a él a través del método `getContentPane` de la clase `JFrame`. Desde el punto de vista técnico, es posible ubicar componentes directamente en el objeto `ContentPane`, aunque es una práctica que Oracle desaconseja. Se prefiere intercalar un contenedor intermedio que contenga los componentes y ubicarlo en el `ContentPane`. Para ello, se suele utilizar el componente `JPanel`.
- Según esto, el escenario clásico de diseño de una interfaz gráfica consiste en crear los diferentes componentes y, a continuación, ubicarlos en un contenedor y, por último, situar este contenedor en el `ContentPane` de la ventana. El ejemplo siguiente lo pone en práctica creando una interfaz usuario compuesta por tres botones.

Las ventanas – Ejemplo

```
public class Principal {  
    public static void main(String[] args) {  
        // creación de la ventana  
        JFrame ventana;  
        ventana=new JFrame();  
        ventana.setTitle("primera ventana en JAVA");  
        ventana.setBounds(0,0,300,100);  
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        // creación de los tres botones  
        JButton b1,b2,b3;  
        b1=new JButton("Rojo");  
        b2=new JButton("Verde");  
        b3=new JButton("Azul");  
        // creación del contenedor intermedio  
        JPanel pano;  
        pano=new JPanel();  
        // agregar los botones en el contenedor intermedio  
        pano.add(b1);  
        pano.add(b2);  
        pano.add(b3);  
        // agregar el contenedor intermedio en el ContentPane  
        ventana.getContentPane().add(pano);  
        // visualizacion de la ventana  
        ventana.setVisible(true);  
    }  
}
```

El thread EDT

- La siguiente etapa de nuestro análisis nos va a permitir determinar lo que debe hacer la aplicación cuando el usuario haga clic en alguno de los botones. Pero antes de eso, es necesario comprender cómo se ejecuta una aplicación swing especificando el rol de los **thread** y más particularmente el **thread EDT (Event Dispatching Thread)**.
- Cuando se ejecuta una aplicación Java sin una interfaz gráfica, se ejecutan al menos dos threads:
 - El **thread principal (el thread main)**, cuya función es ejecutar el método main del programa. Es el punto de entrada obligatorio para ejecutar una aplicación Java SE estándar.
 - El thread **recolector de basura (garbage collector)** para limpiar la memoria cuando está ocupada más allá de cierto umbral.

El thread EDT

- Un **thread** es simplemente una unidad de ejecución para un programa. Un programa puede estar compuesto por uno o más thread. La ventaja de tener múltiples threads es que puede ejecutar múltiples tareas en paralelo. Por ejemplo un navegador: es muy común hacer una solicitud a un sitio y, mientras esperamos su descarga y visualización, consultar otra página, ver un vídeo, etc. Todo esto es posible gracias a uso de diferentes threads.
- En una máquina multiprocesador, los diferentes threads se pueden ejecutar en paralelo, explotando los diferentes procesadores. En una máquina con un solo procesador, es la frecuencia de transición entre los diferentes threads lo que permite al usuario pensar que las tareas se ejecutan en paralelo. De hecho, el procesador divide su tiempo entre los diferentes threads. Solo ejecuta un thread a la vez, pausando otros threads durante este tiempo. Por lo tanto, para ejecutar un thread completo, el procesador puede hacer esto varias veces pausando el thread repetidamente para ejecutar los otros threads.

El thread EDT

- Para ejecutar una aplicación Java con una interfaz gráfica, entra en juego un **tercer thread**: el **thread EDT** (**Event Dispatching Thread**). Este thread es responsable de administrar la interfaz gráfica y manejar los eventos (como hacer clic en un botón).
- En el ejemplo anterior, la interfaz gráfica se creó en el método main, por lo tanto, en el **thread main**. Así, es el thread main el «propietario» de los botones de la interfaz. Si el usuario hace clic en él, se le pide al thread EDT que reaccione a esta acción. Para hacer esto, el thread EDT a menudo necesita acceder a los gráficos. En este caso, es necesaria una operación **interthreads**. Y aquí es donde pueden surgir los problemas. De hecho, los componentes gráficos no son **thread safe**. Esto significa que acceder desde otro thread posiblemente podría causar un problema.
- Para evitar este tipo de situación, es necesario asegurarse de que solo el thread EDT manipule la interfaz gráfica, ya sea para la creación de la interfaz o para la gestión de eventos. Por lo tanto, es necesario que, desde el thread principal que ejecuta el método main, se pueda pedir al thread EDT que construya la interfaz gráfica de la aplicación. Esto es posible utilizando el método **invokeLater** de la clase **SwingUtilities**. Este método espera en el parámetro un objeto de tipo **Runnable** que es una interfaz funcional que contiene un único método abstracto cuya firma es la siguiente: `void run();`
- Este objeto (o expresión lambda) simplemente define lo que debe ser ejecutado por el thread EDT. El código de la siguiente diapositiva realiza una modificación esencial en el ejemplo anterior para garantizar el correcto funcionamiento de la aplicación en todas las situaciones.

El thread EDT

```
public class Principal {
    private JFrame ventana; //Constructor llamado por el thread EDT
    Principal(String[] args) // Construcción de la iIHM
    // creación de la ventana
        this.ventana=new JFrame();
        this.ventana.setTitle("primera ventana en JAVA");
        this.ventana.setBounds(0,0,300,100);
    this.ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    // creación de los tres botones
        JButton b1,b2,b3;
        b1=new JButton("Rojo");
        b2=new JButton("Verde");
        b3=new JButton("Azul");
    // creación del contenedor intermedio
        JPanel pano;
        pano=new JPanel();
    // agregar los botones en el contenedor intermedio
        pano.add(b1);
        pano.add(b2);
        pano.add(b3);
    // agregar el contenedor intermedio en el ContentPane
        this.ventana.getContentPane().add(pano);
}
```

```
//Método llamado por el thread EDT
    public void mostrar() {
        // Visualización de la iIHM
        this.ventana.setVisible(true);
    }
//Punto de entrada ejecutado por el thread main
    public static void main(final String[] args) {

        // Programa una tarea para el thread EDT:
        // Creación y visualización de la interfaz gráfica

        /*SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new Principal3(args).mostrar();
            }
            });
        */
        SwingUtilities.invokeLater(()->new Principal(args).mostrar());
    }
}
```


La gestión de los eventos

- Todos los SSOO que emplean una interfaz gráfica deben vigilar permanentemente los diferentes **periféricos** de introducción de datos para detectar las acciones del usuario y transmitirlos a las diferentes aplicaciones. Para cada acción del usuario, se crea un evento. A continuación, se envían estos eventos a cada aplicación, que determinará si le aplica el evento y determina lo que desea realizar como respuesta.
- Java se encarga de determinar qué **evento** acaba de producirse y sobre qué elemento. El desarrollador es responsable de configurar la sección de código que va a tratar el evento. Desde un punto de vista más técnico, el elemento origen del evento se denomina **fuentes de evento**, y el elemento que contiene la sección de código encargada de gestionar el evento se denomina **receptor de evento**. Las fuentes de eventos gestionan, para cada evento que pueden activar, una lista que les permite saber qué receptores deben ser avisados si el evento se produce. Por supuesto, las fuentes de eventos y los receptores de eventos son objetos. Es necesario prever qué receptores van a gestionar los eventos que les va a transmitir la fuente de eventos.
- Para garantizar esto, a cada tipo de evento le corresponde una interfaz que debe implementar un objeto si quiere ser candidato a gestionar dicho evento.

La gestión de los eventos

- Los eventos se agrupan en categorías. El nombre de estas interfaces siempre respeta la convención siguiente:
- La primera parte del nombre representa la categoría de eventos que los objetos que implementan esta interfaz pueden gestionar. El nombre siempre termina en **Listener**.
- Por ejemplo, tenemos la interfaz **MouseEventListener**, que corresponde a los eventos activados por los movimientos del ratón, o la interfaz **ActionListener**, que corresponde a un clic en un botón. En cada una de estas interfaces encontramos las firmas de los diferentes métodos asociados a cada evento.

```
public interface MouseEventListener extends  
EventListener{  
    void mouseDragged(MouseEvent e);  
    void mouseMoved(MouseEvent e);  
}
```

La gestión de los eventos

- Cada uno de estos métodos recibe como argumento un objeto que representa el propio evento. Este objeto se crea automáticamente en el momento de la activación del evento; a continuación, se pasa como argumento al método encargado de gestionar el evento en el receptor de eventos. En general, contiene información adicional relativa al evento y es específico para cada tipo de evento.
- Es preciso crear clases que implementen estas interfaces. Desde este punto de vista, tenemos una **multitud de posibilidades**:
 - Crear una clase «normal» que implemente la interfaz.
 - Implementar la interfaz en una clase ya existente.
 - Crear una clase interna que implemente la interfaz.
 - Crear una clase interna anónima que implemente la interfaz.
 - Crear eventualmente una expresión lambda si la interfaz es una interfaz funcional.

La gestión de los eventos

- En algunos casos, quizá sea necesario no gestionar todos los eventos presentes en la interfaz. Sin embargo, es obligatorio escribir todos los métodos exigidos por la interfaz incluso si varios de ellos no contienen ningún código. Esto puede perjudicar la legibilidad del código. Para paliar este problema, Java proporciona para casi cada interfaz `XxxListener`, una clase abstracta correspondiente que implementa la interfaz y que contiene los métodos exigidos por esta. Estos métodos no contienen código alguno, ya que el tratamiento de cada evento debe ser específico a cada aplicación. Estas clases emplean la misma nomenclatura que las interfaces, excepto que se sustituye `Listener` por `Adapter`. Tenemos por ejemplo la clase `MouseMotionAdapter`, que implementa la interfaz `MouseMotionListener`. Se pueden utilizar estas clases de varias maneras:
 - Creando una clase «normal» que herede de una de estas clases.
 - Creando una clase interna que herede de una de estas clases.
 - Creando una clase interna anónima que herede de una de estas clases.
- El uso de una clase interna anónima es la solución que más se utiliza, con el pequeño inconveniente de que se obtiene una sintaxis difícil de leer si uno no está acostumbrado.

La gestión de los eventos

- Para aclarar todo esto vamos a ilustrar cada una de estas posibilidades con un pequeño ejemplo que nos va a permitir terminar correctamente la aplicación en el momento del cierre de la ventana principal al invocar al método `System.exit(0)` . Esta solución permite realizar verificaciones antes de detener la aplicación (copia de seguridad, mostrar un mensaje de confirmación, desconectar al usuario...). Es preciso, en este caso, modificar la propiedad `DefaultCloseOperation` de la ventana con el valor `DO_NOTHING_ON_CLOSE` para que no tenga una acción por defecto.
- Se debe invocar este método durante la detección del cierre de la ventana. Para ello, debemos gestionar los eventos relacionados con la ventana y, en particular, el evento `windowClosing`, que se produce cuando el usuario cierra la ventana mediante el menú del sistema. La interfaz `WindowListener` está perfectamente adaptada para este tipo de trabajo.

La gestión de los eventos

```
public class Pantalla extends JFrame {
    public Pantalla() {
        setTitle("primera ventana en JAVA");
        setBounds(0,0,300,100);
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        // creación de los tres botones
        JButton b1,b2,b3;
        b1=new JButton("Rojo");
        b2=new JButton("Verde");
        b3=new JButton("Azul");
        // creación del contenedor intermedio
        JPanel pano;
        pano=new JPanel();
        // agregar los botones en el contenedor intermedio
        pano.add(b1);
        pano.add(b2);
        pano.add(b3);
        // agregar el contenedor en el ContentPane
        getContentPane().add(pano);
    }

    public void mostrar(){
        this.setVisible(true);
    }
}
```

```
public class Principal {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(()->new Pantalla().mostrar());
    }
}
```

Olga M. Moreno Martín

Crear una clase «normal»
que implemente la interfaz.

Si ejecutamos este código, la ventana aparece, pero ya no es posible cerrarla y aún menos detener la aplicación.

Veamos en la diapositiva siguiente cómo remediar este problema con las diferentes soluciones mencionadas anteriormente.

La gestión de los eventos

Implementar la interfaz
en una clase ya existente.

```
public class Pantalla extends JFrame {
    public Pantalla() {
        setTitle("primera ventana en JAVA");
        setBounds(0,0,300,100);
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        // creación de los tres botones
        JButton b1,b2,b3;
        b1=new JButton("Rojo");
        b2=new JButton("Verde");
        b3=new JButton("Azul");
        // creación del contenedor intermedio
        JPanel pano;
        pano=new JPanel();
        // agregar los botones en el contenedor intermedio
        pano.add(b1);
        pano.add(b2);
        pano.add(b3);
        // agregar el contenedor en el ContentPane
        getContentPane().add(pano);
    }
    public void mostrar(){
        this.setVisible(true);
    }
}
```

```
public class OyenteVentana implements WindowListener {

    public void windowActivated(WindowEvent arg0) {}
    public void windowClosed(WindowEvent arg0) {}
    public void windowClosing(WindowEvent arg0) {
        System.exit(0);
    }
    public void windowDeactivated(WindowEvent arg0) {}
    public void windowDeiconified(WindowEvent arg0) {}
    public void windowIconified(WindowEvent arg0) {}
    public void windowOpened(WindowEvent arg0) {}
}

public class Principal {
    public static void main(String[] args){
        SwingUtilities.invokeLater(()-> {
            Pantalla pantalla = new Pantalla();
            // creación de una instancia de la clase encargada
            // de administrar los eventos
            OyenteVentana ef=new OyenteVentana();
            // referencia de esta instancia de clase
            // como listaner de evento para la ventana
            pantalla.addWindowListener(ef);
            //visualizacion de la ventana
            pantalla.mostrar();
        });
    }
}
```

La gestión de los eventos

```
public class Pantalla extends JFrame implements WindowListener {
    public Pantalla() {
        setTitle("primera ventana en JAVA");
        setBounds(0,0,300,100);
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        // creación de los tres botones
        JButton b1,b2,b3;
        b1=new JButton("Rojo"); b2=new JButton("Verde"); b3=new JButton("Azul");
        // creación del contenedor intermedio
        JPanel pano;
        pano=new JPanel();
        // agregar los botones en el contenedor intermedio
        pano.add(b1); pano.add(b2); pano.add(b3);
        // agregar el contenedor intermedio en el ContentPane
        getContentPane().add(pano);
        // referencia de la ventana ella misma como listener de sus propios eventos
        addWindowListener(this);
    }
    public void mostrar(){
        this.setVisible(true);
    }
    public void windowActivated(WindowEvent arg0) {}
    public void windowClosed(WindowEvent arg0) {}
    public void windowClosing(WindowEvent arg0) {
        System.exit(0);
    }
    public void windowDeactivated(WindowEvent arg0) {}
    public void windowDeiconified(WindowEvent arg0) {}
    public void windowIconified(WindowEvent arg0) {}
    public void windowOpened(WindowEvent arg0) {}
}
```

Crear una clase
interna que
implemente la
interfaz

```
public class Principal {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(()->new Pantalla().mostrar());
    }
}
```


La gestión de los eventos

Crear una clase interna anónima que implemente la interfaz

```
public class Pantalla extends JFrame {
    public Pantalla(){
        setTitle("primera ventana en JAVA");
        setBounds(0,0,300,100);
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        // creación de los tres botones
        JButton b1,b2,b3;
        b1=new JButton("Rojo");
        b2=new JButton("Verde");
        b3=new JButton("Azul");
        // creación del contenedor intermedio
        JPanel pano;
        pano=new JPanel();
        // agregar los botones en el contenedor intermedio
        pano.add(b1);
        pano.add(b2);
        pano.add(b3);
        // agregar el contenedor intermedio en el ContentPane
        getContentPane().add(pano);
        // creación de una instancia de la clase encargada
        // de administrar los eventos
        OyenteVentana ef;
        ef=new OyenteVentana();
        // referencia de esta instancia de clase
        // como listener de evento para la ventana
        addWindowListener(ef);
    }
}
```

```
public void mostrar(){
    this.setVisible(true);
}
```

```
public class OyenteVentana implements WindowListener{
    public void windowActivated(WindowEvent arg0) {}
    public void windowClosed(WindowEvent arg0) {}
    public void windowClosing(WindowEvent arg0){
        System.exit(0);
    }
    public void windowDeactivated(WindowEvent arg0) {}
    public void windowDeiconified(WindowEvent arg0) {}
    public void windowIconified(WindowEvent arg0) {}
    public void windowOpened(WindowEvent arg0) {}
}
```

```
public class Principal {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(()->new Pantalla().mostrar());
    }
}
```

La gestión de los eventos

- El único inconveniente que presenta esta última solución reside en la relativa complejidad de su sintaxis. Los comentarios entre las diferentes líneas ofrecen una ayuda valiosa para no perderse entre llaves y paréntesis.
- También existe una problemática general que se le puede reprochar a todas estas soluciones: para un único método realmente útil, tenemos que escribir siete.
- Para evitar este código inútil, podemos trabajar con una clase que implemente ya la interfaz correcta y volver a definir únicamente los métodos que nos interesan.
- Veamos en las próximas diapositivas cómo solucionar esto.

La gestión de los eventos

Crear una clase «normal» que herede de una clase XxxAdapter

```
public class Pantalla extends JFrame {
    public Pantalla() {
        setTitle("primera ventana en JAVA");
        setBounds(0,0,300,100);
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        // creación de los tres botones
        JButton b1,b2,b3;
        b1=new JButton("Rojo");
        b2=new JButton("Verde");
        b3=new JButton("Azul");
        // creación del contenedor intermedio
        JPanel pano;
        pano=new JPanel();
        // agregar los botones en el contenedor intermedio
        pano.add(b1);
        pano.add(b2);
        pano.add(b3);
        // agregar el contenedor en el ContentPane
        getContentPane().add(pano);
    }

    public void mostrar(){
        this.setVisible(true);
    }
}
```

```
public class OyenteVentana extends WindowAdapter {
    public void windowClosing(WindowEvent arg0) {
        System.exit(0);
    }
}

public class Principal {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(()->{
            Pantalla pantalla = new Pantalla();
            // creación de una instancia de la clase encargada
            // de administrar los eventos
            OyenteVentana ef=new OyenteVentana();
            // referencia de esta instancia de clase
            // como listaner de evento para la ventana
            pantalla.addWindowListener(ef);
            //visualizacion de la ventana
            pantalla.mostrar();
        });
    }
}
```

La gestión de los eventos

```
public class Pantalla extends JFrame {  
    public Pantalla() {  
        setTitle("primera ventana en JAVA");  
        setBounds(0,0,300,100);  
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);  
        // creación de los tres botones  
        JButton b1,b2,b3;  
        b1=new JButton("Rojo"); b2=new JButton("Verde"); b3=new JButton("Azul");  
        // creación del contenedor intermedio  
        JPanel pano;  
        pano=new JPanel();  
        // agregar los botones en el contenedor intermedio  
        pano.add(b1); pano.add(b2); pano.add(b3);  
        // agregar el contenedor intermedio en el ContentPane  
        getContentPane().add(pano);  
        // creación de una instancia de la clase encargada  
        // de administrar los eventos  
        ListenerVentana ef;  
        ef=new ListenerVentana();  
        // como listener de evento para la ventana  
        addWindowListener(ef);  
    }  
    public void mostrar() {  
        this.setVisible(true);  
    }  
    public class ListenerVentana extends WindowAdapter {  
        public void windowClosing(WindowEvent arg0) {  
            System.exit(0);  
        }  
    }  
}
```

```
public class Principal {  
    public static void main(String[] args) {  
        SwingUtilities.invokeLater(()->new Pantalla().mostrar());  
    }  
}
```

Crear una clase
interna que
herede de una
clase XxxAdapter

La gestión de los eventos

```
public class Pantalla extends JFrame{
    public Pantalla() {
        setTitle("primera ventana en JAVA");
        setBounds(0,0,300,100);
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        // creación de los tres botones
        JButton b1,b2,b3;
        b1=new JButton("Rojo");b2=new JButton("Verde");b3=new JButton("Azul");
        // creación del contenedor intermedio
        JPanel pano;
        pano=new JPanel();
        // agregar los botones en el contenedor intermedio
        pano.add(b1);pano.add(b2);pano.add(b3);
        // agregar el contenedor intermedio en el ContentPane
        getContentPane().add(pano);
        // creación de una instancia de una clase anonyme
        // encargada de administrar los eventos
        addWindowListener(new WindowAdapter(){
            // inicio de la definición de la clase
            public void windowClosing(WindowEvent arg0) {
                System.exit(0);
            }
        } // fin de la definición de la clase
        ); // fin de la llamada del método addWindowListaner
    } // fin del constructor
    public void mostrar() {
        this.setVisible(true);
    }
} // fin de la clase Pantalla
```

Crear una clase interna anónima que herede de una clase XxxAdapter

```
public class Principal {

    public static void main(String[] args) {

        SwingUtilities.invokeLater(()->new Pantalla().mostrar());
    }
}
```

La gestión de los eventos

- Por supuesto, esta última diapositiva de solución es la más económica en número de líneas; además, es utilizada por numerosas herramientas de desarrollo que generan automáticamente código. La relativa complejidad del código puede intimidar ligeramente cuando uno no está acostumbrado a ello.
- Hasta ahora, tenemos una fuente de eventos y un receptor para esta fuente de eventos. En algunos casos, podemos estar en la situación de tener varias fuentes de eventos y desear utilizar el mismo receptor o tener una fuente de evento y avisar a varios receptores. La situación clásica en la cual tenemos varias fuentes de eventos y un único receptor se da cuando proporcionamos al usuario varias soluciones para lanzar la ejecución de una misma acción (menú y barra de herramientas o botones). Sea cual sea el medio utilizado para lanzar la acción, el código que se ha de ejecutar sigue siendo el mismo.
- En este supuesto, podemos emplear el mismo receptor para las dos fuentes de eventos. Para ilustrar esto, vamos a añadir un menú a la aplicación y hacer que el uso del menú o de uno de los botones ejecute la misma acción modificando el color de fondo correspondiente al botón o al menú usado. Como debemos utilizar el mismo receptor para dos fuentes de eventos, es preferible usar una clase interna para la creación del receptor.
- Con un pequeño truco vamos a poder simplificar el código para obtener una única clase receptora de eventos para los tres botones. Se invocará el método `actionPerformed` con un clic en cualquiera de los botones. La elección de la acción que se ha de ejecutar se realizará en el interior de este método. Para ello, vamos a utilizar el parámetro `ActionEvent` facilitado al propio método. Permite obtener una referencia sobre el objeto que origina el evento a través del método `getSource`.

La gestión de los eventos

- Hay que señalar que para que funcione esta solución, los objetos fuente de eventos deben ser accesibles desde la clase receptor de eventos. Por lo tanto, la declaración de los botones y de los elementos de menú se realiza dentro de la propia clase y no dentro del constructor, como era el caso en la versión anterior. Esta solución es posible únicamente si la clase receptor de eventos es una clase interna. En el caso de que la clase receptor de eventos sea independiente de la clase donde se crean los objetos fuente de eventos, hay que revisar el código del método `actionPerformed`. El parámetro `ActionEvent` del método `actionPerformed` nos proporciona otra solución para resolver este problema. A través del método `getActionCommand` tenemos acceso a una cadena de caracteres que representa al objeto fuente de eventos.
- Por defecto, esta cadena de caracteres corresponde al título del componente que produce el evento, pero se la puede modificar con el método `setActionCommand` de cada componente. De hecho, se recomienda esta práctica, ya que nos permite obtener un código idéntico para una aplicación que funciona en varios idiomas. A continuación mostramos las modificaciones correspondientes.

La gestión de los eventos

- Destacaremos que, en esta solución, la declaración de los botones y de los elementos de menú pueden integrarse en el constructor, ya que no son necesarios a nivel de clase.
- La última etapa de nuestro maratón en los eventos nos va a permitir disponer de varios receptores de eventos para una misma fuente de eventos y, en caso necesario, suprimir un receptor existente. Para ello, vamos a crear una nueva clase receptor que nos va a permitir mostrar en la consola la fecha y la hora del evento y el objeto fuente del evento.

La gestión de los eventos

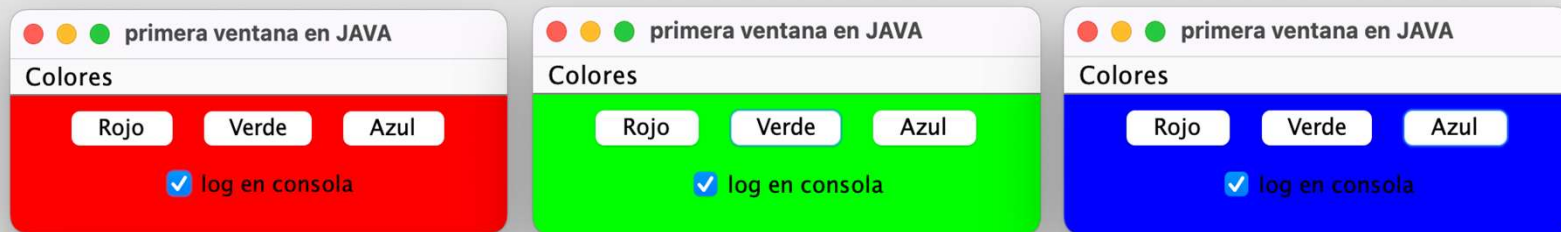
```
public class ConsoleLog implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        String message;
        SimpleDateFormat sdf;
        sdf=new SimpleDateFormat("dd/MM/yyyy hh:mm:ss");
        message=sdf.format(new Date());
        message=message + " click en el ";
        if (e.getSource() instanceof JButton){
            message=message+ "botón ";
        }
        if (e.getSource() instanceof JMenuItem) {
            message=message+ "menú ";
        }
        message=message +
        ((AbstractButton)e.getSource()).getText();
        System.out.println(message);
    }
}
```

Agregamos a continuación, en nuestra aplicación, una casilla que nos permite seleccionar si se visualizan los eventos en la consola. Según la configuración de esta opción, añadimos con el método `addActionListener`, o suprimimos con el método `removeActionListener`, un receptor de eventos a los botones y menús. Estos dos métodos reciben como argumento la instancia del receptor de eventos que se desea agregar o eliminar.

[Ver ejemplo ihm15](#)

La gestión de los eventos

- En tiempo de ejecución y en función de la selección, vemos aparecer los mensajes siguientes en la consola.
- Cabe destacar que, independientemente de la visualización de estos mensajes, el color de la ventana cambia siempre al usar estos botones o menús



28/09/2022 10:01:59 click en el botón Rojo
28/09/2022 10:02:23 click en el botón Verde
28/09/2022 10:02:41 click en el botón Azul

La gestión de los eventos

- Todos estos principios de gestión son idénticos sean cuales sean los eventos y los objetos que los activan. El único problema que podemos encontrar al principio es saber cuáles son los eventos disponibles para un objeto particular. A este nivel, la documentación Java es la única que puede ayudar. Un pequeño truco consiste en buscar en la clase relativa al objeto correspondiente, los métodos llamados `addXxxxListener` y, a continuación, desde este método, subir hasta la interfaz correspondiente y descubrir, gracias a los métodos definidos en esta interfaz, los diferentes eventos posibles. El análisis del tipo de parámetro recibido en los diferentes métodos le permitirá saber qué información adicional está disponible para un evento particular.

La gestión de los eventos

- Por ejemplo, si localizamos el método `addMouseListener`.

```
addMouseListener  
  
public void addMouseListener(MouseListener l)  
  
Adds the specified mouse listener to receive mouse events from this component. If listener l is null, no  
exception is thrown and no action is performed.  
  
Refer to AWT Threading Issues for details on AWT's threading model.  
  
Parameters:  
l - the mouse listener  
  
Since:  
1.1  
  
See Also:  
MouseEvent, MouseListener, removeMouseListener(java.awt.event.MouseListener), getMouseListeners()
```

- Ya se puede determinar que este método va a permitir trabajar con eventos relacionados con el ratón. Para obtener más información relativa a los diferentes eventos, debemos consultar la interfaz `MouseListener`.

La gestión de los eventos

- La documentación de esta interfaz muestra que están previstos **cinco métodos** y, por lo tanto, que es posible gestionar cinco tipos de eventos y en qué circunstancias se producen. Para saber qué información estará disponible cuando se produzca alguno de estos eventos, debe consultar la clase correspondiente al tipo de los argumentos recibidos por estos métodos (**Mouse Event**).

All Methods	Instance Methods	Abstract Methods
Modifier and Type	Method	Description
void	<code>mouseClicked</code> (<code>MouseEvent e</code>)	Invoked when the mouse button has been clicked (pressed and released) on a component.
void	<code>mouseEntered</code> (<code>MouseEvent e</code>)	Invoked when the mouse enters a component.
void	<code>mouseExited</code> (<code>MouseEvent e</code>)	Invoked when the mouse exits a component.
void	<code>mousePressed</code> (<code>MouseEvent e</code>)	Invoked when a mouse button has been pressed on a component.
void	<code>mouseReleased</code> (<code>MouseEvent e</code>)	Invoked when a mouse button has been released on a component.

La gestión de los eventos

All Methods	Static Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description	
int	<code>getButton()</code>	Returns which, if any, of the mouse buttons has changed state.	
int	<code>getClickCount()</code>	Returns the number of mouse clicks associated with this event.	
Point	<code>getLocationOnScreen()</code>	Returns the absolute x, y position of the event.	
static String	<code>getMouseModifiersText(int modifiers)</code>	Returns a String instance describing the modifier keys and mouse buttons that were down during the event, such as "Shift", or "Ctrl+Shift".	
Point	<code>getPoint()</code>	Returns the x,y position of the event relative to the source component.	
int	<code>getX()</code>	Returns the horizontal x position of the event relative to the source component.	
int	<code>getXOnScreen()</code>	Returns the absolute horizontal x position of the event.	
int	<code>getY()</code>	Returns the vertical y position of the event relative to the source component.	
int	<code>getYOnScreen()</code>	Returns the absolute vertical y position of the event.	
boolean	<code>isPopupTrigger()</code>	Returns whether or not this mouse event is the popup menu trigger event for the platform.	
String	<code> paramString()</code>	Returns a parameter string identifying this event.	
void	<code>translatePoint(int x, int y)</code>	Translates the event's coordinates to a new position by adding specified x (horizontal) and y (vertical) offsets.	

Aspecto de los componentes

- Para una aplicación Java susceptible de ejecutarse en **cualquier plataforma**, el aspecto de los componentes debe adaptarse a cada plataforma. Java provee una solución con el mecanismo de **look and feel**. Cada componente gráfico está de hecho formado por dos clases. Una gestiona el aspecto funcional del componente, mientras que la segunda afecta únicamente al aspecto visual de este. Esta última se adapta en función de la plataforma sobre la que se ejecuta la aplicación. La clase **UIManager** es responsable del funcionamiento de este mecanismo. Cuando se crea un componente gráfico, el constructor del componente utiliza la clase UIManager para obtener una instancia de la clase encargada de gestionar el aspecto gráfico del componente. La clase UIManager debe estar informada del look and feel que debe utilizar.

Aspecto de los componentes

- Es posible indicar el **look and feel** que debe utilizarse de tres formas diferentes:
 1. Mediante programación: utilice el método `setLookAndFeel` de la clase `UIManager` pasando como parámetro el nombre completo de la clase que implementa el look and feel deseado.

```
UIManager.setLookAndFeel("javax.swing.plaf.nimbus.NimbusLookAndFeel");
```

- Es aconsejable realizar la modificación del look and feel al comienzo de la aplicación antes incluso de la creación del primer componente gráfico.
- Mediante un parámetro de la línea de comandos utilizada para ejecutar la aplicación.

```
java -Dswing.defaultlaf=javax.swing.plaf.nimbus.NimbusLookAndFeel applicationDemo
```


Aspecto de los componentes

2. Modificando un archivo de configuración.

- También puede utilizar un archivo de configuración `swing.properties` para definir el look and feel por defecto que aplicará la máquina virtual Java durante la ejecución de la aplicación gráfica. Este archivo debe estar situado en la carpeta `lib` de la máquina virtual. Debe contener la siguiente línea:

```
swing.defaultlaf=javax.swing.plaf.nimbus.NimbusLookAndFeel
```

- Los siguientes ejemplos utilizan el look and feel `nimbus` disponible desde la versión 7 de Java. Se distingue de otros look and feel por su modo de gestión gráfica. Las versiones anteriores utilizaban mapas de bits para la representación gráfica de los componentes. Esta nueva versión utiliza un modo gráfico vectorial que proporciona una representación gráfica más precisa que el modo de mapa de bits.

El posicionamiento de los componentes

- ¿No resulta raro que no hayamos definido tamaño ni posición?
- Si redimensiona la ventana de la aplicación, los componentes cambian de sitio.
- Y sin embargo, no se ha previsto ninguna línea de código para efectuar este tratamiento. Este pequeño milagro está relacionado con un concepto muy práctico de Java: los renderizadores (**layout manager**). En realidad, cuando confiamos componentes a un contenedor, este delega a su renderizador la tarea de organizar la disposición de los componentes en su superficie.
- Hay varios tipos de renderizador; cada uno de ellos dispone de una estrategia diferente a la hora de organizar los componentes. Cada tipo de contenedor dispone de un renderizador por defecto, diferente según el tipo de contenedor. Si este renderizador por defecto no le conviene, puede sustituirlo por otro a través de la llamada al método `setLayout`, pasándole el nuevo renderizador que debe utilizar este contenedor.

El posicionamiento de los componentes

- Para poder trabajar y organizar la disposición de los componentes, el renderizador debe conocer el tamaño de cada componente. La mejor manera de obtener esta información es dirigirse directamente al propio componente. Para ello, el renderizador pregunta a cada componente invocando el método `getPreferredSize`. Este método calcula el tamaño del componente según su contenido; por ejemplo, la longitud del título para un botón.
- Para cortocircuitar este cálculo, puede fijar un tamaño por defecto para cada componente con el método `setPreferredSize`.
- El éxito del diseño de una interfaz de usuario supone, por tanto, conocer a fondo cómo funcionan los diferentes renderizadores. Vamos a estudiar las características de los que más se utilizan.

El posicionamiento de los componentes – FlowLayout

a. FlowLayout

- Este renderizador es seguramente el más fácil de utilizar. Está asociado por defecto a un componente **JPanel**. Su estrategia de organización de los componentes consiste en ubicarlos uno tras otro en una línea hasta que no haya más sitio en esta línea. Después de rellenar la primera línea, se ubican los componentes siguientes en una nueva línea, y así sucesivamente. Es el orden en que se añaden los componentes en el contenedor el que determina sus posiciones en las diferentes líneas. Cada componente está separado de manera horizontal de su vecino por un espacio de cinco píxeles predeterminado y cada línea de componentes está separada de su vecina por un espacio de cinco píxeles también por defecto. Cuando este renderizador organiza los componentes, los alinea por defecto en el centro del contenedor. Se pueden modificar todos estos parámetros para cada renderizador **FlowLayout**. Esto se puede hacer desde la creación del FlowLayout indicando en el constructor la información correspondiente. Existen tres constructores para esta clase. El primero no recibe argumento alguno y crea un FlowLayout con las características por defecto descritas más arriba. El segundo recibe como argumento una de las constantes siguientes, que permiten especificar el tipo de alineación.

El posicionamiento de los componentes – FlowLayout

- **FlowLayout.CENTER** : cada línea de componente está centrada en el contenedor (valor por defecto).
- **FlowLayout.LEFT** : cada línea de componente está alineada a la izquierda del contenedor.
- **FlowLayout.RIGHT** : cada línea de componente está alineada a la derecha del contenedor.
- El último constructor disponible recibe como parámetros dos enteros, además de la constante que indica la alineación. Estos dos enteros indican el espaciado horizontal y vertical entre los componentes. La línea de código siguiente crea un FlowLayout que alinea las líneas de componentes en la izquierda del contenedor, deja un espacio horizontal de cincuenta píxeles y un espacio vertical de veinte píxeles entre los componentes.

```
FlowLayout fl=new FlowLayout(FlowLayout.LEFT,50,20);
```

El posicionamiento de los componentes – FlowLayout

- Si en lugar de crear tu propio FlowLayout opta por utilizar el proporcionado por defecto con un JPanel, puede intervenir en estos parámetros de funcionamiento con los métodos siguientes:
- **setAlignment** : para especificar la alineación de los componentes.
- **setHgap** : para especificar el espaciado horizontal entre los componentes.
- **setVgap** : para especificar el espaciado vertical entre los componentes.
- En este caso, es necesario obtener una referencia sobre el FlowLayout asociado al JPanel empleando el método `getLayout` de éste. En este supuesto, se hace obligatorio realizar una operación de tipado dinámico para poder utilizar los métodos de la clase FlowLayout sobre la referencia obtenida.
- De hecho, el método `getLayout` devuelve una variable de tipo `LayoutManager`, el supertipo de todos los tipos de administradores de presentaciones.

El posicionamiento de los componentes – FlowLayout

- Redimensiona la ventana para observar el comportamiento. [Ver ejemplo ihm16](#)

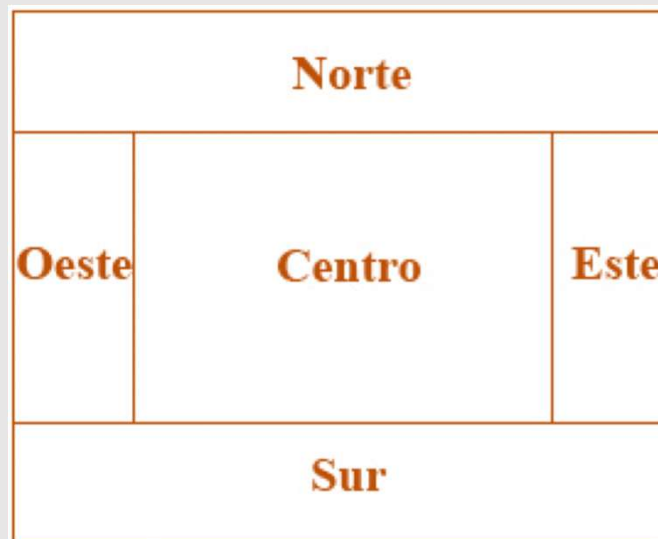
- *//Definición de las características del LayoutManager*
LayoutManager `lm` = `panel.getLayout()`;
`if(lm instanceof FlowLayout){`
 FlowLayout `fl` = (FlowLayout) `lm`;
 `fl.setAlignment(FlowLayout.LEFT);`
 `fl.setHgap(50);`
 `fl.setVgap(20);`
`}`



El posicionamiento de los componentes – BorderLayout

b. BorderLayout

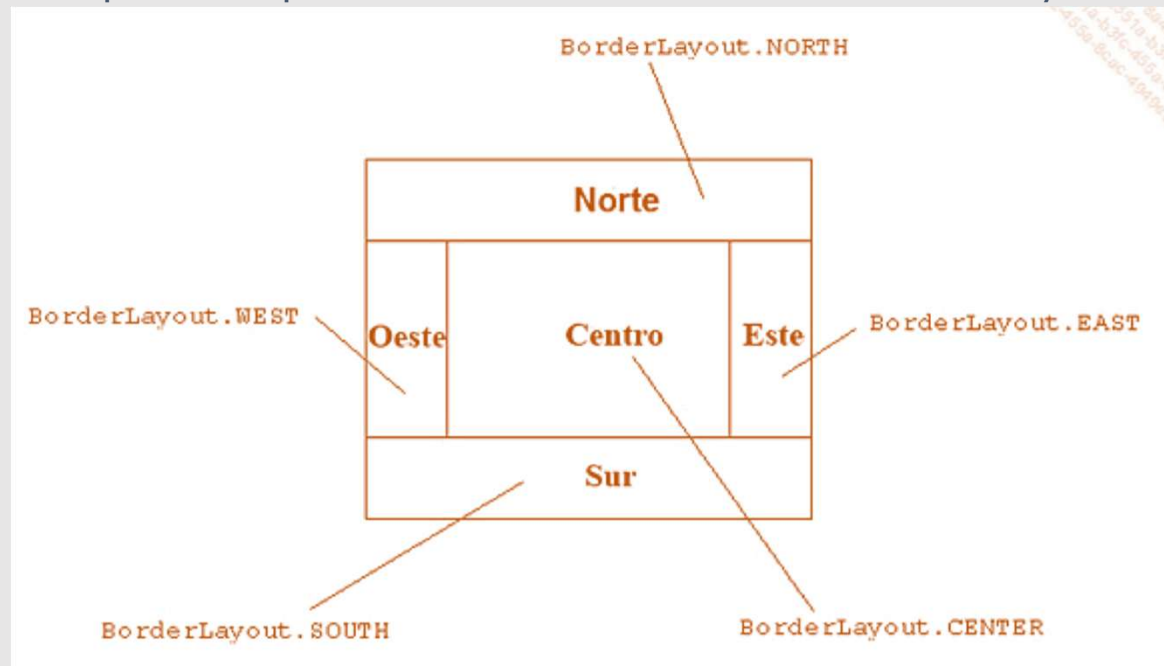
- Este renderizador organiza en cinco zonas la superficie que se le confía, según el esquema siguiente.



- Cuando se confía un componente a un BorderLayout , conviene indicar en qué zona se le debe ubicar.

El posicionamiento de los componentes – BorderLayout

- Se proporciona esta indicación como segundo argumento del método add; el primer argumento siempre es el componente que se debe insertar en el contenedor. Las siguientes constantes están disponibles para identificar una zona del BorderLayout .



El posicionamiento de los componentes – BorderLayout

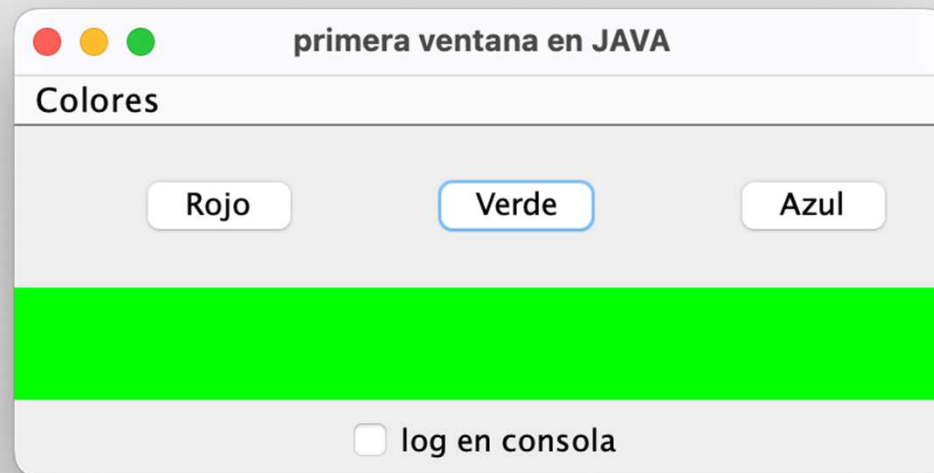
- Si no se indica ninguna información para identificar una zona cuando se agrega un componente, este se ubica de forma **por defecto** en la zona centro (**BorderLayout.CENTER**).
- Al BorderLayout no le gusta el vacío, por lo que redimensiona automáticamente todos los componentes que se le confían para ocupar todo el espacio disponible. Cuando se redimensiona el contenedor, los componentes ubicados en los bordes no cambian de anchura para las zonas Oeste y Este, ni de altura para las zonas Norte y Sur. Por lo tanto, se gana o se pierde espacio en la zona Centro. Cada zona del BorderLayout solo puede contener un único componente. Si, a pesar de todo, se añaden varios componentes a una misma zona de un BorderLayout, este solo se encargará del último añadido y los demás no serán visibles.
- Por esta limitación, se suele utilizar el BorderLayout para posicionar contenedores en relación con otros elementos, más que para posicionar componentes aislados.

El posicionamiento de los componentes – BorderLayout

- El **BorderLayout** es el renderizador por defecto del elemento **ContentPane** de una JFrame. El uso clásico consiste en colocar en la zona Norte la barra o las barras de herramientas, en la zona Sur la barra de estado de la aplicación y en la zona Centro el documento sobre el que debe trabajar el usuario. Como para el FlowLayout , es posible indicar al BorderLayout que debe reservar un espacio vertical u horizontal entre las distintas zonas. Esto se puede llevar a cabo en el momento de la creación del BorderLayout
- BorderLayout indicando el espacio horizontal y vertical en la llamada al constructor o, si utilizamos un existente, invocando los métodos setHgap y setVgap del BorderLayout . Para ilustrar todo ello, vamos a retomar la interfaz de nuestra aplicación ubicando los botones en la zona Norte, la opción marcapunto en la zona Sur y modificando el color de la zona Centro en el momento del clic en los botones o menús.

El posicionamiento de los componentes – BorderLayout

[Ver ejemplo ihm17](#)



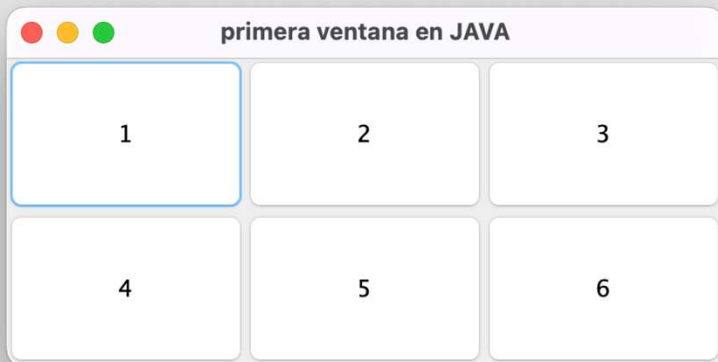
El posicionamiento de los componentes – GridLayout

- Este renderizador organiza la superficie que se le confía mediante una **mallla invisible**. Se debe indicar el número de filas y columnas en la llamada al constructor. Cuando este renderizador ubica dichos componentes, empieza por rellenar todas las casillas de la primera fila y, a continuación, sigue con la fila siguiente y así sucesivamente. Todas las casillas tienen un tamaño idéntico y por lo tanto su contenido se redimensiona para ocupar todo el espacio disponible en cada casilla. Por defecto, se colocan los componentes uno al lado del otro. Es posible insertar un espacio vertical u horizontal entre cada componente especificando dos argumentos adicionales en la llamada al constructor. Estos dos argumentos indican el espaciado horizontal y el espaciado vertical. Estos espaciados también pueden modificarse gracias a los métodos **setHgap** y **setVgap**.
- Si se insertan más componentes que casillas existentes en el **GridLayout**, este creará casillas adicionales aumentando el número de columnas pero respetando el número de filas de origen. Por ejemplo, con el código siguiente prevemos una malla de 2x2 casillas, pero añadimos seis componentes.

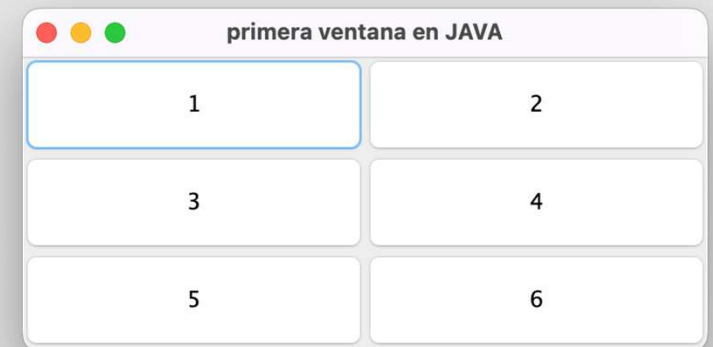
[Ver ejemplo ihm18](#)

El posicionamiento de los componentes – GridLayout

- Los componentes están colocados según el esquema siguiente:
- Si deseamos que el GridLayout respete el número de columnas, debemos indicar que no importa el número de filas y, para ello, debemos especificar el valor 0 para este argumento en el constructor. En este caso, el renderizador añade filas adicionales para los componentes en exceso.



`GridLayout grl;
//grl=new
GridLayout(2,2);
grl=new
GridLayout(0,2);`



El posicionamiento de los componentes – BoxLayout

d. BoxLayout

- Se utiliza este renderizador cuando se necesita organizar componentes en una fila o una columna.
- Aunque se pueda obtener el mismo resultado con un GridLayout de una única fila o de una única columna, este renderizador ofrece más funcionalidades. El precio que se debe pagar a cambio de estas funcionalidades adicionales es el de una relativa complejidad de utilización. El constructor de la clase BoxLayout recibe como primer argumento el contenedor cuyo contenido va a gestionar, y como segundo argumento, una constante que informa de si este contenedor gestiona una fila (`BoxLayout.X_AXIS`) o una columna (`BoxLayout.Y_AXIS`).
- Aunque proporcionemos una referencia al contenedor en el momento de la creación del renderizador, resulta obligatorio asociar el renderizador al contenedor mediante el método `setLayout`, como ya hemos hecho con otros renderizadores. Por lo tanto, se implementa una relación bidireccional.

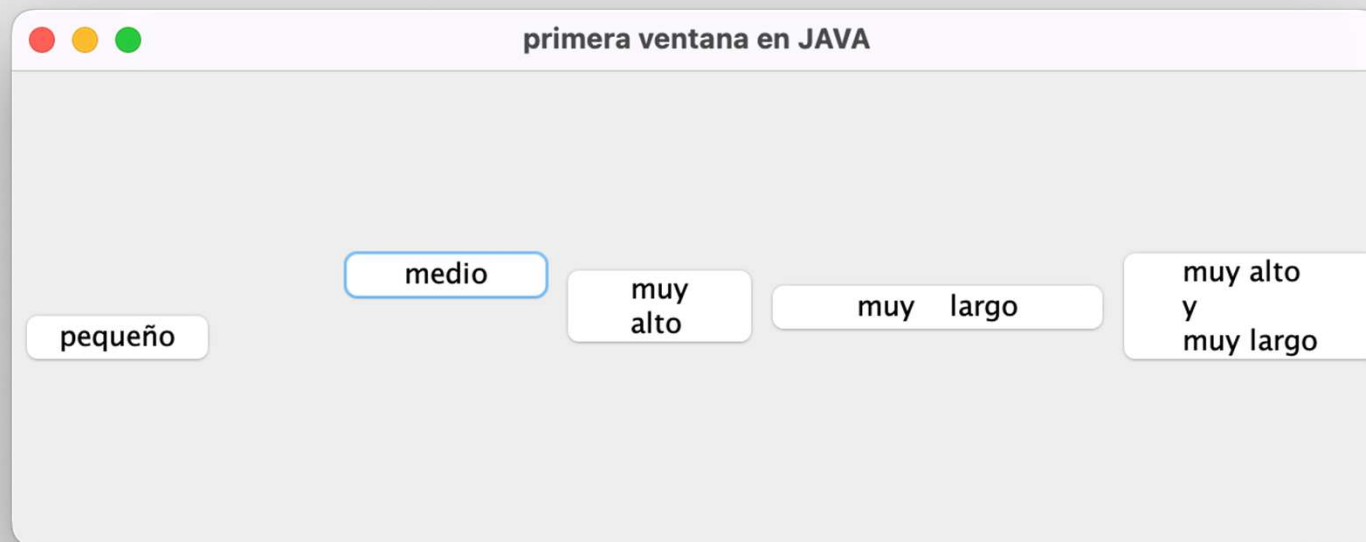
El posicionamiento de los componentes – BoxLayout

Estudiamos ahora la estrategia de funcionamiento de este renderizador. Veamos el ejemplo de un renderizador horizontal.

- El renderizador busca el componente más alto.
- Intenta ampliar los demás componentes hasta esa altura.
- Si un componente no puede alcanzar esa altura, se alinea verticalmente sobre los demás componentes según el valor devuelto por el método `SetAlignmentY` del componente.
 - 0 para una alineación por encima de los demás componentes.
 - 1 para una alineación por debajo de los demás componentes.
 - 0.5 para que esté centrado respecto a los demás componentes.
- Se suman las anchuras preferidas de cada componente.
- Si la suma es inferior a la anchura del contenedor, se amplían los componentes hasta su anchura máxima.
- Si la suma es superior a la anchura del contenedor, se reducen los componentes hasta su anchura mínima. Si no resulta suficiente, no se visualizarán los últimos componentes.
- A continuación, se organizan los componentes en el contenedor de izquierda a derecha sin espacio de separación.
- El funcionamiento es muy similar para un renderizador vertical.

El posicionamiento de los componentes – BorderLayout

[Ver ejemplo ihm19](#)



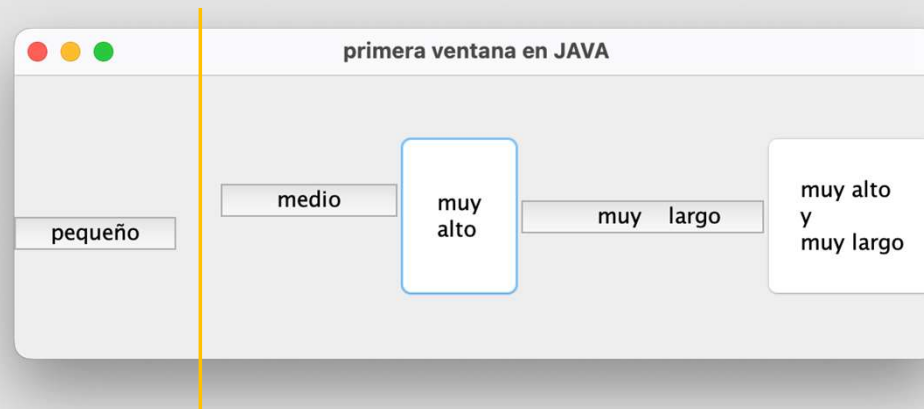
El posicionamiento de los componentes - BoxLayout

- Por defecto no hay espacio entre los componentes gestionados por un BoxLayout y el aspecto final no es gran cosa. Para poder añadir un espacio entre los componentes, debemos insertar componentes de reserva de espacio, que pueden ser de tres tipos:
- **Strut**: este elemento se utiliza para añadir un espacio fijo entre dos componentes. Un Strut puede ser horizontal o vertical según el tipo de BoxLayout al cual se destina. Estos dos tipos de elementos se crean mediante los métodos estáticos `createVerticalStrut` y `createHorizontalStrut` de la clase `Box`. Ambos métodos reciben como argumento el número de píxeles para la anchura o la altura.

```
// creación de un botón con
// alineación en la parte superior
b1=new JButton("pequeño");
b1.setAlignmentY(0);
línea.add(b1);
línea.add(Box.createHorizontalStrut(10));
// creación de un botón con
// alineación en la parte inferior
b2=new JButton(" medio ");
b2.setAlignmentY(1);
línea.add(b2);
```

El posicionamiento de los componentes - BorderLayout

- **Glue:** la meta de este elemento sigue siendo la de separar los componentes, pero, esta vez, el espacio entre los componentes no tiene un tamaño fijo. Se puede comparar el objeto Glue con un muelle que se coloca entre dos componentes y que los aleja siempre lo más posible. Para obtener un funcionamiento correcto del objeto Glue, los componentes deben tener un tamaño fijo o un tamaño máximo fijado. Como los demás elementos de separación, se crea a partir del método estático `createGlue` de la clase `Box`.
- El siguiente código es una adaptación de los ejemplos anteriores, con la instalación de un componente de tipo Glue entre el primer y el segundo botón. [Ver ejemplo ihm20](#)



El posicionamiento de los componentes - GridBagLayout

e GridBagLayout

- Puede considerar el GridBagLayout como el «**súperrenderizador**» tanto desde el punto de vista de las posibilidades que ofrece como de la complejidad de utilización. Funciona básicamente como un GridLayout , ubicando los componentes en el interior de una malla, pero aquí acaba la comparación. En un GridBagLayout las filas y las columnas tienen tamaños variables, las celdas adyacentes se pueden fusionar para acoger los componentes más grandes. Los componentes no ocupan obligatoriamente toda la superficie de sus celdas y es posible modificar su posición en el interior de la celda. La especificidad de este renderizador se sitúa principalmente en el método que permite agregar componentes. Este método recibe como parámetro, por supuesto, el componente que se ha de añadir, pero también un objeto, que indica la manera de colocar el componente en el contenedor.
- Por lo tanto, son las características de este objeto las que van a permitir el posicionamiento del componente por el GridBagLayout . Veamos las características principales:
 - **gridx**: número de la columna a partir de la esquina superior izquierda del componente.
 - **gridy**: número de la fila a partir de la esquina superior izquierda del componente.
- Para estas dos propiedades, la numeración empieza por cero.

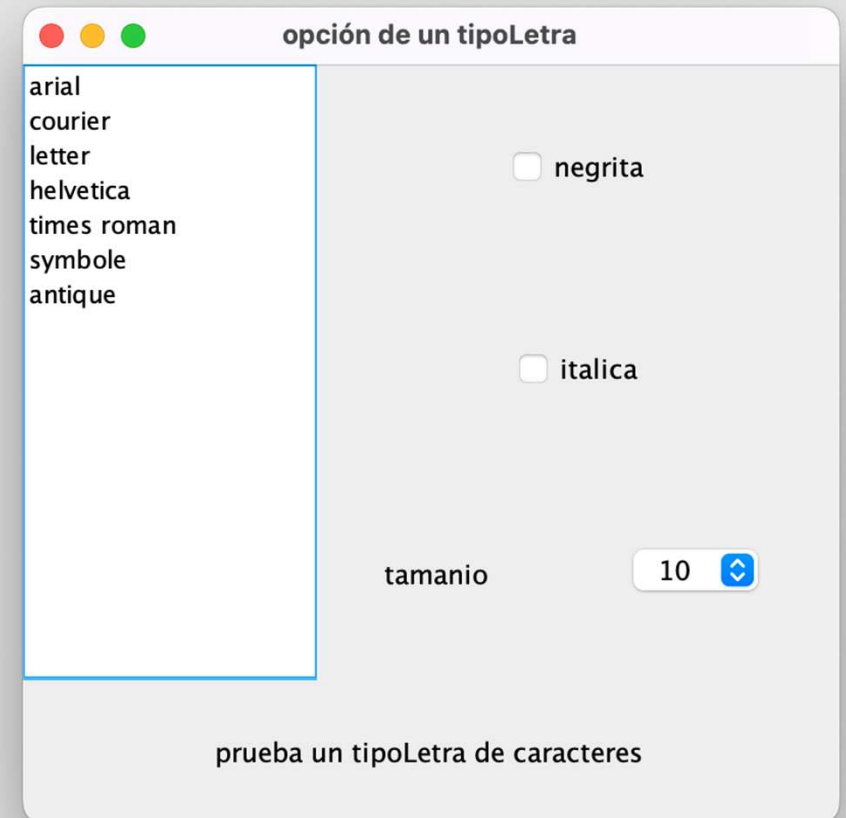
El posicionamiento de los componentes - GridBagLayout

- **gridwidth**: número de columnas ocupadas por el componente.
- **gridheight**: número de filas ocupadas por el componente.
- **weightx**: indica cómo se reparte el espacio adicional disponible en anchura cuando se redimensiona el contenedor. El reparto se hace repartiendo este valor. Si el valor es igual a cero, no se redimensiona. Si es idéntico para todos los componentes, se reparte el espacio de manera proporcional.
- **weighty**: esta propiedad tiene el mismo papel que la anterior, pero en el eje vertical.
- **fill**: se utiliza esta propiedad cuando la zona asignada a un componente es superior a su tamaño. Determina si se redimensiona el componente y la manera de hacerlo. Puede adoptar los siguientes valores constantes:
 - **NONE**: no se redimensiona el componente.
 - **HORIZONTAL**: se redimensiona el componente en anchura y no se cambia su altura.
 - **VERTICAL**: se redimensiona el componente en altura y no se cambia su anchura.
 - **BOTH**: se redimensiona el componente en anchura y altura para rellenar completamente la superficie disponible.
- **anchor**: indica la manera en que está posicionado el componente en el espacio disponible si no se redimensiona. Puede adoptar los siguientes valores constantes.

El posicionamiento de los componentes - GridBagLayout

NORTHWEST	NORTH	NORTHEAST
WEST	CENTER	EAST
SOUTHWEST	SOUTH	SOUTHEAST

- Se puede usar un mismo objeto `GridBagConstraints` para organizar varios componentes en un `GridBagLayout`. En este caso, no hay que olvidar inicializar correctamente los diferentes campos en el momento de la inserción de cada componente. [Ver ejemplo ihm21](#)



El posicionamiento de los componentes - GridBagLayout

f. Sin renderizador

- Si no te convence la utilidad de un renderizador, tienes la posibilidad de gestionar el tamaño y la posición de los componentes de la interfaz de usuario. Primero se debe indicar que se renuncia a emplear un renderizador llamando al método `setLayout` del contenedor y pasándole el valor `null`. A continuación, puede añadir los componentes al contenedor con el método `add` de este. Y, para terminar, debe posicionar y dimensionar los componentes. Estas dos operaciones pueden realizarse en dos etapas mediante los métodos `setLocation` y `setSize` o en una sola con el método `setBounds`.

[Ver ejemplo ihm22](#)

- Nuestra aplicación presenta el mismo aspecto que si empleara un renderizador.
- En cambio, si se redimensiona la ventana, los componentes conservan sus tamaños y posiciones y la interfaz puede presentar un aspecto incoherente.

Los componentes gráficos

- Cada componente utilizable con Java está representado por una clase desde la cual vamos a poder crear instancias para diseñar la interfaz de la aplicación. La mayoría de los componentes Swing proceden de la clase **JComponent** y por este motivo heredan una buena cantidad de propiedades y métodos de esta clase o de las clases situadas por encima en la jerarquía de herencia.

```
Module java.desktop
Package javax.swing

Class JComponent

java.lang.Object
  java.awt.Component
    java.awt.Container
      javax.swing.JComponent

All Implemented Interfaces:
ImageObserver, MenuContainer, Serializable

Direct Known Subclasses:
AbstractButton, BasicInternalFrameTitlePane, Box, Box.Filler, JColorChooser, JComboBox, JFileChooser,
JInternalFrame, JInternalFrame.JDesktopIcon, JLabel, JLayer, JLayeredPane, JList, JMenuBar, JOptionPane, JPanel,
JPopupMenu, JProgressBar, JRootPane, JScrollbar, JScrollPane, JSeparator, JSlider, JSpinner, JSplitPane,
JTabbedPane, JTable, JTableHeader, JTextComponent, JToolBar, JToolTip, JTree, JViewport
```

- Por lo tanto, vamos a estudiar los elementos más útiles de la clase JComponent .

Los componentes gráficos – JComponent

• Dimensiones y posición

Hay dos categorías de información disponibles para el tamaño de los componentes:

1. Se accede al tamaño actual por el método `getSize`. Obtiene el tamaño del componente en el momento de la llamada al método. Si un renderizador se encarga del componente, es susceptible de modificar el tamaño del componente y, por lo tanto, la llamada de este método puede devolver un resultado diferente tras cada llamada. También es posible modificar el tamaño mediante el método `setSize`. Los efectos de este método pueden tener poca duración, ya que si un renderizador se encarga del componente puede redibujarlo en cualquier momento.
2. El tamaño favorito es accesible por el método `getPreferredSize`. Por defecto este tamaño se calcula en función del contenido del componente. Los renderizadores suelen utilizar este método sobre todo a nivel interno. Para evitar que el tamaño del componente se recalcule según su contenido, puede emplear el método `setPreferredSize`. La información facilitada por este método la utiliza, a continuación, el método `getPreferredSize` cuando el contenedor necesita información sobre el tamaño del componente.

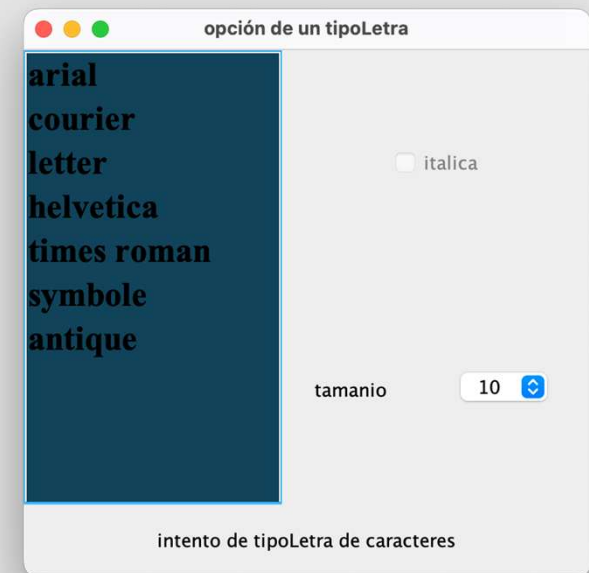
Los componentes gráficos – JComponent

- La posición de un componente en el interior de su contenedor se puede obtener o definir con los métodos `getLocation` o `setLocation`. Como para el método `setSize`, el renderizador puede cancelar en cualquier momento el efecto del método `setLocation` si debe volver a dibujar los componentes.
- Los métodos `getBounds` y `setBounds` permiten combinar la modificación del tamaño y de la posición del componente. Estos métodos solo tienen realmente interés cuando el interior del contenedor asume por completo la presentación de los componentes, sin pasar por los servicios de un renderizador.

Los componentes gráficos – JComponent

• Apariencia de los componentes

- Es posible modificar el **color de fondo** del componente mediante el método **setBackground**, mientras que el **color del texto** del componente se modifica mediante la propiedad **setForeground**. Estos métodos reciben como parámetro un objeto **Color**. Se puede obtener este objeto **Color** con las constantes definidas en la clase **Color**. La clase **Color** propone también numerosos constructores que permiten obtener un color particular al efectuar una mezcla de los colores básicos.
- Se puede modificar la **fuentes** con el método **setFont** del componente. Para la ocasión, podemos crear una nueva instancia de la clase **Font** y asignarla al componente. Para ello, utilizaremos uno de los constructores de la clase **Font** indicando el nombre de la fuente, su estilo y su tamaño.
- Estas dos propiedades se aplican sobre el conjunto del texto que se muestra en el componente. **Ver ejemplo ihm23**



Los componentes gráficos – JComponent

- Es posible utilizar varios colores y tipos de letra para formatear el texto del componente mediante etiquetas **HTML**. Basta, simplemente, con incluir el texto del componente entre las etiquetas `<html>` y `</html>`. En su interior, puede utilizar cualquier etiqueta de formato HTML. El siguiente ejemplo modifica el color del texto y muestra en itálica la etiqueta del botón.
- El método **setCursor** permite elegir la apariencia del cursor cuando se pasa el ratón sobre el componente. Existen varios cursores predefinidos, que son accesibles creando una instancia de la clase **Cursor** con una de las constantes predefinidas.

```
lstFuentes.setCursor(new Cursor(Cursor.HAND_CURSOR));
```

- La detección de la entrada y salida del ratón sobre el componente y la modificación del cursor como consecuencia de ello la gestiona automáticamente el propio componente.

Los componentes gráficos – JComponent

- **Comportamiento de los componentes**

- Se puede ocultar cualquiera de los componentes ubicados en un contenedor invocando el método `setVisible` con el argumento `false` o desactivarlo mediante el método `setEnabled` con el argumento `false`. En este caso, el componente sigue visible, pero aparece con un aspecto gris para indicar al usuario que está inactivo de momento.
- Los componentes que se encuentren ocultos o deshabilitados no podrán recibir el foco de la aplicación.
- Se puede comprobar llamando al método `isFocusable`, que devuelve un booleano. También se puede verificar si un componente tiene actualmente el foco utilizando el método `isFocusOwner`. Se puede colocar el foco sobre un componente sin la intervención del usuario, llamando al método del componente.
- Para manejar el paso del foco de un componente al otro, puede aprovechar dos eventos:
 - `focusLost`: indica que un componente perdió el foco.
 - `focusGained`: indica que un componente particular recibió el foco.

Los componentes gráficos – JComponent

- Por ejemplo, para visualizar correctamente que un componente tiene el foco, se puede emplear el código siguiente que modifica el color del texto cuando el componente recibe o pierde el foco:

```
IstFuentes.addFocusListener(new FocusListener() {  
    public void focusGained (FocusEvent arg0)  
    {  
        IstFuentes.setForeground(Color.RED);  
    }  
    public void focusLost (FocusEvent arg0)  
    {  
        IstFuentes.setForeground(Color.BLACK);  
    }  
});
```

[Ver ejemplo ihm23](#)

Los componentes gráficos – JLabel

Visualización de la información – El componente JLabel

- Se utiliza el componente `JLabel` para visualizar en un formulario un texto que no podrá modificar el usuario. Sirve básicamente para facilitar en una leyenda los controles disponibles (por ejemplo: zonas de texto, listas desplegables...). En este caso, también puede proporcionar un acceso rápido desde el teclado para alcanzar ese componente.
- El texto que muestra el componente se indica en el momento de su creación o mediante el método `setText`. Por defecto, el componente `JLabel` no dispone de borde. Se puede añadir uno invocando el método `setBorder` y pasándole el borde que se quiere emplear.

```
JLabel leyenda;  
leyenda=new JLabel("nombre");  
leyenda.setBorder(BorderFactory.createEtchedBorder());
```

- También puede indicar la posición del texto en el componente mediante los métodos `setHorizontalAlignment` predefinidas.

Los componentes gráficos – JLabel

- Estos métodos no tienen efecto visible alguno si el contenedor dimensiona automáticamente el componente, ya que en este caso su tamaño se ajusta automáticamente a su contenido. Para que estos métodos tengan un efecto visible, deberá indicar un tamaño favorito para el componente.

SwingConstants.RIGHT: alineación a la derecha

SwingConstants.CENTER: alineación en el centro

SwingConstants.LEFT: alineación a la izquierda

SwingConstants.TOP: alineación en la parte superior

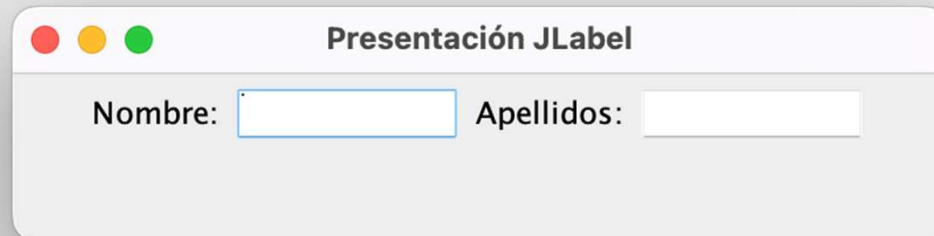
SwingConstants.BOTTOM: alineación en la parte inferior

Ver ejemplos ihm24 y
ihm25

- Los componentes `JLabel` también pueden mostrar imágenes. Podemos indicar la imagen que se quiere visualizar mediante el método `setIcon`. Por supuesto, la imagen debe existir previamente. Si también se visualiza algo de texto en el componente, se puede determinar su posición en relación con la imagen mediante los métodos `setHorizontalTextPosition` y `setVerticalTextPosition`, y también el espacio entre el texto y la imagen con el método `setIconTextGap`.

Los componentes gráficos - JLabel

- Hemos indicado que es posible utilizar el componente JLabel como atajo de teclado para otro componente. Para ello, se deben tomar dos precauciones:
 - Indicar, con el método `setDisplayMnemonic`, el carácter utilizado como atajo.
 - Indicar al JLabel para qué componente juega el papel de leyenda utilizando el método `setLabelFor`.
- El siguiente ejemplo muestra un formulario para introducir un nombre y apellido. Puede acceder a las áreas de entrada con los accesos directos [Alt] n (para el nombre) y [Alt] a (para el apellido).



Ver ejemplo
ihm26

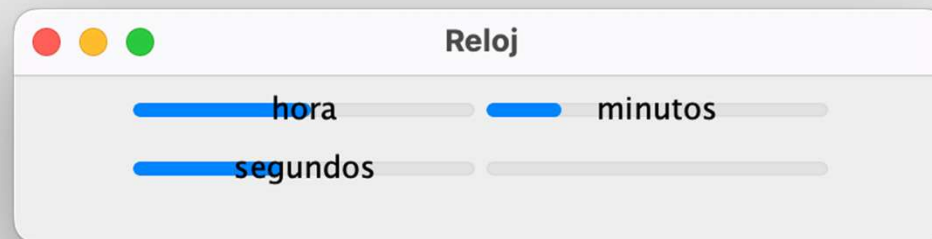
Los componentes gráficos – JProgressBar

- **Visualización de la información – Jprogressbar**

- Se utiliza este componente para informar al usuario del progreso de una acción iniciada en la aplicación.
- Esta información se muestra como una zona rectangular, que estará más o menos llena según el estado de progreso de la acción ejecutada.
- La posición de la barra de progreso está controlada por el método `setValue` . Este método recibe un argumento que permite establecer el progreso entre los dos extremos indicados por los métodos `setMinimum` y `setMaximum`.
- También se puede visualizar una leyenda incrustada en la barra de progreso. Se indica el texto de la leyenda con el método `setString` y se controla su visualización con el método `setStringPainted` .
- Si resulta imposible evaluar el progreso de una operación y por lo tanto obtener un valor para facilitárselo a la barra de progreso, es posible configurar la barra en un estado indeterminado con el método `setIndeterminate` . En este caso, se muestra un cursor que se desplaza entre los dos extremos.

Los componentes gráficos – JProgressBar

- Hay que tener en cuenta que, en el código, la presencia de un thread hijo permite que el reloj funcione. Este thread ejecuta un bucle infinito para actualizar la interfaz gráfica con el paso del tiempo. Es necesario realizar este procesamiento en un thread hijo; de lo contrario, no se realiza ninguna visualización. De hecho, el thread EDT procesa las tareas que se realizarán secuencialmente. Hasta que se complete una tarea, no realiza el resto de las tareas. Si el bucle infinito se ejecutó en el thread EDT, este thread solo haría esta tarea y, por lo tanto, no realizaría las otras, como actualizar la interfaz gráfica.
- También en el thread hijo se realizan operaciones para actualizar las diferentes barras de progreso. Es posible que esto cause excepciones, ya que estas son operaciones de interthread.
- Sería más razonable delegar la actualización de la GUI al thread EDT. La sección sobre operaciones largas, al final de este apartado, cubre este tema.

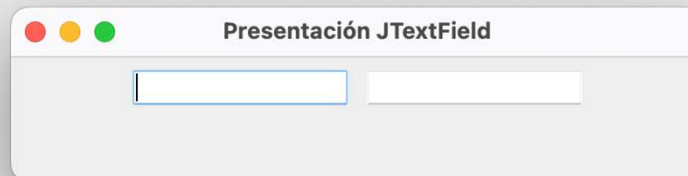


[Ver ejemplo ihm27](#)

Los componentes gráficos – JTextField

Los componentes de edición de texto – JTextField

- Se utiliza el componente `JTextField` para permitir al usuario la introducción de datos. Este componente solo permite la introducción de texto en una única línea. Este componente también es capaz de gestionar la selección de texto y las operaciones con el Portapapeles. Hay distintos métodos disponibles para trabajar con este componente. Por defecto, adapta automáticamente su tamaño a su contenido, lo que puede provocar una revisualización permanente del componente. Para resolver este problema, es preferible especificar un tamaño para el componente utilizando su método `setPreferredSize` o indicando el número de columnas deseado para la visualización del texto en este componente. Esto no limita para nada el número de caracteres que se pueden introducir, sino únicamente la anchura del componente. Es posible modificar o recuperar el texto mostrado en el componente mediante los métodos `setText` o `getText`.



[Ver ejemplo ihm28](#)

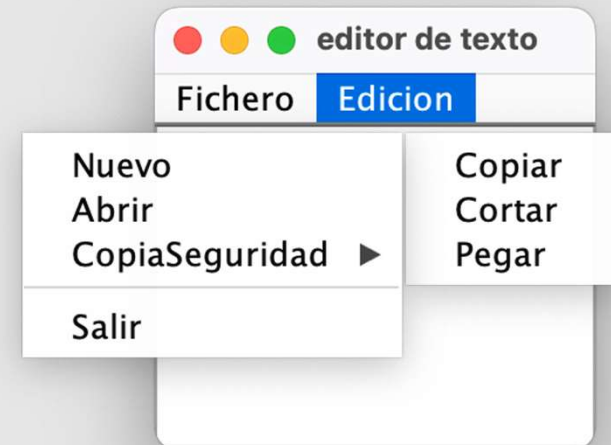
Los componentes gráficos – JTextField

- La gestión de la selección del texto la hace automáticamente el componente. El método `getSelectedText` permite recuperar la cadena de caracteres actualmente seleccionada en el control.
- Los métodos `getSelectionStart` y `getSelectionEnd` indican respectivamente el carácter de principio de la selección (el primer carácter tiene el índice 0) y el último carácter de la selección.
- También se puede efectuar la selección de texto con el método `select`, indicando el carácter de principio y de fin de la selección.
- Se puede seleccionar la totalidad del texto con el método `selectAll`. Por ejemplo, se puede forzar la selección de todo el texto cuando el componente recibe el foco.

```
JTextField txt=new JTextField(10);
txt.addFocusListener(new FocusListener(){
    //Selección del texto en el foco
    public void focusGained(FocusEvent e){
        txt.selectAll();
    }
    //Deselección del texto durante la pérdida del foco
    public void focusLost(FocusEvent e){
        txt.setSelectionStart(0);
        txt.setSelectionEnd(0);
    }
});
```

Los componentes gráficos – JTextField

- Para la gestión del Portapapeles, el componente JTextField gestiona automáticamente los atajos de teclado del sistema para las operaciones de copiar, cortar y pegar. Sin embargo, tenemos la posibilidad de invocar los métodos `copy`, `cut` y `paste` para gestionar las operaciones de copiar, cortar y pegar de otra manera, como por ejemplo mediante un menú de la aplicación o un menú contextual.
- Sin embargo, las operaciones cortar y pegar no serán posibles si el componente `JTextField` está configurado en solo lectura con el método `setEditable(false)`, en cuyo caso, resulta imposible que el usuario modifique el texto.
- Es posible provocar la cancelación de la última modificación de texto efectuada en el control. Para ello, debemos incorporar la ayuda de un objeto `UndoManager`. Efectivamente, este último se encargará de la cancelación de la última modificación gracias a su método `undo`. Este método se invoca mediante una opción del menú contextual o mediante el atajo de teclado `[Ctrl] Z`. Solo existe un nivel de «Undo», ¡No es posible volver al texto introducido hace más de dos horas!
- El método `getKeyCode` se usa para encontrar el código ASCII de la letra mayúscula, correspondiente a la tecla presionada. El método `isControlDown` le permite saber si se presiona la tecla `[Ctrl]`. La combinación de las dos pruebas verifica que se ha usado la combinación `[Ctrl] Z`.



[Ver ejemplo ihm29](#)

Los componentes gráficos – JPasswordField

- **Los componentes de edición de texto – JPasswordField**
- Este componente es en esencia un `JTextField` normal ligeramente modificado para especializarse en la introducción de contraseñas. Las dos diferencias principales realmente útiles son el método `setEchoChar`, que permite indicar qué carácter se utiliza como carácter de sustitución durante la visualización, y el método `getPassword`, que permite obtener la contraseña introducida por el usuario.

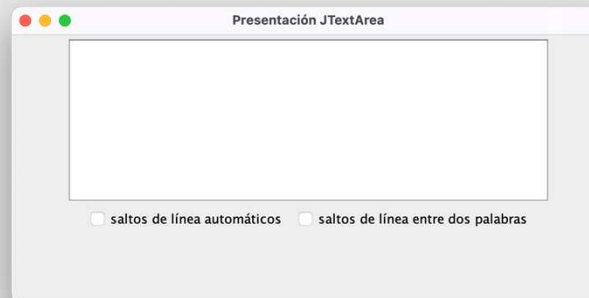
Los componentes gráficos – JTextArea

- **Los componentes de edición de texto – JTextArea**

- Este componente ofrece funcionalidades más avanzadas que un mero `TextField`. La primera mejora importante aportada por este componente reside en su capacidad de gestionar la introducción de datos en varias filas. Por este motivo, en el momento de la construcción de un objeto `TextArea`, debemos especificar el número de filas y de columnas que va contener. Sólo se usan estos dos datos para el dimensionamiento del componente y no influyen en la cantidad de texto que puede contener el componente.
- En cambio, este componente no gestiona por sí mismo el desplazamiento del texto que se introduce. Para añadirle esta funcionalidad, debe estar ubicado en un contenedor de tipo `ScrollPane`, que se encargará del desplazamiento del contenido del `TextArea`.
- Si no se utiliza esta solución, se puede configurar el componente `TextArea` para dividir automáticamente una fila en el momento de su visualización. El método `setLineWrap` permite activar este modo de funcionamiento. Por defecto el recorte se produce en el último carácter visible de una fila, con el riesgo de ver que algunas palabras se muestran en dos filas. Para evitar este problema, se puede configurar el componente `TextArea` para efectuar su recorte en los espacios que separan las palabras con el método `setWrapStyleWord`. Los saltos de línea añadidos por el componente no forman parte del texto y se utilizan únicamente para su visualización.

Los componentes gráficos – JTextArea

- Con las opciones por defecto, las líneas demasiado largas no son visibles en su totalidad y las barras de desplazamiento se activan automáticamente aunque sean necesarias.
- Con la opción `LineWrap`, se cortan las líneas automáticamente a la anchura del componente. Ya no hace falta la barra de desplazamiento horizontal. En cambio, eventualmente es necesaria una barra de desplazamiento vertical porque el número de líneas es superior. Para mejorar la legibilidad del texto, es posible recortar palabras enteras con la opción `WrapStyleWord`.
- La gestión del texto contenido en este componente se ve facilitada gracias a varios métodos específicos.
- `append(String cadena)` : añade la cadena de caracteres que se pasa como argumento al texto ya presente en el componente.
- `insert(String cadena, int position)`: inserta la cadena de caracteres que se pasa como primer argumento a la posición indicada por el segundo argumento.
- `replaceRange(String cadena, int principio, int fin)`: reemplaza la porción de texto comprendida entre el valor facilitado por el segundo argumento y el proporcionado por el tercer argumento por la cadena que se pasa como primer argumento.



Ver ejemplo
[ihmjtextarea](#)

Los componentes gráficos – JButton

Los componentes de activación de acciones – JButton

- Se utiliza principalmente el componente `JButton` en una aplicación para lanzar la ejecución de una acción. Esta acción puede ser la ejecución de una sección de código o el cierre de un cuadro de diálogo. Como para los controles estudiados hasta ahora, el título del botón se puede modificar con el método `setText` del componente.
- Este componente puede también contener una imagen. Se gestiona dicha imagen de la misma manera que para el componente `JLabel`. Se suele asociar casi siempre este componente a un receptor de eventos que implementa la interfaz `ActionListener` con el fin de gestionar los clics en el botón.

Los componentes gráficos

Jmenu, JMenuBar, JMenuItem, JPopupMenu, JSeparator

Los componentes de activación de acciones

- JMenuBar, JMenu, JMenuItem, JPopupMenu, JSeparator
- Este conjunto de componentes va a permitir gestionar los menús de aplicación o los menús contextuales.
- El componente JMenuBar constituye el contenedor que va a acoger los menús representados por JMenu, que, a su vez, van a contener elementos de menús representados por JMenuItem. El componente JMenuBar se comporta como un contenedor para los JMenu. Los JMenu también se van a comportar como un contenedor para JMenuItem u otros JMenu. Por lo tanto, el diseño de un menú va a consistir en crear instancias de estas diferentes clases y a continuación asociarlas entre sí. Para terminar, el JMenuBar obtenido se ubica en su contenedor, que no es otro que el JFrame de la aplicación.
- Naturalmente también hace falta pensar en tratar los eventos producidos por sus elementos. En general, solo se asocia los JMenuItem a receptores de eventos. Se comportan como los JButton. De hecho, tienen un pariente común, ya que los dos heredan de la clase AbstractButton. El diseño de un menú es muy complejo. La única dificultad reside en la cantidad de código relativamente importante necesaria.

[Ver ejemplo ihm30](#)

Los componentes gráficos – JToolBar

- **Los componentes de activación de acciones – JToolBar**

- Este componente sirve de contenedor para los elementos que constituyen una barra de herramientas. Se añaden elementos al componente `JToolBar` como para cualquier otro contenedor. Se suelen colocar en este componente unos `JButton` sin título, pero que muestran imágenes. Facilitan el acceso a alguna funcionalidad ya disponible, generalmente, a través de algún elemento de menú. En este sentido, suelen compartir el mismo receptor de eventos. Este componente presenta la particularidad de que puede desplazarlo el usuario hacia cualquiera de los bordes de la ventana, e incluso en algunos casos al exterior de la ventana. Para que este mecanismo funcione correctamente, el componente `JToolBar` debe ubicarse dentro de un contenedor que utilice un `BorderLayout` como renderizador. En este caso, se le ubica en alguno de los bordes (NORTH, SOUTH, WEST, EAST) y debe ser el único colocado en esta zona. Se puede desactivar esta funcionalidad invocando el método `setFloatable(false)`. La barra de herramientas se queda entonces en su posición de origen. El código siguiente añade una barra de herramientas al editor de texto de la sección anterior.

[Ver ejemplo ihm31](#)

Los componentes gráficos - JCheckBox

Los componentes de selección

JCheckBox

- Se utiliza el componente `JCheckBox` para proponer al usuario diferentes opciones, entre las cuales podrá elegir ninguna, una o varias. El componente `JCheckBox` presenta dos estados: marcado, cuando la opción está seleccionada, o no marcado, cuando la opción no está seleccionada. Es posible verificar o modificar el estado de la opción marcapable mediante los métodos `isSelected` y `setSelected`. Como otros muchos componentes, puede tener un título y una imagen. Sin embargo, la gestión de la imagen difiere un poco de los `JLabel`, ya que viene a remplazar el dibujo de la opción marcapable. En este caso, para poder distinguir entre una `JCheckBox` marcada y otra no marcada, debe asociar una segunda imagen correspondiente al estado marcado. Se indica esta segunda imagen con el método `setSelectedIcon`.

[Ver ejemplo ihm32](#)

Los componentes gráficos – JRadioButton

Los componentes de selección

JRadioButton

- El componente `JRadioButton` permite también proponer al usuario diferentes opciones entre las cuales solo podrá seleccionar una. Como su nombre indica, este control funciona como los botones que permiten seleccionar una emisora en una radio (¡no puede escuchar tres emisoras a la vez!). Las características de este componente son idénticas a las disponibles en el componente `JCheckBox`.
- Para poder funcionar correctamente, los componentes `JRadioButton` se deben agrupar, como es lógico. La clase `ButtonGroup` proporciona esta funcionalidad. Funciona de modo que, entre todos los `JRadioButton` que se le han confiado, mediante su método `add`, solo uno podrá estar seleccionado.
- Para agrupar físicamente unos `JRadioButton`, estos deben estar ubicados en un contenedor, como, por ejemplo, un `JPanel`. Para visualizar correctamente esta agrupación, se suele añadir un borde en el `JPanel` y, de manera opcional, una leyenda.

[Ver ejemplo ihm33](#)

Los componentes gráficos – JList

Los componentes de selección

JList

- El componente **JList** propone al usuario una lista de opciones en la cual podrá seleccionar una o varias de ellas. El componente gestiona automáticamente la visualización de los elementos; sin embargo, no es capaz de asegurar su desplazamiento. Este componente debe ubicarse dentro de un contenedor que pueda encargarse de esta funcionalidad, como por ejemplo un **JScrollPane**. Se pueden proporcionar los elementos visualizados en la lista en el momento de la creación del componente pasando como parámetro al constructor un array de objetos o un Vector. Los métodos **toString** de cada uno de los objetos serán usados por el componente **JList** para poder mostrar el elemento. Con esta solución, la lista se crea en solo lectura y no es posible añadir otros elementos después. Para disponer de una lista en la cual sea posible añadir otros elementos, hay que emplear un objeto de tipo **DefaultListModel** al que se le va a confiar la tarea de gestionar los elementos de la lista. Este objeto posee numerosos métodos, similares a los métodos disponibles para un Vector, que permiten gestionar los elementos. Si deseamos usar una lista dinámica, debemos pasar al constructor de la clase **JList** un objeto de este tipo.

Los componentes gráficos – JList

- Se puede diseñar la lista para autorizar diferentes tipos de selección. La elección del tipo de selección se realiza con el método `setSelectionMode`, al cual se le pasa como argumento una de las constantes siguientes::
- `ListSelectionModel.SINGLE_SELECTION`: solo se puede seleccionar un único elemento de la lista a la vez.
- `ListSelectionModel.SINGLE_INTERVAL_SELECTION`: se pueden seleccionar varios elementos, pero deben ir seguidos en la lista.
- `ListSelectionModel.MULTIPLE_INTERVAL_SELECTION`: se pueden seleccionar varios elementos en la lista y no tienen que ser consecutivos.
- La recuperación del elemento o de los elementos seleccionados puede efectuarse de dos maneras diferentes. Podemos obtener el índice o los índices de los elementos seleccionados con los métodos `getSelectedIndex` o `getSelectedIndices`. Estos métodos devuelven un entero o un array de enteros que representan el índice o los índices de los elementos seleccionados. El objeto o los objetos seleccionados están disponibles mediante los métodos `getSelectedValue` y `getSelectedValues`.

Los componentes gráficos – JList

- De manera similar, estos dos métodos devuelven el objeto seleccionado o un array que contiene los objetos seleccionados. Es posible comprobar la presencia de un elemento seleccionado en la lista mediante el método `isSelectedEmpty`. Se puede cancelar la selección con el método `clearSelection`. La modificación de la selección en la lista activa un evento de tipo `ValueChanged`.
- Es posible gestionar este evento asociando a la lista un receptor de eventos de tipo `ListSelectionListener` con el método `addListSelectionListener`.
- Hay que tener cuidado con este evento, ya que se produce varias veces seguidas cuando se selecciona un elemento. El primer evento corresponde a la desección del elemento anterior, y el segundo, a la selección del elemento actual. Por lo tanto, es este último el que hay que tener en cuenta. Para ello, el argumento `ListSelectionEvent` disponible con este evento dispone del método `getValueIsAdjusting`, que permite saber si la selección terminó o si se encuentra en un estado transitorio. Para probar el funcionamiento, vamos a añadir a nuestro editor de texto una lista que permite seleccionar una fuente de caracteres.

[Ver ejemplo ihm34](#)

Los componentes gráficos - JComboBox

Los componentes de selección

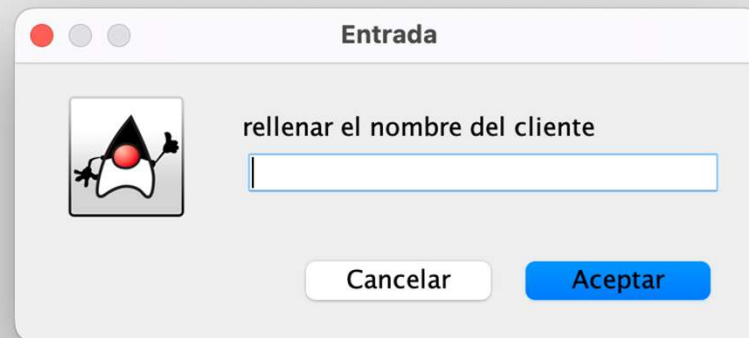
JComboBox

Ver ejemplo ihm35

- Se puede asimilar este componente a la asociación de otros tres componentes:
 - un `JTextField`
 - un `Jbutton`
 - un `JList`
- El usuario puede elegir un elemento en la lista que aparece cuando hace clic en el botón o bien introducir directamente en la zona de texto la información esperada si esta no está disponible en la lista. Este control presenta dos modos de funcionamiento. Podemos autorizar la selección de un único elemento en la lista al configurar la zona de texto en modo de solo lectura con el método `setEditable(false)` , o bien autorizar la selección en la lista, o la introducción en la zona de texto con el método `setEditable(true)`. La creación de una instancia de este componente sigue las mismas pautas que la creación de un `JList`. La recuperación del elemento seleccionado se efectúa de manera similar a un `JList`. Hay que tener cuidado, sin embargo, ya que no hay selecciones múltiples en este tipo de componente.
- La detección de la selección de un nuevo elemento se puede hacer gestionando el evento `actionPerformed` . Este evento se activa en el momento de la selección de un elemento en la lista o cuando el usuario valida la introducción con la tecla [Intro]. Terminamos el editor de texto proponiendo al usuario la selección de un tamaño de fuente de caracteres.

Los cuadros de diálogo

- Los cuadros de diálogo son ventanas que tienen una función especial en la aplicación. Se utilizan, por lo general, para solicitar al usuario que introduzca información, presentarle un mensaje o hacerle una pregunta. Para asegurarse de que el usuario ha procesado el cuadro de diálogo antes de continuar con la aplicación, este se muestra en modo modal, es decir, el resto de la aplicación se bloquea mientras se está mostrando el cuadro de diálogo. Para evitar tener que recrear cada vez un nuevo cuadro de diálogo, podemos acceder a varios cuadros de diálogo predefinidos. Están disponibles mediante métodos `static` que se hallan en la clase `JOptionPane`.



Ver ejemplo
ihm36

Los cuadros de diálogo

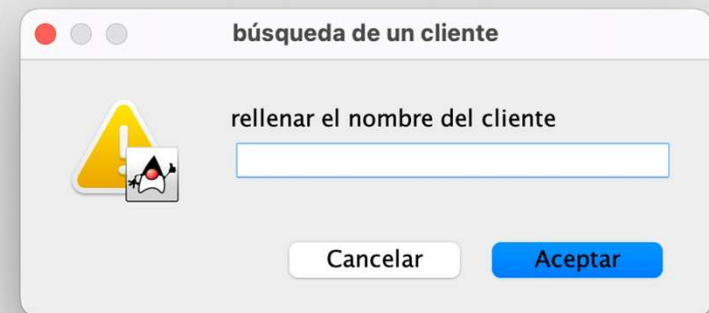
a. El cuadro para introducir información

- El cuadro para introducir información permite solicitar al usuario que introduzca una cadena de caracteres. Esta funcionalidad está disponible mediante la función `showInputDialog` de la clase `JOptionPane`. Tenemos a nuestra disposición varias versiones de esta función, que nos permiten configurar la representación del cuadro de diálogo. La más sencilla recibe como parámetro una cadena de caracteres que representa el mensaje que muestra el cuadro de diálogo. Generalmente, este mensaje indica la naturaleza de la información que debe introducir el usuario.

Los cuadros de diálogo

- Una versión más completa permite escoger:
 - **el modo de visualización:** ventana autónoma o ventana interna de un componente. En este caso, se le pasa como primer parámetro el componente afectado, o el valor null si la ventana es autónoma.
 - **el mensaje** que se muestra en el cuadro de diálogo. Es el segundo parámetro que se utiliza para este propósito.
 - **el título** del cuadro de diálogo como tercer parámetro.
 - **el tipo de icono** que se visualizará en el cuadro de diálogo. Una serie de constantes definidas en la clase permiten realizar dicha selección.

ERROR_MESSAGE	
INFORMATION_MESSAGE	
WARNING_MESSAGE	
QUESTION_MESSAGE	
PLAIN_MESSAGE	Ningún icono

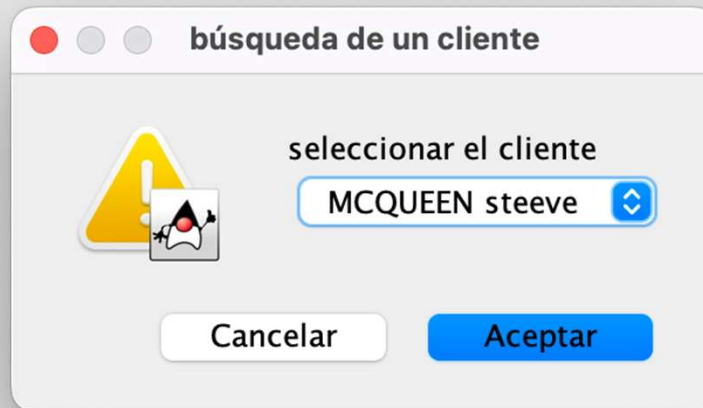


Los cuadros de diálogo

- Existe otra versión que tiene un funcionamiento ligeramente distinto. No proporciona una zona de libre introducción de texto, sino una lista de objetos entre los que el usuario debe realizar su elección. La lista representa los objetos invocando al método `toString` de cada uno de ellos. Recibe como parámetros:
 - El **modo de visualización**: ventana autónoma o ventana interna a un componente. En este caso, se le pasa como primer parámetro el componente afectado o el valor null si la ventana es autónoma.
 - El **mensaje** que muestra el cuadro de diálogo. El segundo parámetro se utiliza con este propósito.
 - El **título** del cuadro de diálogo como tercer parámetro.
 - El **tipo de icono** que se visualizará en el cuadro de diálogo. Una serie de constantes definidas en la clase permiten realizar dicha selección.
 - Un **icono** que permite remplazar al icono por defecto o null para conservar el icono por defecto.
 - Una **tabla de objetos** que forman la lista de opciones que se presentan al usuario.
 - El **objeto** que representa la opción por defecto seleccionada en la visualización del cuadro de diálogo.
- Esta función devuelve el objeto seleccionado o null si se utiliza el botón Cancelar para cerrar el cuadro de diálogo.

Los cuadros de diálogo

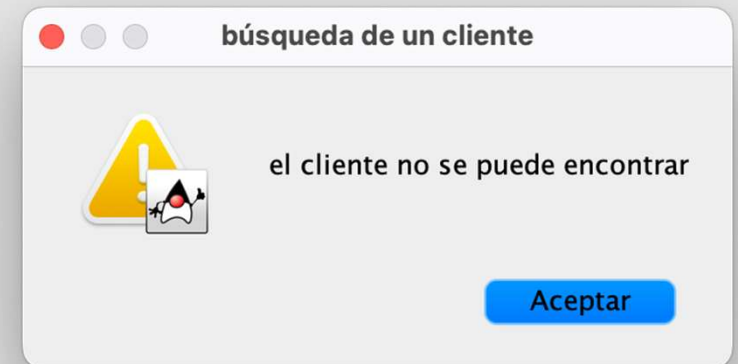
- El siguiente ejemplo permite seleccionar una persona entre las propuestas en la lista.



Los cuadros de diálogo

b. El cuadro de mensaje

- El cuadro de mensaje permite mostrar una información al usuario; está disponible mediante la función `showMessageDialog`.
- Esta función recibe los siguientes parámetros:
 - El **modo de visualización**: como ventana autónoma o interna a un componente. En este caso, se pasa como primer parámetro el componente afectado o el valor null si la ventana es autónoma.
 - El **mensaje** que se muestra en el cuadro de mensaje. El segundo parámetro se utiliza con este propósito.
 - El **título** del cuadro de diálogo como tercer parámetro.
 - El tipo de icono que se muestra en el cuadro de mensaje. Una serie de constantes definidas en la clase permiten realizar dicha selección. Estas **constantes** son idénticas a las que se utilizan en el cuadro de diálogo para introducir información.



Los cuadros de diálogo

c. El cuadro de confirmación

- Este cuadro de diálogo se utiliza para realizar una pregunta al usuario y permitirle responder mediante uno de los botones que proporciona. Los botones disponibles están predefinidos por el cuadro de confirmación. La función `showConfirmDialog` provoca la apertura del cuadro de confirmación. Devuelve un valor entero que permite identificar el botón utilizado para cerrar el cuadro de diálogo y, en consecuencia, obtener la respuesta del usuario.
- Recibe los siguientes parámetros:
 - El **modo de visualización**: como ventana autónoma o interna a un componente. En este caso, se pasa como primer parámetro el componente afectado o el valor null si la ventana es autónoma.
 - El **mensaje** que se muestra en el cuadro de confirmación. El segundo parámetro se utiliza con este propósito.
 - El título del cuadro de diálogo como tercer parámetro.
 - La combinación de botones que se ha de mostrar. Las constantes `YES_NO_OPTION`, `YES_NO_CANCEL_OPTION` u `OK_CANCEL_OPTION` permiten configurar este parámetro.
 - El tipo de icono que se muestra en el cuadro de mensaje. Una serie de constantes definidas en la clase permiten realizar dicha selección. Estas constantes son idénticas a las que se utilizan en el cuadro de diálogo para introducir información.
- Esta función devuelve un valor entero que se compara, a continuación, con las siguientes constantes para determinar la `YES_OPTION`, `OK_OPTION`, `NO_OPTION`, `CANCEL_OPTION` o `CLOSE_OPTION` respuesta por parte del usuario.



¿Preguntas?

Planificación de Interfaces Gráficas y Fundamentos del Diseño UI/UX

Módulo: 2º DAM – Desarrollo de Interfaces





Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Índice

**THINK TWICE
CODE ONCE!**

PLANIFICACIÓN DE INTERFACES

1. Introducción a las interfaces gráficas : Las leyes de Gestalt
2. El papel del diseñador de interfaces
3. Trabajo colaborativo en proyectos digitales
4. Objetivos de una interfaz gráfica
5. Tipos de interfaces gráficas
6. Proceso de diseño de una interfaz
7. Principios fundamentales del diseño de interfaces
8. Elementos gráficos de una interfaz
9. Accesibilidad en interfaces gráficas
10. Documentación del diseño

FUNDAMENTOS DEL DISEÑO UI/UX

1. Introducción al diseño UI/UX
2. Principios de Diseño de Interfaces Gráficas
3. Arquitectura Emocional: Diseñar para Conectar
4. Modelos Mentales y Diseño Intuitivo
5. Jerarquía Visual e Interacción
6. Diseño Centrado en Tareas y Objetivos del Usuario
7. UI y UX: Colaboración Perfecta
8. Recursos

01 Planificación de Interfaces Gráficas

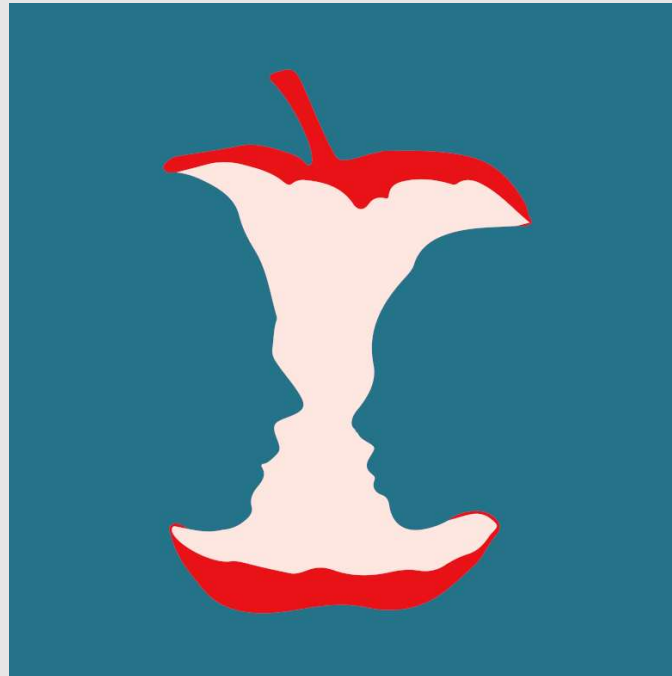
Interfaces

- ¿Qué ocurre cuando un usuario observa uno de nuestros diseños?



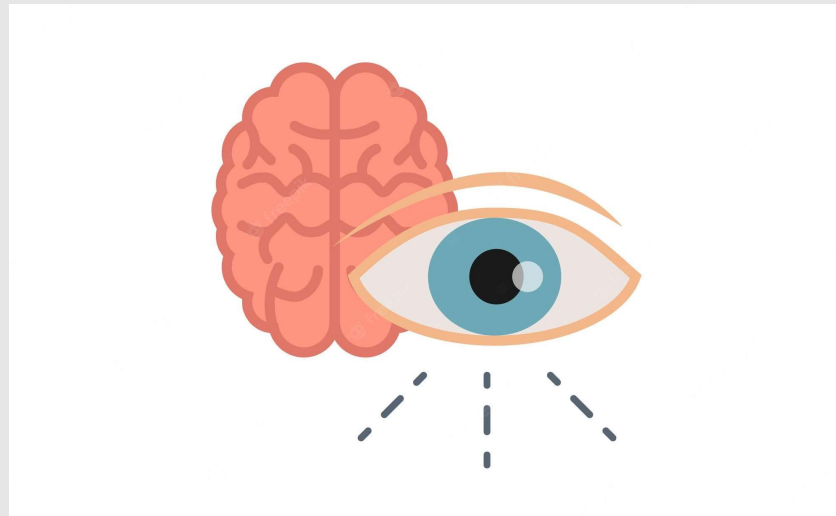
Interfaces

- ¿Cómo podemos saber cómo se está percibiendo nuestro trabajo?



Interfaces

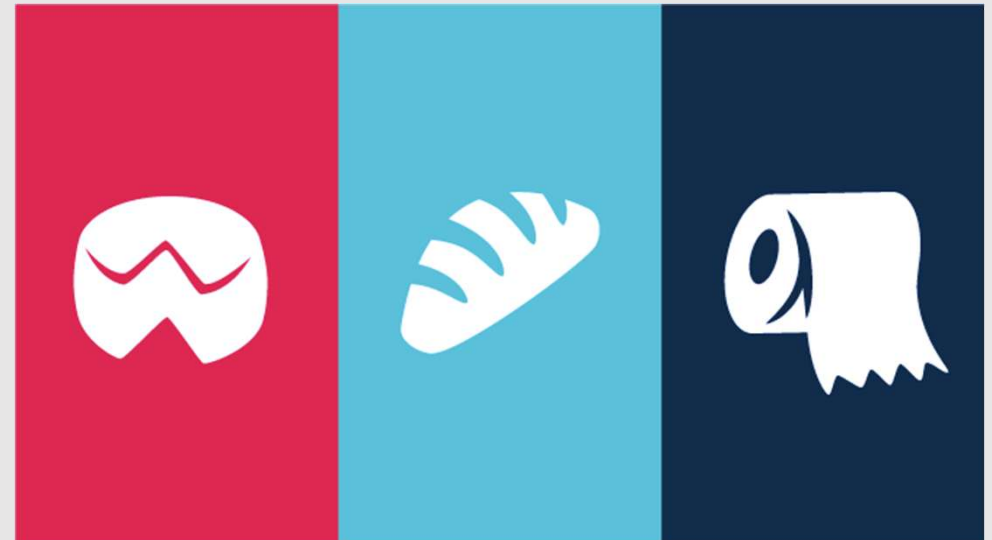
- La psicología siempre va a tener un papel importante en diseño gráfico, porque estamos buscando impactar, comunicar un mensaje y queremos garantizar que se percibe correctamente por nuestro espectador.
- Por eso, el diseño gráfico no sólo tiene que ver con las emociones, también con la percepción.



Teoría de Gestalt



- Conocer la Teoría de Gestalt y sus principios o leyes, nos va ayudar a llamar la atención de nuestros espectadores, a estimular nuestra creatividad y a mejorar la funcionalidad de nuestros diseños.
- En diseño gráfico, buscamos influir o impactar visualmente a nuestro espectador. Para conseguirlo mejor, necesitamos conocer cómo funciona la percepción humana. Y aquí es donde entra en juego la Teoría de la Gestalt.



¿Qué es la Teoría de Gestalt?



- La Teoría o psicología de la Gestalt, fue un movimiento que se inició en la década de 1920 en Berlín, Alemania.
- Gestalt es una palabra alemana que significa patrón, figura o forma.
- Este movimiento buscaba encontrar un sentido a cómo nuestras mentes perciben las cosas como un todo, en lugar de individualmente cada elemento.
- La corriente estudiaba cómo nuestra mente configura, mediante ciertas leyes o principios básicos, los elementos que llegan a ella a través de los canales sensoriales o de la memoria. Demostraron que nuestro cerebro siempre va a intentar simplificar y organizar imágenes complejas, porque está preparado para comprender mejor el mundo que nos rodea.



Relación Teoría de Gestalt con el diseño gráfico

- Según Gestalt cuando percibimos el mundo, a pesar de que existen distintos estímulos al mismo tiempo, para organizarlos, nuestra mente percibe el entorno como formas o grupos de elementos. **Lo simplifica.**
- Por tanto, la Teoría de la Gestalt también es aplicable al diseño de interfaces o usabilidad web. Para que un diseño funcione bien se debe considerar siempre, no sólo los elementos visuales por separado, sino también cómo se percibe la totalidad del diseño.
- Antes de profundizar en la teoría o principios gestálticos vamos a conocer cómo funciona nuestra percepción de la realidad visual, y por qué tendemos a agrupar los elementos, necesitamos conocer los **4 conceptos clave** desde los que se parte para desarrollar precisamente estos principios.

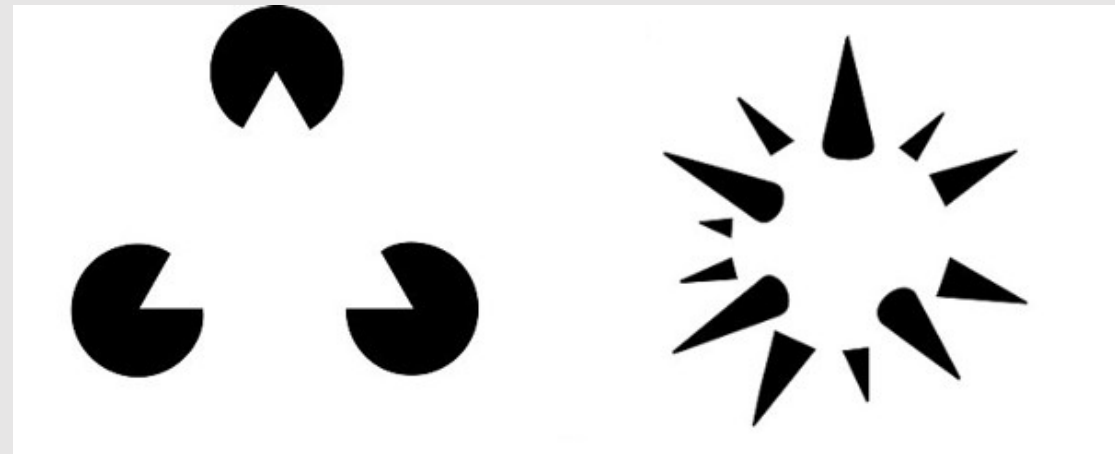
Aparición

- Aparición
- La idea de aparición surge cuando se percibe un objeto y le damos una interpretación a partir de la información previa que ya conocemos. Suele emplearse con frecuencia en el diseño de logos para las empresas.
- En el ejemplo que podéis ver arriba, nuestra mente va a visualizar antes un perro que las formas individuales, porque es una silueta que ya conocemos.



Cosificación

- **Cosificación**
- Las personas pueden reconocer objetos incluso cuando faltan partes de ellos. Nuestro cerebro hace coincidir lo que vemos con patrones familiares almacenados en nuestra memoria para llenar los vacíos.
- Aquí podéis ver un nuevo ejemplo. En cada una de estas dos figuras, nuestra mente construye nuevas formas en el espacio negativo a partir de las que percibimos visualmente. En el primer ejemplo percibiríamos un círculo, en el segundo un cuadrado y en el tercero un triángulo.



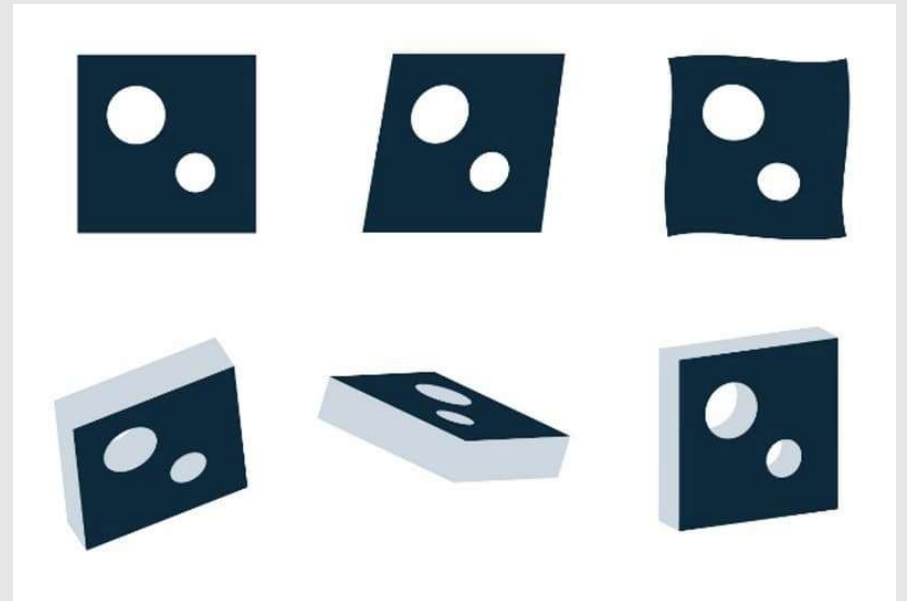
Multiestabilidad

- **Multiestabilidad**
- La idea de la Multiestabilidad se basa en que la gente a menudo interpreta objetos ambiguos en más de una forma. Nuestros cerebros van de un lado a otro buscando la certeza.
- Un ejemplo muy conocido que nos puede servir para demostrar esta idea clave de la Teoría de la Gestalt, sería la famosa ilustración desarrollada por el psicólogo danés Edgar Rubin. Todos la conocemos. Al espectador se le presentan dos formas: dos rostros o un jarrón. Puedes preferir ver una u otra forma y es probable que te quedes un rato observándola intentando analizar cuál es la forma acertada.



Invariabilidad

- Invariabilidad
- Las personas pueden reconocer objetos simples independientemente de su rotación, de su escala o de traslación. Nuestro cerebro puede percibir objetos desde diferentes perspectivas a pesar de que cuenten con apariencia diferente.
- En este ejemplo podemos percibir que se trata exactamente de la misma forma, a pesar de que el objeto se ha rotado, escalado o trasladado.



Las Leyes de la Gestalt

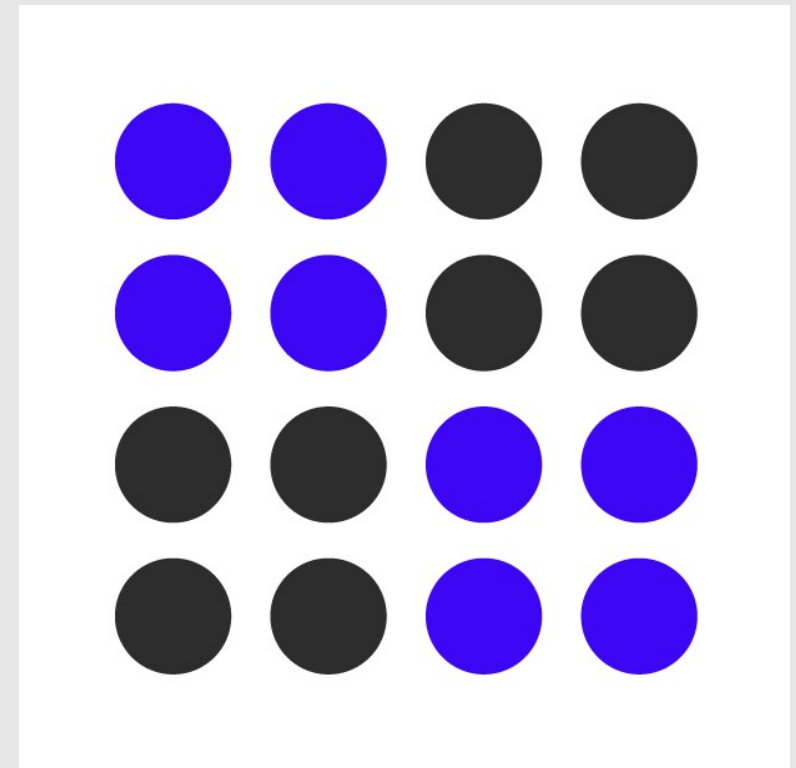


- Ahora que ya conocemos las 4 ideas clave desde las que partieron los miembros de la Teoría de la Gestalt, para desarrollar las leyes, ya podemos aprender estos principios y conocer cómo se relacionan con el diseño gráfico.
- Como he mencionado antes, estos principios intentan explicar cómo y cuándo nuestras mentes perciben diferentes componentes visuales como parte de un todo.
- Si uno se quiere dedicar al diseño gráfico, necesitamos conocer estos principios, porque nos van a ayudar a crear un trabajo estructurado y coherente, que atrape la atención del espectador.



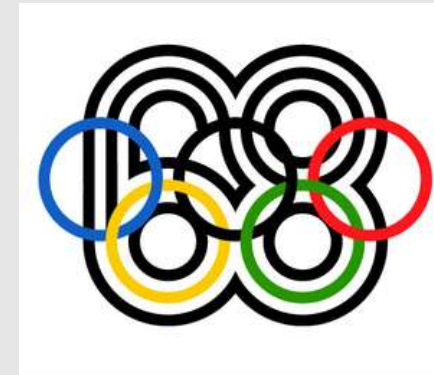
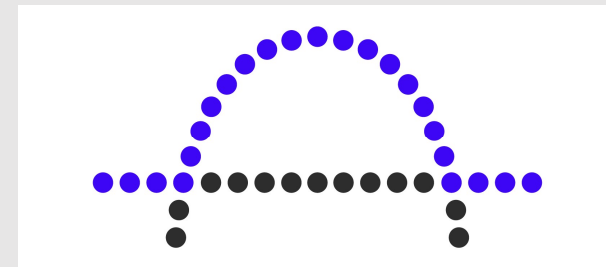
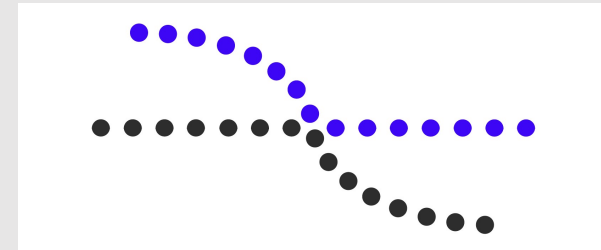
Principio de Semejanza o Similitud

- El principio de semejanza se utiliza para unir elementos que estén o no próximos entre sí. Si los encontramos similares, tenderemos a verlos como parte de un grupo. Además, si un diseño está formada por objetos similares, el espectador va a percibir el conjunto como algo equilibrado y coherente. Cuanto mayor es la similitud entre los objetos, más coherencia transmitirá la obra en su conjunto.
- La similitud entre los diferentes elementos puede ser de forma, color, tamaño o textura.
- Recordad, que gracias al principio de semejanza, podemos generar énfasis sobre un elemento, rompiendo el patrón de similitud dentro del grupo. A estos casos donde un objeto destaca del resto y no concuerda con la similitud, se denomina Anomalía.
- En el ejemplo, los círculos se encuentran equidistantes unos de otros, con exactamente el mismo tamaño. Sin embargo, nuestra mente pasa a agruparlos por color, y distinguimos dos conjuntos, por un lado los azules y por otro los negros. Esto es debido a la semejanza de los colores.



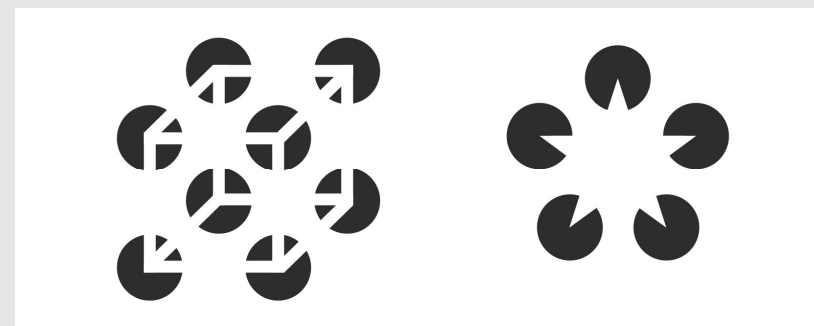
Principio de Continuidad

- El ojo puede crear visualmente la continuidad de una línea. Este principio se basa en la idea de que el ojo humano va a seguir siempre el camino visual más suave, menos forzado y más coherente.
- Por ejemplo, en estas ilustraciones que podéis ver bajo estas líneas, nuestro ojo va a querer seguir la línea recta o curva, de un extremo a otro, incluso aunque las líneas cambien de color a mitad de su recorrido. En este caso, tendemos a agrupar los elementos por la continuidad que percibe nuestro ojo.
- El principio de Continuidad puede resultarnos de mucha utilidad, para por ejemplo, apuntar hacia un elemento del diseño o cuando necesitamos guiar la mirada de nuestro espectador.



Principio de Cierre

- Cuando percibimos una figura que no está cerrada o delimitada por líneas, nuestra mente tiende a transmitir una sensación de cierre o forma completa.
- Según la Teoría de la Gestalt, las formas cerradas se perciben como formas más estables. Tendemos a cerrar o completar las formas abiertas, buscando otorgarles más estabilidad. Por eso, tendemos a agrupar esas formas individuales, como parte de un grupo, buscando componer una forma cerrada y completa.
- El principio de cierre es una técnica bastante recurrida en diseño gráfico, especialmente en el diseño de logos. Al percibir un elemento como incompleto, nuestro cerebro trabaja rellenando la información que falta.



Principio de Proximidad

- El principio de proximidad teoriza cómo tendemos a agrupar las formas o elementos visuales que están próximos entre sí. Para ello, dichos objetos deben compartir alguna de estas características en común: forma, color, tamaño o textura.
- Por otro lado, si aumentamos el espacio entre los elementos podemos conseguir una sensación de separación, aún cuando los elementos tengan características en común.
- En el ejemplo que podéis ver aquí comprobamos que la única diferencia entre la imagen de la izquierda o la imagen de la derecha es la proximidad de las líneas. Sin embargo, nuestra mente interpreta la imagen de la derecha como 3 grupos diferentes de elementos.
- El logo de Unilever puede ser también un buen ejemplo de cómo aplicar el principio de proximidad sobre un diseño. Gracias a la proximidad de los elementos podemos acabar distinguiendo la letra U.



Principio de Figura y Fondo

- El principio de Figura y Fondo es bastante similar al principio de cierre, porque aprovecha también la forma en que el cerebro procesa el espacio negativo. Este principio describe la tendencia que tiene el ojo humano a ver un objeto separado de lo que le rodea. Nuestra mente no es capaz de interpretar un mismo elemento como figura o como fondo al mismo tiempo.

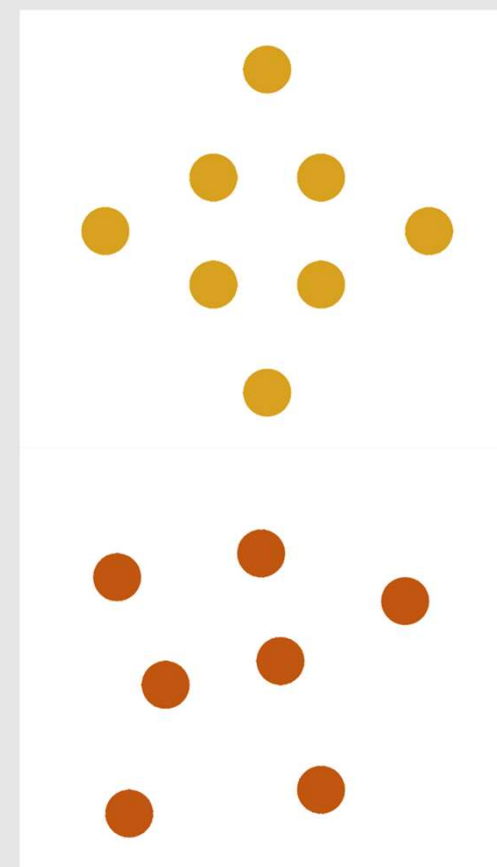
GESTALT



PROMETEO

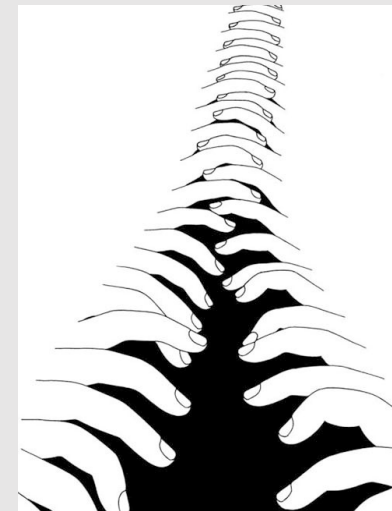
Principio de Simetría y Orden

- Este principio, también conocido como prägnanz (que significa buena figura), teoriza que los elementos que cuentan con simetría y orden, son también percibidos como parte del mismo grupo.
- A la izquierda podéis ver como la disposición de los elementos simétrica y ordenada, favorece a que los percibamos como parte de un grupo, mientras que a la derecha se han dispuesto los elementos de forma aleatoria.
- Además, este principio establece que nuestra mente tiende a analizar siempre de la forma más simple, organizando los elementos en figuras simétricas, regulares y estables. Según este principio una composición no debe proporcionar nunca una sensación de desorden o falta de equilibrio.
- En Diseño gráfico esto es clave, ya que de lo contrario el espectador perderá el tiempo tratando de buscar el equilibrio o de localizar lo que falta, centrándose en la composición en lugar de en el mensaje que estamos buscando transmitir.

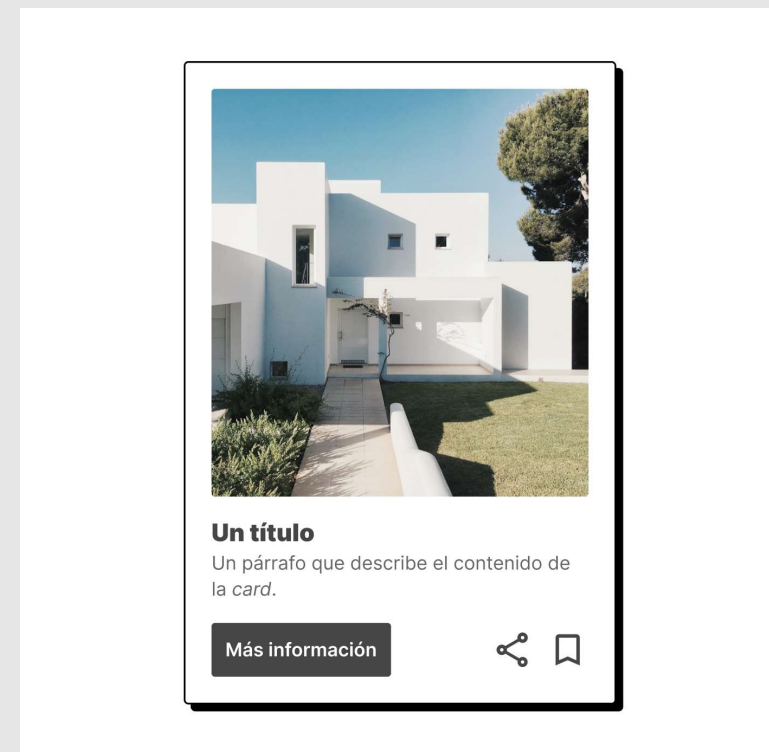
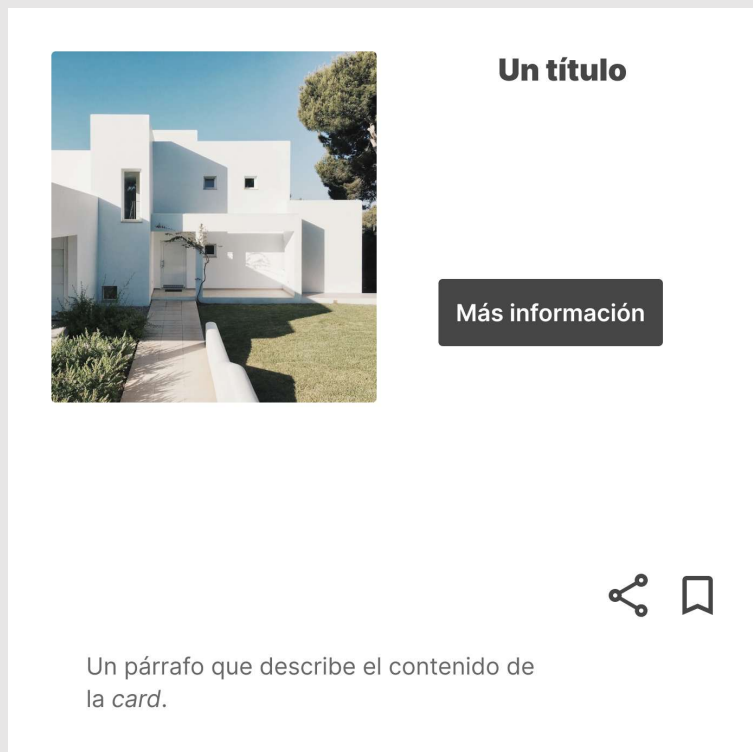


Principio de Dirección Común

- En los últimos años se habla también de este principio de dirección común. Esta última ley, teoriza que los objetos y elementos que parece que construyen un patrón o flujo en la misma dirección, son una vez mas percibidos como parte de un grupo.
- Aquí os dejo algunos últimos ejemplos del principio de dirección, que como podéis comprobar no cuenta con ninguna dificultad.



Aplicamos a la web/app



Semejanza

[Vende tu tech](#)[Reacondicionado por profesionales](#)[BackStories](#)[¿Necesitas ayuda?](#)

[Ofertones](#)[Móviles](#)[Portátiles](#)[Tablets](#)[Consolas y videojuegos](#)[Smartwatches](#)[Audio](#)[Electrodomésticos](#)[Más](#)

Sácale partido a la tecnología con nosotros

Compra y vende tecnología que es mejor para el planeta.

[Reacondicionado por profesionales](#)

[Hazte un renove](#)

[30 días de prueba](#)

[Ofertas exclusivas en la app](#)

Compra los "más buscados"


iPhone


Macbook


iPad


Consolas

Compra nuestras mejores ofertas




Android


MacBook


iPad


Portátiles

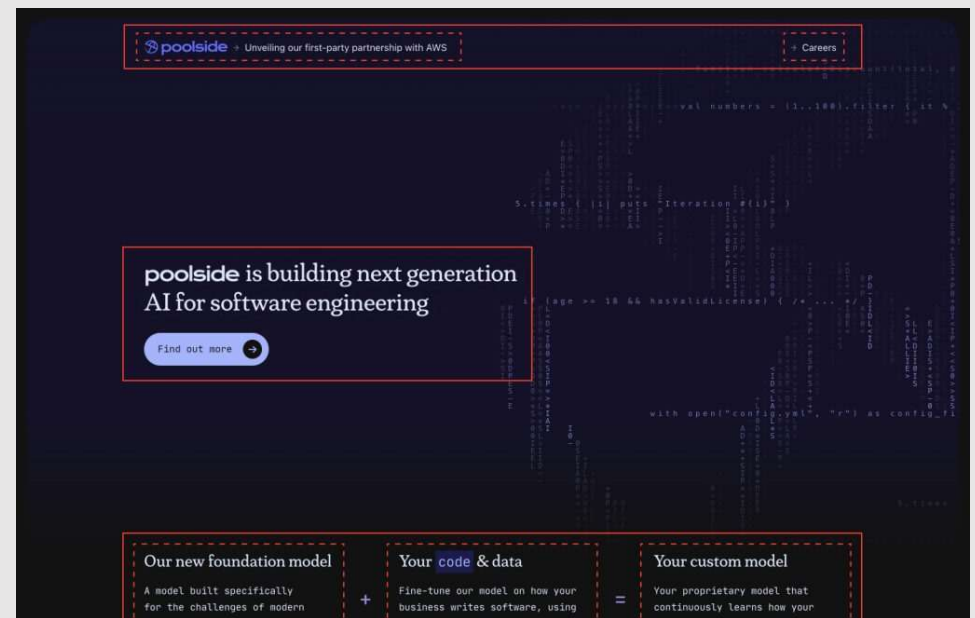
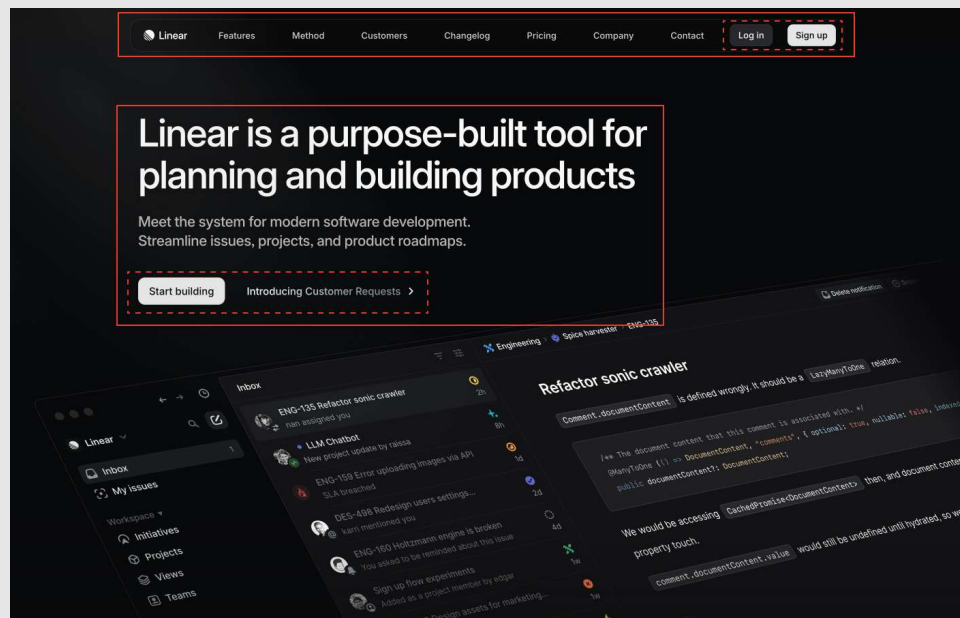

Altavoces


Consolas


Cámaras


Ocio en ci

Proximidad



Continuidad

stripe

Start your company in a few clicks

Incorporate in the US from anywhere in the world. Join more than 50,000 companies already using Atlas.

Get everything you need to run your business

Set up banking, payroll, accounting, cloud computing, and more with \$100K in exclusive discounts.

Charge your customers—for free

Share a payment link or embed checkout to accept payments. We'll waive your first year of Stripe Payments fees.

Create your Stripe account

Email

Full name

Password

☒ Get emails from Stripe about product updates, industry news, and events. You can [unsubscribe](#) at any time. [Privacy Policy](#)

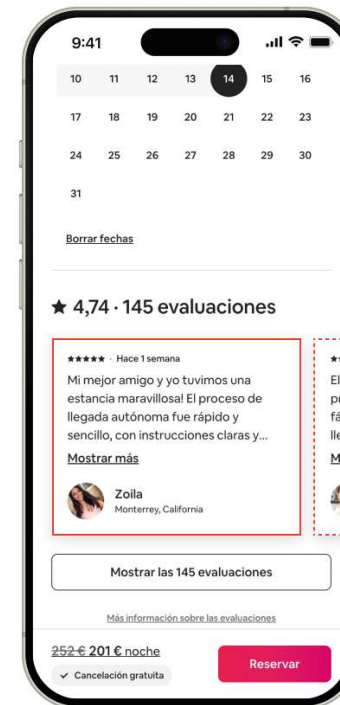
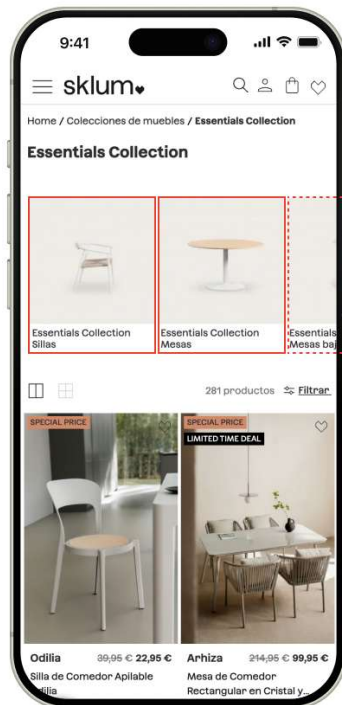
Create account

Already have an account? [Sign in](#)

© Stripe

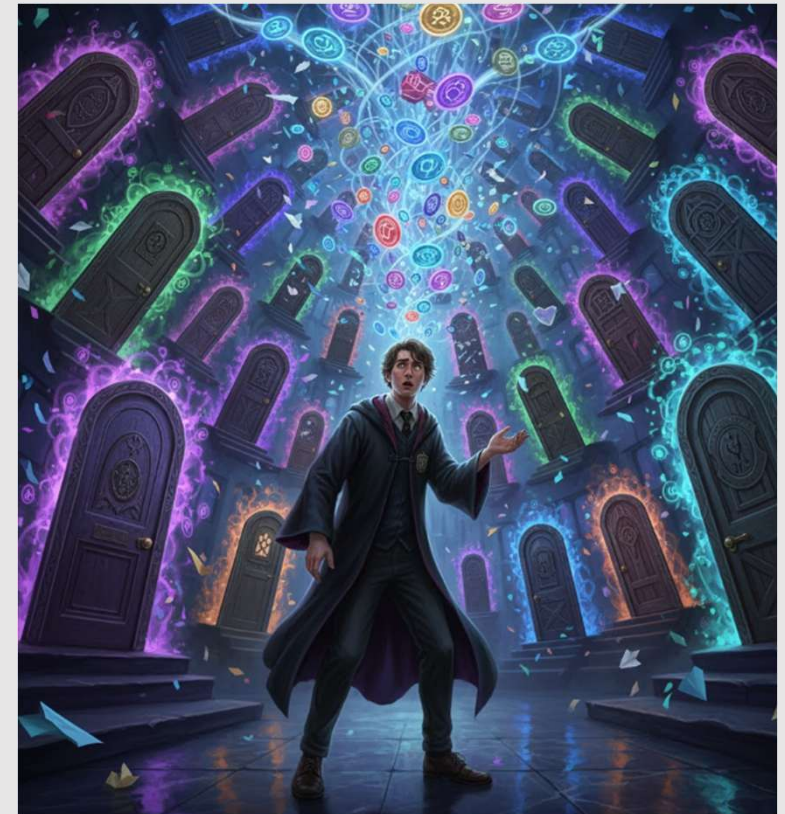
[Privacy & terms](#)

Cierre



¿Qué es una Interfaz gráfica?

- Una interfaz gráfica de usuario (**GUI - Graphical User Interface**) es el espacio visual y funcional donde el usuario interactúa con un sistema informático, una aplicación o un dispositivo. Está formada por elementos visuales como ventanas, botones, menús, iconos, formularios, textos y otros componentes gráficos que permiten que el usuario controle y reciba información del sistema de manera sencilla, intuitiva y eficiente.
- **Ejemplo :** como cuando en el Ministerio de Magia aparecen mil puertas y botones mágicos, pero tú solo quieres llegar al despacho de Arthur Weasley.



Características clave de las interfaces gráficas

Características de una Interfaz de Usuario Efectiva

Compatibilidad de Dispositivos

Funciona en varios dispositivos como ordenadores y móviles



Interacción Visual y Táctil

Permite a los usuarios interactuar a través de pantallas táctiles y elementos visuales



Interactividad

Responde a las acciones del usuario con retroalimentación en tiempo real



Metáforas Visuales

Utiliza iconos y símbolos reconocibles para la navegación



El papel del diseñador de interfaces

El **diseñador de interfaces gráficas** es el profesional encargado de crear estos espacios visuales donde los usuarios interactúan con la tecnología.

Su trabajo permite que la experiencia sea:

- **Fácil de entender:** que el usuario sepa qué hacer sin explicaciones complicadas.
- **Eficiente:** que pueda realizar sus tareas con el mínimo esfuerzo y tiempo.
- **Atractiva:** que el diseño sea visualmente agradable y coherente
- **Accesible:** que pueda usarla cualquier persona, independientemente de sus capacidades.

Funciones:

- **Investigar y comprender** las necesidades y comportamientos de los usuarios.
- **Diseñar estructuras, componentes y flujos de navegación.**
- **Elegir colores, tipografías, iconos y estilos** que transmitan la identidad del producto y faciliten su uso.
- **Elaborar prototipos**, wireframes y mockups para mostrar la interfaz antes de desarrollarla.
- **Colaborar con desarrolladores** para asegurar que el diseño se implemente correctamente.
- **Realizar pruebas y mejoras** continuas basadas en el feedback de usuarios.

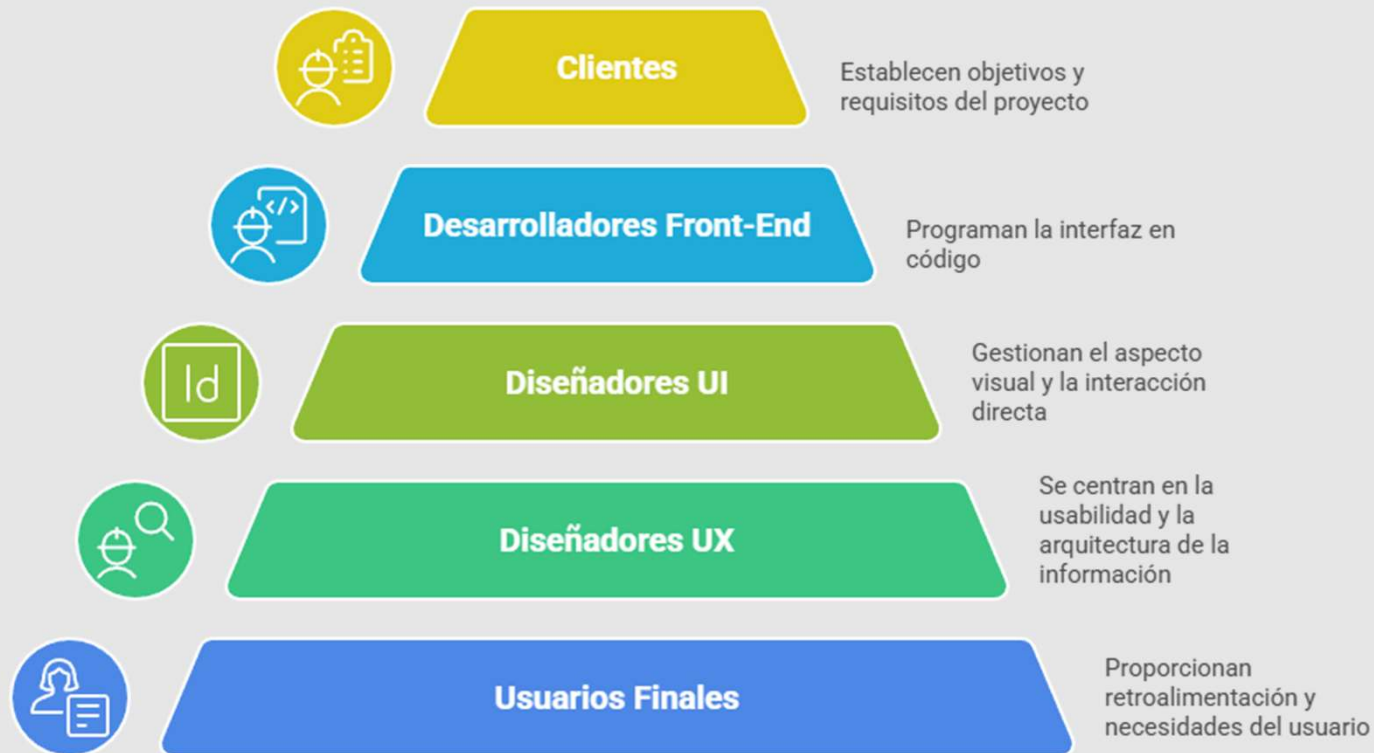
Colaboración con equipos de desarrollo, UX y cliente

- El diseño de interfaces es un proceso colaborativo que involucra varios perfiles profesionales:
 - **Diseñadores UX (Experiencia de Usuario):** Se centran en la investigación del usuario, análisis de necesidades, arquitectura de la información y la usabilidad general.
 - **Diseñadores UI (Interfaz de Usuario):** Se encargan del aspecto visual y la interacción directa.
 - **Desarrolladores front-end:** Programan la interfaz en código.
 - **Clientes y usuarios finales.**

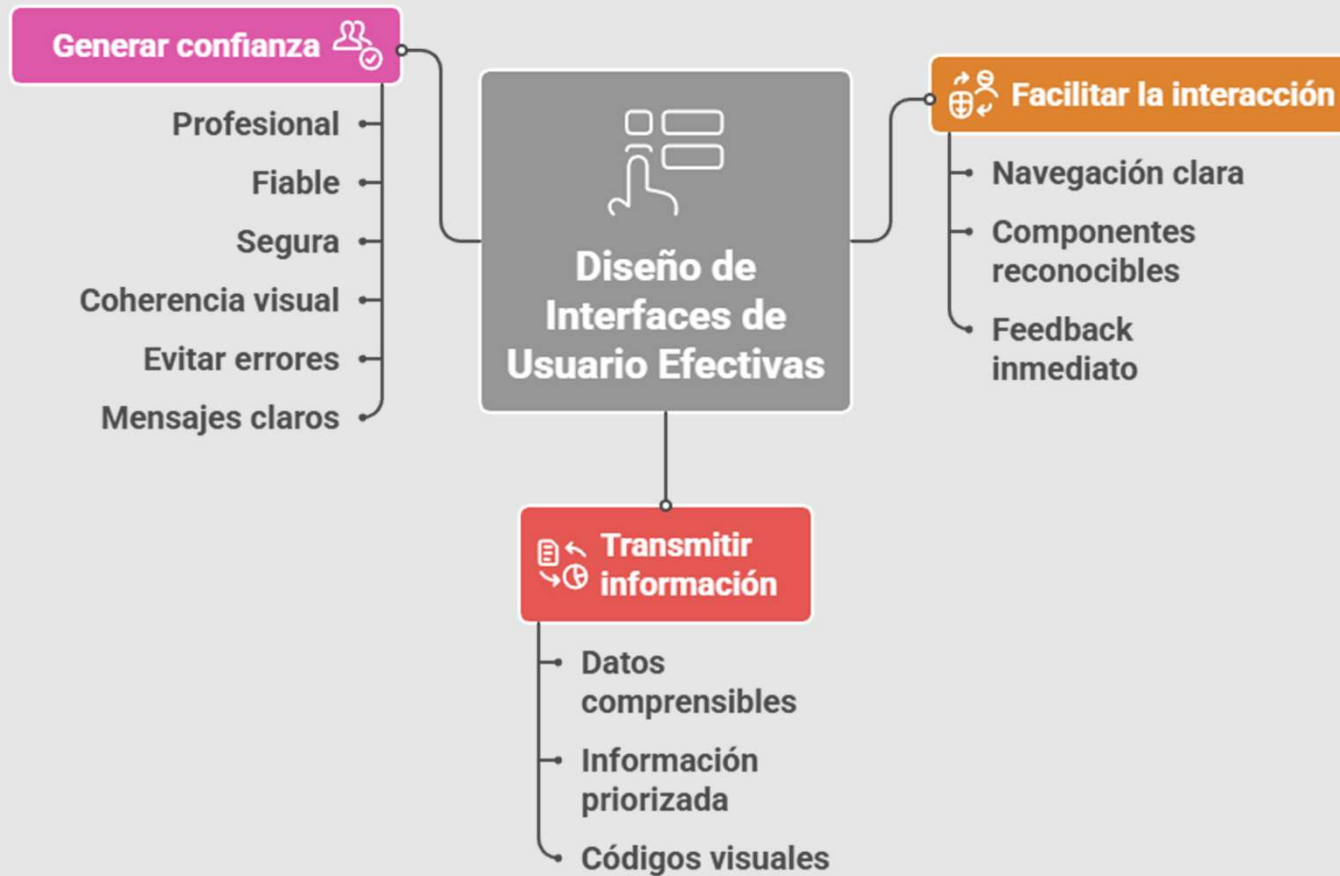
El diseñador de interfaces actúa como puente entre los objetivos del cliente, las necesidades reales del usuario y las capacidades técnicas del equipo de desarrollo.

Colaboración con equipos de desarrollo, UX y cliente

Jerarquía de Diseño de Interfaces



Objetivos de una interfaz gráfica



Olga M. Moreno Martín

PROMETEO

Tipos de interfaces Gráficas

Interfaces de escritorio

- Utilizadas en sistemas operativos como Windows, macOS o Linux.
- Basadas en ventanas, menús, iconos y barras de herramientas.
- Adecuadas para aplicaciones complejas o profesionales.

Interfaces móviles

- Diseñadas para pantallas pequeñas y control táctil.
- Basadas en gestos, botones grandes y navegación simplificada.
- Necesitan adaptarse a multitud de dispositivos.

Interfaces web

- Accesibles desde navegadores.
- No requieren instalación.
- Compatibles con múltiples plataformas y tamaños de pantalla.

Interfaces embebidas

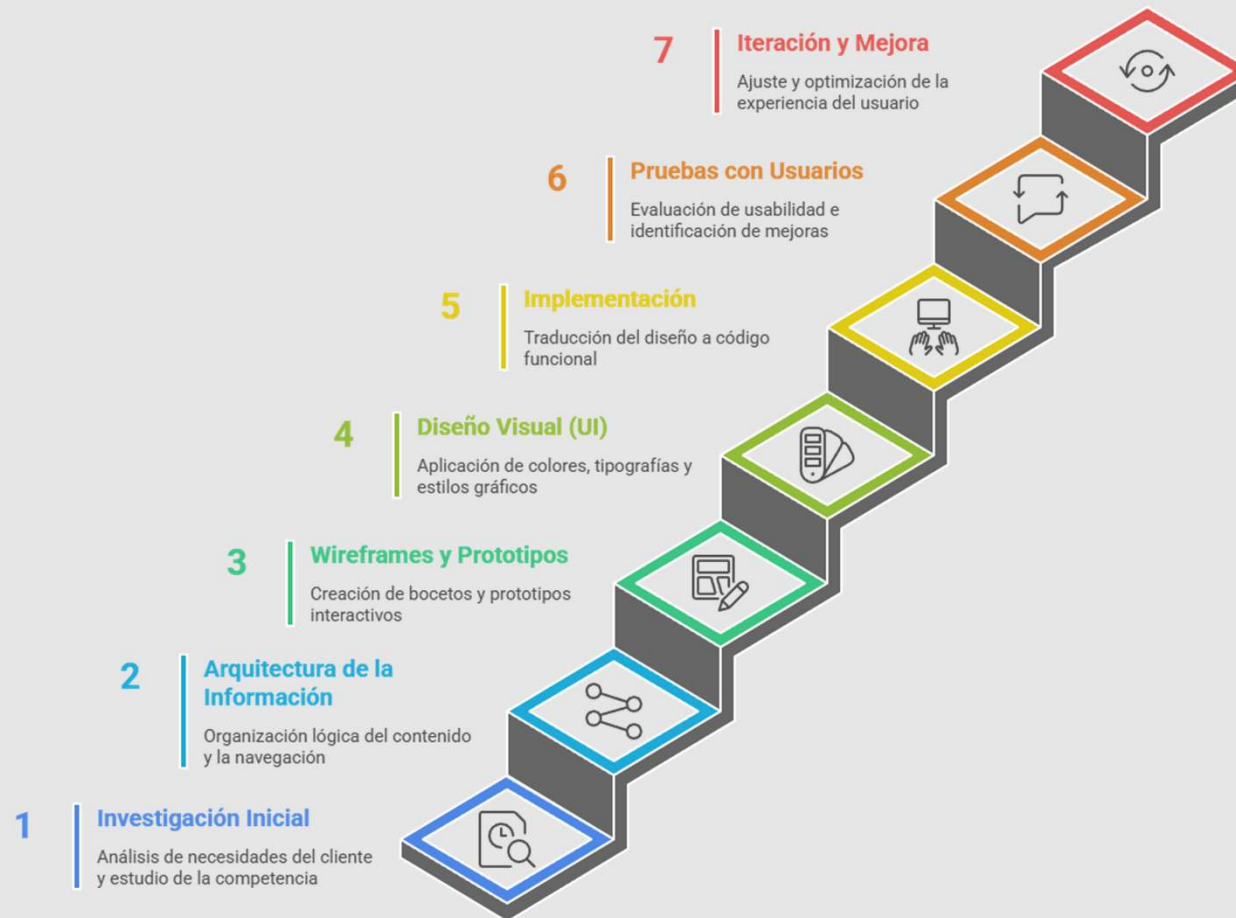
- Presentes en electrodomésticos, vehículos, cajeros automáticos, etc.
- Deben ser extremadamente simples y resistentes a errores.

Proceso de diseño de una interfaz gráfica

El diseño de interfaces implica una serie de pasos clave:

- **1. Investigación inicial**
 - Análisis de necesidades del cliente.
 - Perfiles de usuarios.
 - Estudio de productos similares o competencia.
- **2. Arquitectura de la información**
 - Organización lógica del contenido y las funcionalidades.
 - Definición de jerarquías, categorías y rutas de navegación.
- **3. Wireframes y prototipos**
 - Bocetos de baja o media fidelidad para visualizar estructura.
 - Prototipos interactivos para mostrar flujo y comportamiento.
- **4. Diseño visual (UI)**
 - Aplicación de colores, tipografías, iconos y estilos gráficos.
 - Construcción de elementos y componentes visuales coherentes.
- **5. Implementación**
 - Traducción del diseño a código funcional por parte del equipo técnico.
 - Revisión continua entre diseño y desarrollo.
- **6. Pruebas con usuarios**
 - Evaluación de usabilidad.
 - Identificación de errores, confusiones o mejoras potenciales.
- **7. Iteración y mejora**
 - Ajuste de componentes.
 - Optimización de experiencia y accesibilidad.

Proceso de diseño de una interfaz gráfica



Olga M. Moreno Martín

PROMETEO

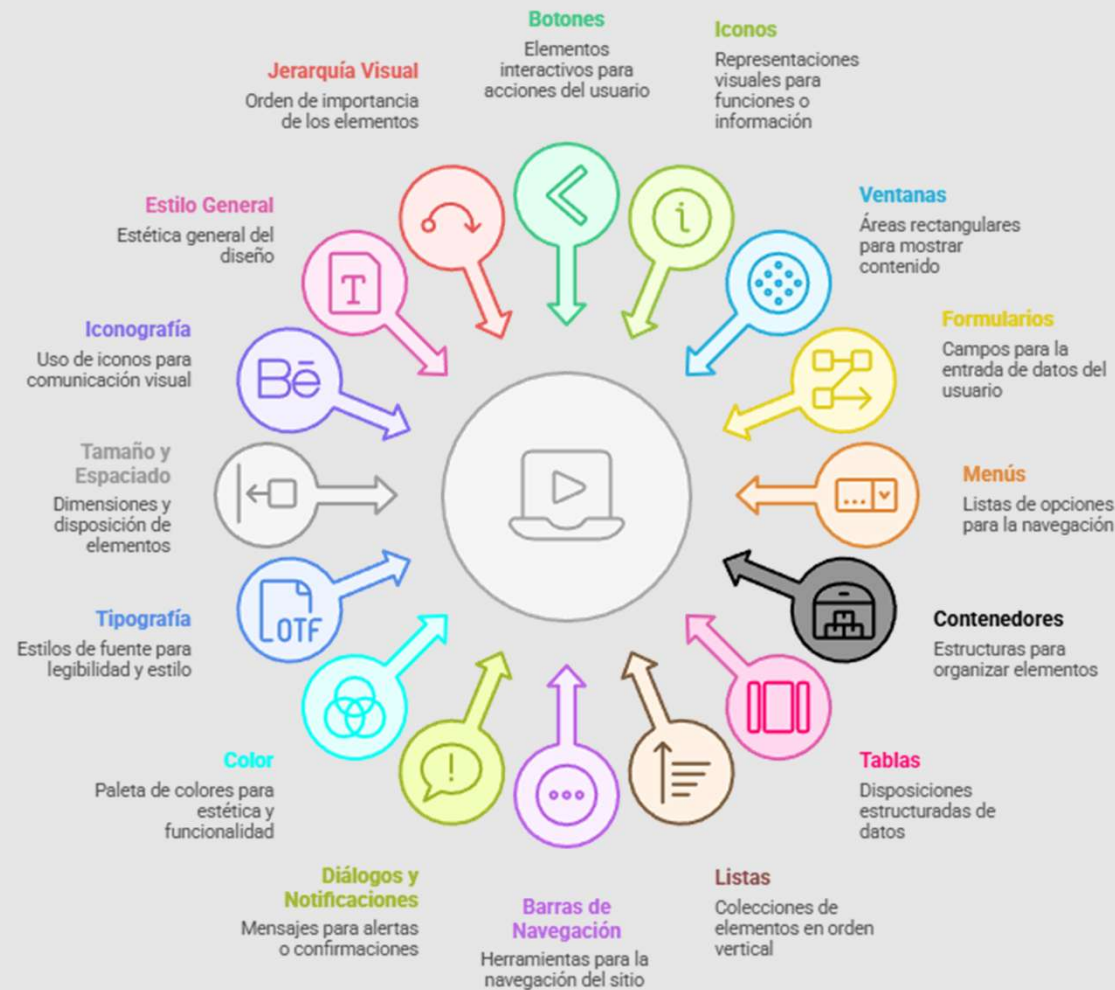
Principios fundamentales de diseño de interfaces



Olga M. Moreno Martín

PROMETEO

Elementos gráficos de una interfaz



Olga M. Moreno Martín

PROMETEO

Accesibilidad en interfaces gráficas

Principios clave de accesibilidad:

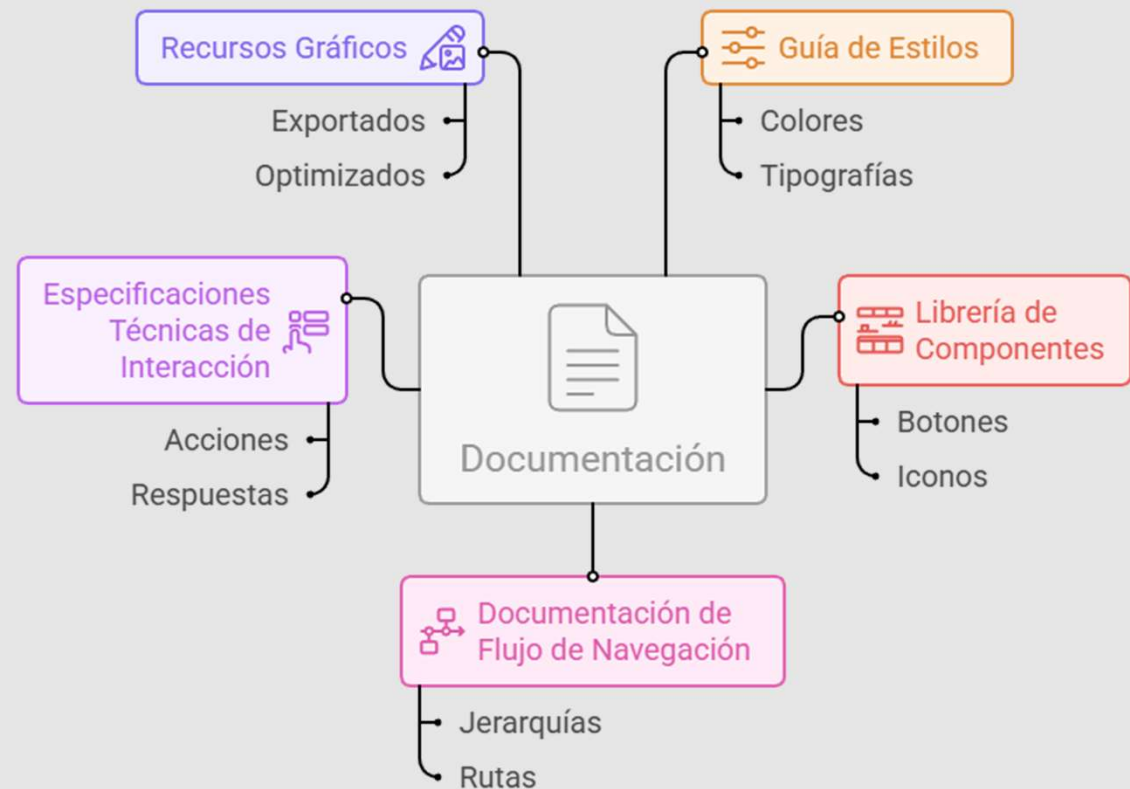
- Contrastes adecuados de color.
- Tamaño de texto legible.
- Elementos táctiles suficientemente grandes.
- Navegación mediante teclado para usuarios que no usan ratón.
- Textos alternativos en imágenes.
- Evitar contenido que pueda provocar problemas (parpadeos, luces).

Beneficios de una interfaz accesible:

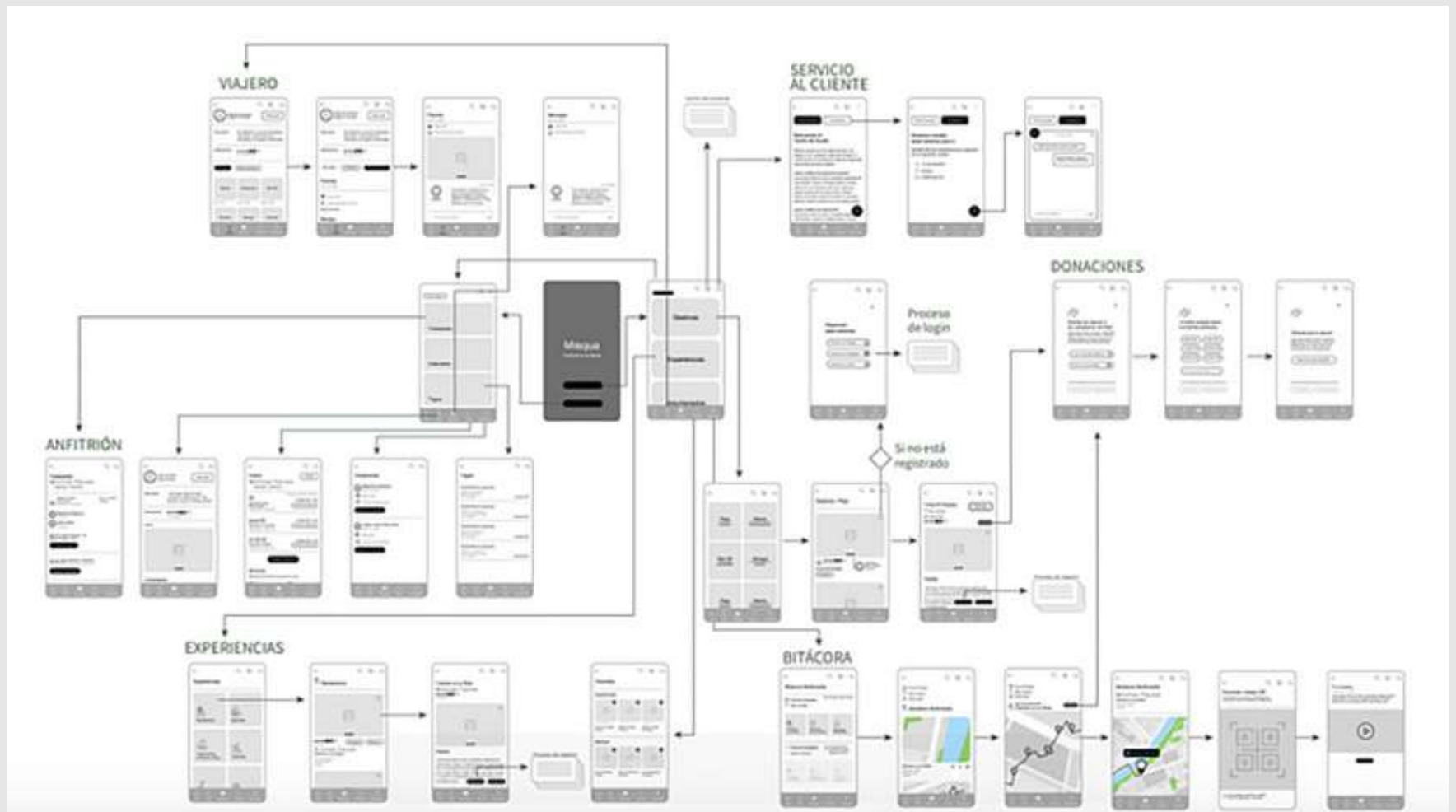
- Más personas pueden usar el producto.
- Cumplimiento de normativas y estándares.
- Mejora la experiencia general del usuario.
- Incrementa la calidad y profesionalidad del producto final.

Documentación de diseño

La documentación es clave para transmitir correctamente la intención del diseño al equipo técnico y al cliente.



Ejemplo flujo de navegación



02 Fundamentos del diseño UX/UI

Introducción al Diseño UX/UI

Dos caras de una misma moneda

- UI y UX funcionan como cerebro y corazón de un producto.
- Disciplinas distintas, pero conectadas.
- UI = apariencia
- UX = experiencia completa



¿UI o UX? Diferencias clave

UI (User Interface)

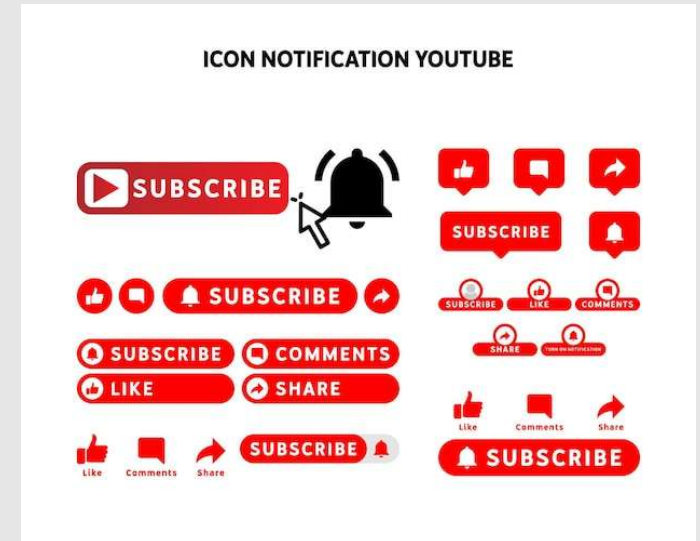
- Colores, tipografías, botones, iconos, layouts.
- Es el “rostro” del producto.

Ejemplo: Botón rojo brillante con animación.

UX (User Experience)

- Emociones, eficiencia, accesibilidad, satisfacción.
- Es el “viaje” del usuario.

Ejemplo: El botón está donde el usuario lo espera.



Digitaliza tu formación con Foxize Cloud

Tu plataforma LMS fácil de usar y sin conocimiento técnico. Descubre todo lo que puede ofrecerte

[Solicita una demo](#)

Sin compromiso

¿Por qué son complementarios?

UI bonito + UX malo = producto inútil.

UX sólido + UI débil = frustración

El equilibrio es clave.



El usuario en el centro

Regla de oro: “No diseñes para ti, diseña para quienes usarán tu producto”.

Implica:

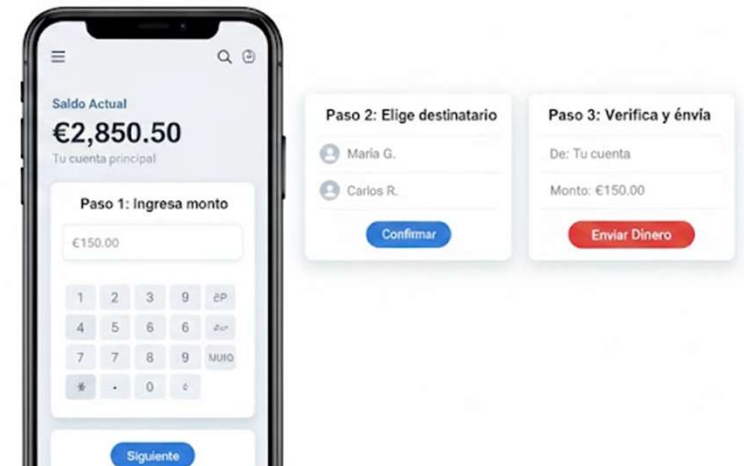
- Investigar usuarios reales.
- Prototipar y testear.
- Priorizar usabilidad.

Ejemplo: App de banca móvil

- UI decide cómo se ve el saldo.
- UX optimiza transferencias en 3 pasos.

Ejemplo: App de Banca Móvil

Transferencias en 3 Pasos



Impacto en el negocio

Un buen diseño UI/UX:

- ✓ Reduce costos
- ✓ Aumenta retención
- ✓ Diferencia del mercado

<https://stripe.com/>

Metáfora del restaurante

UI: presentación del plato, decoración, menú.

UX: experiencia completa: sabor, servicio, tiempos, ambiente.

Comparaciones:

- UI bueno + UX malo → decepción.
- UX bueno + UI malo → rechazo inicial.

UI/UX en acción

Fast food:

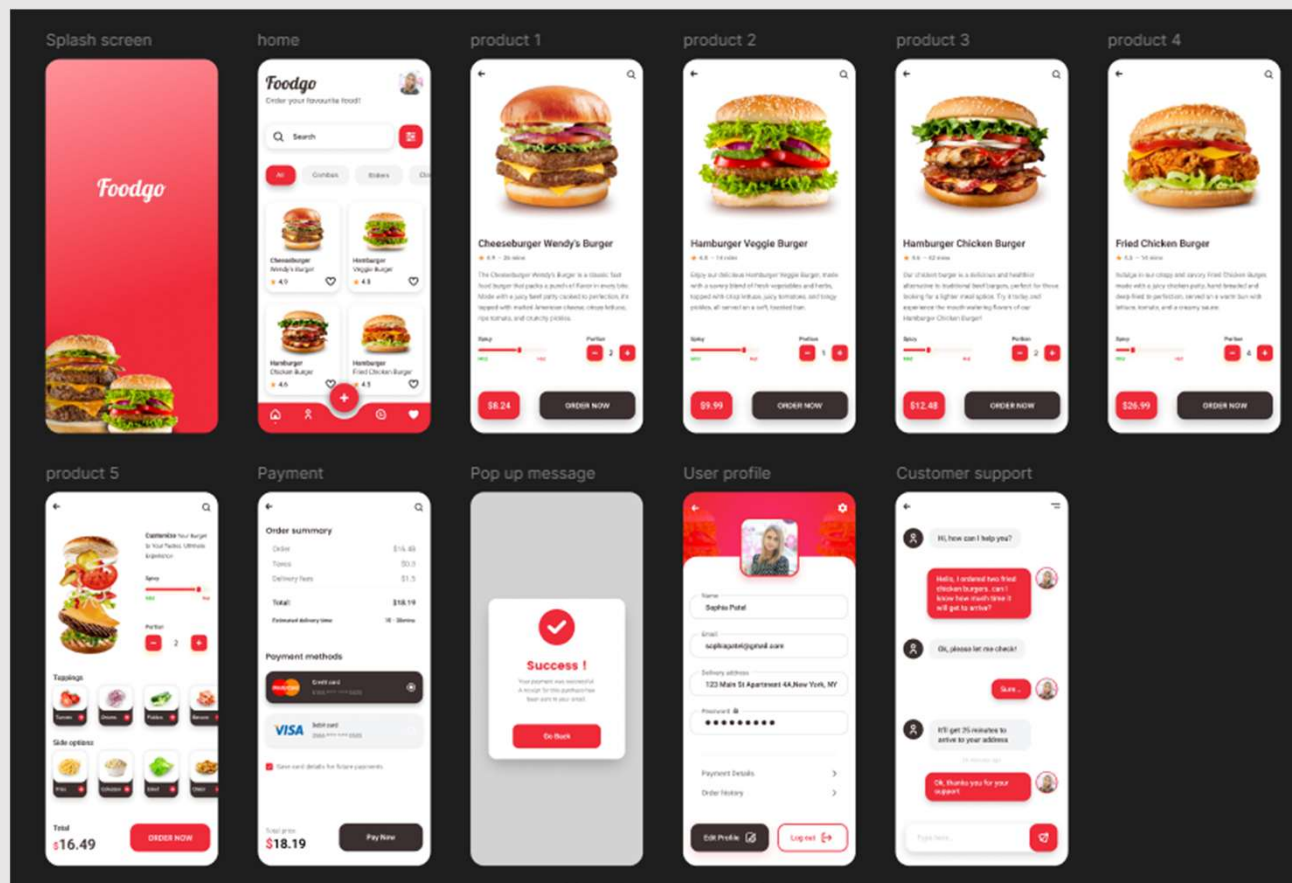
- UI simple y vibrante.
- UX rápida y eficiente.

Gourmet:

- UI elegante.
- UX memorable.

Ejemplo Fast Food

<https://www.figma.com/community/file/1362393407429980800>

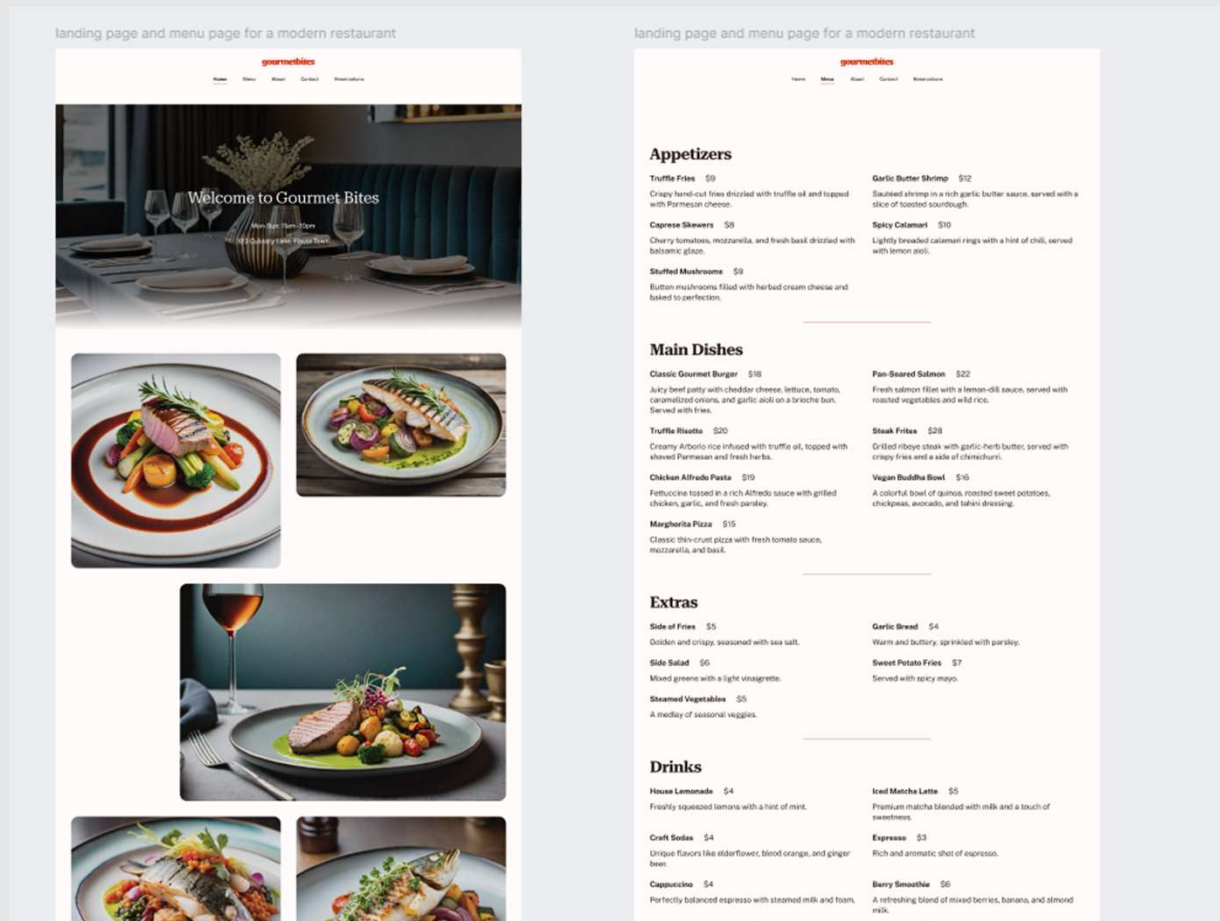


Orga M. Moreno Martin

PROMETEO

- <https://www.figma.com/community/file/1459604905771610527>

Ejemplo Gourmet



Olga M. Moreno Martín

PROMETEO

Diseño centrado en el usuario

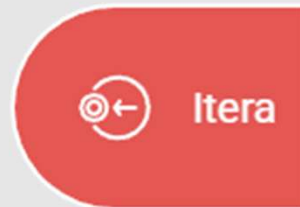
Ejemplo digital:

- UI: botón “Pedir ahora” llamativo.
- UX: proceso de pedido en pocos clics.



Recopila información y analiza las necesidades del usuario.

Genera modelos o representaciones del producto.



Realiza ajustes y mejoras basadas en la retroalimentación.

Principios de Diseño de Interfaces

Estos principios son tu GPS de usabilidad:



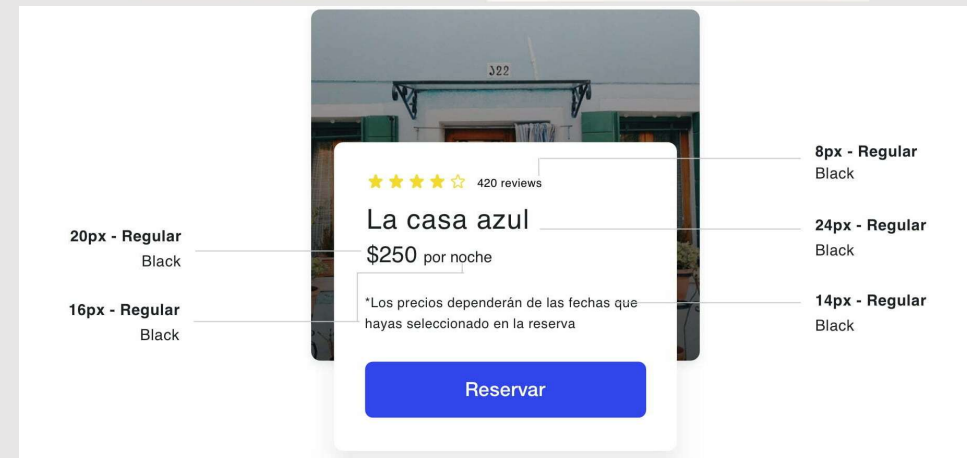
10 principios de usabilidad de Jakob Nielsen



Consistencia

¿Por qué?

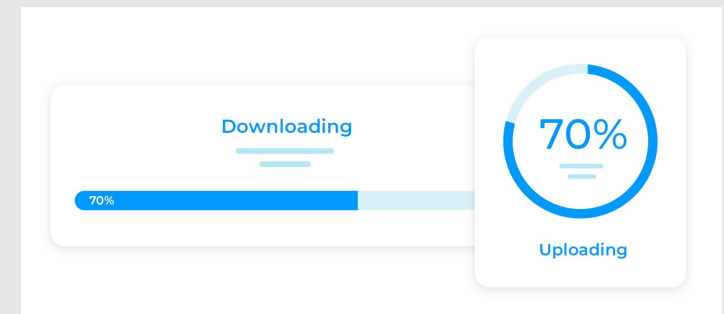
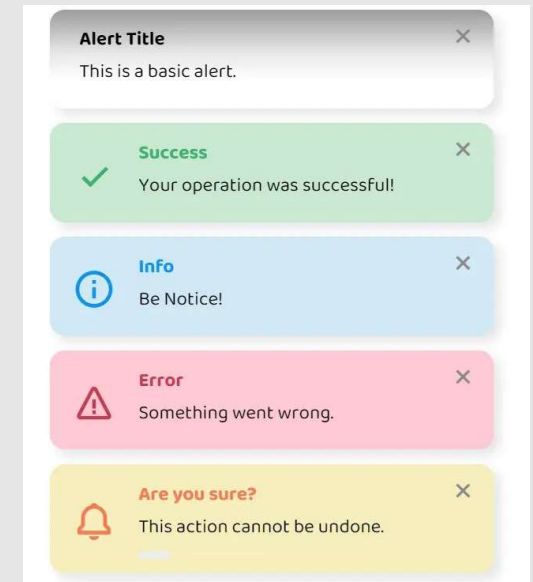
- Evita confusiones.
- Reglas:
 - Mismos colores para acciones similares.
 - Tipografías uniformes.
 - Iconos coherentes.
- Errores comunes:
 - Cambiar el lugar del carrito en un e-commerce.



Visibilidad del estado del sistema

¿Por qué?

- Evita ansiedad del usuario.
- Tácticas:
 - Feedback inmediato.
 - Indicadores de carga, éxito y error.
- Errores comunes:
 - Formularios sin indicador de envío.



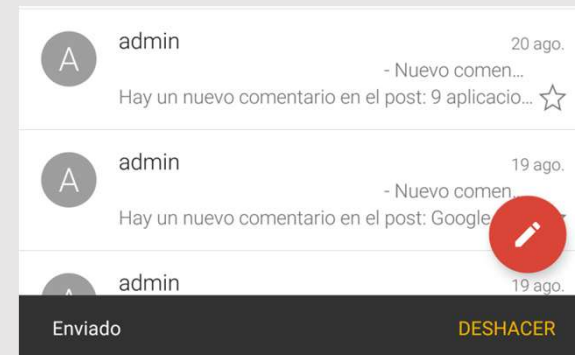
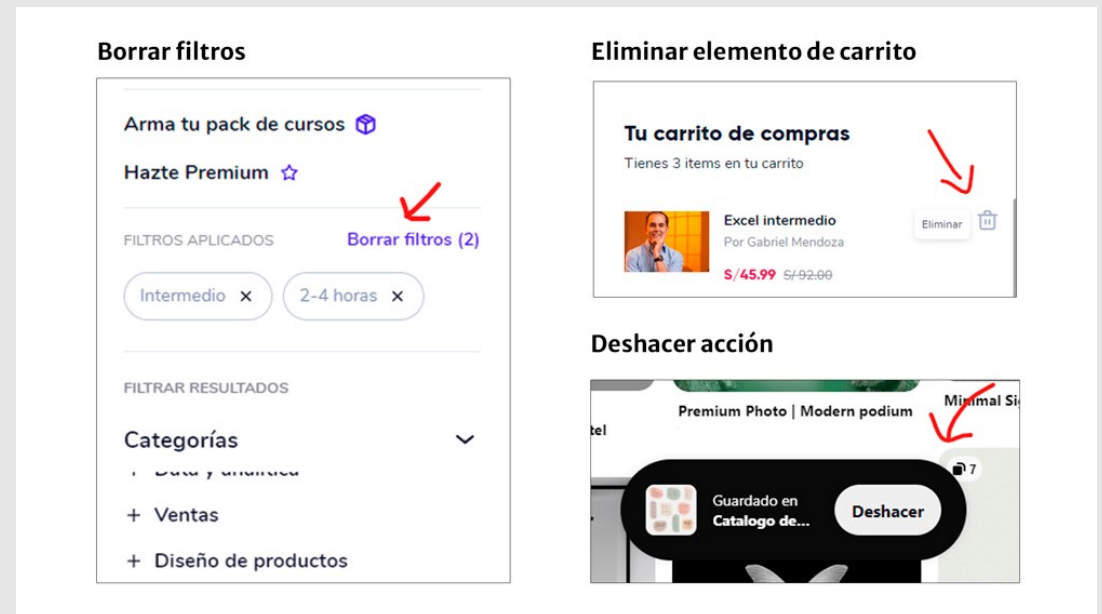
Relación entre el sistema y el mundo real

-  Tácticas:
- Utilizar el lenguaje del usuario, con expresiones y palabras que le resulten familiares.
- La información debe aparecer en un orden lógico y natural.



Control y libertad del usuario

-  Claves:
 - Deshacer/rehacer.
 - Botones de cancelar y volver.
- Ejemplo:
 - Gmail permite recuperar emails.



Prevención de errores

- ✂ Estrategias:
 - Desactivar botones cuando faltan campos.
 - Validación en tiempo real.
- ❌ Error común:
 - Mostrar errores solo tras enviar.

Get started with a free account

Find your people. Engage your customers. Build your brand. Do it all with MailChimp's marketing tools. Already have an account? [Log in](#)

Email

Username

jagriti

Password

..|

● One lowercase character

● One uppercase character

● One number

● One special character

● 8 characters minimum

Get Started!

By clicking this button, you agree to MailChimp's [Anti-spam Policy & Terms of Use.](#)

Validación en tiempo real

Nombre completo

Diego

✓

DNI

Ejemplo: 300650890

DNI obligatorio

Validar fechas no anteriores

DOM	LUN	MAR	MIE	JUE	VIE	SAB
		1	2	3	4	5 HOY
6	7	8	9	10	11	12
13	14	15	16	17	18	19

Placeholders descriptivos

Sobre mí

GUARDAR 

Ej. Soy fotógrafo aficionado, me gusta viajar y capturar los mejores lugares del planeta.

0/600

En Crehana desde Abril 2021

Perú

▼

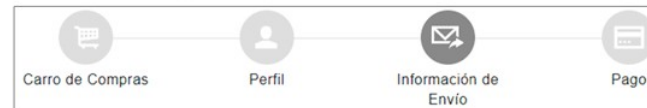
Fecha de nacimiento

DD/MM/AAAA

Reconocimiento > Recuerdo

- 🤔 ¿Cómo aplicarlo?
 - Menús visibles.
 - Jerarquía clara.
- ❌ Error común:
 - Esconder opciones importantes en menús ocultos.

Pasos de compra



Historial de reproducciones

Hoy



Papillón ft. Corazón Serrano - Concierto Virtual 2021 (Solo...

La Karibeña • 4,3 M de visualizaciones

Papillón ft. Corazón Serrano - Concierto Virtual 2021 (Producción General: Radio Karibeña Producción Audió...

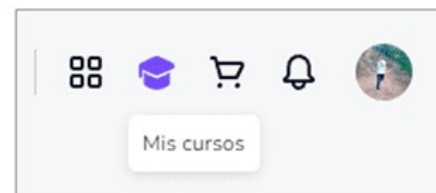


Tiësto & KSHMR ft. Vassy - Secrets (UK Extended Radio Mix)

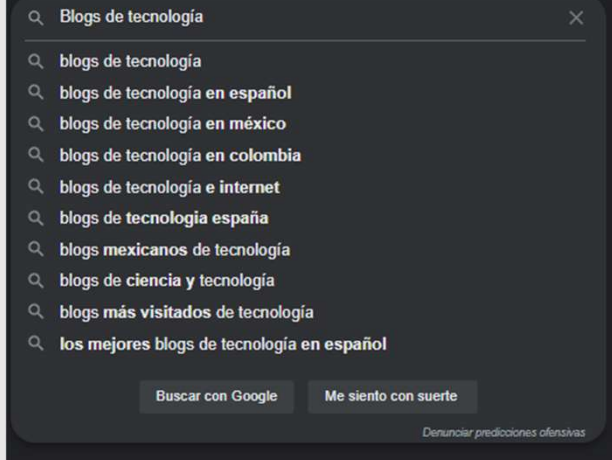
François Brille • 1,6 M de visualizaciones

PLEASE READ DESCRIPTION Tiësto & KSHMR ft. Vassy Extended Radio Mix) Please listen to the entire song b...

Tooltips



Google

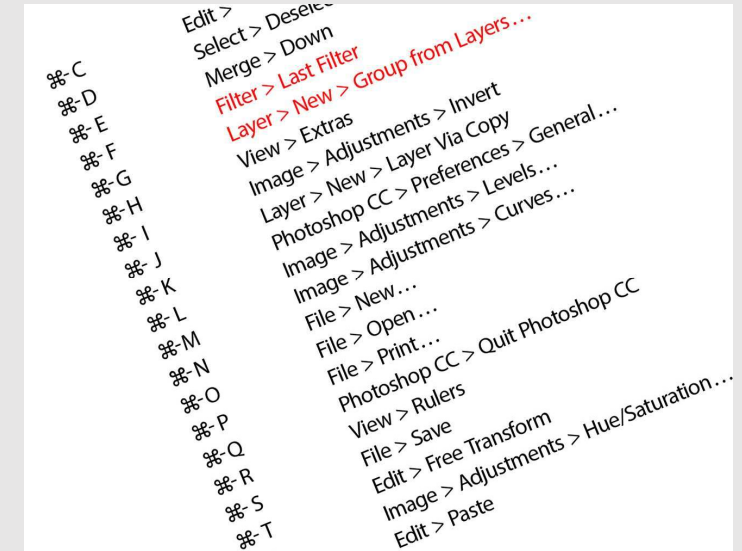


Barra de progreso en un video no terminado

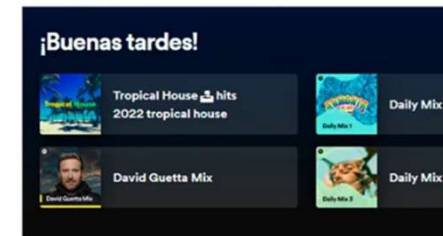


Flexibilidad y eficiencia de uso

- Los usuarios más experimentados, deben poseer atajos y aceleradores para poder realizar sus operaciones mas habituales. De esta forma, tiene alternativas para personalizar sus acciones frecuentes.
-  Claves:
- Personalización del contenido
- Personalización de la funcionalidad
- Para cada usuario.



Contenido frecuente



Atajos de teclado

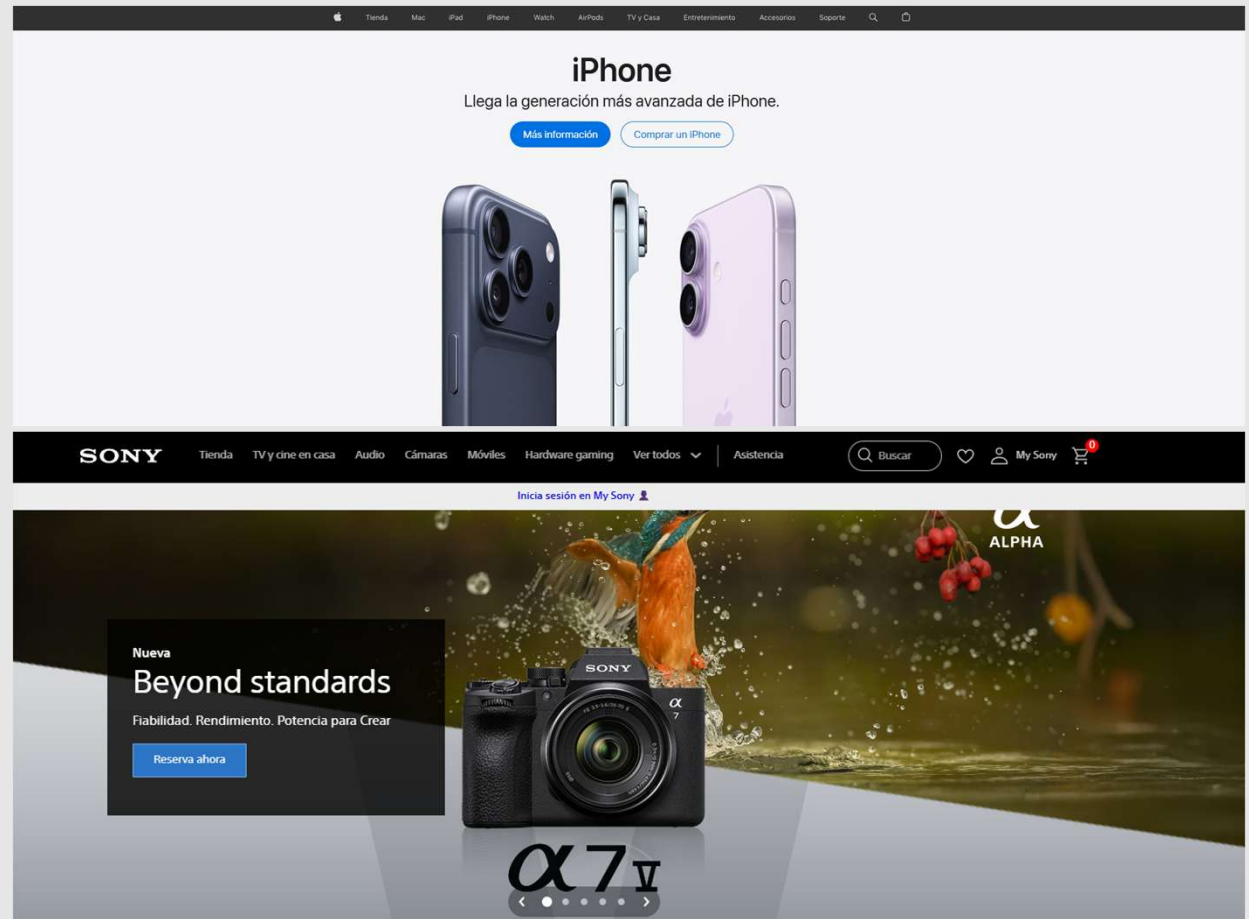


Configuración avanzada



Diseño estético y minimalista

- Las interfaces no deben contener información irrelevante.
- Cada elemento extra compite con la información relevante y disminuye su visibilidad.
- **Recuerda, menos es más.**
- ¿Qué necesita mi usuario?
- ¿Qué valor le aporta lo que le estoy mostrando?



Ayudar a los usuarios a reconocer, diagnosticar y recuperarse de los errores

⚙️ Técnicas:

- Mensajes de error en un lenguaje entendible, sin códigos de error.
- Se debe describir con precisión el problema y sugerir constructivamente una solución.

Los peores mensajes de error son aquellos que no existen. – Jakob Nielsen

Error al iniciar sesión

Iniciar sesión en Facebook

Correo electrónico o número de teléfono 

El correo electrónico o número de celular que ingresaste no está conectado a una cuenta. Encuentra tu cuenta e inicia sesión.

Contraseña

Informar error 404

Esta página no está disponible

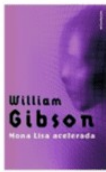
Es posible que el enlace esté roto o que se haya eliminado la página. Verifica que el enlace que quieres abrir es correcto.

[Ir a la sección de noticias](#)

[Volver](#)

[Ir al servicio de ayuda](#)

Sin stock & Botón para notificar



Mona Lisa acelerada (Trilogía del Sprawl 3)

william gibson · booket

 Sin Stock

 [Avisarme al correo cuando vuelva a estar disponible](#)

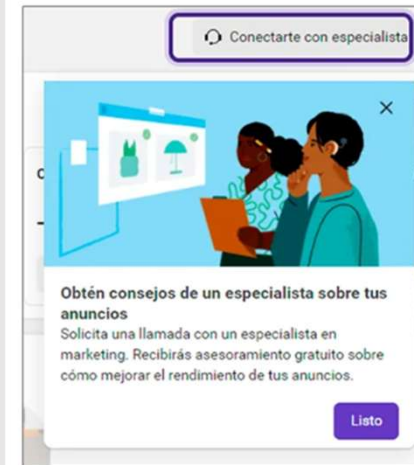
Ayuda y documentación

- Lo ideal es que un sistema no necesite ninguna explicación adicional.
- Proporcionar documentación de ayuda, fácil de encontrar. Pasos definidos de manera concisa

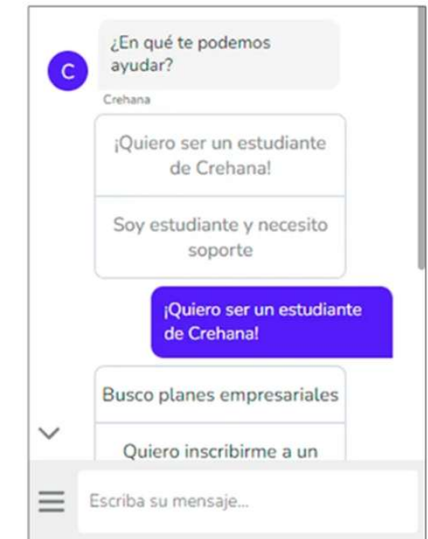
Técnicas:

- Sección de Preguntas frecuentes (FAQs).
- Minitours en la primera visita del usuario a una interfaz.
- Chats de ayuda o chatbots.
- Tutorial o guías para herramientas web.
- Mostrar información en los iconos cuando se pasa sobre ellos (tooltips)

Miniguías



Chatbots



Sección de FAQ's

Aquí hay algunas otras preguntas frecuentes que recibimos mucho:

¿Tengo que pagar por las actualizaciones?

¡No! ¡Actualizamos constantemente nuestro software, agregamos nuevas plantillas y lo obtienes todo GRATIS! Debido a que ClickFunnels está en la nube, cuando hacemos una

¿Tengo que instalar algo?

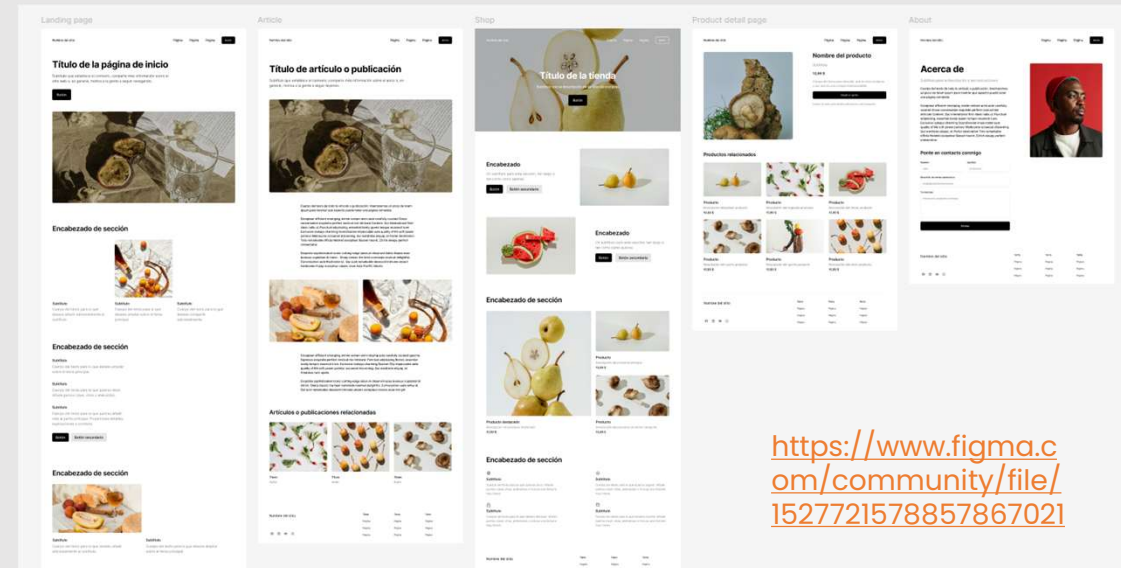
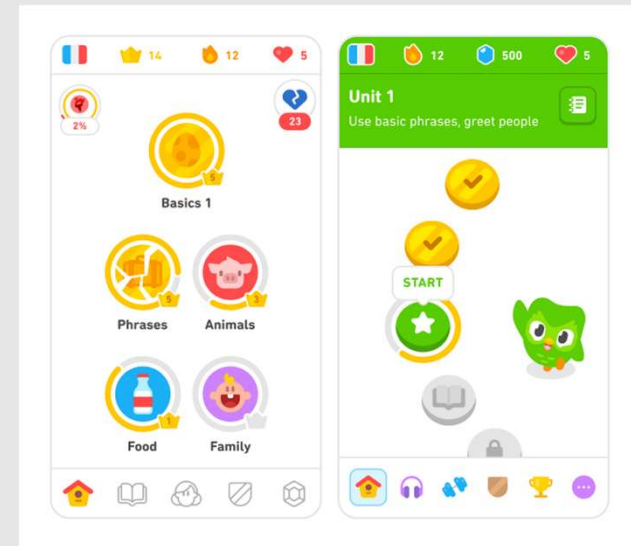
¡No! ¡Creamos ClickFunnels para que no tenga que tener todo un departamento de TI para ejecutar su sitio web! ¡Simplemente inicie sesión, comience a hacer clic y podrá crear

Diseño emocional

El objetivo es conectar con el usuario.

Datos clave:

- 95% de decisiones son subconscientes.
- Ejemplos:
 - Apple vende simplicidad.
 - Duolingo gamifica el aprendizaje.



<https://www.figma.com/community/file/1527721578857867021>

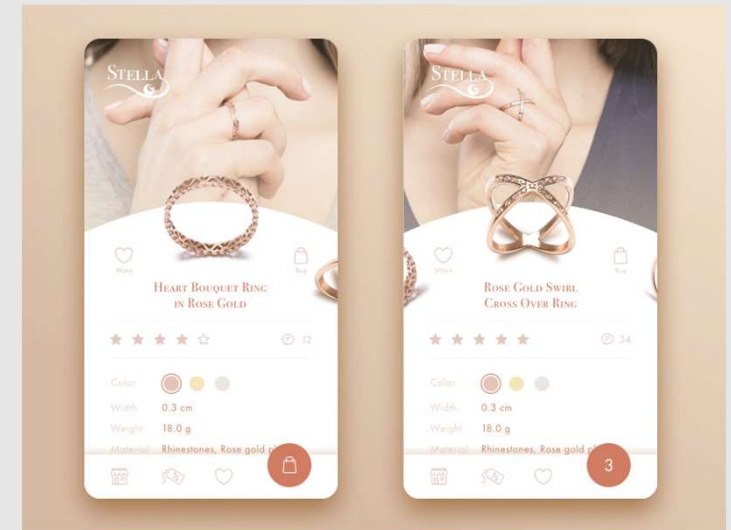
Claves para diseñar emociones

1. Empatizar

2. Crear experiencias memorables

3. Eliminar fricciones

- Ejemplos reales:
 - Microinteracciones.
 - Doodles de Google.
 - “1-Click Order” de Amazon.

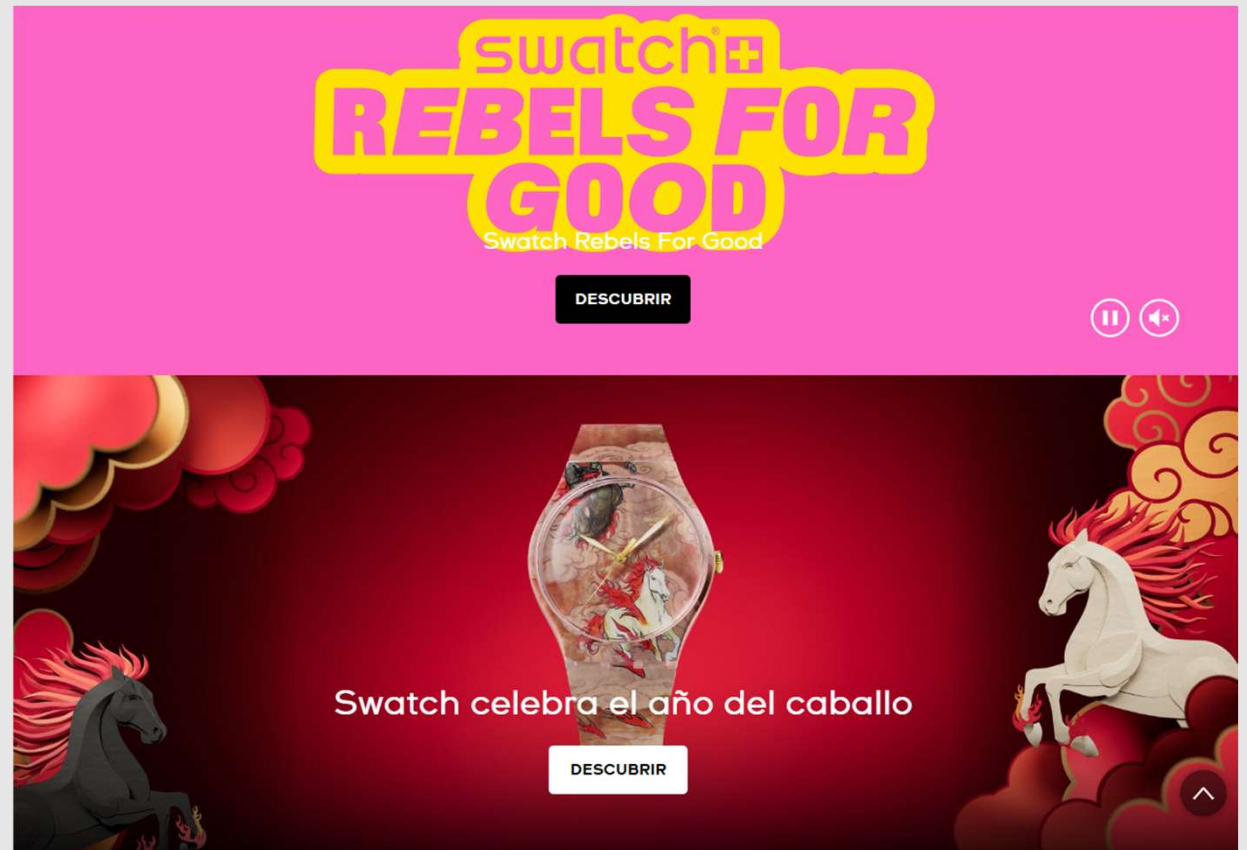


Olga M. Moreno Martín

PROMETEO

Ejemplo profundo: Botón emocional

- Color cálido
- Animación suave
- Mensaje positivo
- Feedback inmediato



Técnicas avanzadas del diseño emocional

- Psicología del color
- Storytelling
- Sonidos emocionales



Modelos mentales

Definición: **cómo el usuario espera que funcione algo.**

- Ejemplo:
 - Disquete = guardar (aunque ya no exista).

A. Metáforas visuales

- Funciona porque el cerebro procesa imágenes rápido.
- Ejemplos:
 - Papelera = eliminar
 - Carpeta = agrupar
 - Lupa = buscar
- Error: iconos abstractos sin referencia.

B. Patrones de interacción

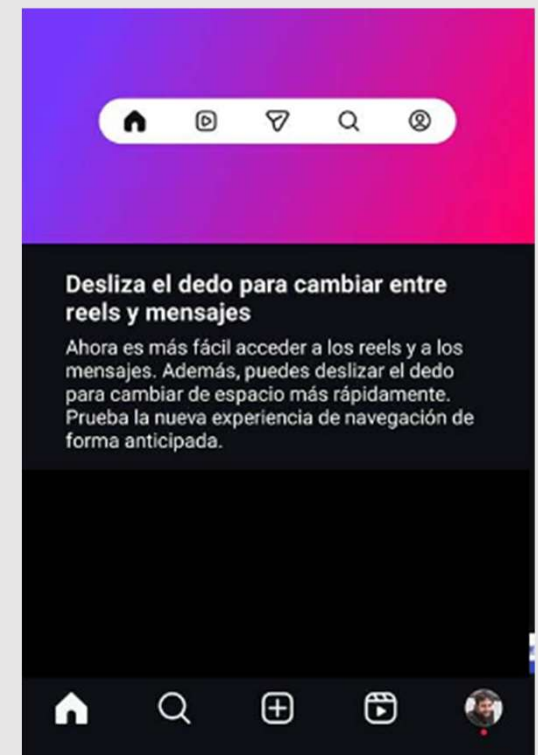
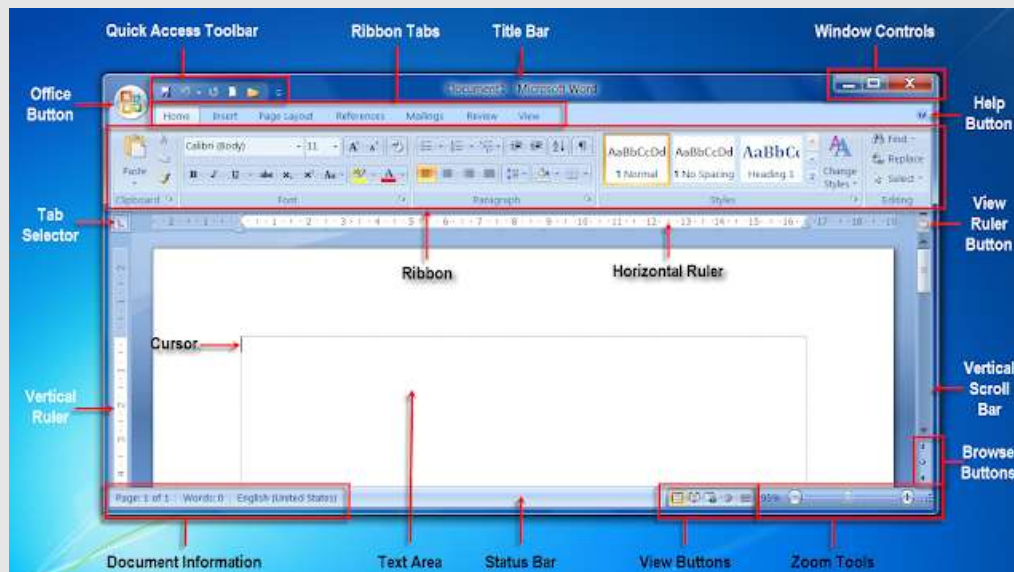
- Los usuarios esperan:
- Menu hamburguesa
- Swipe para eliminar
- Arrastrar y soltar

C. Diseño sin instrucciones

- Regla de oro:
Si necesita tutorial, falló el diseño.
- Técnicas:
- Jerarquía
- Affordances
- Agrupación lógica

Casos de fracaso por no cumplir el modelo mental

- Microsoft Ribbon (2007)
- Instagram cambiando el botón Crear (2020)



Jerarquía Visual

Importancia

- 50ms para evaluar una interfaz
- 94% abandona si no se escanea bien

Técnicas de jerarquía visual

1. Tamaño
2. Color y contraste
3. Ubicación
4. Espaciado
5. Tipografía

Ejemplo práctico

- Landing page de newsletter:
- Título grande
- CTA destacado
- Pocos distracciones

Errores comunes

- ✗ Todo en negrita
- ✗ Bajo contraste
- ✗ Espaciado inconsistente
- ✗ Demasiadas fuentes

Diseño Centrado En Tareas

Por qué centrarse en tareas

- 55% abandona si no encuentra lo que busca.
Ejemplo: Uber muestra “Pedir viaje”.

• Pilares

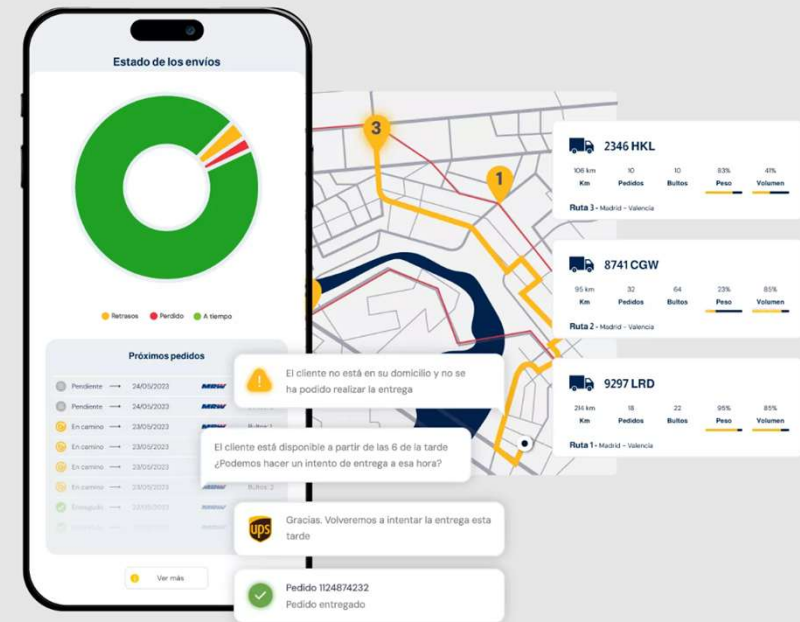
1. Contextos de uso
2. Priorizar lo relevante
3. Minimizar pasos

Ejemplo: App de reparto

- Pantalla principal:
- Mapa
- Tiempo estimado
- Botón contactar
- Secundario:
- Ajustes y promociones

• Errores graves

- ✗ Ocultar función principal
- ✗ Sobrecargar con info
- ✗ Pasos redundantes



UI vs UX

Diferencias clave

- Tabla comparativa: enfoque, objetivo, proceso, herramientas, métricas.

Cómo colaboran

1. UX define
2. UI diseña
3. Testean juntos
4. Iteran

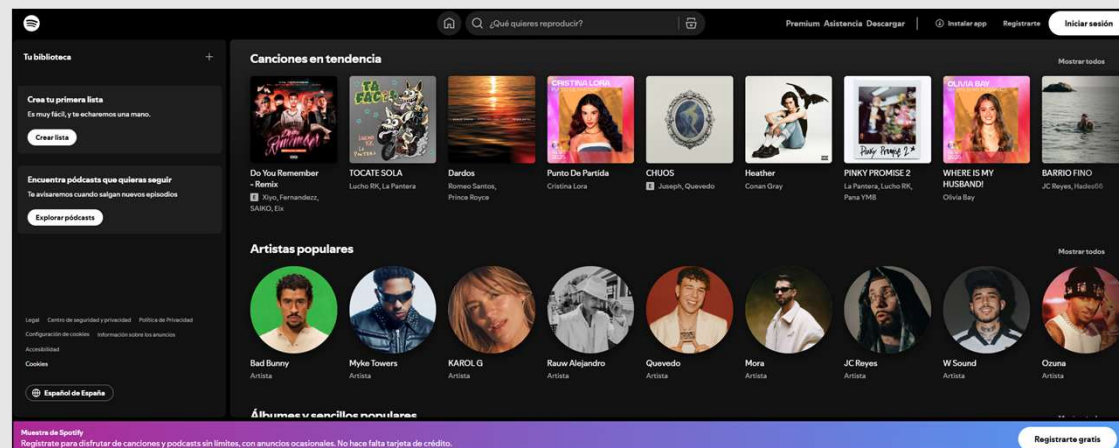
- Ejemplo: Spotify

Errores comunes

- ✗ Trabajar en silos
- ✗ Saltar investigación
- ✗ Priorizar estética sobre uso

- Caso real: Snapchat 2018

<https://www.youtube.com/watch?v=WuLPiKtN7oI>



Recursos

Libros

1. No me hagas pensar – Steve Krug
2. The Design of Everyday Things – Don Norman
3. Articulating Design Decisions – Tom Greever

Cursos

- Google UX
- CalArts UI/UX
- Georgia Tech UX

Herramientas

- Figma
- Adobe XD
- Maze

Blogs y comunidades

NNGroup (Nielsen Norman Group)

- <https://www.nngroup.com/articles/>

Smashing Magazine

- <https://www.smashingmagazine.com/>

UX Collective (Medium)

- <https://uxdesign.cc/>

Canales de YouTube

AJ&Smart

- <https://www.youtube.com/c/AJSmartDesign>

- Design Sprints y metodologías UX

Flux

- <https://www.youtube.com/c/FluxWithRanSegall>

- Diseño UI/UX y freelancing

Olga M. Moreno Martín

Figma

- <https://www.youtube.com/c/Figmadesign>
- Tutoriales oficiales

Herramientas de Accesibilidad

WAVE

- <https://wave.webaim.org/>
- Evaluador de accesibilidad web

Stark

- <https://www.getstark.co/>
- Plugin para verificar contraste y más



¿Preguntas?

Introducción al “Design Thinking” y Diseño de interfaces de usuario

Módulo: 2º DAM – Desarrollo de Interfaces





Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Índice

**THINK TWICE
CODE ONCE!**

INTRODUCCIÓN AL DESIGN THINKING

1. ¿Qué es el Design Thinking?
2. Características del Design Thinking
3. Las 5 fases del proceso de Design Thinking
4. Design Thinking aplicado al diseño de interfaces
5. ¿Cuándo usar Design Thinking?
6. Ejemplo y recursos Design Thinking

DISEÑO DE INTERFACES DE USUARIO

1. Introducción al diseño de interfaces de usuario
2. ¿Qué es el diseño de interfaz de usuario?
3. El Color en el Diseño UI
4. Tipografías en Interfaces
5. Distribución Visual Layout
6. Diseño Responsive y mobile-first
7. Ejemplo práctico de elementos clave del diseño UI
8. Recursos

03 **Introducción al “Design Thinking”**

¿Qué es el Design Thinking?

¿Qué es?



Olga M. Moreno Martín

PROMETEO

¿Qué es el Design Thinking?

Características clave:



Centrado en el usuario

Pone las necesidades del usuario en primer lugar.



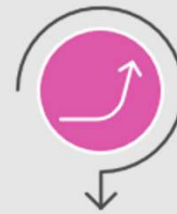
Altamente colaborativo

Enfatiza el trabajo en equipo y la comunicación.



Promueve la experimentación

Fomenta probar nuevas ideas y enfoques.



Iterativo

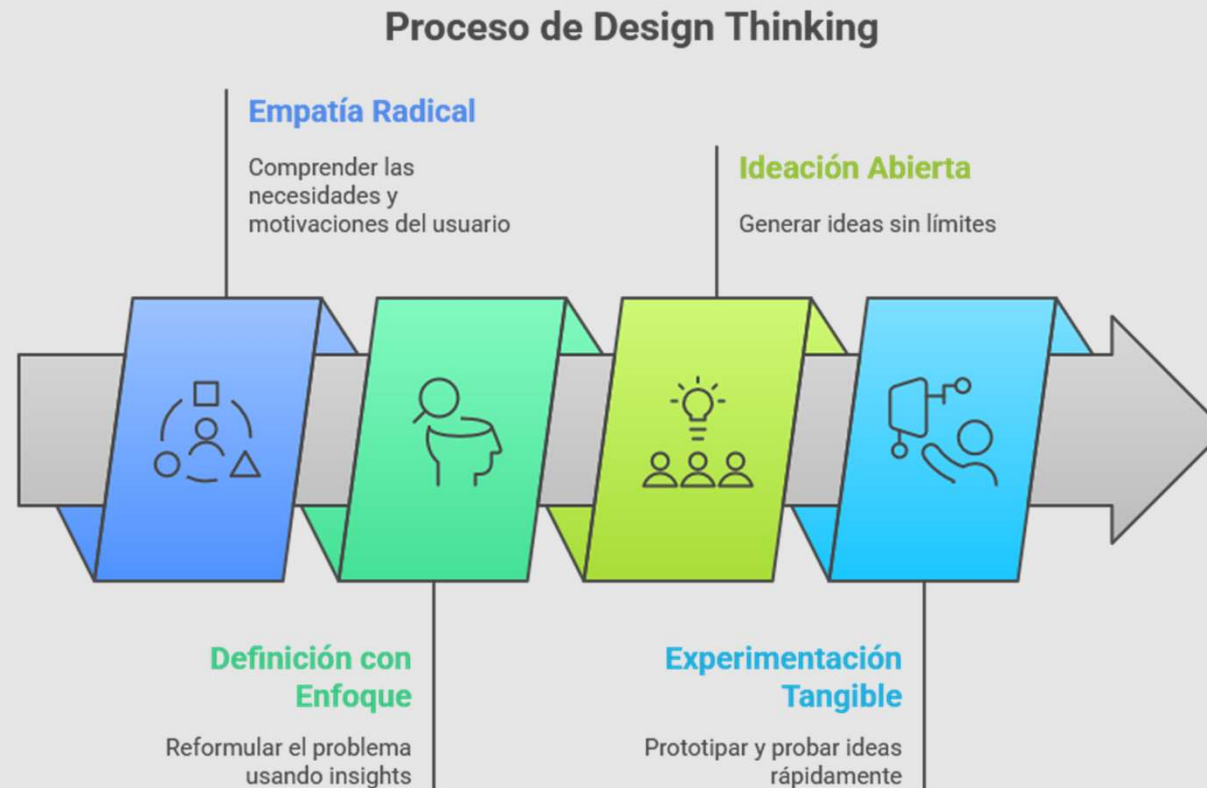
Probar, mejorar y repetir el proceso.



Combina creatividad y análisis

Mezcla el pensamiento innovador con la toma de decisiones basada en datos.

Esencia Del Design Thinking



Made with Napkin

Esencia Del Design Thinking

1. Empatía radical

Comprender motivaciones reales, frustraciones y comportamientos.

- Sumergirse completamente en el mundo del usuario. Observar, entrevistar, escuchar, analizar sentimientos y comportamientos ocultos.
- **Objetivo:** comprender necesidades no expresadas.

2. Definición con enfoque

Reformular el problema verdadero usando insights.

- Convertir hallazgos en retos claros.
- Técnica principal: **How Might We (HMW)**
- Transforma problemas en oportunidades.
- Ejemplo de estructura:
¿Cómo podríamos... [acción] para [usuario] que [necesidad]?

3. Ideación abierta

Generar ideas sin límites ni juicios.

Reglas:

- Cantidad > calidad
- Aplazar juicio
- Construir sobre ideas de otros
- **Técnicas:** brainstorming, SCAMPER, mapas mentales, Crazy 8s.

4. Experimentación tangible

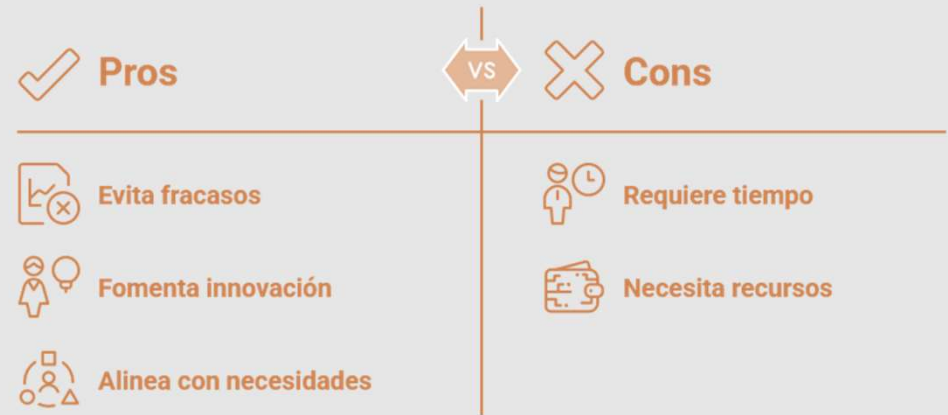
Prototipos rápidos → aprender rápido. Fallar barato.

Niveles de prototipos:

- Baja fidelidad (papel)
- Media fidelidad (maquetas estáticas)
- Alta fidelidad (interactivas)
- **Wizard of Oz:** El usuario cree que interactúa con un sistema automatizado, pero en realidad hay una persona detrás haciendo que funcione.

¿POR QUÉ USAR DESIGN THINKING EN INTERFACES?

1. Evita fracasos costosos
2. Fomenta innovación
3. Alinea diseño con necesidades reales



 **"No diseñamos para nosotros mismos, diseñamos para quienes usarán lo que creamos."**

- Evita sesgos personales y prioriza accesibilidad sobre estética cuando sea necesario.

¿Por qué usarlo en diseño de interfaces?

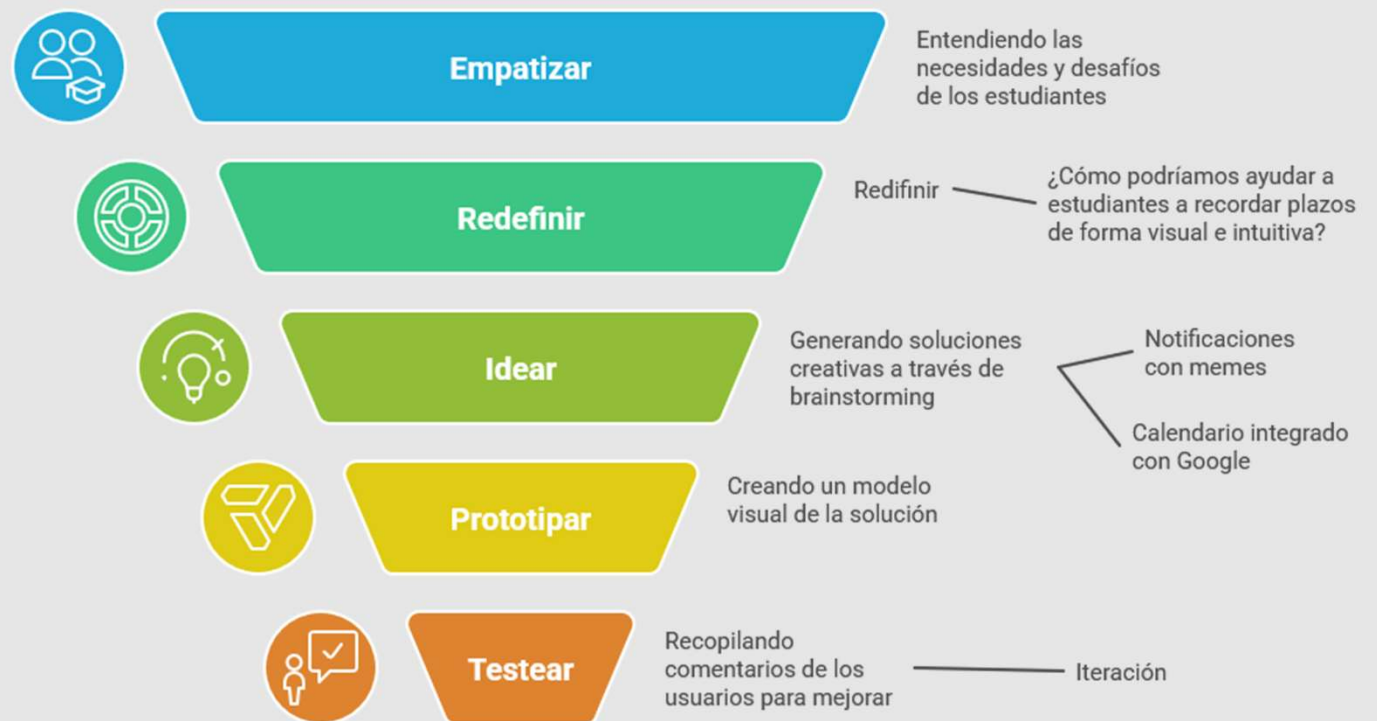
Evita el "fracaso costoso": El 70% de los proyectos digitales fallan por no entender al usuario.

Fomenta la innovación: Soluciones como el scroll infinito (Pinterest) o el carrito de compra persistente surgieron de observar usuarios.

EJEMPLO APLICADO (APP EDUCATIVA)

Problema: alumnos no usan la plataforma

- Problema: alumnos no usan la plataforma.
1. Empatizar: entrevistas revelan que olvidan fechas de entrega y la interfaz es confusa.
 2. Redefinir: "¿Cómo podríamos ayudar a estudiantes a recordar plazos de forma visual e intuitiva?"
 3. Idear: brainstorming genera ideas como:
 - Notificaciones con memes.
 - Calendario integrado con Google.
 4. Prototipar: Se crea un wireframe de calendario con colores por urgencia.
 5. Testear: Usuarios prefieren recordatorios por voz → iteración.



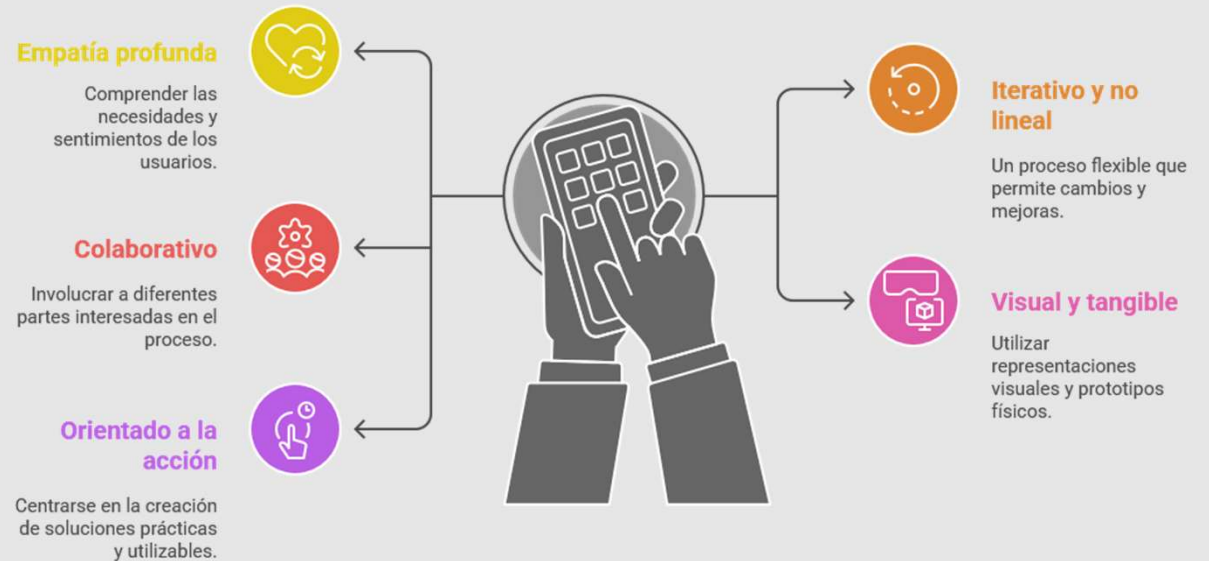
CARACTERÍSTICAS DEL DESIGN THINKING

Diferenciadores clave:

1. Empatía profunda
2. Iterativo y no lineal
3. Colaborativo
4. Visual y tangible
5. Orientado a la acción

Comparado con métodos tradicionales:

- Prototipa rápido vs esperar a MVP
- Test temprano vs test final
- Fallar rápido vs evitar error a toda costa



1. Empatía como punto de partida

¿Por qué es crítica?

- El 85% de los errores en diseño de interfaces surgen de no entender al usuario (Nielsen Norman Group).
- Herramientas para practicarla:
- Mapa de Empatía:
 - **Qué PIENSA y SIENTE** -> "Me estresa perder tiempo"
 - **Qué DICE y HACE** -> "Cierra la app tras 3 intentos" |
- **Entrevistas etnográficas:** Preguntas abiertas como "Cuéntame sobre la última vez que usaste esta app y te frustraste".
- **User "Personas":** Personajes ficticios basados en datos reales (ej.: "Ana, 35 años, usa apps de salud por ansiedad laboral").
- **Ejemplo práctico:** Spotify descubrió que los usuarios querían compartir música en el momento (no solo guardar playlists), lo que llevó a crear la función "Compartir en Stories".

2. Iterativo y no lineal

- El diseño UX no sigue una línea recta (paso 1 → paso 2 → paso 3), sino que **avanza, retrocede y se repite** según lo que se descubre en cada fase; **probar, fallar y mejorar** forma parte natural del proceso.
- **Casos donde se retrocede:**
 - Si al testear un prototipo de e-commerce, los usuarios no encuentran el botón de compra, se vuelve a idear soluciones.
 - Si al definir el problema se descubre un sesgo (ej.: solo se entrevistó a jóvenes), se regresa a empatizar.
- **Atención:** Ningún prototipo es sagrado; si los datos dicen que hay que cambiar, se cambia.

3. Colaborativo

¿Quiénes participan?

- Diseñadores, desarrolladores, stakeholders, y usuarios finales.
- **Técnicas para fomentarlo:**
 - Workshops con Lego Serious Play: Construir modelos físicos de ideas.
 - Brainwriting: Cada miembro escribe 3 ideas en silencio, luego se discuten en grupo.
 - Criticas en "Sandwich": Feedback = *1 positivo + 1 mejora + 1 positivo*.

Caso de éxito: Airbnb involucró a anfitriones reales en el rediseño de su plataforma, logrando un 30% más de reservas.

4. Visual y tangible

- Herramientas clave:

Fase	Herramienta visual	Ejemplo
Empatizar	Customer Journey Map	Mapa de emociones al usar una app bancaria.
Idear	Mapa mental o esquemas en pizarra	Post-its con ideas para menú de restaurante.
Prototipar	Wireframes en papel	Boceto de un chatbot para servicio al cliente.

- **Ventajas:**

- Rompe la barrera del lenguaje técnico.
- Facilita la detección de errores tempranos (ej.: un botón mal ubicado en un sketch).

5. Orientado a la acción

Métodos para evitar el "parálisis por análisis":

- Timeboxing: Dedicar 20 minutos a generar ideas, sin debate.
- Prototipos en 1 hora: Usar materiales simples (cartón, Figma básico).
- Pruebas "guerrilla": Testear con 5 usuarios en un café (sin laboratorio costoso).

Importante: En Design Thinking, un prototipo feo, pero bien testeado, vale más que un diseño perfecto en un cajón

- **Ejemplo integrador: Rediseño de una app de reparto de comida**

Ejemplo integrador: Rediseño de una app de reparto de comida

1. **Empatía:** Entrevistas revelan que los repartidores se sienten presionados por el tiempo.
2. **Iteración:** Tras testear un prototipo de ruta optimizada, se descubre que prefieren flexibilidad → vuelta a idear.
3. **Colaboración:** Cocineros, repartidores y UX designers crean un sistema de priorización.
4. **Visualización:** Se usa un storyboard para mostrar el flujo de pedidos.
5. **Acción:** En 2 días se prueba un cambio mínimo (ej.: botón "Pausar entrega") con 10 repartidores.

Comparativa con otros enfoques

Design Thinking	Metodología Tradicional
Prototipa con post-its	Espera a tener un MVP completo
Testea con 5 usuarios	Encuesta a 100 personas al final
"Fallar rápido es aprender"	"El error es costoso"

Las 5 fases del proceso de Design Thinking

Metodología iterativa y centrada en el usuario

1. Empatizar
2. Definir el Problema
3. Idear
4. Prototipar
5. Testear

Fase 1: Empatizar

Objetivo: Sumergirse en el mundo del usuario para entender sus necesidades reales (no las supuestas).

Técnicas clave:

1. Entrevistas en profundidad:

- Preguntas abiertas: "¿Qué te frustra al usar [producto]?"
- Evitar preguntas sesgadas: ❌ "¿Te gustaría una función de...?" → ✅ "¿Cómo resuelves actualmente...?"

2. Observación contextual:

- Ejemplo: Ver cómo los ancianos usan una app de banca (¿hacen zoom en los botones? ¿Confunden iconos?).

3. Mapa de Empatía:

- Qué PIENSA → "Quiero ahorrar tiempo"
- Qué SIENTE → "Me estresa lo complicado"

4. User Persona:

- Perfil ficticio basado en datos: "Carlos, 40 años, prefiere videollamadas antes que chatbots para servicio al cliente."

Ejemplo aplicado:

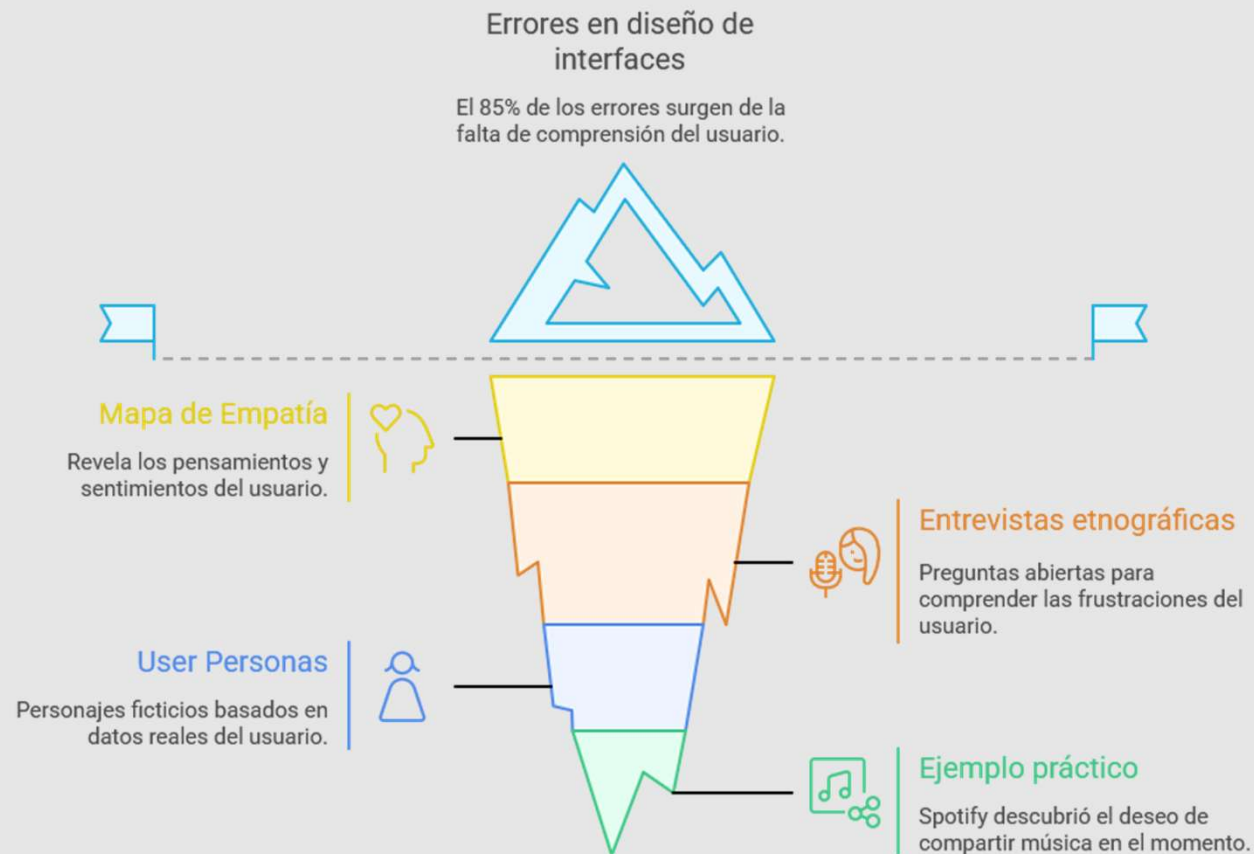
Antes de diseñar una app de comida saludable, se observa que los usuarios:

- Tiran verduras porque se olvidan de usarlas → Necesidad real: recordatorios de caducidad.

Olga M. Moreno Martín

PROMETEO

Entender al usuario es crucial para el diseño de interfaces.



Fase 2: Definir el problema

- **Objetivo:** Reformular el problema desde la perspectiva del usuario.
- **Técnicas:**
 - **Pregunta "How Might We" (HMW):**
"¿Cómo podríamos ayudar a [usuarios] a [necesidad] para que [beneficio]?"
 - **Ejemplo:** "¿Cómo podríamos ayudar a padres primerizos a rastrear las vacunas de su bebé para que reduzcan su ansiedad?"
 - **Point of View (POV):**
"[Usuario] necesita [necesidad] porque [insight]."
 - **Ejemplo:** "Los estudiantes necesitan recordatorios visuales de plazos porque olvidan revisar emails."
- **Error común: Confundir síntomas con problemas.**
 "Los usuarios no completan el registro" →  "El registro pide demasiados datos personales."

Reformulación del problema



Fase 3: Idear

Técnicas de ideación:

1. Brainstorming:

- Regla: Ninguna idea es mala.
- Ejemplo: Para una app de transporte, ideas como "viajar en teleférico" inspiran "rutas turísticas en Uber".

2. Crazy 8s:

- Dibujar 8 ideas en 8 minutos (incluso absurdas).

3. SCAMPER:

- Modificar ideas existentes:
 - Sustituir: ¿Cambiar texto por iconos?
 - Combinar: ¿Integrar calendario con lista de tareas?

4. Mapas mentales:

- Centralizar el problema y ramificar soluciones.

Ejemplo: Para una app de estudio, ideas como:

- "Notificaciones con memes motivacionales"
- "Modo oscuro para leer de noche"

Técnicas de ideación

Mapas mentales

Organizar ideas visualmente alrededor de un problema central.



SCAMPER

Modificar ideas existentes usando una lista de verificación.



Brainstorming

Generar muchas ideas sin juzgarlas.



Crazy 8s

Dibujar rápidamente ocho ideas en ocho minutos.

Fase 4: Prototipar

Objetivo: Materializar ideas rápidamente para validarlas.

Niveles de prototipado:

Tipo	Herramientas	Cuándo usarlo
Boceto en papel	Lápiz y post-its	Feedback inicial
Wireframe	Figma, Balsamiq	Estructura básica
Prototipo digital	Adobe XD, Proto.io	Interacciones realistas

Regla del 80/20: Un prototipo debe ser suficientemente bueno para testear, no perfecto.

Ejemplo: Un prototipo en papel de un menú de restaurante se prueba con clientes para ver si entienden los íconos de alérgenos.

Desarrollo de prototipos



Boceto en papel

Diseño inicial en papel



Wireframe

Estructura básica de la interfaz



Prototipo digital

Versión interactiva para pruebas



Prototipo testeable

Listo para la retroalimentación del usuario

Fase 5: Tetear

Objetivo: Validar suposiciones con usuarios reales.

Métodos de testing:

1. Think Aloud (Pensar en voz alta):

El usuario verbaliza sus pensamientos: "No encuentro el botón de 'guardar'..."

2. Test A/B:

Comparar dos versiones: ¿Botón verde o azul genera más clicks?

3. Test remoto no moderado:

Herramientas como UserTesting graban pantallas y comentarios.

4. Feedback con escala:

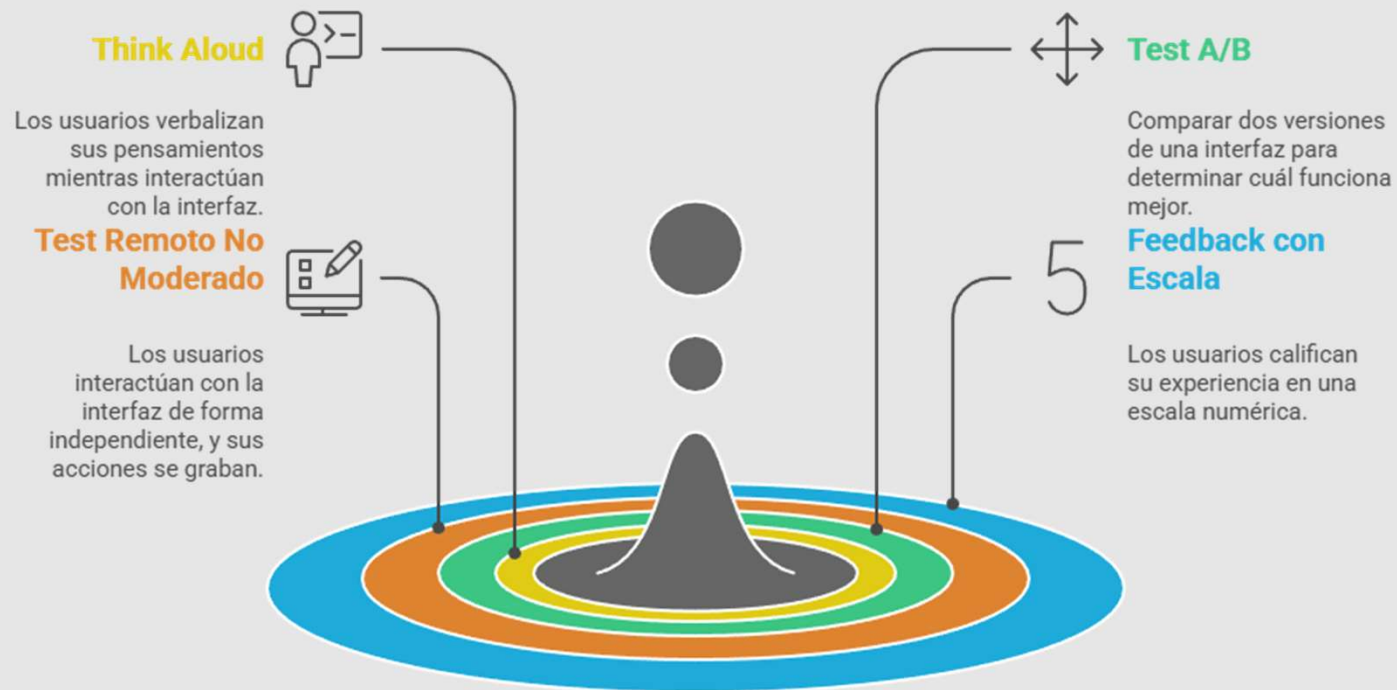
Preguntar: "Del 1 al 5, ¿qué tan fácil fue completar la tarea?"

Clave: Observar lo que hacen, no solo lo que dicen.

💡 **Ejemplo:** Al testear una app de fitness, se descubre que los usuarios:

- Ignoran el tutorial → Solución: Onboarding interactivo.

Métodos de Prueba de Interfaz de Usuario



Design Thinking Aplicado Al Diseño De Interfaces

- Cómo transformar necesidades de usuarios en interfaces intuitivas y efectivas

1. Comprender los objetivos del usuario (Fase de Empatía)

Herramientas específicas para UI/UX:

- **User Journey Maps:** Visualizar cada paso del usuario en la interfaz, identificando:
 - **Paso** -> "Buscar libro" . **Emoción** -> "Frustración". **Lo que piensa** -> "No sé si está disponible"
 - **Paso** -> "Reservar". **Emoción** -> "Alivio". **Lo que piensa** -> "El botón es claro"
- **Card Sorting:** Pedir a usuarios que agrupen funciones (ej.: ¿"Renovar préstamo" va en "Mi perfil" o "Catálogo"?).

Ejemplo: Para una app de banca móvil:

- Entrevistas revelan que usuarios mayores prefieren iconos grandes antes que menús textuales.

Proceso de Diseño de Interfaz



Crear Mapa de Viaje del Usuario

Visualizar los pasos del usuario en la interfaz



Identificar Emociones del Usuario

Reconocer las emociones experimentadas por el usuario en cada paso



Comprender Pensamientos del Usuario

Obtener información sobre los pensamientos del usuario durante la interacción



Realizar Clasificación de Tarjetas

Involucrar a los usuarios en la agrupación de funciones

Design Thinking Aplicado Al Diseño De Interfaces

2. Diseñar flujos de interacción coherentes (Definición + Ideación)

Técnicas clave:

- **Flujos de navegación:**

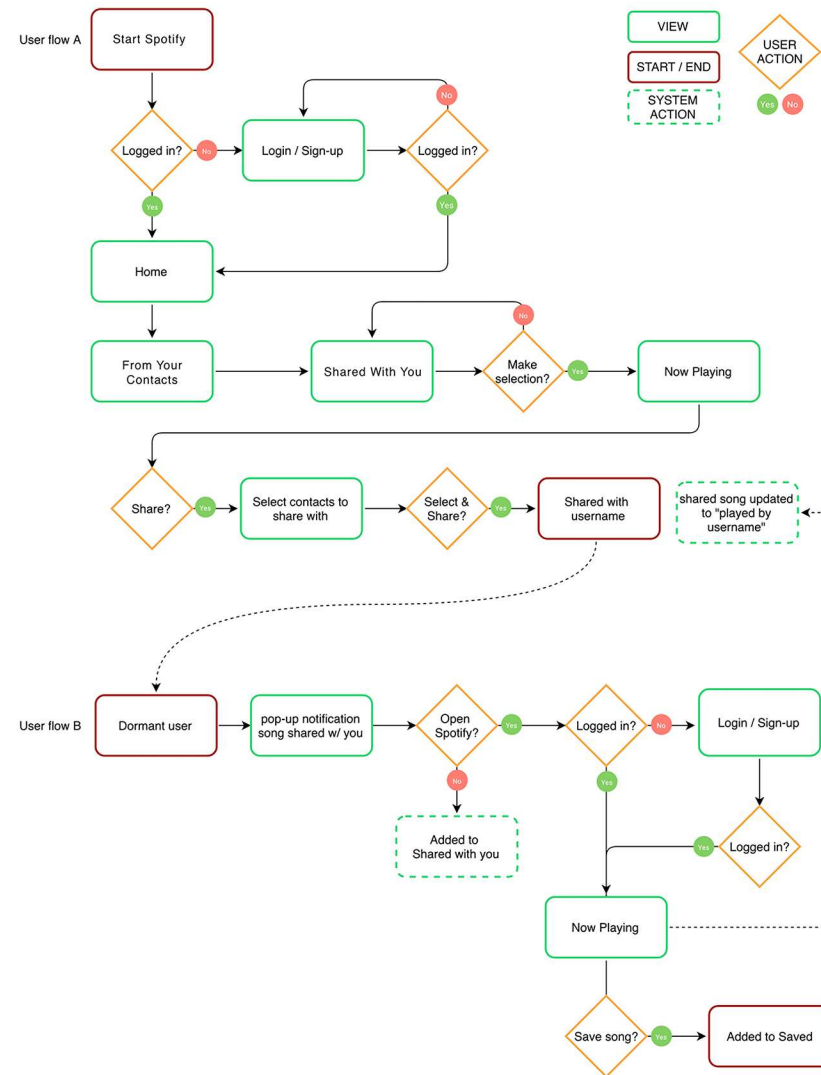
Diagramar cómo se mueve el usuario entre pantallas (ej.: Home → Catálogo → Detalle libro → Reserva).

- **Arquitectura de la información:**

Usar árboles de contenido para organizar jerarquías (ej.: Evitar más de 3 clics para reservar).

Ejemplo práctico: En una app de reparto de paquetes:

- Definir el problema: "¿Cómo ayudar a usuarios a reordenar su pedido anterior en menos pasos?"
- Idear soluciones: "Botón 'Reordenar' en historial" + "Guardar combinaciones frecuentes".



Design Thinking Aplicado Al Diseño De Interfaces

3. Probar alternativas visuales antes de programar (Prototipado)

- Estrategias para interfaces:

Tipo de prototipo	Uso ideal	Herramienta
Sketch en papel	Validar disposición básica	Lápiz + plantillas
Wireframe digital	Testear jerarquía visual	Figma, Balsamiq
Prototipo interactivo	Simular interacciones reales	Adobe XD, Proto.io

Caso real: Spotify probó 50 versiones de su botón "Play" antes de elegir la más reconocible.

Desarrollo de prototipos



Design Thinking Aplicado Al Diseño De Interfaces

4. Corregir errores antes de codificar (Testeo)

Métodos ágiles para UI:

1. Test de los 5 segundos:

- Mostrar la interfaz brevemente: ¿El usuario entiende su propósito?

2. Heatmaps:

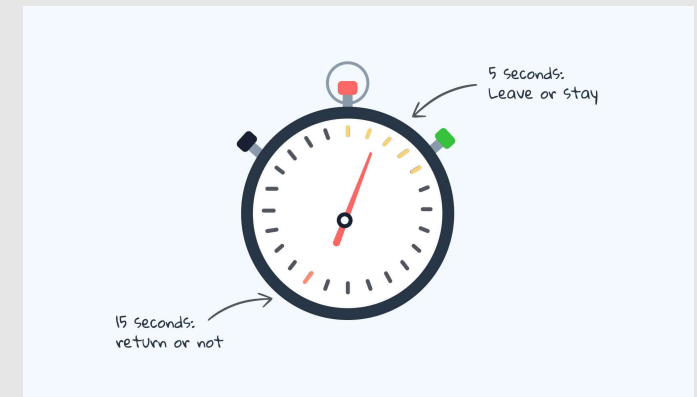
- Herramientas como "Hotjar" muestran dónde los usuarios hacen clic.

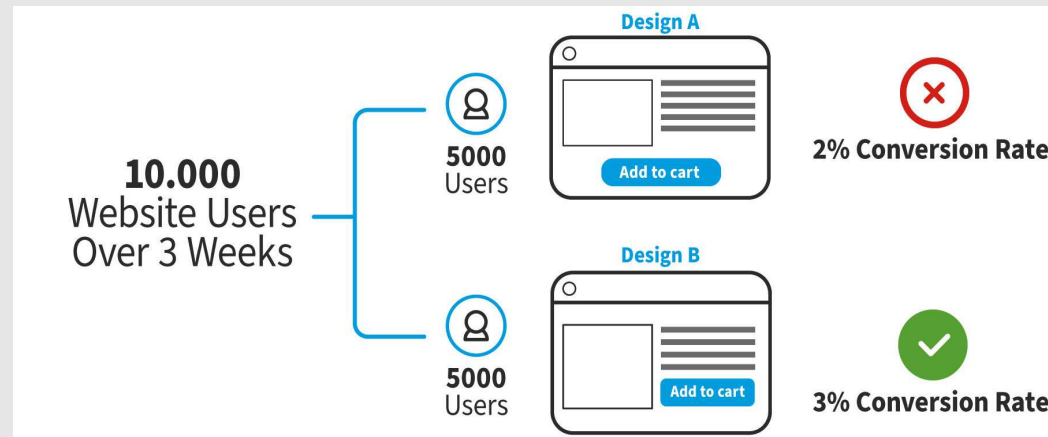
3. A/B Testing de microinteracciones:

- Ejemplo: ¿Un botón animado aumenta las conversiones?

Errores comunes detectables:

- Botones "invisibles" (bajo contraste).
- Pasos redundantes (ej.: Doble confirmación de correo).





Design Thinking Aplicado Al Diseño De Interfaces

Ejemplo aplicado al aula: App para biblioteca de instituto

Paso a paso con Design Thinking:

1. Empatizar:

- Entrevistas con estudiantes: "Odio hacer cola para preguntar si un libro está disponible".
- Insight clave: Necesitan ver el estado del libro en tiempo real.

2. Definir:

- Pregunta HMW: "¿Cómo podríamos ayudar a estudiantes a reservar libros en 1 clic desde su móvil?"

3. Idear:

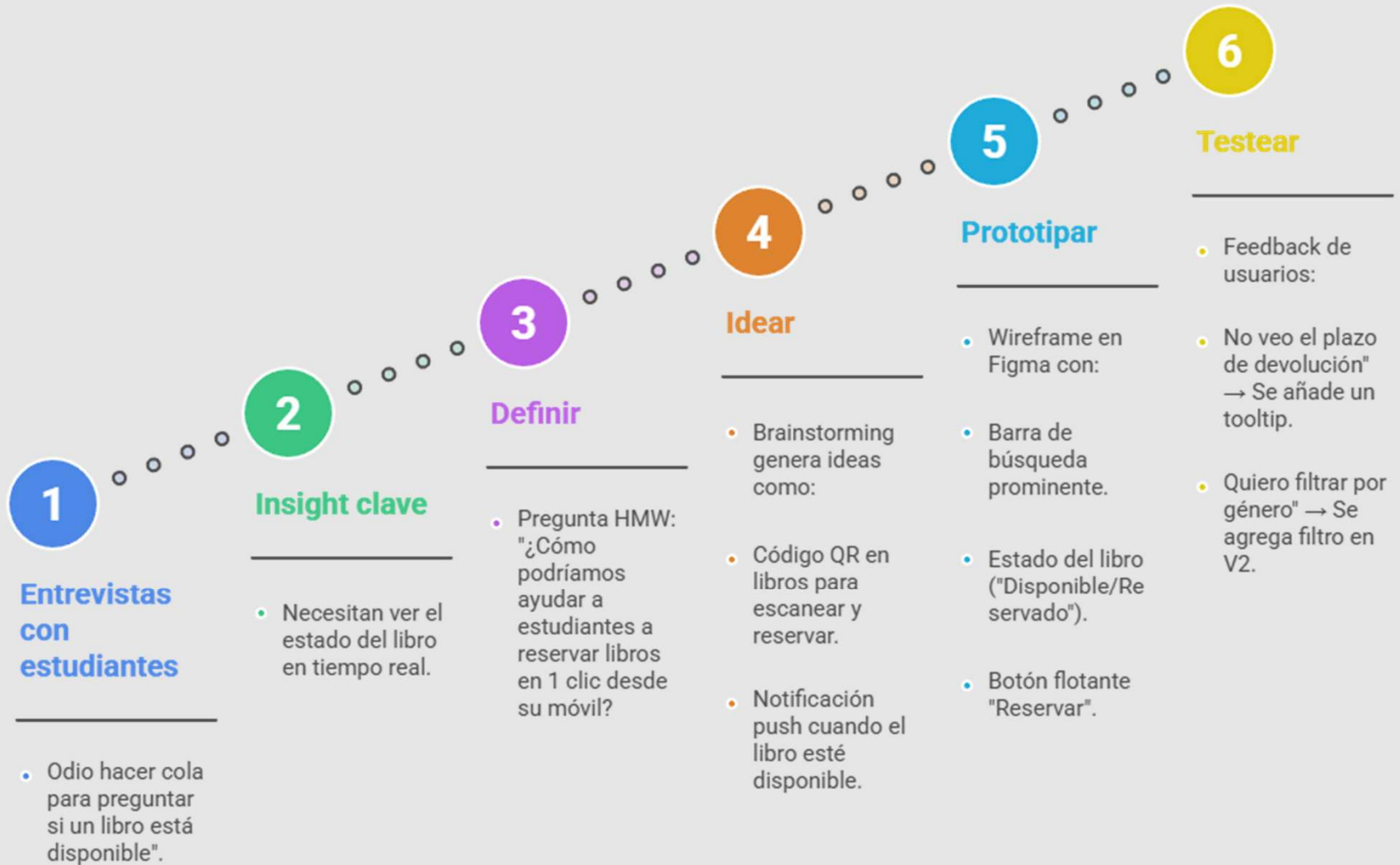
- Brainstorming genera ideas como:
 - Código QR en libros para escanear y reservar.
 - Notificación push cuando el libro esté disponible.

4. Prototipar:

- Wireframe en Figma con:
 - Barra de búsqueda prominente.
 - Estado del libro ("Disponible/Reservado").
 - Botón flotante "Reservar".

5. Testear:

- Feedback de usuarios:
 - "No veo el plazo de devolución" → Se añade un tooltip.
 - "Quiero filtrar por género" → Se agrega filtro en V2.



Beneficios clave para el diseño de interfaces

1. Reduce costos:

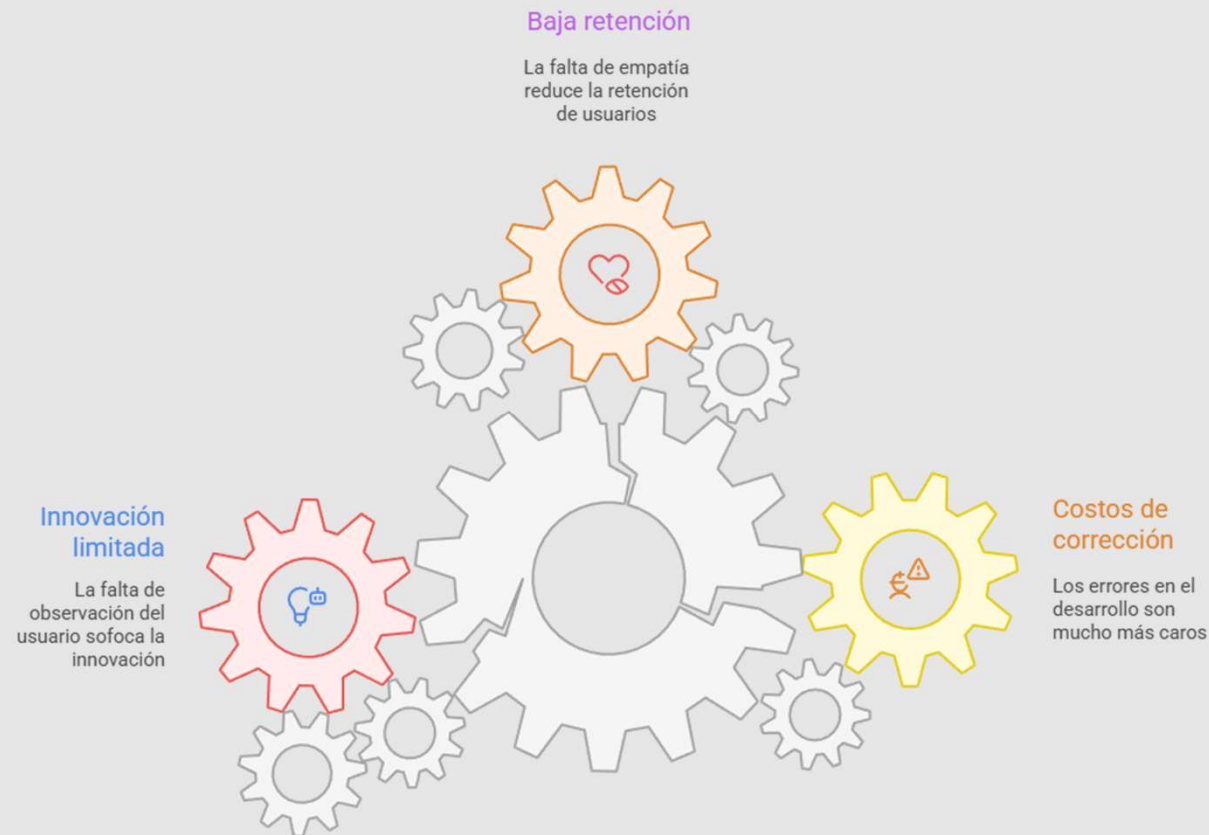
- Corregir un error en prototipo cuesta un 90% menos que en desarrollo (IBM Design Study).

2. Aumenta la adopción:

- Interfaces basadas en empatía tienen un 60% más de retención (Adobe Report).

3. Fomenta la innovación:

- Soluciones como el scroll infinito (Pinterest) nacieron de observar usuarios.



¿CUÁNDO USAR DESING THINKING?

Design Thinking	Enfoque tradicional
Proyectos innovadores (ej.: apps).	Sistemas con regulaciones estrictas (ej.: banca).
Problemas ambiguos (ej.: mejorar UX).	Requisitos claros y estables (ej.: migración de datos).
Equipos multidisciplinarios.	Equipos técnicos especializados.

Ejemplo práctico: App de reservas de hotel

Design Thinking:

1. Empatizar: Observar que los usuarios comparan precios en 5 pestañas.
2. Definir: "¿Cómo ayudar a viajeros a comparar hoteles sin cambiar de app?"
3. Idear: Integrar un comparador visual con filtros táctiles.

Enfoque tradicional:

1. Requisito: "La app debe mostrar hoteles por precio y ubicación."
2. Desarrollo: Lista tabular con columnas ordenables.

Resultado:

- DT genera una interfaz más intuitiva (arrastrar tarjetas para comparar).
- Enfoque tradicional cumple con funcionalidad básica, pero menos innovadora.

Conclusión

- Elige Design Thinking si buscas innovación, empatía con usuarios y flexibilidad.
- Opta por metodologías tradicionales cuando los requisitos son fijos y el riesgo de error debe minimizarse.

Ejemplos y Herramientas de Desing Thinking

- Ver documentación de la Plataforma para ejemplo detallado: “FocusStudyFlow”

Kit de Inicio Rápido:

- Descarga el PDF gratuito de la universidad de Stanford con ejercicios prácticos:
- https://static1.squarespace.com/static/57c6b79629687fde090a0fdd/t/5b19b2f2aa4a99e99b26b6bb/1528410876119/dschool_bootleg_deck_2018_final_sm.pdf

Herramientas Prácticas

- Kit de Herramientas de Design Thinking – Stanford d.school (Gratis)
<https://dschool.stanford.edu/resources>
- Plantillas para Mapa de Empatía y User Journey (Miro): <https://miro.com/templates/design-thinking/>
- Guía de Prototipado Rápido (Figma): <https://www.figma.com/resource-library/what-is-design-thinking/>

04 Diseño de Interfaces de Usuario (UI)

Introducción al diseño de interfaz de usuario (UI)

El Diseño de Interfaz de Usuario (UI) es la disciplina que se encarga de diseñar la presentación y la interactividad de un producto digital. Es la capa visual con la que los usuarios interactúan directamente. Un buen diseño UI no es solo hacerlo "bonito"; es una herramienta estratégica fundamental para el éxito.

¿Por qué es esencial el Diseño UI?

1. Primeras impresiones:

- El 94% de los usuarios juzgan la credibilidad de un sitio web por su diseño visual (Stanford Research).
- Ejemplo: ¿Confiarías en un banco con colores fluorescentes y tipografía cómica?

2. Guía al usuario:

- El color y la tipografía dirigen la atención a lo importante (ej.: botones de acción en rojo).
- Caso real: Spotify usa verde brillante para el botón "Play" (acción principal).

3. Refuerza la marca:

- Los colores y fuentes consistentes hacen reconocible una app entre miles.
- Dato: El 80% del reconocimiento de marca depende del color (Color Marketing Group).

4. Mejora la accesibilidad:

- Un buen contraste ayuda a usuarios con discapacidad visual.
- Regla: Texto negro sobre fondo blanco tiene mejor legibilidad.

Los 3 Pilares del UI Design

Color

El color es un sistema de comunicación visual.



Tipografía

La tipografía es la arquitectura de la información.



Layout y Espaciado

El layout y el espaciado son la retícula estructural.



1. Color: Sistema de Comunicación Visual

El color es una propiedad perceptual que se gestiona a través de sistemas de diseño (Design Systems) mediante paletas codificadas (primaria, secundaria, neutral y semántica) que garantizan la coherencia y la accesibilidad.

¿Para qué sirve en términos prácticos?

- **Psicología del Color (Función Semántica):** Transmite emociones y significados predefinidos.
 - **AZUL** (#2563EB) → Confianza y estabilidad. Se utiliza en entornos financieros (bancos) y profesionales (LinkedIn).
 - **ROJO** (#DC2626) → Urgencia o error. Ideal para destacar acciones críticas (promociones limitadas, mensajes de alerta).
 - **VERDE** (#16A34A) → Éxito y confirmación. Se emplea para indicar acciones positivas (éxito, avance, validación).
- **Jerarquía Visual (Función Pragmática):** El contraste de color dirige la atención del usuario hacia elementos interactivos primarios (Botones CTA - Call To Action) y establece un orden de lectura.
- **Entendiendo bien el concepto:**

Imagina que el color es el código de colores de una fábrica. El rojo significa "peligro" o "parar", el verde significa "avanzar" o "seguro". En tu UI, usas este mismo código: un botón rojo para "Eliminar" (peligro) y uno verde para "Aceptar" (avanzar). Elige una paleta limitada y úsala siempre con el mismo significado.

2. Tipografía: Arquitectura de la Información

La tipografía es el arte de disponer el sistema de fuentes para crear una jerarquía clara, mejorar la legibilidad y comunicar la personalidad de la marca. Se implementa mediante una escala tipográfica predefinida (usando una base de 8pt) para todos los elementos de texto (H1, H2, Body, Caption...).

¿Para qué sirve en términos prácticos?

- **Jerarquía Visual:** Define el orden de importancia de la información.
 - H1 / Título Principal: Montserrat Bold, 2rem (32px) → Máximo impacto, para títulos únicos.
 - H2 / Subtítulo: Montserrat SemiBold, 1.5rem (24px) → Para dividir secciones.
 - Body / Párrafo: Open Sans Regular, 1rem (16px), line-height: 1.5 → Para bloques de texto legibles.
- **Legibilidad:** La correcta elección de la fuente, el tamaño, el interlineado (line-height) y el tracking (espaciado entre letras) asegura que el texto sea fácil de leer durante largos períodos.
- **Branding:** La tipografía transmite la voz de la marca (ej.: una fuente serif para tradición, una sans-serif para modernidad).
- **Entendiendo bien el concepto:**

Piensa en la tipografía como la voz de tu app. Los títulos (H1) son como gritar una noticia importante. Los subtítulos (H2) son como hablar en un tono firme pero normal. El texto del cuerpo (Body) es como conversar tranquilamente. Usa diferentes "tonos de voz" (tamaños y pesos de letra) para que el usuario sepa qué es lo más importante de un vistazo.

3. Layout (Composición) y Espaciado

El layout se construye sobre una retícula (grid system) que define columnas, márgenes y gutters (canalillos). El espaciado se rige por un múltiplo base (ej: 8px) para crear consistencia rhythm y alineación visual (regla de proximidad). Esto se conoce como Ley de Espaciado Uniforme.

¿Para qué sirve en términos prácticos?

- Organización y Escaneo: La retícula (ej: 12 columnas en web, 4 en móvil) permite alinear los elementos de forma coherente, facilitando al usuario escanear la información rápidamente.
- Respiración Visual (White Space): El espaciado consistente (ej: 16px, 24px, 32px) entre grupos de elementos reduce la densidad cognitiva, mejora la legibilidad y genera una sensación de orden y equilibrio.
- Responsividad: Un sistema de grid bien definido permite que la interfaz se adapte de manera predecible a diferentes tamaños de pantalla.

• Entendiendo bien el concepto:

El layout es el plano de tu casa. La retícula son las paredes y habitaciones que mantienen todo ordenado y en su sitio. El espaciado es el "aire" entre los muebles. Si amontonas todos los muebles en el centro de la sala, es un caos y no se puede pasar. En UI, el espacio en blanco ("white space") es ese aire vital que hace que tu diseño se vea limpio, ordenado y fácil de "caminar" por él. Usa siempre los mismos "metros" (múltiplos de 8) para medir las distancias

Impacto en la Experiencia de Usuario



Olga M. Moreno Martín

PROMETEO

airbnb



Airbnb aumentó sus reservas un 25% tras rediseñar su UI con colores más cálidos y tipografía clara (TechCrunch).

¿Qué es el diseño de interfaz de usuario (UI)?

- El Diseño de Interfaz de Usuario (UI Design) es la disciplina que se enfoca en la presentación visual e interactiva de un producto digital. Transforma la estructura funcional (definida por el UX) en una experiencia atractiva, intuitiva y emocionalmente resonante.

- **UI vs. UX: Diferencias clave:**

UI Design	UX Design
Se ocupa del aspecto visual (colores, tipografía, iconos).	Define la estructura y flujos (arquitectura de información, usabilidad).
Responde: "¿Cómo se ve?"	Responde: "¿Cómo funciona?"
Ejemplo: Diseñar un botón atractivo.	Ejemplo: Decidir dónde colocar ese botón para optimizar el flujo.

Analogía:

- Si una app fuera un restaurante...
- UX sería el diseño del menú, la disposición de las mesas y el proceso de pedido.
- UI sería la presentación del plato, los colores del local y la vajilla utilizada.

Objetivos Fundamentales del UI Design

1. Comunicar información de forma clara

- **Jerarquía visual:** Usar tamaño, color y contraste para guiar la atención.
 - Ejemplo: Títulos en negrita (24px) vs. texto cuerpo (16px).
- **Lenguaje visual intuitivo:** Iconos reconocibles (🔍 para búsqueda, + para añadir).

2. Mejorar la interacción con el usuario

- **Feedback visual:** Mostrar cambios al interactuar (ej.: botón que cambia de color al presionarse).
- **Microinteracciones:** Pequeñas animaciones que confirman acciones (like en Instagram).

3. Crear una identidad visual coherente

- **Branding consistente:** Mismos colores, tipografías y estilos en toda la app.
 - Caso: El azul de Facebook se usa en todos sus botones primarios.
- **Guías de estilo:** Documentar patrones de diseño para equipos.

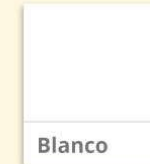
4. Facilitar la navegación y comprensión

- **Consistencia:** Mismos patrones en todas las pantallas (ej.: menú siempre en la parte inferior).
- **Accesibilidad:** Contraste adecuado para texto, tamaños legibles.

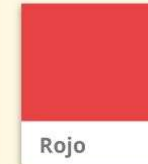
El Color en el Diseño UI

Color	Emoción/Asociación	Usos Recomendados	Ejemplos Reales
Azul	Confianza, seguridad, calma	Aplicaciones bancarias, redes sociales, tecnología	Facebook (#1877F2), LinkedIn (#0A66C2)
Rojo	Urgencia, pasión, error	Botones de acción, promociones, alertas de error	Netflix (#E50914), YouTube (#FF0000)
Verde	Éxito, naturaleza, salud	Confirmaciones, apps de sostenibilidad, salud	WhatsApp (#25D366), Spotify (#1DB954)
Amarillo	Optimismo, advertencia	Avisos temporales, elementos gamificados	Snapchat (#FFFC00), Ikea (#FFD700)
Gris	Neutralidad, elegancia	Fondos, texto secundario, bordes	Apple (#F5F5F7), Slack (#E8E8E8)
Negro	Sofisticación, lujo	Modo oscuro, marcas premium	Chanel (#000000), Nike (#111111)

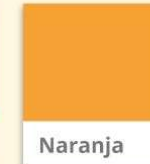
LA PSICOLOGÍA DEL COLOR



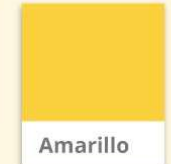
Pureza, limpieza, simplicidad o frescura.



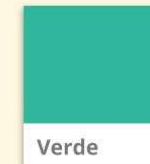
Revolución, pasión, energía o fuerza.



Creatividad, éxito, aventura o lujuria.



Energía, alegría, optimismo o verano.



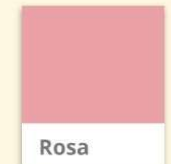
Naturaleza, esperanza, dinero o crecimiento.



Confianza, calma, frescura, lealtad o inteligencia.



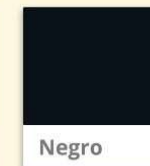
Elegancia, lujo, poder, misterio o espiritualidad.



Femenino, dulce, inmadurez o delicadeza.



Paz, tranquilidad, o tecnología.



Poder, elegancia, o formalidad.

Creación de paletas de color

Estructura profesional para interfaces:

1. Color primario:

- Representa la marca.
- Uso: Botones principales, elementos clave.
- Ejemplo: Azul (#3366FF) para un botón "Guardar".

2. Color secundario:

- Complementa al primario.
- Uso: Botones secundarios, highlights.
- Ejemplo: Naranja (#FF9900) para un botón "Cancelar".

3. Colores neutros:

- Fondos, texto y contenedores.
- Escala recomendada:
 - i. Blanco (#FFFFFF) → Fondos

ii. Gris claro (#F8F9FA) → Cards

iii. Gris medio (#E9ECEF) → Bordes

iv. Gris oscuro (#6C757D) → Texto secundario

v. Negro (#212529) → Texto principal

4. Colores de estado:

- Éxito (#28A745), Error (#DC3545), Advertencia (#FFC107).

Herramientas para generar paletas:

- Adobe Color: <https://color.adobe.com>
- Colors: <https://colors.co>
- Eye Dropper <https://www eyedropper.org/>

Regla de oro:

- Máximo 5 colores principales (incluyendo neutros).
- Coherencia en toda la app.

Creación de paletas de color: consejos de la profe

Para aplicaciones:

- **Color primario** (en general es un color que consideraremos claro y que llamará la atención)
- **Color secundario** (en general es un color que consideraremos oscuro, pero puede ser un color con contraste entre primario y acentuación)
- **Color acentuación** (color llamativo y claro, suele contrastar al primario, son opuestos en el círculo cromático)
- **Color sombra** (color oscuro cercano al negro más oscuro que el secundario en la misma tonalidad)
- **Color neutro** (color cercano al primario pero con muy poca saturación y cercano al blanco suele actuar de fondo)



Contraste y accesibilidad

Normas WCAG (Web Content Accessibility Guidelines):

1. Contraste mínimo:

- Texto normal: 4.5:1 vs fondo.
- Texto grande (18px+): 3:1.

2. Herramientas de verificación:

- WebAIM Contrast Checker: <https://webaim.org/resources/contrastchecker/>
- Plugin Stark para Figma: Detecta problemas de contraste.

3. Prácticas inclusivas:

- Nunca uses solo color para comunicar información.
 -  Rojo = Error →  Rojo + ícono "  " + texto "Error".
- Evita combinaciones problemáticas:
 - Rojo/verde (para daltónicos).
 - Texto amarillo sobre blanco.

Tipografías en interfaces

Legibilidad ante todo:

- Evitar fuentes decorativas o con serifs finas en interfaces.
- Ejemplo de mala práctica: Usar Brush Script en un formulario médico.

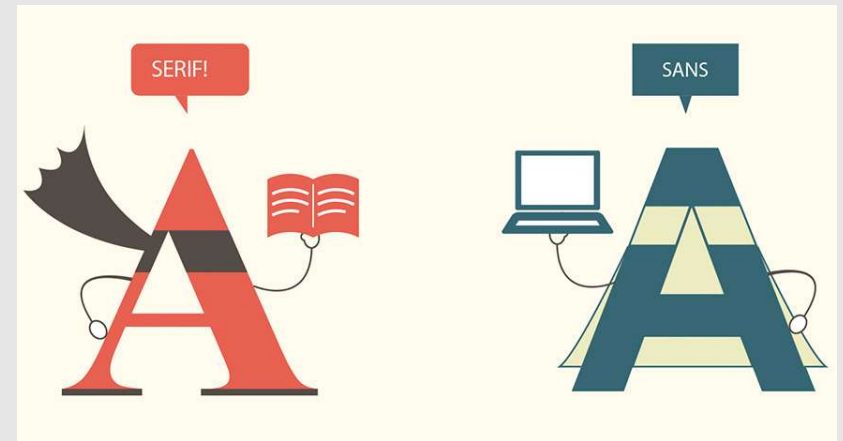
Preferir Sans-Serif para pantallas:

Top 5 fuentes para UI:

- Roboto (Android) → Neutral y versátil.
- Open Sans → Excelente legibilidad en pequeños tamaños.
- Montserrat → Moderna para títulos.
- Inter → Optimizada para pantallas de baja resolución.
- SF Pro (iOS) → Alta legibilidad en móviles.

Recursos gratuitos:

- Google Fonts: <https://fonts.google.com>
- Font Pair: <https://fontpair.co> (combinaciones predefinidas).



I am sans.
I am serif.

Tipografía

No uses muchas fuentes diferentes con 1 ó 2 tipos basta

- No te compliques, utiliza una o dos tipos de fuentes como máximo y juega con los diferentes estilos.
- La mayoría de tipografías actuales cuentan con varios estilos, me refiero a al grosor o peso de la fuente. Lo que viene siendo el texto en negrita y cursiva.
- Usa el **font weight** para conseguir resaltar algunas palabras, títulos, botones, etc.
- Te pongo como ejemplo las variantes de la tipografía Roboto, aunque hay tipografías con hasta 9 variantes.

Roboto Thin
Roboto Light
Roboto Regular
Roboto Medium
Roboto Bold
Roboto Black

Tipografía

Fuentes complementarias

- Si optas por usar más de una fuente, asegúrate de que se complementen una a otra.
- Cada tipografía transmite una personalidad o estado diferente: elegante, casual, infantil, serio, moderno, futurista, etc.
- Tienes que identificar qué estado quieres transmitir a través de tu aplicación y elegir la tipografía acorde a ello.
- Font Pair: es una página web donde hay un unas pocas de combinaciones muy buenas organizadas por el tipo de fuente.

The spectacle
before us was
indeed sublime.

My two natures had
memory in
common.

LA
TIPOGRAFÍA
en
diseño WEB

Tipografía: jerarquía visual

El tamaño de letra adecuado para la aplicación

- El tamaño de la tipografía de la web/app tiene mucha importancia en la lectura.
- Un texto demasiado pequeño hace que el usuario se tenga que concentrar más y es más fácil perder la línea por la que ibas leyendo. Esto afecta especialmente a dispositivos móviles.
- Como resultado, lo más probable es que el usuario acabe saltándose gran parte del contenido y con suerte se leerá los encabezados.
- Usar un texto demasiado grande tampoco es bueno porque puede resultar demasiado llamativo.
- Por eso, lo recomendable es empezar estableciendo el tamaño de letra para el cuerpo de la página.

Tipografía: jerarquía visual

- Escala tipográfica estándar para móvil/web:

Elemento	Tamaño	Peso	Color	Uso
H1 (Título)	24-32 px	Bold	#000000	Nombres de pantalla
H2 (Subtítulo)	18-22 px	SemiBold	#333333	Secciones
Body (Texto)	16 -18 px	Regular	#555555	Párrafos, botones
Footnote	12-14 px	Light	#777777	Leyendas, ayudas

- **Técnicas para reforzar jerarquía:**
- Contraste de peso: H1 en Bold (700) vs Body en Regular (400).
- Color estratégico: Títulos en negro (#000), texto secundario en gris (#6C757D).
- Espaciado vertical: Doble espacio entre títulos y cuerpo.

Tipografía: Espaciado Y Alineación

Buenas prácticas profesionales:

- **Interlineado (Line-height):**

Móvil: 1.5x el tamaño de fuente (ej.: 16px → 24px).

Desktop: 1.4x (para líneas más largas).

- **Alineación:**

Texto largo: Siempre alineado a la izquierda (mejor legibilidad).

Centrado: Solo para títulos breves (<2 líneas) o frases destacadas.

Justificado: ❌ Evitar en interfaces (crea "ríos" de espacio blanco).

- **Errores comunes:**

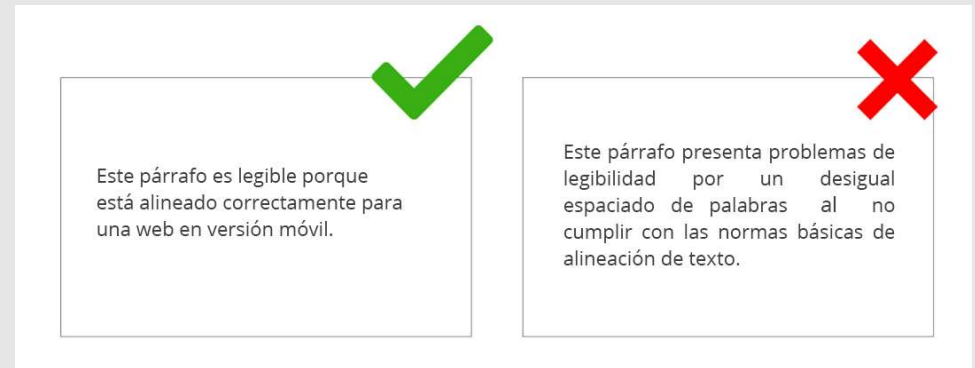
❌ Interlineado <1.2x (texto apretado).

❌ Mezclar alineaciones en una misma pantalla.

- **Plantillas listas para usar:**

Escala tipográfica en Figma: <https://www.figma.com/community/file/888356646442115999>

Guía de accesibilidad WCAG: <https://www.w3.org/WAI/tips/designing/>



Distribución Visual (Layout)

- En este punto veremos como organizar elementos para crear experiencias intuitivas, equilibradas y eficaces. Un buen layout guía al usuario de forma natural a través de la interfaz.

Principios fundamentales de diseño visual

- Estas reglas universales, derivadas de la Psicología de la Gestalt, son la base para crear interfaces comprensibles y agradables (Las hemos visto anteriormente).



Proximidad (Ley de la Agrupación)

- Qué es: Agrupar visualmente los elementos que están relacionados entre sí.
- Por qué funciona: Nuestro cerebro asume que los elementos cercanos pertenecen a un mismo grupo o concepto.
- **Ejemplo Práctico:**
 - Formulario: Los campos "Nombre", "Apellido" y "Email" deben tener 8px de separación entre ellos, pero el grupo debe estar separado del siguiente (ej: "Dirección") por al menos 24px. Esto reduce la carga cognitiva.
 - Tarjeta de producto: La imagen, el título, el precio y el botón "Añadir" deben estar agrupados con espaciado consistente, separándose claramente de la tarjeta contigua.

Rejillas Grid

Tipo de Grid	Uso	Ejemplo
12-Columnas	Diseño web complejo y flexible	- Máxima flexibilidad para layouts adaptativos - Permite múltiples combinaciones (ej: 4+4+4, 3+6+3)
8px Grid	Alineación precisa en apps móviles	- Todos los elementos y espacios en múltiplos de 8 - Garantiza consistencia visual
Baseline Grid	Alineación tipográfica perfecta	- Textos alineados a línea base invisible - Interlineado consistente (ej: 4px o 8px)

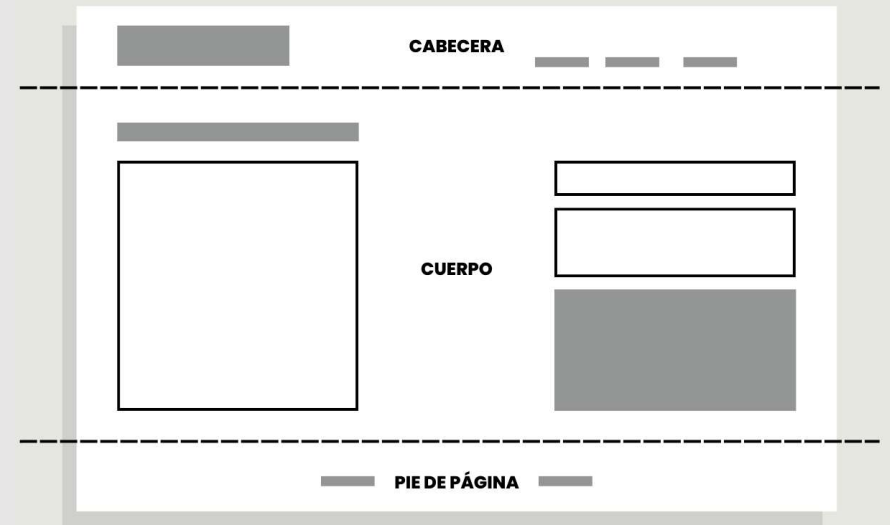
Herramientas:

- Figma Layout Grids: Configurar columnas/gutters.
- CSS Grid Generator: <https://cssgrid-generator.netlify.app/>

Layouts comunes

1. Patrón Cabecera – Cuerpo – Pie (Header-Body-Footer)

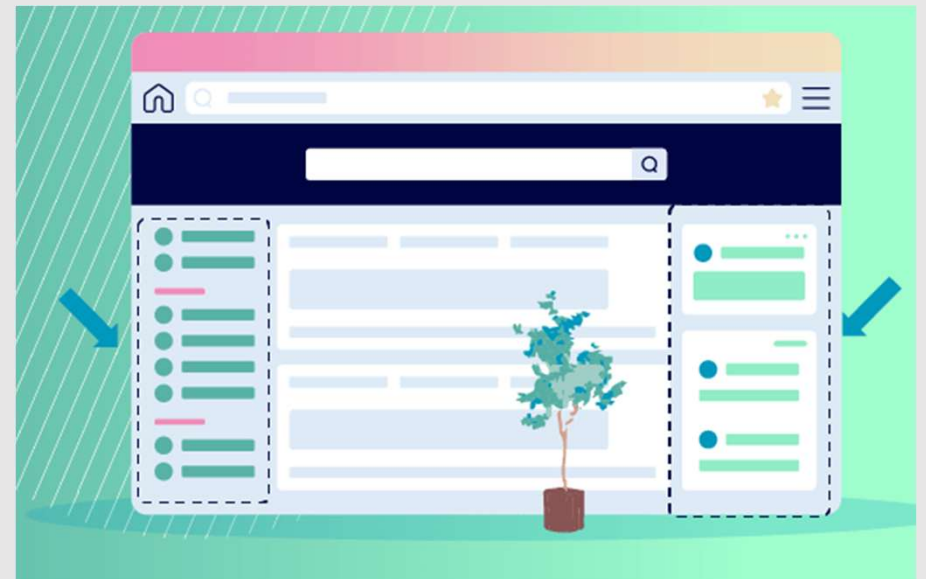
- Estructura clásica y verticalmente segmentada donde la cabecera contiene navegación principal, el cuerpo alberga el contenido central y el pie incluye información secundaria.
- Ejemplo representativo: Google Search.
- Ventaja principal: extremadamente familiar para los usuarios, lo que reduce la curva de aprendizaje y ofrece una sensación de inmediata usabilidad.
- Ideal para: sitios web corporativos, blogs y páginas de destino (landing pages) donde la simplicidad y la claridad son primordiales.



Layouts comunes

2. Patrón menú lateral + Contenido (Sidebar + Content)

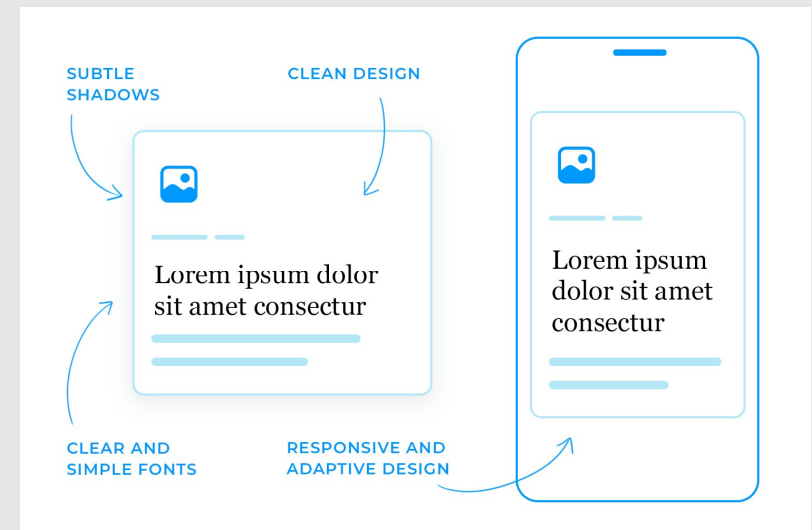
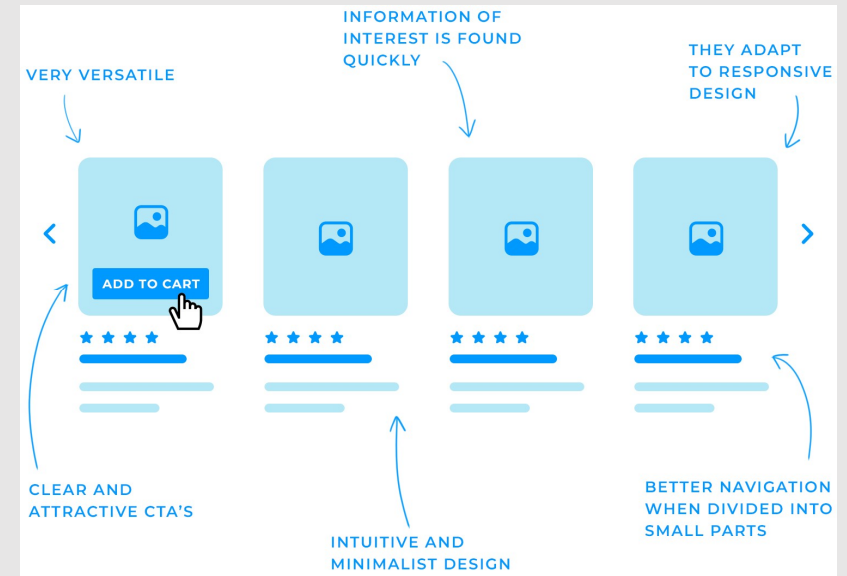
- Layout que divide la pantalla en dos áreas: una barra lateral (sidebar) fija o colapsable para la navegación y una sección principal dinámica para el contenido.
- Ejemplo representativo: Gmail (con su menú izquierdo).
- Ventaja principal: maximiza el espacio horizontal y permite un acceso rápido y constante a un gran número de secciones o herramientas.
- Ideal para: aplicaciones web complejas, dashboards de administración, herramientas de productividad y plataformas de correo electrónico que requieren una navegación extensa.



Layouts comunes

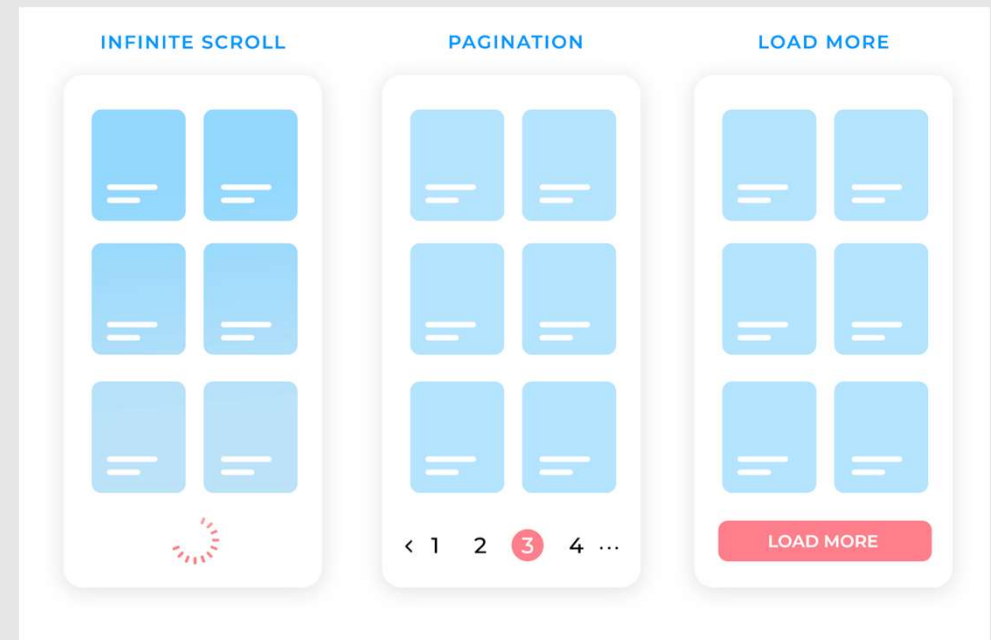
• 3. Patrón de tarjetas (Cards)

- Organiza el contenido en módulos rectangulares independientes (tarjetas) que contienen información relacionada, como una imagen, título y texto.
- Ejemplo representativo: Pinterest o Netflix.
- Ventaja Principal: escala de forma excepcional en diseños responsivos, reorganizándose fluidamente en diferentes tamaños de pantalla. Es visualmente atractivo y favorece el "escaneo" del contenido por parte del usuario.
- Ideal para: galerías, marketplaces, catálogos de productos y cualquier interfaz que priorice el contenido visual



Layouts comunes

- **4. Patrón de desplazamiento a pantalla completa (Full-Screen Scroll)**
 - Cada sección o bloque de contenido ocupa el 100% del viewport (altura y anchura de la pantalla), creando una experiencia narrativa e inmersiva.
 - Ejemplo representativo: páginas de producto de Apple.
 - Truco clave: utilizar la propiedad CSS scroll-snap-type para lograr un "ajuste" o snapping suave entre secciones, mejorando la navegación y la experiencia en dispositivos táctiles.
 - Ideal para: sitios de portafolio, narrativas digitales, landing pages de alto impacto y presentaciones de productos únicos.



Layouts comunes

5. Diseño responsive y adaptativo

- Estrategia de diseño que permite que la interfaz se adapte y responda al tamaño y orientación del dispositivo del usuario, modificando su layout y componentes.
- Técnicas comunes de implementación:
 - Patrón de navegación adaptable: menú hamburguesa (ícono de tres barras) en la versión móvil → Se transforma en una barra de navegación horizontal completa en desktop.
 - Rejilla adaptativa (Adaptive Grid): una cuadrícula (grid) de 1 columna en móvil → 2 columnas en tablet → 4 columnas en desktop, reordenando el contenido de forma óptima.
- Objetivo principal: garantizar una experiencia de usuario óptima, legible y usable en cualquier dispositivo, desde un teléfono móvil hasta un monitor de escritorio.



Diseño Responsive Y Mobile-first

- Estrategias para interfaces adaptables y centradas en móvil.

Conceptos Clave:

Término	Definición	Ejemplo
Responsive	Diseño que se ajusta automáticamente al tamaño y orientación de la pantalla.	Una web que pasa de 4 columnas (desktop) a 1 (móvil).
Mobile-First	Metodología que prioriza el diseño para móvil antes que para desktop.	Comenzar con un menú hamburguesa en móvil y luego adaptarlo a una barra superior en desktop.

- 📖 Importante: El 58% del tráfico web global proviene de móvil y esta cifra aumenta año a año.

Buenas Prácticas para Mobile-First

1. Menús hamburguesa

- ¿Por qué? Ahorra espacio en pantallas pequeñas.
- Cómo implementarlo:
 - Icono ☰ en la esquina superior.
 - Menú desplegable con opciones clave.
 - Ejemplo: Spotify móvil.

2. Botones Táctiles

- Tamaño mínimo: 48x48 píxeles (recomendación de Apple/Google).
- Espaciado: 8-12px entre botones para evitar errores de clic.
- Ejemplo de error: ❌ Botón de 32px en móvil (difícil de tocar).

3. Contenido Prioritario

- Regla 80/20: Mostrar solo el 20% de funcionalidades clave en móvil.
- Técnicas:
 - Ocultar elementos secundarios bajo pestañas o "Ver más".
 - Eliminar imágenes decorativas en móvil.

4. Tipografía Responsive

- Escala adaptable:
 - Móvil: Títulos 22px, cuerpo 16px.
 - Desktop: Títulos 28px, cuerpo 18px.

5. Media Queries (CSS)

Puntos de corte típicos. Ejemplo en CSS

```
/* Móvil: <768px */
@media (max-width: 768px) { ... }

/* Tablet: 768px-1024px */
@media (min-width: 768px) and (max-width: 1024px) { ... }

/* Desktop: >1024px */
@media (min-width: 1024px) { ... }
```

LAS WCAG

- La Iniciativa de Accesibilidad a la Web del W3C (WAI), fundada en 1997, es un grupo de trabajo permanente del W3C (World Wide Web Consortium - <http://www.w3c.es>).
- La W3C es una organización internacional que trabaja en el desarrollo de estándares web, y que recibe el apoyo de los principales actores de la industria y los gobiernos del mundo.
- LA WAI se dedica a promover soluciones de accesibilidad en la web para personas con discapacidades. Actúa principalmente sobre cinco áreas de trabajo:
 - o Asegurar que las tecnologías web den soporte a la accesibilidad
 - o Desarrollar pautas de accesibilidad
 - o Crear herramientas de evaluación y corrección de la accesibilidad web
 - o Desarrollar materiales para la educación y difusión
 - o Coordinar proyectos de investigación y desarrollo

LAS WCAG

- La accesibilidad web incluye los contenidos y aplicaciones, los navegadores y reproductores multimedia, las herramientas de autor y las tecnologías XML. La WAI ha propuesto para cada una de estas necesidades unas pautas a seguir.
- En 1999 se publica las Pautas de Accesibilidad para el Contenido Web, WCAG 1.0 (Web Content Accessibility Guidelines, WCAG), que representan el primer y más grande esfuerzo por establecer unas pautas de diseño **ACCESIBLE**.
- En 2008 se publica las WCAG 2.0.
- En 2019 se publica las WCAG 2.1.
- **En 2023 se publica las WCAG 2.2.** <https://www.w3.org/WAI/standards-guidelines/wcag/glance/es>
- En desarrollo : WCAG 3.0

LAS WCAG

- Las WCAG se organizan en 5 principios fundamentales para la accesibilidad del contenido:
- **PERCEPTIBLE:** La información y los componentes de la interfaz de usuario deben ser presentados a los usuarios de modo que ellos puedan percibirlos.
- **OPERABLE:** Los componentes de la interfaz de usuario y la navegación deben ser operables.
- **COMPENSIBLE:** La información y el manejo de la interfaz de usuario deben ser comprensibles.
- **ROBUSTO:** El contenido debe ser suficientemente robusto como para ser interpretado de forma fiable por una amplia variedad de aplicaciones de usuario, incluyendo los productos de apoyo.
- Estas pautas no son testables en sí, sino que proporcionan las metas básicas para hacer el **CONTENIDO ACCESIBLE**, y sirven para comprender los criterios de conformidad e implementarlos.

LAS WCAG

Pauta 1.3 Adaptable

- 1.3.4 Orientación (AA)
- 1.3.5 Identificación del Propósito de Entrada (AA)
- 1.3.6 Identificación del Propósito (AAA)

Pauta 1.4 Distinguible

- 1.4.10 Reajuste (AA)
- 1.4.11 Contraste no Textual (AA)
- 1.4.12 Espaciado de Texto (AA)
- 1.4.13 Contenido en Puntero Flotante o en Foco (AA)

Pauta 2.1 Accesible por Teclado

- 2.1.4 Atajos de Teclado (A)

Pauta 2.2 Tiempo Suficiente

- 2.2.6 Límites Temporales (AAA)

Pauta 2.3 Convulsiones y Reacciones Físicas

- 2.3.3 Interacciones Animadas (AAA)

Pauta 2.5 Modalidades de Entrada

- 2.5.1 Gestos del Puntero (A)
- 2.5.2 Cancelación del Puntero (A)
- 2.5.3 Etiqueta en el Nombre (A)
- 2.5.4 Actuación del Movimiento (A)
- 2.5.5 Tamaño del Objetivo (AAA)
- 2.5.6 Mecanismos de Entrada Concurrentes (AAA)

Recursos

Kit de Recursos Prácticos

1. Plantillas de Grids:
 - [Figma UI Grids](#)
2. Ejemplos de Layouts:
 - [Land-book Inspo](#)
3. Guía Responsive:
 - [MDN Web Docs](#)

Libros

- "Designing Interfaces" (Jenifer Tidwell)
- "Refactoring UI" (Adam Wathan & Steve Schoger)
- "The Non-Designer's Design Book" (Robin Williams)

Cursos en Línea

1. "UI Design Fundamentals" (DesignCourse)
<https://www.youtube.com/playlist?list=PL0eyrZgxdwhxNGMWROnaY35NLrEjTdybM>

2. "UI Design Patterns for Successful Software" (Interaction Design Foundation) <https://www.interaction-design.org/courses/ui-design-patterns-for-successful-software>

Tutoriales en Video

1. "Typography in UI Design" (Flux Academy)
 - <https://www.youtube.com/watch?v=agbh1wbJt8>
2. "8-Point Grid System" (UXcel)
 - <https://www.youtube.com/watch?v=EyD6nFI-9po>
3. "Mobile UI Design Tutorial" (Dansky)
 - https://www.youtube.com/watch?v=WzWXpnQX_bM

WCAG:

- Guía básica del estándar: <https://www.w3.org/WAI/standards-guidelines/es>
- Resumen en español: <https://guia-wcag.com/es/>

Introducción a Dart

Fundamentos del lenguaje de programación de Google para desarrollo multiplataforma

Módulo: 2º DAM – Desarrollo de Interfaces





Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Índice

01

Introducción a Dart

Rendimiento, ecosistema y simplicidad

02

Configuración y Herramientas

SDK, IDEs y primer proyecto

03

Sintaxis Básica

Variables, tipos y control de flujo

04

Funciones Avanzadas

Parámetros, closures y arrow functions

05

Colecciones y Null Safety

Listas, Sets, Maps y seguridad de nulos

06

POO en Dart

Clases, herencia, mixins e interfaces

07

Características Avanzadas

Extensions, generators y callable classes

08

Asincronía y Concurrencia

Futures, Streams e Isolates

09

Buenas Prácticas

Convenciones, código limpio y organización

10

Documentación y Pruebas

Testing unitario y cobertura de código

¿Por qué Dart?

Rendimiento

JIT (Just-In-Time): Compila durante la ejecución, ideal para desarrollo con Hot Reload en Flutter.

AOT (Ahead-Of-Time): Compila a código máquina nativo antes de ejecutarse, optimizando el rendimiento en producción.

Ecosistema Flutter

Dart es el lenguaje oficial de Flutter, el framework de Google para crear aplicaciones móviles, web y de escritorio con un solo código.

UI consistente

Comunidad activa

Simplicidad

Sintaxis clara y moderna con null safety integrada. Combina programación orientada a objetos con influencias funcionales.

dart pub

dart compile

Dart vs Otros Lenguajes

 **vs JavaScript/TypeScript:** Sintaxis similar (var, async/await, funciones flecha), pero con null safety de serie y mejor rendimiento.

 **vs Java:** Tipado fuerte, clases e interfaces similares, pero con compilación nativa y código multiplataforma.

Historia: Dart nació en 2011, pero su popularidad explotó en 2017 con la llegada de Flutter.

Flujo de Compilación

Desarrollo

Dart → JIT → Ejecución rápida

Producción

Dart → AOT → Binario nativo

Configuración del Entorno

Instalación del SDK

1. Descarga desde dart.dev/get-dart
2. Extrae y añade la carpeta `bin` al PATH
3. Verifica la instalación:

```
dart --version
```

Herramientas Incluidas

dart pub

Gestión de paquetes y dependencias

dart compile

Generación de ejecutables nativos

IDEs Recomendados

VS Code + Extensión Dart

Ligero y potente

Android Studio + Plugin

Ideal para Flutter

DartPad (Online)

Sin instalación: dartpad.dev

Crear Primer Proyecto CLI

```
dart create mi_proyecto
```

```
cd mi_proyecto
```

```
dart run
```

Estructura de Carpetas

```
mi_proyecto/  
├─ bin/ → Código principal  
│   └─ mi_proyecto.dart  
├─ lib/ → Librerías y módulos  
│   └─ pubspec.yaml → Dependencias
```

Compilar a ejecutable:

```
dart compile exe main.dart
```

Sintaxis Básica de Dart

Declaración de Variables

```
var nombre = "Ana"; // Tipo inferido
dynamic dato = 42; // Puede cambiar
final fecha = DateTime.now(); // Asignación única
const pi = 3.1416; // Compilación
```

Tipos Básicos

int: Números enteros

double: Decimales

bool: true/false

String: Texto

List: Lista ordenada

Set: Sin duplicados

Map: Clave-valor

dynamic: Cualquier tipo

Operadores Null-Aware

```
?? → Valor por defecto
?. → Acceso seguro
! → Afirmación (cuidado)
```

Control de Flujo

If-Else

```
if (edad >= 18) { print("Mayor"); }
```

Switch con Pattern Matching

```
switch (dia) { case "Lunes"|"Martes": ... }
```

Bucles

```
for (var i = 0; i < 5; i++) { ... }
for (var fruta in frutas) { ... }
while (contador < 3) { ... }
```

Operador Cascade (..)

```
var buffer = StringBuffer()
..write("Hola ")
..write("Mundo");
```


Funciones y Parametrización Avanzada

Parámetros Opcionales

Se colocan entre corchetes y pueden tener valores por defecto.

```
void mostrar(String texto, [int veces = 1]) { ... }  
// Uso:  
mostrar("Hola"); // 1 vez  
mostrar("Hola", 3); // 3 veces
```

Parámetros Nombrados

Se especifican por nombre, más legibles.

```
void crearUsuario({  
  required String nombre,  
  int edad = 18,  
}) { ... }  
// Uso:  
crearUsuario(nombre: "Ana");
```

Combinando Ambos

Parámetros posicionales y nombrados juntos.

```
void registrar(String id, {  
  String? nombre,  
  int edad = 0,  
}) { ... }
```

Funciones Anónimas (Lambdas)

```
var sumar = (int a, int b) {  
  return a + b;  
};  
print(sumar(3, 4)); // 7
```

Uso común: Como argumentos de funciones de orden superior.

Lexical Scope

Las funciones internas acceden a variables de funciones externas.

Arrow Functions (=>)

Sintaxis reducida para funciones que retornan una expresión.

Closures

```
Function contador() {  
  int cuenta = 0;  
  return () {  
    cuenta++;  
    print(cuenta);  
  };  
}  
  
var c = contador();  
c(); // 1  
c(); // 2
```

Concepto clave: La función interna "recuerda" el estado externo.

Colecciones y Null Safety

List (Listas)

Ordenadas, permiten duplicados, indexadas desde 0.

```
var frutas = ["Manzana", "Pera"];
frutas.add("Uva");
print(frutas[0]); // Manzana
```

Métodos útiles:

map: Transforma

where: Filtra

fold: Reduce

Set (Conjuntos)

No permiten duplicados, ideales para valores únicos.

```
var numeros = {1, 2, 3, 3};
print(numeros); // {1, 2, 3}
numeros.add(4);
```

Map (Diccionarios)

Pares clave-valor, claves únicas.

```
var persona = {
  "nombre": "Ana",
  "edad": 30
};
print(persona["nombre"]); // Ana
```

Null Safety: Seguridad de Nulos Integrada

Dart utiliza null safety para evitar errores en tiempo de ejecución al acceder a variables que pueden ser nulas.

Tipos Anulables (?)

```
String? nombre;
nombre = null; // Válido
```

Operador ! (Afirmación)

```
String? texto = "Hola";
print(texto!.length); // 4
```

⚠ Usar con cuidado

Operador ?. (Acceso Seguro)

```
String? saludo;
print(saludo?.length); // null
```

✅ No lanza error

Operador ?? (Valor por Defecto)

```
String? usuario;
print(usuario ?? "Invitado");
// "Invitado"
```

Ejemplo Combinado:

```
String? nombre;
print((nombre ?? "Desconocido").toUpperCase());
```

Programación Orientada a Objetos en Dart

Clases y Constructores

```
class Persona {  
  String nombre;  
  int edad;  
  Persona(this.nombre, this.edad);  
  void saludar() => print("Hola, soy $nombre");  
}
```

Constructores con Nombre

```
class Punto {  
  double x, y;  
  Punto(this.x, this.y);  
  Punto.origen(): x = 0, y = 0;  
}  
  
var p1 = Punto(5, 3);  
var p2 = Punto.origen();
```

Factory (Singleton)

```
class Conexion {  
  static final Conexion _instancia = Conexion._interna();  
  Factory Conexion() => _instancia;  
  Conexion._interna();  
}
```

Getters y Setters: Propiedades calculadas con get y set.

Herencia (extends)

```
class Animal {  
  void hacerSonido() => print("Sonido");  
}  
class Perro extends Animal {  
  @override  
  void hacerSonido() => print("Guau");  
}
```

Mixins (with)

```
mixin Volador {  
  void volar() => print("Volando");  
}  
class Pajaro with Volador {}
```

Interfaces

```
class Documento  
  Implements Imprimible { ... }
```

Clases Abstractas

```
abstract class Figura {  
  double area();  
}
```

Características Avanzadas del Lenguaje

Extensions

Añaden métodos o propiedades a clases existentes sin modificarlas.

```
extension StringMayus onString {
  String primeraMayus() {
    if (isEmpty) return this;
    return this[0].toUpperCase() + substring(1);
  }
}

// Uso:
print("hola mundo".primeraMayus());
// Hola mundo
```

Ventaja: Añade utilidades a tipos nativos o de librerías externas.

Generators

Producen elementos bajo demanda (lazy evaluation).

```
sync* (Síncrono):
Iterable contarHasta(int max) sync* {
  for (int i = 1; i <= max; i++) {
    yield i; // Devuelve y pausa
  }
}
```

```
async* (Asíncrono):
Stream contarAsync(int max) async* { ... }
```

Usos: Lectura progresiva, streams de eventos, grandes listas.

Callable Classes

Permiten usar objetos como funciones.

```
class Suma {
  int call(int a, int b) => a + b;
}

var sumar = Suma();
print(sumar(3, 4)); // 7
```

Ventaja: Objetos que representan operaciones, más naturales de usar.

Ejemplo: Generador Síncrono

```
void main() {
  for (var n in contarHasta(5)) {
    print(n); // 1 2 3 4 5
  }
}
```

Ejemplo: Generador Asíncrono

```
void main() async {
  awaitfor (var n in contarAsync(3)) {
    print(n); // 1 por segundo
  }
}
```

Concepto clave: `yield` devuelve un valor y "pausa" la función, manteniendo su estado para la próxima llamada.

Olga M. Moreno Martín

PROMETEO

Asincronía y concurrencia

Futures

Representan un valor que estará disponible en el futuro.

```
Future obtenerDatos() async {
  await Future.delayed(Duration(seconds: 2));
  return "Datos recibidos";
}

// Uso:
var datos = await obtenerDatos();
```

Manejo de errores:

```
try { ... } catch (e) { ... }
```

Streams

Secuencia de datos que llega con el tiempo.

```
Stream contadorStream(int max) async* {
  for (int i = 1; i <= max; i++) {
    await Future.delayed(Duration(seconds: 1));
    yield i;
  }
}
```

Uso común:

Eventos de usuario, lectura de sensores, comunicación en tiempo real.

Isolates

Hilos independientes con memoria propia.

```
import 'dart:isolate';
void tareaPesada(SendPort sendPort) {
  var resultado = 0;
  for (var i = 0; i < 1000000000; i++) {
    resultado += i;
  }
  sendPort.send(resultado);
}
```

Ventaja:

Evita bloqueos en operaciones intensivas de CPU.

Stream con StreamController

```
import 'dart:async';
void main() {
  final controller = StreamController<String>();
  controller.stream.listen((dato) {
    print("Recibido: $dato");
  });
  controller.add("Hola");
  controller.close();
}
```

Comparativa de Enfoques

Futures → Un valor futuro. Único resultado.

Streams → Múltiples valores en el tiempo.

Isolates → Paralelismo real con memoria separada.

Tip: Usa `await` para escribir código asíncrono como si fuera síncrono, mejorando la legibilidad.

Olga M. Moreno Martín

PROMETEO

Buenas prácticas y documentación

Convenciones de Nombres

lowerCamelCase: variables y funciones

UpperCamelCase: clases y enums

lowerCamelCase: constantes

_privado: antepone _ para privacidad

Código Limpio

- Funciones cortas: una función = una cosa
- Evitar "números mágicos": usar constantes
- Comentarios claros: explicar el "por qué"
- Agrupar código relacionado

Organización de Archivos

```
lib/  
├── models/ # Clases de datos  
├── services/ # Lógica  
├── utils/ # Funciones de ayuda  
└── main.dart
```

Null Safety Correcto

✗ **Evitar:**

```
print(nombre!.length);
```

✓ **Preferir:**

```
print(nombre?.length ?? 0);
```

Documentación ///

Comentarios procesados por dart doc para generar documentación HTML.

Linting

Reglas automáticas para mantener estilo uniforme.
Archivo: analysis_options.yaml

Principios SOLID

DRY, KISS, YAGNI para mantenibilidad.

Documentación con Comentarios ///

```
/// Clase que representa un usuario en el sistema.  
/// Contiene información básica como [nombre] y [edad].  
class Usuario {  
  /// Nombre completo del usuario.  
  final String nombre;  
  /// Edad del usuario en años.  
  final int edad;  
}
```

Genera documentación automática

Mejora la comprensión del código

M. Moreno Martín

PROMETEO

Pruebas en Dart

Configuración

Añade la dependencia en pubspec.yaml:

```
dev_dependencies:  
  test: ^1.25.0
```

Estructura de Archivos

```
mi_proyecto/  
├── lib/calculadora.dart  
└── test/calculadora_test.dart
```

Ejecución

```
dart test
```

Los archivos deben terminar en `_test.dart`

Ejemplo Completo de Test

```
import 'package:test/test.dart';  
import '../lib/calculadora.dart';  
  
void main() {  
  group('Pruebas calculadora', () {  
    test('Suma correcta', () {  
      expect(sumar(2, 3), equals(5));  
    });  
  });  
}
```

setUp()

Se ejecuta antes de cada test.

tearDown()

Se ejecuta después de cada test.

Cobertura de Pruebas

```
dart test --coverage=coverage
```

¿Preguntas?

Introducción al framework Flutter



Flutter

Módulo: 2º DAM – Desarrollo de Interfaces

Olga M. Moreno Martín

PROMETEO





Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Repositorio Github

<https://github.com/olga3emes/Flutter-2DAM/>



Para trabajar con los ejemplos descritos en estas diapositivas se debe acceder a la **rama** del repositorio citada con una etiqueta del estilo que aparece aquí:

EJX donde X es el nombre de la rama

Para acceder a todas las ramas disponibles con un solo click:

<https://github.com/olga3emes/Flutter-2DAM/branches>

Olga M. Moreno Martín

01 Puesta en marcha

¿Qué es Flutter?

- Flutter es un framework de código abierto desarrollado por Google para crear interfaces nativas multiplataforma (móvil, web y escritorio) desde una única base de código. Su primera versión estable se lanzó en diciembre de 2018, y desde entonces ha ganado popularidad por su rendimiento, flexibilidad y productividad.
- **Características clave:**
 - ✓ UI declarativa: La interfaz se construye en función del estado actual de la aplicación.
 - ✓ Motor de renderizado propio (Skia): No depende de componentes nativos del sistema, lo que garantiza consistencia visual en todas las plataformas.
 - ✓ Alto rendimiento: Compila a código nativo (ARM, x64) y JavaScript (para web).
 - ✓ Hot Reload: Permite ver cambios al instante sin reiniciar la app, acelerando el desarrollo.
 - ✓ Widgets altamente personalizables: Ofrece una amplia biblioteca de componentes adaptables.

¿Por qué Flutter eligió Dart?

- Dart es el lenguaje de programación detrás de Flutter, diseñado por Google para ser:
 - ✓ Rápido: Compilación AOT (ahead-of-time) para producción y JIT (just-in-time) para desarrollo ágil.
 - ✓ Productivo: Sintaxis clara y moderna, similar a JavaScript/TypeScript, Java y C#.
 - ✓ Orientado a UI: Ideal para aplicaciones reactivas gracias a su manejo eficiente de estados y eventos.

¿Por qué Flutter eligió Dart?

- Rendimiento cercano al nativo (evita el "puente JavaScript" de otros frameworks).
- Capacidad de compilación multiplataforma (móvil, web y desktop).
- Hot Reload nativo, algo difícil de lograr con otros lenguajes.

Ventajas del desarrollo con Flutter

- **Código único para múltiples plataformas:**
 - Escribe una vez y despliega en Android, iOS, web, Windows, macOS y Linux.
-  **Productividad elevada:**
 - Hot Reload acelera las iteraciones de desarrollo.
 - Amplio ecosistema de paquetes (pub.dev) y herramientas integradas (Dart DevTools).
-  **UI consistente y personalizable:**
 - Los widgets de Flutter se ven y funcionan igual en todas las plataformas, sin inconsistencias entre Android/iOS.
-  **Comunidad activa y soporte de Google:**
 - Más de 150,000 paquetes disponibles en pub.dev.
 - Documentación oficial detallada y actualizada.
-  **Rendimiento competitivo:**
 - Supera a soluciones basadas en JavaScript (como React Native) al evitar el "puente" entre lenguajes

Flutter vs React Native vs Apps Nativas

Característica	Flutter	React Native	Nativo (Kotlin/Swift)
Lenguaje	Dart	JavaScript	Kotlin / Swift
Rendimiento	Alto (compilación nativa)	Medio-Alto	Excelente
UI	100% personalizada con widgets	Bridged con componentes nativos	Componentes nativos
Hot Reload	Sí	Sí	No
Comunidad y soporte	Alta y creciendo	Muy grande	Alta pero separada por plataforma
Acceso a funciones nativas	Completo con plugins y canales	Requiere puente con código nativo	Directo
Estabilidad	Alta	Media-Alta	Alta

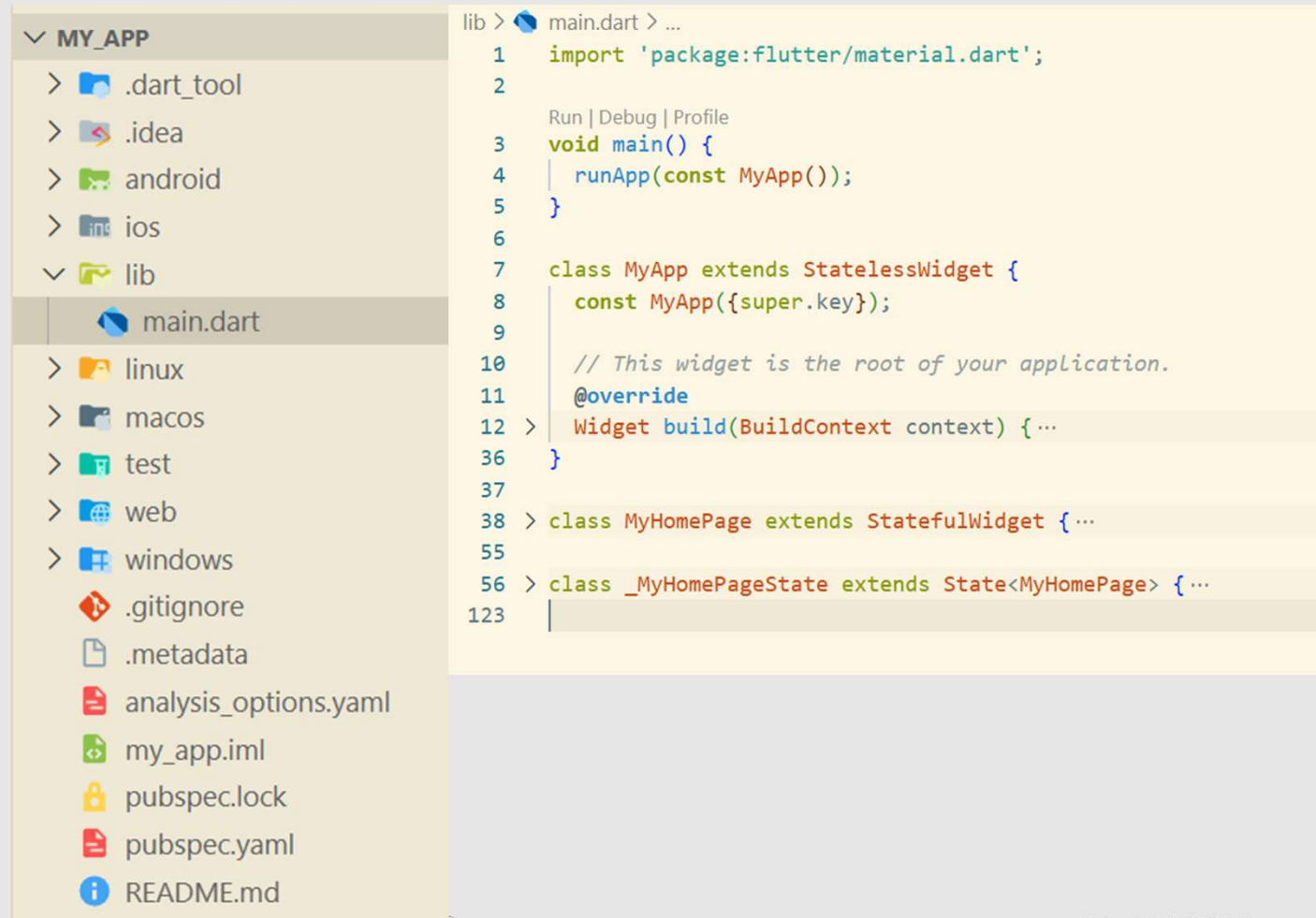
Instalación de Flutter y Configuración del Entorno

Requisitos generales

- Los requisitos para instalar Flutter son:
 - Un sistema operativo compatible: Windows, macOS o Linux
 - Espacio en disco: Al menos 2.8 GB (sin contar dependencias)
 - Un editor de texto o IDE: Visual Studio Code, Android Studio, etc.
 - Git instalado y accesible desde la terminal
- Para instalar Flutter, sigue los pasos actualizados en: <https://docs.flutter.dev/install>

Estructura Básica De Una App Flutter

- En Flutter, toda aplicación empieza por el archivo `main.dart` que se encuentra dentro de la carpeta `lib/`.
- Este archivo contiene la función principal (`main`) que arranca la aplicación con el método `runApp()`.



Widgets principales: MaterialApp y Scaffold

- En este ejemplo aparecen dos widgets fundamentales:

1. MaterialApp

- Es el contenedor raíz de toda la aplicación.
- Define la configuración global:
 - Título
 - Tema (colores, tipografías)
 - Rutas de navegación
 - Pantalla inicial (propiedad home)
- Analogía: Piensa en MaterialApp como el diseñador de interiores de una casa: establece el estilo, los colores y la distribución general.

2. Scaffold

- Proporciona la estructura visual estándar de cada pantalla.
- Incluye los elementos más comunes de una interfaz en Material Design:
 - AppBar: barra superior
 - Body: contenido principal
 - Drawer: menú lateral
 - FloatingActionButton: botón flotante
- Analogía: Scaffold es como el andamiaje de una casa: paredes, techo y suelo donde colocas los muebles (otros widgets).

Analogías prácticas

1. La casa 🏠

- main.dart → La puerta de entrada de la casa.
- MaterialApp → El interiorista que decide colores, distribución y estilo.
- Scaffold → Las paredes y el esqueleto de cada habitación.
- Widgets (Text, Button, etc.) → Los muebles y objetos dentro de la casa.

2. Las muñecas rusas 🧸

- Los widgets se anidan unos dentro de otros.
- Ejemplo:
- Aquí vemos:
 - MaterialApp contiene → Scaffold
 - Scaffold contiene → Center
 - Center contiene → Text

📌 Como las muñecas rusas, cada widget es una pieza que envuelve a otra hasta llegar al más pequeño.

```
MaterialApp(  
  home: Scaffold(  
    body: Center(  
      child: Text('Hola'),  
    ),  
  ),  
)
```

Ideas clave para recordar:

1. El archivo `main.dart` siempre es el inicio de la aplicación.
2. `MaterialApp` configura la app entera.
3. Cada pantalla suele estar envuelta en un `Scaffold`.
4. Los widgets se organizan en forma de árbol, de padres a hijos.

Comandos principales en Flutter

Flutter ofrece una CLI (Command Line Interface) muy potente para gestionar todo el ciclo de vida de una aplicación: desde la creación del proyecto hasta el despliegue en las diferentes plataformas.

A continuación, repasamos los comandos más importantes:

flutter create → Crear un proyecto

- Este comando genera la estructura básica de una aplicación Flutter.

`flutter create mi_app`

Esto crea una carpeta con:

- `lib/main.dart` → Punto de entrada de la app
- `pubspec.yaml` → Archivo de configuración de dependencias y assets
- Carpetas específicas para Android (`android/`), iOS (`ios/`), web (`web/`), etc.

flutter run → Ejecutar en dispositivo/emulador

- Permite compilar y lanzar la app en un emulador o dispositivo físico conectado.

`flutter run`

Opciones comunes:

- `flutter run -d chrome` → Ejecutar en navegador Chrome
- `flutter run -d emulator-5554` → Lanzar en un emulador Android específico
- `flutter run -d ios` → Ejecutar en un simulador de iOS

✚ Además, soporta Hot Reload (r) y Hot Restart (R) directamente desde la terminal.

Comandos principales en Flutter

flutter build → Generar binarios para distintas plataformas

- Compila la aplicación en modo release para distribuirla.

Ejemplos más usados:

- `flutter build apk` # Genera un APK para Android
- `flutter build appbundle` # Genera un AAB (obligatorio en Play Store)
- `flutter build ios` # Genera un IPA para iOS
- `flutter build web` # Compila para web (HTML/CSS/JS)
- `flutter build windows` # Compila ejecutable para Windows
- `flutter build linux` # Compila para Linux
- `flutter build macos` # Compila para macOS

Importante:

- Para Android → necesitarás configurar firma en `key.properties`
- Para iOS → requiere Xcode y cuenta de desarrollador de Apple

flutter pub get y **flutter pub add** → Gestión de dependencias

- Las dependencias en Flutter se gestionan en el archivo `pubspec.yaml`.

Actualizar dependencias `flutter pub get`

- Descarga las librerías definidas en `pubspec.yaml`.

Añadir una dependencia `flutter pub add http`

- Añade el paquete `http` automáticamente al `pubspec.yaml` e instala la librería.

Ejemplo manual:

Fichero `pubspec.yaml`

`dependencies:`

`http: ^0.13.6`

`provider: ^6.1.1`

Luego ejecutar:

- `flutter pub get`

Las librerías se obtienen del repositorio oficial: <https://pub.dev>

Comandos principales en Flutter

flutter test → Ejecución de tests unitarios

Al ejecutar flutter test, la salida mostrará si el test pasó o falló  .

- Flutter incluye un framework de testing integrado.

flutter test

-  Busca automáticamente todos los archivos terminados en `_test.dart` dentro de la carpeta `test/`.
- Ejemplo de test unitario (`test/suma_test.dart`):

```
import 'package:flutter_test/flutter_test.dart';

int sumar(int a, int b) => a + b;

void main() {

  test('La función sumar debería retornar la suma de dos enteros', () {

    expect(sumar(2, 3), 5);

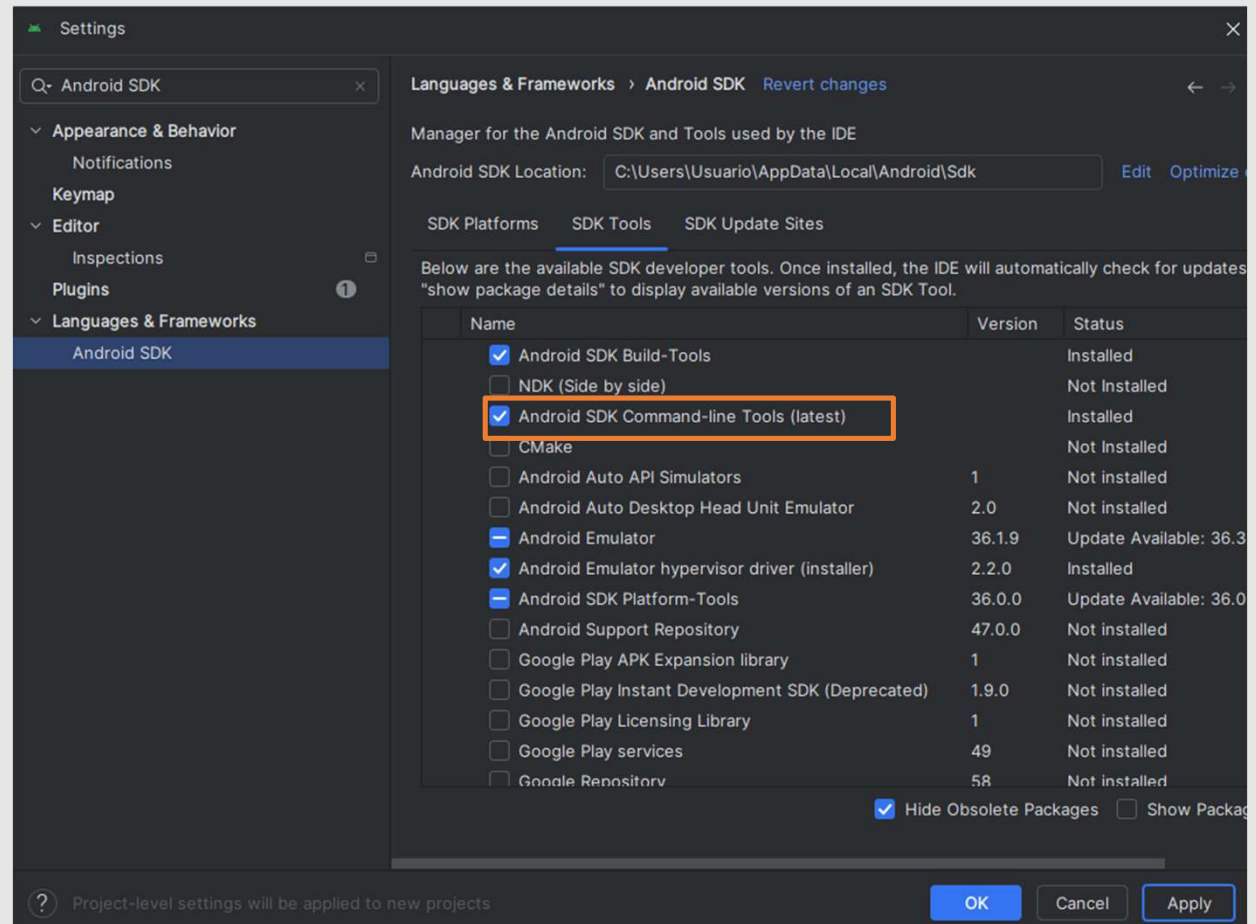
  });

}
```

Resumen comandos

Comando	Uso principal	Ejemplo
flutter create	Crear un nuevo proyecto	flutter create mi_app
flutter run	Ejecutar en dispositivo/emulador	flutter run -d chrome
flutter build	Generar binarios para distribución	flutter build apk
flutter pub get	Descargar dependencias definidas	flutter pub get
flutter pub add	Añadir nueva dependencia	flutter pub add provider
flutter test	Ejecutar tests unitarios	flutter test
flutter doctor	Ejecutar diagnóstico	Flutter doctor

Flutter doctor



flutter doctor --android-licenses

Gestión de Paquetes y Herramientas en Flutter

- Una de las mayores ventajas de Flutter es su ecosistema de paquetes en pub.dev, donde encontramos más de 150.000 librerías que extienden las funcionalidades de nuestras aplicaciones (bases de datos, networking, autenticación, animaciones, etc.).

Añadir dependencias al pubspec.yaml

- El archivo pubspec.yaml es el corazón de la configuración de un proyecto Flutter. Aquí definimos:
 - Nombre y versión de la app
 - Dependencias externas (paquetes de pub.dev)
 - Assets (imágenes, fuentes, traducciones)

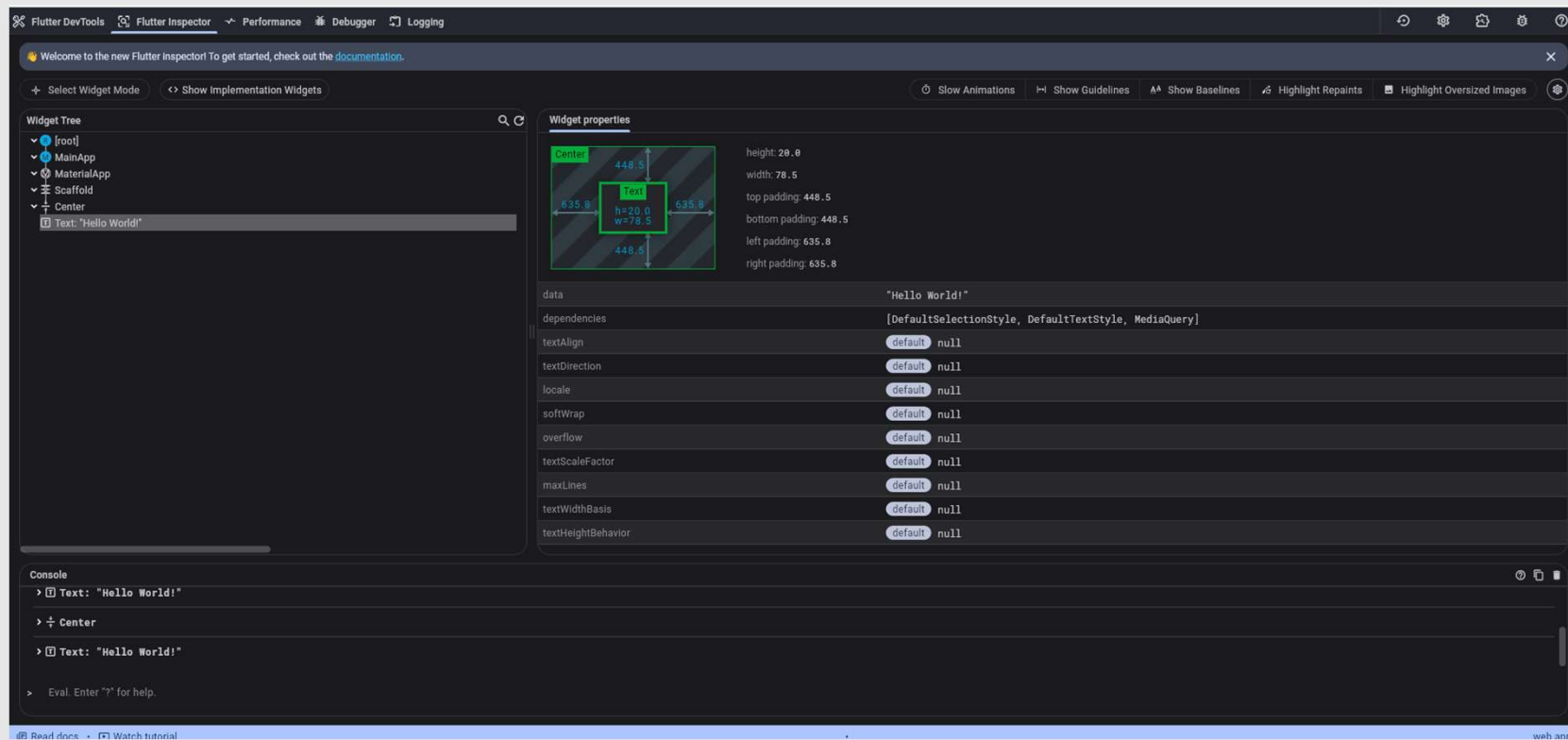
Cada vez que edites el archivo debes ejecutar:

flutter pub get

Extensiones VSCode

- Flutter ([Dart-Code.flutter](#)): soporte completo para Flutter (snippets, autocompletado, debugging).
- Dart ([Dart-Code.dart-code](#)): soporte para Dart.
- [Flutter Widget Snippets](#): autocompletado rápido de widgets.
- [Pubspec Assist](#): facilita añadir paquetes de pub.dev.

DevTools



Despliegue de Flutter en diferentes plataformas

Android

- Documentación oficial para publicar en Google Play Store
<https://docs.flutter.dev/deployment/android>
- Cómo generar APK/AAB: flutter build appbundle o flutter build apk
- Configuración de firma: keytool y build.gradle
- Proceso de subida a Play Console

Despliegue de Flutter en diferentes plataformas

- 🍏 iOS
- Guía completa para App Store
<https://docs.flutter.dev/deployment/ios>
- Requisitos: Cuenta de desarrollador Apple (\$99/año)
- Generar IPA: flutter build ipa
- Configurar Xcode: Certificados y provisionamiento

Despliegue de Flutter en diferentes plataformas

Windows

- Compilación para Windows
<https://docs.flutter.dev/deployment/windows>
- Requisitos: Visual Studio 2022 con workloads específicos
- Comando: flutter build windows
- Opciones de empaquetado: MSIX, EXE o instalador

Despliegue de Flutter en diferentes plataformas

Linux

- Despliegue en Linux
<https://docs.flutter.dev/deployment/linux>
- Dependencias: GTK, CMake y clang
- Formatos soportados: .deb, .rpm y Snap
- Personalización: Íconos y metadatos

Despliegue de Flutter en diferentes plataformas

Web

-  Publicación para web
<https://docs.flutter.dev/deployment/web>
- Optimización: `flutter build web --web-renderer canvaskit`
- Plataformas de hosting recomendadas:
 - Firebase: <https://firebase.google.com/docs/hosting>
 - GitHub Pages: gh-pages branch
 - Netlify/Vercel: Drag-and-drop deployment

02 Widgets en Flutter

Todo es un widget

En Flutter, absolutamente todo lo que ves en pantalla es un widget.

- Un texto es un widget.
- Un botón es un widget.
- La pantalla entera también es un widget.
- Incluso elementos que no son visibles, como espaciadores (`SizeBox`), alineaciones (`Align`) o márgenes (`Padding`), se representan como widgets.
- Esto significa que la UI se construye combinando widgets, de la misma forma que construyes un objeto más complejo a partir de piezas más simples.

Analogía con LEGO

- Piensa en Flutter como un juego de LEGO:
- Cada pieza pequeña (bloques de colores, ruedas, ventanas) es un widget simple (`Text`, `Icon`, `Image`).
- Al combinar piezas, puedes formar estructuras más complejas (una casa → `Scaffold`, un coche → `Row` con `Icons`).
- Igual que en LEGO, no necesitas reinventar la rueda: tienes piezas listas que puedes reutilizar y apilar.

Árbol de widgets y eficiencia en renderizado

EJ1

- Cuando Flutter dibuja la interfaz, organiza los widgets en un árbol jerárquico conocido como Widget Tree.

- Ejemplo de código:

```
MaterialApp(  
  home: Scaffold(  
    body: Center(  
      child: Text("Hola Flutter"),  
    ),  
  ),  
);
```

- Representación del árbol:

MaterialApp

└─ Scaffold

└─ Center

└─ Text("Hola Flutter")

Importancia del árbol de widgets:

- Cada widget tiene un padre y puede tener hijos.
- Cuando algo cambia en la interfaz (ej: texto o color), Flutter no redibuja todo, solo los widgets afectados.
- Esto hace que el renderizado sea muy eficiente:
 - Si cambias solo un Text, no se vuelve a renderizar toda la pantalla, únicamente ese widget.
 - La eficiencia viene de este mecanismo de reutilización de widgets en el árbol.

Estructura Básica De Una App Flutter

- Toda app Flutter comienza en el **archivo main.dart** dentro del directorio **lib/**. Este es el punto de entrada



Estructura básica de una app Flutter

- Imagina que construir una app es como armar una casa. Necesitas planos (código) y materiales (widgets). Todo comienza en el archivo main.dart (¡la puerta de entrada!).
- `import 'package:flutter/material.dart';`

Esto es como: Abrir tu caja de herramientas antes de construir.

material.dart contiene todos los widgets básicos (botones, textos, diseños).

Estructura Básica De Una App Flutter

EJ2

- **Función main()** – El motor de la app

```
void main() {  
  runApp(const MyApp());  
}
```

Así funciona:

- main() es como el interruptor de luz de tu casa (lo primero que se ejecuta).
- runApp() "enciende" la aplicación usando MyApp (el widget principal).

Estructura Básica De Una App Flutter

MainApp – La raíz de todo

```
class MainApp extends StatelessWidget {  
  const MainApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: HomePage(),  
    );  
  }  
}
```

Claves:

- MaterialApp es el "diseñador de interiores":
 - Configura temas, rutas y la pantalla inicial (home).
- Es Stateless porque no cambia después de crearse.

Estructura Básica De Una App Flutter

- **HomePage – La pantalla principal**

```
class HomePage extends StatelessWidget {  
  const HomePage({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Home Page'),  
      ),  
      body: Center(  
        child: Text('Welcome to the Home Page!'),  
      ),  
    );  
  }  
}
```

- **Partes del Scaffold (andamiaje):**

Widget	Función	Ejemplo real
AppBar	Barra superior con título	Como el nombre en WhatsApp
Body	Área principal de la pantalla	El chat en WhatsApp
(Opcionales) Drawer, FloatingActionButton	Menú lateral o botón flotante	El menú de Gmail

Estructura Básica De Una App Flutter

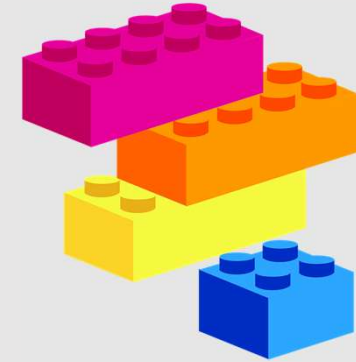
Consejos para recordar:

- 1) main.dart siempre es el punto de entrada.
- 2) MaterialApp envuelve toda la app (como un contenedor gigante).
- 3) Scaffold da estructura a cada pantalla (como paredes y techo).
- 4) Los widgets se anidan como muñecas rusas:

MaterialApp > Scaffold > Center > Text.



Widgets En Flutter



¿Qué es un Widget en Flutter?

- En Flutter, todo es un widget. Pero, ¿qué significa esto exactamente? Imagina que estás construyendo una casa de LEGO:
- **Widget = Pieza de LEGO**
Cada elemento visual o funcional en tu app (botones, textos, imágenes, pantallas, incluso la estructura completa) es un widget. Se combinan como bloques para crear interfaces complejas.

Definición Técnica

- Un widget es:
 1. Componente reutilizable: Pequeña unidad de UI (Interfaz de Usuario) o lógica.
 2. Configurable: Recibe parámetros (props) para personalizar su comportamiento/apariencia.
 3. Anidable: Los widgets se componen unos dentro de otros (como muñecas matrioskas).

Widgets En Flutter

Tipos:

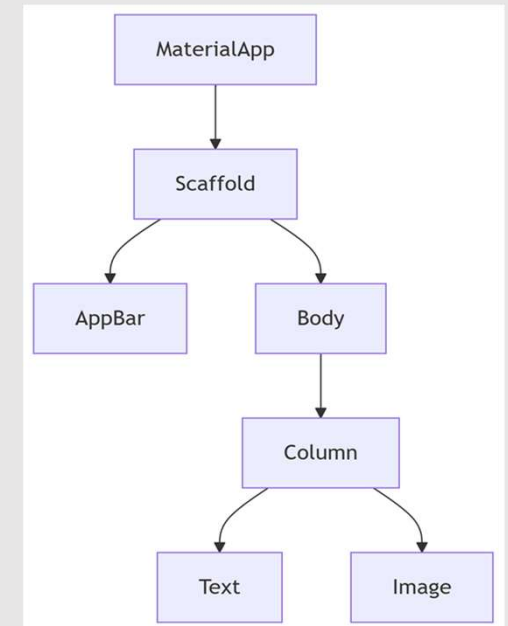
Tipo	Ejemplos comunes	¿Para qué sirve?
Layout	Row, Column, Stack	Organizar otros widgets en pantalla.
Visual	Text, Image, Icon	Mostrar contenido estático.
Interactivo	ElevatedButton, TextField	Responder a acciones del usuario.
Estructural	Scaffold, Container	Proporcionar "esqueleto" a la UI.

El Árbol de Widgets

- Flutter organiza los widgets en una jerarquía de padres e hijos:

¿Por qué importa?

- Eficiencia:** Flutter solo redibuja los widgets que cambian (¡no toda la pantalla!).
- Legibilidad:** El código sigue la estructura visual.



Widgets En Flutter

- **¿Por qué Flutter usa widgets?**

1. Modularidad: Puedes reusar widgets en cualquier parte.
2. Declarativo: Describe QUÉ quieres mostrar (no CÓMO hacerlo paso a paso).
3. Optimización: Flutter gestiona automáticamente su renderizado.

Regla de Oro

- "En Flutter, si lo ves en pantalla, es un widget. Si no se ve, pero afecta a otros (como gestores de estado), también puede ser un widget."

Widgets En Flutter: Widgets con y sin Estado

EJ3

- En Flutter, la interfaz de usuario se construye a partir de widgets, que se dividen en dos grandes categorías:

StatelessWidget

- Inmutables: No almacenan datos que cambien después de su creación.
- Renderizado: Solo se reconstruyen cuando reciben nuevos parámetros (desde el padre).
- Ciclo de vida: Muy simple → únicamente implementan el método build().

StatefulWidget

- Mutables: Mantienen un estado interno mediante la clase State.
- Renderizado: Se actualizan al invocar setState().
- Ciclo de vida: Más complejo, incluye métodos como:
 - initState() → inicialización.
 - dispose() → liberación de recursos.

Comprender esta distinción es clave para diseñar interfaces eficientes y mantenibles.

Widgets En Flutter: Widgets con y sin Estado

Tipo	Estado	Ejemplo	Ciclo de vida
StatelessWidget	✗ No guarda estado	Text, Icon	Solo build()
StatefulWidget	✓ Guarda estado	Checkbox, TextField	initState(), setState(), dispose()

En conclusión:

- Usa StatelessWidget cuando la UI no dependa de datos que cambian.
- Usa StatefulWidget cuando necesites dinamismo o interacción con el usuario.

Widgets En Flutter: Crear un Widget Personalizado

¿Por qué crear widgets personalizados?

- En Flutter, la interfaz se construye a base de widgets. Sin embargo, cuando un diseño se repite muchas veces en una app (por ejemplo, un mismo estilo de botón, tarjeta o cabecera), lo ideal es encapsularlo en un widget propio.
- Esto nos da tres ventajas principales:
 1. Reutilización → Evitamos repetir código en múltiples pantallas.
 2. Legibilidad → El código queda más limpio y fácil de mantener.
 3. Escalabilidad → Si necesitamos cambiar el diseño, lo hacemos en un solo lugar y se actualiza en toda la app.

Widgets En Flutter: Crear un Widget Personalizado

EJ4

- **Ejemplo: BotonPersonalizado**
- Aquí creamos un widget que recibe texto y color como parámetros y renderiza un botón con un estilo definido:

```
// Definición del widget personalizado
class BotonPersonalizado extends StatelessWidget {

  final String texto;
  final Color color;

  const BotonPersonalizado({super.key, required this.
texto, required this.color});
```

```
@override
Widget build(BuildContext context) {
  return Container(
    padding: EdgeInsets.all(12),
    decoration: BoxDecoration(
      color: color,
      borderRadius: BorderRadius.circular(8), // Bordes redondeados
    ),
    child: Text(
      texto,
      style: const TextStyle(
        color: Colors.white,
        fontSize: 16,
        fontWeight: FontWeight.bold,
      ),
      textAlign: TextAlign.center,
    ),
  );
}
```

Widgets En Flutter: Crear un Widget Personalizado

Uso en una pantalla del Widget personalizado de ejemplo

- Podemos instanciar el botón en cualquier parte de nuestra app simplemente pasando el texto y color deseado:

```
class MainApp extends StatelessWidget {  
  const MainApp({super.key});  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(  
          title: const Text('Ejemplo de Widget Personalizado'),  
        ),  
        body: Center(  
          child: BotonPersonalizado(  
            texto: 'Presióname',  
            color: Colors.blue,  
          ),  
        ),  
      ),  
    );  
  }  
}
```

Widgets En Flutter: Los fundamentales

<https://api.flutter.dev/flutter/widgets/>

- Estos son los "**ladrillos esenciales**" para construir interfaces en Flutter. Cada uno resuelve necesidades específicas de diseño y funcionalidad.
- Text – RichText
- Row & Column
- Container
- Image
- Flexible
- Extended

Widgets En Flutter: Los fundamentales

<https://api.flutter.dev/flutter/widgets/Text-class.html>

EJ5 y EJ6

Text

- Muestra texto en pantalla con estilo.

Propiedades clave:

- `style`: Fuente, tamaño, color (usar `TextStyle`).
- `textAlign`: Alineación (centro, izquierda, etc.).

- **`Text.rich()`**

Widgets En Flutter: Los fundamentales

<https://api.flutter.dev/flutter/widgets/Row-class.html>

EJ7

Row / Column

- Descripción:
 - Row: Organiza widgets horizontalmente.
 - Column: Organiza widgets verticalmente.
- Propiedades clave:
 - `mainAxisAlignment`: Alineación en el eje principal (ej: `MainAxisAlignment.center`).
 - `crossAxisAlignment`: Alineación en el eje secundario.
 - `children`: Lista de widgets hijos.

Widgets En Flutter: Los fundamentales

EJ8

Container

- Descripción: "Caja" personalizable con decoración y espaciado.
- Propiedades clave:
 - padding: Espacio interno.
 - margin: Espacio externo.
 - decoration: Color, bordes, sombras (BoxDecoration).
 - width/height: Tamaño fijo.

Padding
Añade **espacio interno** alrededor de un widget.

```
Padding(  
  padding: EdgeInsets.all(16),  
  child: Text("Texto con padding"),  
);
```

Widgets En Flutter: Los fundamentales

EJ9

Image

- Descripción: Muestra imágenes desde diferentes fuentes.
- Tipos de carga:
 - `Image.asset('ruta/local')`: Desde archivos del proyecto.
 - Meter la ruta local en el fichero de dependencias y actualizar el proyecto:
 - `flutter pub get`
 - `Image.network('URL')`: Desde internet.

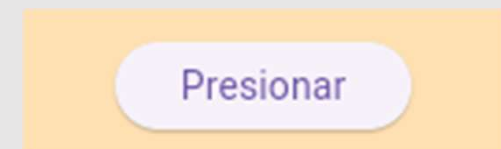
```
flutter:  
  uses-material-design: true  
  assets:  
    - assets/images/
```


Widgets En Flutter: Los fundamentales

EJ10

ElevatedButton

- Descripción: Botón con elevación visual (Material Design).
- Propiedades clave:
 - onPressed: Función al presionar (si es null, se deshabilita).
 - child: Widget hijo (texto, icono, etc.).



Widgets En Flutter: Los fundamentales

EJ11

ListView

- Descripción: Lista desplazable (vertical u horizontal).

Casos de uso:

- **Lista básica:**

```
ListView(  
  children: [  
    ListTile(title: Text('Item 1')),  
    ListTile(title: Text('Item 2')),  
  ],  
)
```

- **Dinámica (para muchos elementos):**

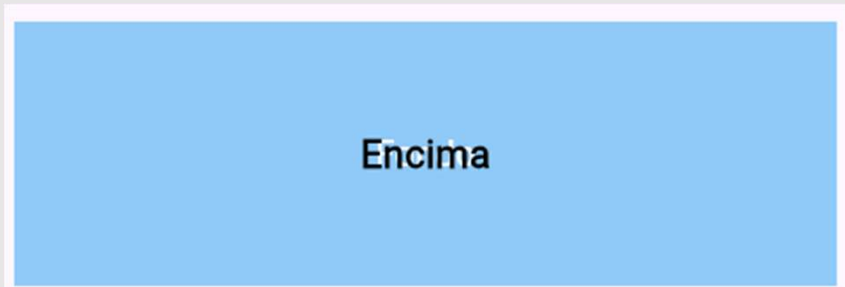
```
ListView.builder(  
  itemCount: 100,  
  itemBuilder: (context, index) {  
    return ListTile(title: Text('Item $index'));  
  },  
)
```

Widgets En Flutter: Los fundamentales

EJ12

Stack (Pila)

- Descripción: Superpone widgets (útil para elementos en capas).
- Ejemplo común: Texto sobre imagen o sobre otro texto, botones, etc.



Widgets En Flutter: Los fundamentales

EJ13

Expanded

- Descripción: Ocupa el espacio **disponible** en Row/Column.
- Regla clave: Solo funciona dentro de Row o Column.
- Ejemplo:

```
Row(  
  children: [  
    Expanded( // ← Ocupa 70% del espacio  
      flex: 7,  
      child: Container(color: Colors.red),  
    ),  
    Expanded( // ← Ocupa 30%  
      flex: 3,  
      child: Container(color: Colors.blue),  
    ),  
  ],  
)
```

Widgets En Flutter: Los fundamentales

EJ14

Flexible

- El widget Flexible se usa dentro de un Row o Column para controlar cómo los hijos ocupan el espacio disponible.
 - Permite que un widget crezca o se encoja según el espacio libre.
 - Se diferencia de Expanded:
 - Expanded ocupa todo el espacio disponible.
 - **Flexible ocupa solo lo necesario**, pero puede estirarse o encogerse según sea necesario.

```
Row(  
  children: [  
    Flexible(  
      flex: 2,  
      child: Container(color: Colors.red, height: 50),  
    ),  
    Flexible(  
      flex: 1,  
      child: Container(color: Colors.blue, height: 50),  
    ),  
  ],  
)
```

Widgets En Flutter: Los fundamentales

EJ15

AnimatedSwitcher

- Permite animar transiciones cuando cambia un widget hijo.
- Detecta si son diferentes usando la key (ValueKey).
- Hace una transición con la duración que le indiques

Cómo animar al cambiar estados:

- Se usa setState() para alternar entre:
 - Un Text
 - Un Icon
- Y el switcher suaviza ese cambio con animación

transitionBuilder

- Puedes personalizar cómo cambia el widget:
 - Fade
 - Scale
 - Slide
 - Fade + Scale (como en el ejemplo)
- Muy útil para cambios de texto, iconos o pantallas pequeñas con animación suave.

Widgets En Flutter: Los fundamentales

EJ16

FloatingActionButton (FAB)

- Botón circular flotante, normalmente para la acción principal de la pantalla.
- 1) FAB básico: El clásico botón flotante con icono.
- 2) FAB animado: Cambia de icono usando AnimatedSwitcher.
- 3) FAB extendido: Tiene texto e icono, útil para acciones importantes.
- 4) Speed Dial casero: Muestra FABs adicionales con animación. Sin librerías externas.
- 5) FAB docked con BottomAppBar: Una barra inferior que deja espacio al FAB central. Diseño moderno tipo "Google apps".

Widgets En Flutter: Los fundamentales

EJ17

DropDownButton, Checkbox, RadioButton, Switch and ListTile.

- Crean elementos para seleccionar entre varias opciones.
- DropDownButton es ideal si no necesitas validación; si vas a usar formularios y validar, mejor DropDownButtonFormField.
- Para Radio, usa un groupValue compartido (puede ser enum), así garantizas que solo uno esté seleccionado.
- CheckboxListTile y RadioListTile ahorran trabajo: incluyen título, subtítulo, icono secundario y gestionan la interacción.
- SwitchListTile es perfecto para preferencias de usuario (ajustes).



¿Preguntas?

Bibliografía y referencias

- Ediciones ENI – Los fundamentos del lenguaje Java Ed. 2025
- Ed. O'Really – Java Cookbook última actualización
- Ediciones ENI – Java Ed. 2021
- Pearson, 2016 – Estructuras de Datos con Java: Diseño de estructuras y algoritmos.
- Concepto de Programación: <https://concepto.de/programacion/>
- Documentos de la API de Java:
<https://docs.oracle.com/en/java/javase/17/docs/api/index.html>